Build your first WinUI app

Build your first WinUI app and see it come to life on Windows. This quickstart takes you from tool setup to a running app, then guides you through your first interface update.

Before you start

What you'll learn

- ✓ Choose and set up your development tools.
- ✓ Create and run a WinUI app.
- ✓ Update the interface and see your change in the running app.

Choose your development tools

Follow either recommended path through this quickstart:

- **winapp CLI**: Work from a terminal in your preferred editor. This command-line workflow also works well for agentic coding.
- **Visual Studio**: Use an integrated development and debugging experience.

If your tools are already installed, go to Creating your first app.

Installing the tools

winapp CLI

1. Enable Developer Mode. You can open its settings with Developer settings.

2. Install the .NET 10 SDK and the Windows App Development CLI. If you use WinGet, run:

   PowerShell

   ```
   winget install --id Microsoft.DotNet.SDK.10 --exact --source winget
   winget install --id Microsoft.WinAppCli --exact --source winget
   ```

   Skip an installation if the required tool is already installed.

3. Open a new terminal so it picks up the installed commands, then check the versions:

   PowerShell

   ```
   dotnet --version
   winapp --version
   ```

   Use .NET SDK 10 or later and winapp CLI 0.6 or later for this workflow.

> **Note**
>
> winapp CLI is in public preview. `winapp new` manages the WinUI project templates, but it does not install the .NET SDK. See the CLI 0.6 announcement for the new project and run commands.

Creating your first app

Use the workflow you installed above to create and launch the project.

winapp CLI

From a folder where you keep your projects, run:

PowerShell

```
winapp new --name MyWinUIApp --template winui --use-defaults
cd MyWinUIApp
winapp run
```

`winapp new` creates the blank WinUI app. `winapp run` builds the project and launches it; you don't need a separate build or package-registration command.

Your app opens a desktop window. Continue with **Make a change** below.

Make a change

Keep the project you just created. Add a message to its existing interface:

1. Close the running app. Open `MainPage.xaml` in your editor. If your template doesn't contain that file, open `MainWindow.xaml` instead.

2. Find the layout panel that holds the app's content. In the current CLI blank template, `MainPage.xaml` contains an empty `<Grid />`. Replace that empty grid with:

   XAML

   ```
   <Grid>
       <TextBlock
           Text="Hello, WinUI!"
           HorizontalAlignment="Center"
           VerticalAlignment="Center" />
   </Grid>
   ```

   If the panel already contains controls, add only the `TextBlock` inside that panel. Keep the existing controls, names, and event handlers. Don't replace the surrounding `Page` or `Window`, or the frame and title bar in `MainWindow.xaml`.

3. Save the file. Run `winapp run` again from the project folder, or press **F5** in Visual Studio.

Congratulations! You've created, run, and updated your first WinUI app. The window now displays **Hello, WinUI!**

Troubleshooting

Expand table

Problem	What to check
`winapp` or `dotnet` is not recognized.	Reopen the terminal after installation. Use the version commands in the CLI tab to confirm both tools are available.
`winapp new` is not recognized.	This workflow requires winapp CLI 0.6 or later. Update the CLI using the installation method you chose.
WinUI templates don't appear in Visual Studio.	Open the Visual Studio Installer, select **Modify**, and confirm that the **WinUI application development** workload is installed. Restart Visual Studio afterward.
The build reports an SDK or restore error.	Check the selected workflow's prerequisites and the first error in the build output. Use Windows App SDK support to check version requirements before changing project properties.
Deployment reports that Developer Mode is disabled.	Enable Developer Mode, then run the app again.
Your new message does not appear.	Save the XAML file and relaunch the project. In templates with a `MainPage.xaml`, add the message to that page rather than replacing the

Problem	What to check
	window's hosting frame.

Next steps

Build a notes app

The Notes tutorial uses a separate `WinUINotes` project to teach pages, navigation, data binding, and local storage. You can also find focused control examples in Code samples and the WinUI Gallery.

Last updated on 10/08/2026

Build a WinUI 3 notes app

This tutorial series demonstrates how to create a WinUI app using XAML and C#. The app you'll create is a note app, where the user can create, save, and load multiple notes. You can download or view the code for this tutorial from the GitHub repo.

Start here after you can build and run a blank project. This is a step-by-step introduction to WinUI 3 fundamentals using Visual Studio; no AI coding assistant or MVVM Toolkit is required.

In this tutorial, you learn how to:

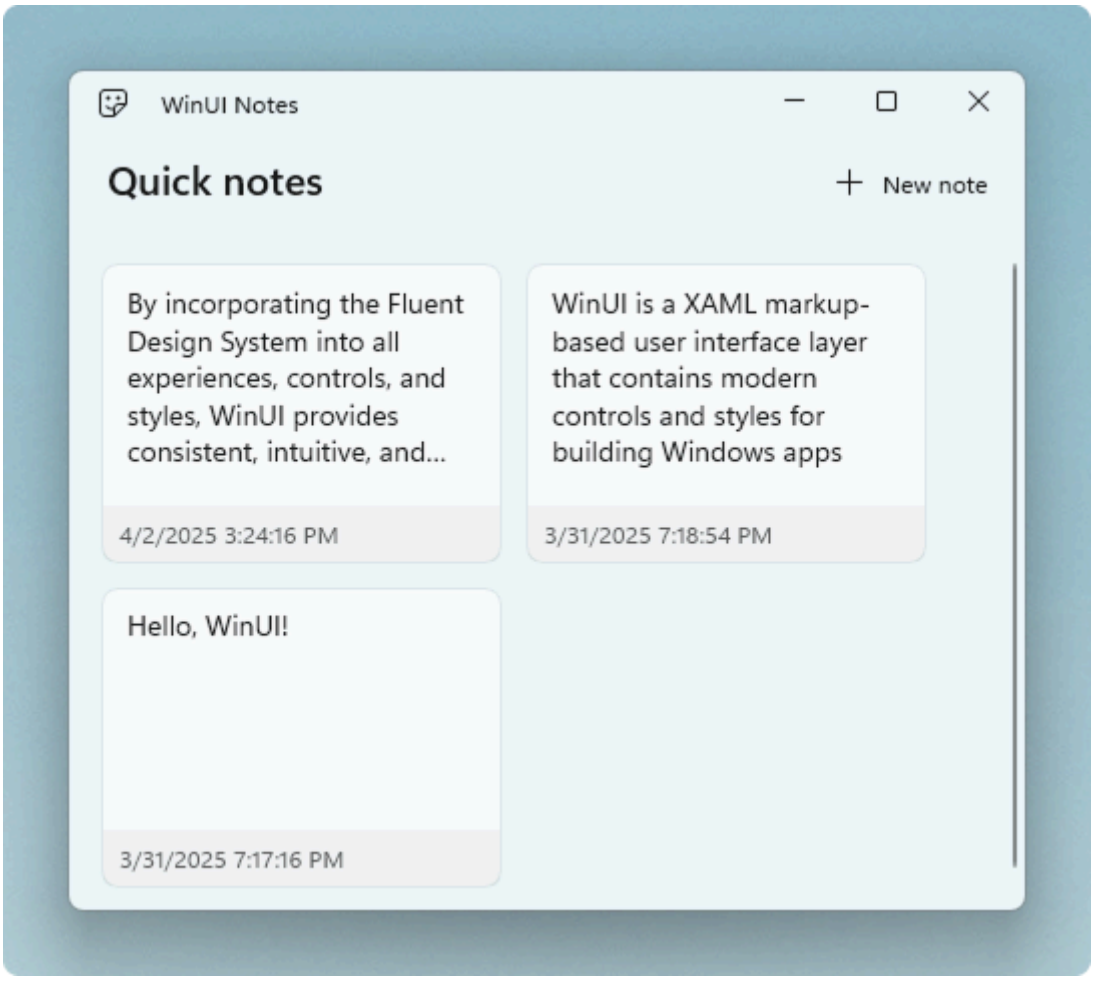
- ✓ Use XAML markup to define the user interface of the app.
- ✓ Interact with XAML elements through C# code.
- ✓ Save and load files from the local file system.
- ✓ Create views and bind them to data.
- ✓ Use navigation to move to and from pages in the app.
- ✓ Use resources like documentation and sample apps to create your own app.

You'll create an app that you can use to enter a note and save it to local app storage. The app will have two pages:

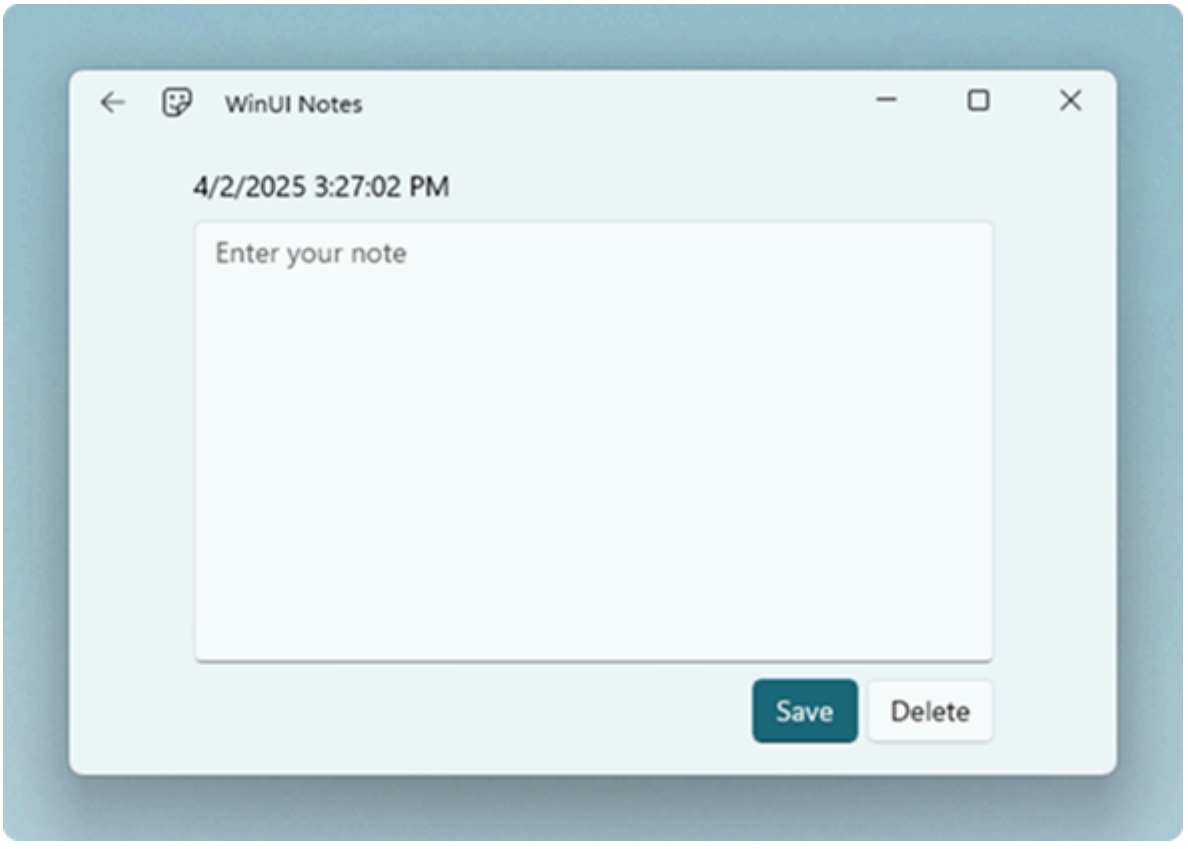
- `NotePage` - a page for editing a single note.
- `AllNotesPage` - a page to show all the saved notes.

The final application is shown below:

AllNotesPage



NotePage



Create the Visual Studio project

Follow the Visual Studio path in Quickstart: Create and run a WinUI 3 project for tool installation and project creation. Use a packaged C# project with these tutorial-specific settings:

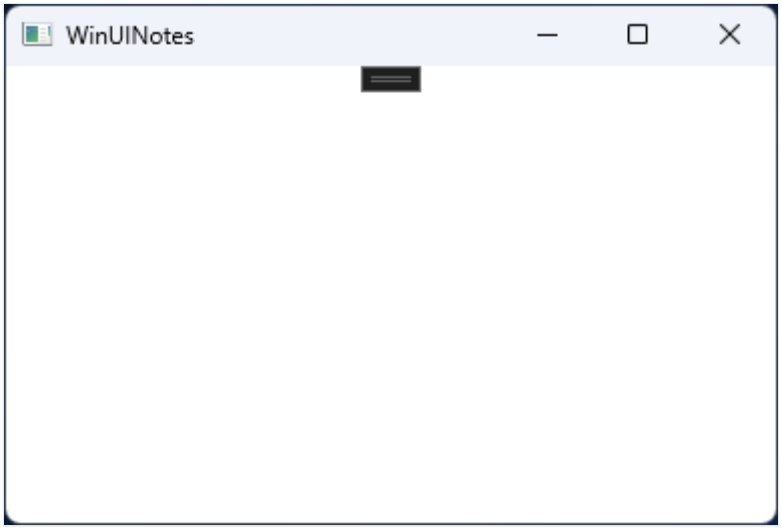
- **Project Name**

  This must be set to `WinUINotes`. If the project is named something different, the code you copy and paste from this tutorial may result in build errors. This is because Visual Studio uses the project name as the default *namespace* for your app code.

- **Windows App SDK**

  This tutorial uses features that are available in Windows App SDK 1.7 and later. Make sure the Windows App SDK NuGet package is updated to version 1.7 or later.

Build and launch the blank `WinUINotes` project before continuing. You should see an empty window that looks like this:



> **Tip**
>
> You'll frequently refer to API reference docs and conceptual docs while building Windows apps. In this tutorial, you'll see links inline in the text, and in groups labeled, "Learn more in the docs:". These links are optional; you don't need to follow them to complete the tutorial. They're provided in case you want to make note of where to find the information you'll need when you start to create your own apps.

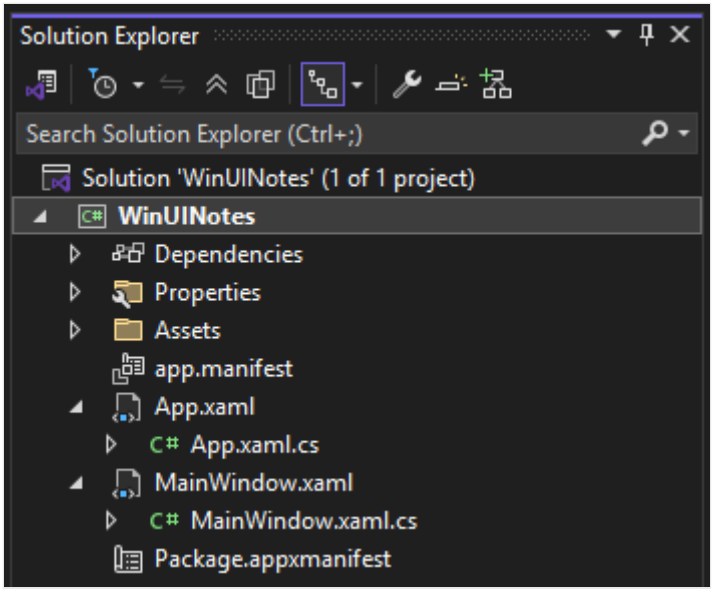
Continue to step 1 - Project setup

Last updated on 09/10/2026

Project setup

09/02/2025

Before you get into coding the app, we'll take a quick look at the Visual Studio project and take care of some project setup. When Visual Studio creates a WinUI 3 project, several important folders and code files are generated. These can be seen in the **Solution Explorer** pane of Visual Studio:



The items listed here are the ones you'll primarily interact with. These files help get the WinUI 3 app configured and running. Each file serves a different purpose, described below:

- **Assets** folder

  This folder contains your app's logo, images, and other media assets. It starts out populated with placeholder files for your app's logo. This logo represents your app in the Windows Start Menu, the Windows taskbar, and in the Microsoft Store when your app is published there.

- **App.xaml** and **App.xaml.cs**

  The **App.xaml** file contains app-wide XAML resources, such as colors, styles, or templates. The **App.xaml.cs** file generally contains code that instantiates and activates the application window. In this project, it points to the `MainWindow` class.

- **MainWindow.xaml** and **MainWindow.xaml.cs**

  These files represent your app's window.

- **Package.appxmanifest**

This package manifest file lets you configure publisher information, logos, processor architectures, and other details that determine how your app appears in the Microsoft Store.

XAML files and partial classes

Extensible Application Markup Language (XAML) is a declarative language that can initialize objects and set properties of objects. You can create visible UI elements in the declarative XAML markup. You can then associate a separate code file for each XAML file (typically called a *code-behind* file) that can respond to events and manipulate the objects that you originally declare in XAML.

There are generally two files with any XAML file, the `.xaml` file itself, and a corresponding code-behind file that is a child item of it in the **Solution Explorer**.

- The `.xaml` file contains XAML markup that defines your app UI. The class name is declared with the `x:Class` attribute.
- The code file contains code you create to interact with the XAML markup and a call to the `InitializeComponent` method. The class is defined as a `partial class`.

When you build your project, the `InitializeComponent` method is called to parse the `.xaml` file and generate code that's joined with the code `partial class` to create the complete class.

 Learn more in the docs:

- XAML Overview
- Partial Classes and Methods (C# Programming Guide)
- x:Class attribute, x:Class Directive

Update MainWindow

The `MainWindow` class included with the project is a sub-class of the XAML Window class, which is used to define the shell of the app. The app's window has two parts:

- The *client* portion of the window is where your content goes.
- The *non-client* portion is the part controlled by the Windows Operating System. It includes the title bar, where the caption controls (Min/Max/Close buttons), app icon, title, and drag area are. It also includes the frame around the outside of the window.

To make the WinUI Notes app consistent with Fluent Design guidelines, you'll make a few modifications to `MainWindow`. First, you'll apply Mica material as the window backdrop. Mica is an opaque, dynamic material that incorporates theme and desktop wallpaper to paint the

background of the window. Then, you'll extend your app's content into the title bar area and replace the system title bar with a XAML TitleBar control. This makes better use of space and gives you more control over the design, while providing all the functionality required of the title bar.

You'll also add a Frame as the content of the window. The `Frame` class works with the Page class to let you navigate between pages of content in your app. You'll add the pages in a later step.

> **Tip**
>
> You can download or view the code for this tutorial from the **GitHub repo**. To see the code as it is in this step, see this commit: **note page - initial**.

1. Double-click **MainWindow.xaml** in **Solution Explorer** to open it.
2. Between the opening and closing `<Window.. >` tags, replace any existing XAML with this:

XAML

```
<Window.SystemBackdrop>
    <MicaBackdrop Kind="Base"/>
</Window.SystemBackdrop>

<Grid>
    <Grid.RowDefinitions>
        <!-- Title Bar -->
        <RowDefinition Height="Auto" />
        <!-- App Content -->
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <TitleBar x:Name="AppTitleBar"
              Title="WinUI Notes">
        <TitleBar.IconSource>
            <FontIconSource Glyph="&#xF4AA;"/>
        </TitleBar.IconSource>
    </TitleBar>

    <!-- App content -->
    <Frame x:Name="rootFrame" Grid.Row="1"/>
</Grid>
```

3. Save the file by pressing `CTRL + S`, clicking the Save icon in the tool bar, or by selecting the menu **File** > **Save MainPage.xaml**.

Don't worry if you don't understand what all this XAML markup does right now. As you build the rest of the app UI, we'll explain XAML concepts in more detail.

> **Note**
>
> In this markup, you see two different ways to set properties in XAML. The first and shortest way is to use XAML *attribute syntax*, like this: `<object attribute="value">`. This works great for simple values, such as `<MicaBackdrop Kind="Base"/>`. But it only works for values where the XAML parser knows how to convert the string to the expected value type.
>
> If the property value is more complex, or the XAML parser doesn't know how to convert it, you must use *property element syntax* to create the object. like this:
>
> XAML
>
> ```
> <object ... >
>     <object.property>
>         value
>     </object.property>
> </object>
> ```
>
> For example, to set the `Window.SystemBackdrop` property, you have to use property element syntax and explicitly create the `MicaBackdrop` element. But you can use simple attribute syntax to set the `MicaBackdrop.Kind` property.
>
> XAML
>
> ```
> <Window ... >
>    <Window.SystemBackdrop>
>        <MicaBackdrop Kind="Base"/>
>    </Window.SystemBackdrop>
>     ...
> </Window>
> ```
>
> In `MainWindow.xaml`, `<Window.SystemBackdrop>`, `<Grid.RowDefinitions>`, and `<TitleBar.IconSource>` contain complex values that must be set with property element syntax.
>
> Learn more in the docs:
>
> - XAML syntax guide

If you run the app now, you'll see the XAML `TitleBar` element you added, but it will be below the system title bar, which is still showing.



You need to write a bit of code to finish replacing the system title bar.

1. Open **MainWindow.xaml.cs**. You can open the code-behind for **MainWindow.xaml** (or any XAML file) in three ways:

   - If the **MainWindow.xaml** file is open and is the active document being edited, press `F7`.
   - If the **MainWindow.xaml** file is open and is the active document being edited, right-click in the text editor and select **View Code**.
   - Use **Solution Explorer** to expand the **MainWindow.xaml** entry, revealing the **MainWindow.xaml.cs** file. Double-click the file to open it.

2. In the constructor for `MainWindow`, add this code after the call to `InitializeComponent`:

   C#

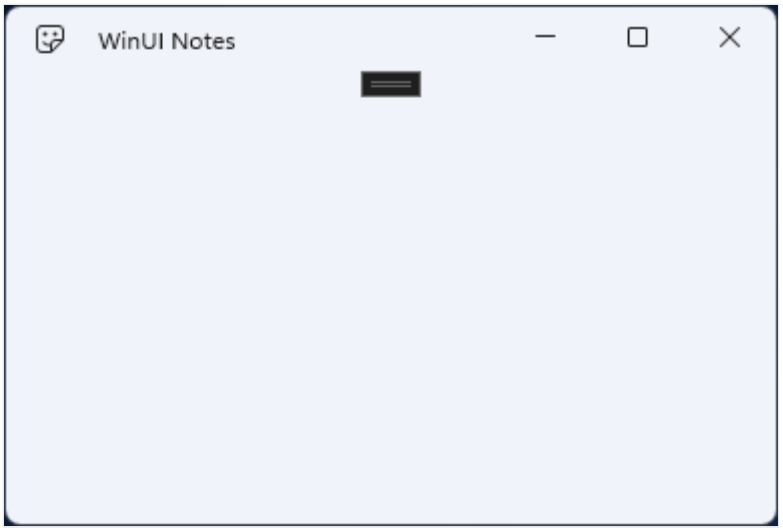
   ```
   public MainWindow()
   {
       this.InitializeComponent();

       // ↓ Add this. ↓
       // Hide the default system title bar.
       ExtendsContentIntoTitleBar = true;
       // Replace system title bar with the WinUI TitleBar.
       SetTitleBar(AppTitleBar);
       // ↑ Add this. ↑
   }
   ```

   The ExtendsContentIntoTitleBar property hides the default system title bar and extends your app XAML into that area. The call to SetTitleBar then tells the system to treat the XAML element you specified (`AppTitleBar`) as the title bar for the app.

3. Build and run the project by pressing `F5`, clicking the Debug "Start" button in the tool bar, or by selecting the menu **Run** > **Start Debugging**.

When you run the app now, you'll see an empty window with a mica background and the XAML title bar that's replaced the system title bar.



Continue to step 2 - Add a note page

Create a page for a note

09/02/2025

Now you'll create a page that allows a user to edit a note, and then you'll write the code to save or delete the note.

> **Tip**
>
> You can download or view the code for this tutorial from the GitHub repo. To see the code as it is in this step, see this commit: note page - initial.

First, add the new page to the project:

1. In the **Solution Explorer** pane of Visual Studio, right-click on the **WinUINotes** project > **Add** > **New Item...**.
2. In the **Add New Item** dialog, select **WinUI** in the template list on the left-side of the window. Next, select the **Blank Page (WinUI 3)** template. Name the file *NotePage.xaml*, and then select **Add**.
3. The **NotePage.xaml** file will open in a new tab, displaying all of the XAML markup that represents the UI of the page. Replace the `<Grid> ... </Grid>` element in the XAML with the following markup:

```
XAML
<Grid Padding="16" RowSpacing="8">
    <Grid.RowDefinitions>
        <RowDefinition Height="*"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*"/>
        <ColumnDefinition Width="400"/>
        <ColumnDefinition Width="*"/>
    </Grid.ColumnDefinitions>

    <TextBox x:Name="NoteEditor"
         AcceptsReturn="True"
         TextWrapping="Wrap"
         PlaceholderText="Enter your note"
         Header="New note"
         ScrollViewer.VerticalScrollBarVisibility="Auto"
         Width="400"
         Grid.Column="1"/>

    <StackPanel Orientation="Horizontal"
```

```
            HorizontalAlignment="Right"
            Spacing="4"
            Grid.Row="1" Grid.Column="1">
        <Button Content="Save" Style="{StaticResource AccentButtonStyle}"/>
        <Button Content="Delete"/>
    </StackPanel>
</Grid>
```

4. Save the file by pressing `CTRL + S`, clicking the Save icon in the tool bar, or by selecting the menu **File** > **Save NotePage.xaml**.

   If you run the app right now, you won't see the note page you just created. That's because you still need to set it as the content of the `Frame` control in `MainWindow`.

5. Open **MainWindow.xaml** and set `NotePage` as the SourcePageType on the `Frame`, like this:

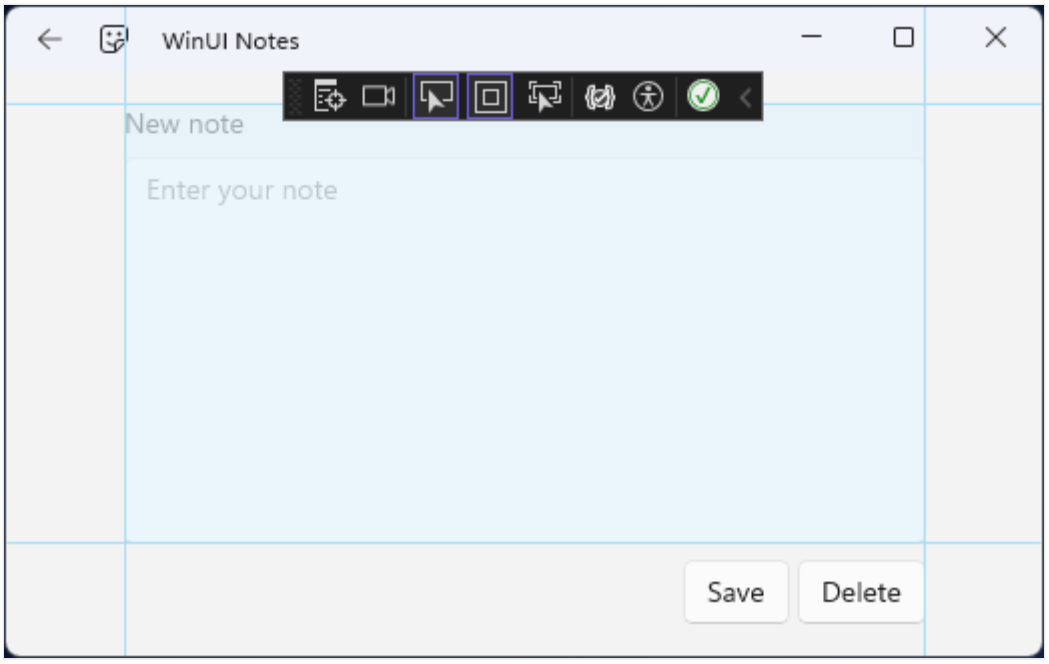
   XAML

   ```
   <Frame x:Name="rootFrame" Grid.Row="1"
          SourcePageType="local:NotePage"/>
   ```

   Now when you run the app, the `Frame` will load an instance of `NotePage` and show it to the user.

> ⓘ **Important**
>
> XAML namespace (xmlns) mappings are the XAML counterpart to the C# `using` statement. `local:` is a prefix that is mapped for you within the XAML pages for your app project (`xmlns:local="using:WinUINotes"`). It's mapped to refer to the same namespace that's created to contain the `x:Class` attribute and code for all the XAML files including **App.xaml**. As long as you define any custom classes you want to use in XAML in this same namespace, you can use the `local:` prefix to refer to your custom types in XAML.

Let's break down the key parts of the XAML controls placed on the page:



- The Grid.RowDefinitions and Grid.ColumnDefinitions define a grid with 2 rows and 3 columns (placed below the title bar).
  - The bottom row is automatically (`Auto`) sized to fit its content, the two buttons. The top row uses all the remaining vertical space (`*`).
  - The middle column is `400` epx wide and is where the note editor goes. The columns on either side are empty and split all the remaining horizontal space between them (`*`).

  > ⓘ Note
  >
  > Because of how the scaling system works, when you design your XAML app, you're designing in effective pixels, not actual physical pixels. Effective pixels (epx) are a virtual unit of measurement, and they're used to express layout dimensions and spacing, independent of screen density.

- `<TextBox x:Name="NoteEditor" ... > ... </TextBox>` is a text entry control (TextBox) configured for multi-line text entry, and is placed in the top center cell of the `Grid` (`Grid.Column="1"`). Row and column indexes are 0-based, and by default, controls are placed in row 0 and column 0 of the parent `Grid`. So this is the equivalent of specifying Row 0, Column 1.

- `<StackPanel Orientation="Horizontal" ... > ... </StackPanel>` defines a layout control (StackPanel) that stacks its children either vertically (default) or horizontally. It's placed in the bottom center cell of the `Grid` (`Grid.Row="1" Grid.Column="1"`).

  > ⓘ Note

> `Grid.Row="1" Grid.Column="1"` is an example of XAML attached properties. Attached properties let one XAML object set a property that belongs to a different XAML object. Often, as in this case, child elements can use attached properties to inform their parent element of how they are to be presented in the UI.

- Two `<Button>` controls are inside the `<StackPanel>` and arranged horizontally. You'll add the code to handle the buttons' Click events in the next section.

 Learn more in the docs:

- Responsive layouts with XAML
- Layout panels
- XAML namespaces and namespace mapping

Load and save a note

Open the **NotePage.xaml.cs** code-behind file. When you add a new XAML file, the code-behind contains a single line in the constructor, a call to the `InitializeComponent` method:

C#

```
namespace WinUINotes
{
    public sealed partial class NotePage : Page
    {
        public NotePage()
        {
            this.InitializeComponent();
        }
    }
}
```

The `InitializeComponent` method reads the XAML markup and initializes all of the objects defined by the markup. The objects are connected in their parent-child relationships, and the event handlers defined in code are attached to events set in the XAML.

Now you're going to add code to the **NotePage.xaml.cs** code-behind file to handle loading and saving notes.

1. Add the following variable declarations to the `NotePage` class:

   C#

```
public sealed partial class NotePage : Page
{
    private StorageFolder storageFolder =
ApplicationData.Current.LocalFolder;
    private StorageFile? noteFile = null;
    private string fileName = "note.txt";
```

When a note is saved, it's saved to the app's local storage as a text file.

You use the StorageFolder class to access the app's local data folder. This folder is specific to your app, so notes saved here can't be accessed by other apps. You use the StorageFile class to access the text file saved in this folder. The name of the file is represented by the `fileName` variable. For now, set `fileName` to "*note.txt*".

2. Create an event handler for the note page's Loaded event.

C#

```
public NotePage()
{
    this.InitializeComponent();
    // ↓ Add this. ↓
    Loaded += NotePage_Loaded;
}

// ↓ Add this event handler method. ↓
private async void NotePage_Loaded(object sender, RoutedEventArgs e)
{
    noteFile = (StorageFile)await storageFolder.TryGetItemAsync(fileName);
    if (noteFile is not null)
    {
        NoteEditor.Text = await FileIO.ReadTextAsync(noteFile);
    }
}
```

In this method, you call TryGetItemAsync to retrieve the text file from the folder. If the file doesn't exist, it returns `null`. If the file does exist, call ReadTextAsync to read the text from the file into the `NoteEditor` control's Text property. (Remember, `NoteEditor` is the `TextBox` control you created in the XAML file. You reference it here in your code-behind file using the `x:Name` you assigned to it.)

> **ⓘ Important**
>
> You need to mark this method with the `async` keyword because the file access calls are asynchronous. In short, if you call a method that ends in `...Async` (like `TryGetItemAsync`), you can add the **await** operator to the call. This keeps subsequent

code from executing until the awaited call completes and keeps your UI responsive. When you use `await`, the method that you're calling from needs to be marked with the async keyword. For more info, see Call asynchronous APIs in C#.

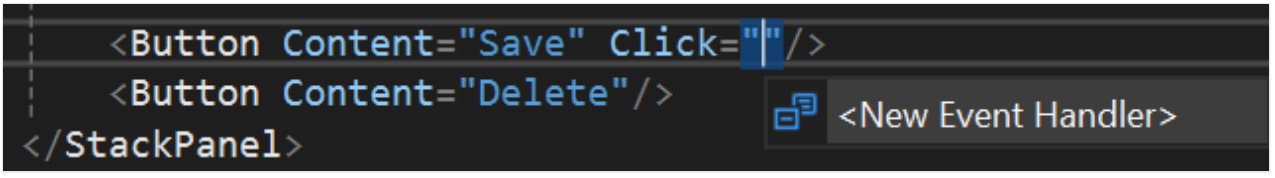
Learn more in the docs:

- Access files and folders with WinRT APIs
- Call asynchronous APIs in C#

Add event handlers

Next, add the Click event handlers for the for the **Save** and **Delete** buttons. Adding event handlers is something that you'll do often while creating your apps, so Visual Studio provides several features to make it easier.

1. In the **NotePage.xaml** file, place your cursor after the `Content` attribute in the **Save** `Button` control. Type `Click=`. At this point, Visual Studio should pop up an auto-complete UI that looks like this:



   - Press the down-arrow key to select **<New Event Handler>**, then press `Tab`. Visual Studio will complete the attribute with `Click="Button_Click"` and add an event handler method named `Button_Click` in the **NotePage.xaml.cs** code-behind file.

   Now, you should rename the `Button_Click` method to something more meaningful. You'll do that in the following steps.

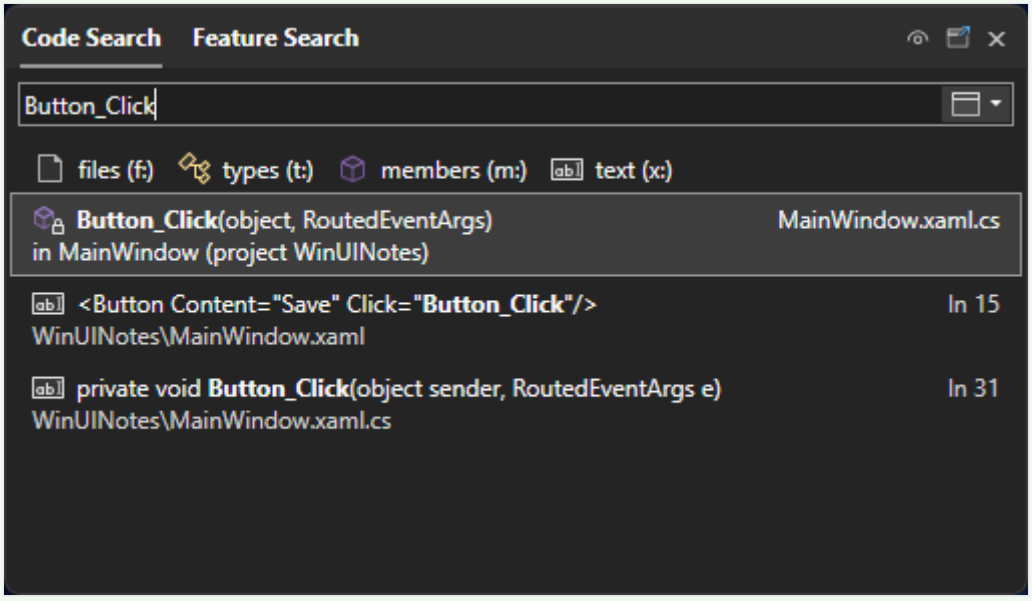
2. In **NotePage.xaml.cs**, find the method that was added for you:

```C#
private void Button_Click(object sender, RoutedEventArgs e)
{

}
```

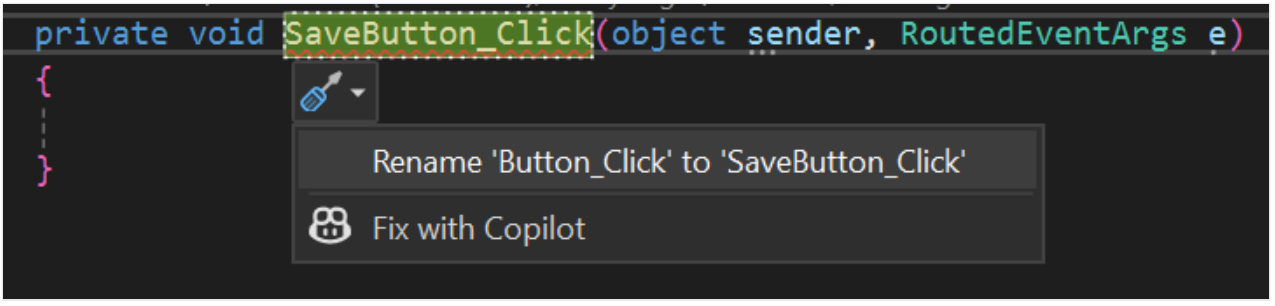
> **Tip**

To locate code in your app, click **Search** in the Visual Studio title bar and use the **Code Search** option. Double-click the search result to open the code in the code editor.



3. Place your cursor before the "B" in `Button` and type `Save`. Wait a moment, and the method name will be highlighted in green.

4. When you hover over the method name, Visual Studio will show a tooltip with a screwdriver or lightbulb icon. Click the down-arrow button next to the icon, then click **Rename 'Button_Click' to 'SaveButton_Click'**.



   Visual Studio will rename the method everywhere in your app, including in the XAML file where you first added it to the `Button`.

5. Repeat these steps for the **Delete** button, and rename the method to `DeleteButton_Click`.

Now that the event handlers are hooked up, you can add the code to save and delete the note file.

1. Add this code in the `SaveButton_Click` method to save the file. Notice that you also need to update the method signature with the `async` keyword.

```
C#
```

```
private async void SaveButton_Click(object sender, RoutedEventArgs e)
{
    if (noteFile is null)
    {
        noteFile = await storageFolder.CreateFileAsync(fileName,
CreationCollisionOption.ReplaceExisting);
    }
    await FileIO.WriteTextAsync(noteFile, NoteEditor.Text);
}
```

In the `SaveButton_Click` method, you first check to see if `noteFile` has been created. If it's `null`, then you have to create a new file in the local storage folder with the name represented by the `fileName` variable, and assign the file to the `noteFile` variable. Then, you write the text in the `TextBox` control to the file represented by `noteFile`.

2. Add this code in the `DeleteButton_Click` method to delete the file. You need to update the method signature with the `async` keyword here, too.

C#

```
private async void DeleteButton_Click(object sender, RoutedEventArgs e)
{
    if (noteFile is not null)
    {
        await noteFile.DeleteAsync();
        noteFile = null;
        NoteEditor.Text = string.Empty;
    }
}
```

In the `DeleteButton_Click` method, you first check to see if `noteFile` exists. If it does, delete the file represented by `noteFile` from the local storage folder and set `noteFile` to `null`. Then, reset the text in the `TextBox` control to an empty string.

> **ⓘ Important**
>
> After the text file is deleted from the file system, it's important to set `noteFile` to `null`. Remember that `noteFile` is a StorageFile that provides access to the system file in your app. After the system file is deleted, `noteFile` still points to where the system file was, but doesn't know that it no longer exists. If you try to read, write, or delete the system file now, you'll get an error.

3. Save the file by pressing `CTRL + S`, clicking the Save icon in the tool bar, or by selecting the menu **File** > **Save NotePage.xaml.cs**.

The final code for the code-behind file should look like this:

C#

```
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using System;
using Windows.Storage;

namespace WinUINotes
{
    public sealed partial class NotePage : Page
    {
        private StorageFolder storageFolder = ApplicationData.Current.LocalFolder;
        private StorageFile? noteFile = null;
        private string fileName = "note.txt";

        public NotePage()
        {
            this.InitializeComponent();
            Loaded += NotePage_Loaded;
        }

        private async void NotePage_Loaded(object sender, RoutedEventArgs e)
        {
            noteFile = (StorageFile)await storageFolder.TryGetItemAsync(fileName);
            if (noteFile is not null)
            {
                NoteEditor.Text = await FileIO.ReadTextAsync(noteFile);
            }
        }

        private async void SaveButton_Click(object sender, RoutedEventArgs e)
        {
            if (noteFile is null)
            {
                noteFile = await storageFolder.CreateFileAsync(fileName,
CreationCollisionOption.ReplaceExisting);
            }
            await FileIO.WriteTextAsync(noteFile, NoteEditor.Text);
        }

        private async void DeleteButton_Click(object sender, RoutedEventArgs e)
        {
            if (noteFile is not null)
            {
                await noteFile.DeleteAsync();
                noteFile = null;
                NoteEditor.Text = string.Empty;
            }
        }
    }
}
```

Test the note

With this code in place, you can test the app to make sure the note saves and loads correctly.

1. Build and run the project by pressing F5, clicking the Debug "Start" button in the tool bar, or by selecting the menu **Run** > **Start Debugging**.
2. Type into the text entry box and press the **Save** button.
3. Close the app, then restart it. The note you entered should be loaded from the device's storage.
4. Press the **Delete** button.
5. Close the app, restart it. You should be presented with a new blank note.

> ⓘ **Important**
>
> After you've confirmed that saving and deleting a note works correctly, create and save a new note again. You'll want to have a saved note to test the app in later steps.

Continue to step 3 - Add a view and model for the note

Add a view and model for the note

09/02/2025

This portion of the tutorial introduces the concepts of data views and models.

In the previous steps of the tutorial, you added a new page to the project that lets the user save, edit, or delete a single note. However, because the app needs to handle more than one note, you need to add another page that displays all the notes (call it `AllNotesPage`). This page let's the user choose a note to open in the editor page so they can view, edit, or delete it. It should also let the user create a new note.

To accomplish this, `AllNotesPage` needs to have a collection of notes, and a way to display the collection. This is where the app runs into trouble because the note data is tightly bound to the `NotePage` file. In `AllNotesPage`, you just want to display all the notes in a list or other collection view, with information about each note, like the date it was created and a preview of the text. With the note text being tightly bound to the `TextBox` control, there's no way to do this.

Before you add a page to show all the notes, let's make some changes to separate the note data from the note presentation.

Views and models

Typically, a WinUI app has at least a *view layer* and a *data layer*.

The view layer defines the UI using XAML markup. The markup includes data binding expressions (such as x:Bind) that define the connection between specific UI components and data members. Code-behind files are sometimes used as part of the view layer to contain additional code needed to customize or manipulate the UI, or to extract data from event handler arguments before calling a method that performs the work on the data.

The data layer, or *model*, defines the types that represent your app data and related logic. This layer is independent of the view layer, and you can create multiple different views that interact with the data.

Currently, the `NotePage` represents a view of data (the note text). However, after the data is read into the app from the system file, it exists only in the `Text` property of the `TextBox` in `NotePage`. It's not represented in the app in a way that lets you present the data in different ways or in different places; that is, the app doesn't have a data layer. You'll restructure the project now to create the data layer.

Separate the view and model

> **Tip**
>
> You can download or view the code for this tutorial from the GitHub repo. To see the code as it is in this step, see this commit: note page - view-model.

Refactor the existing code to separate the model from the view. The next few steps will organize the code so that views and models are defined separately from each other.

1. In **Solution Explorer**, right-click on the **WinUINotes** project and select **Add** > **New Folder**. Name the folder *Models*.
2. Right-click on the **WinUINotes** project again and select **Add** > **New Folder**. Name the folder *Views*.
3. Find the **NotePage.xaml** item and drag it to the **Views** folder. The **NotePage.xaml.cs** file should move with it.

   > **Note**
   >
   > When you move a file, Visual Studio usually prompts you with a warning about how the move operation may take a long time. This shouldn't be a problem here, press **OK** if you see this warning.
   >
   > Visual Studio may also ask you if you want to adjust the namespace of the moved file. Select **No**. You'll change the namespace in the next steps.

Update the view namespace

Now that the view has been moved to the **Views** folder, you'll need to update the namespaces to match. The namespace for the XAML and code-behind files of the pages is set to `WinUINotes`. This needs to be updated to `WinUINotes.Views`.

1. In the **Solution Explorer** pane, expand **NotePage.xaml** to reveal the code-behind file.
2. Double-click on the **NotePage.xaml.cs** item to open the code editor if it's not already open. Change the namespace to `WinUINotes.Views`:

   C#

   ```
   namespace WinUINotes.Views
   ```

3. Double-click on the **NotePage.xaml** item to open the XAML editor if it's not already open. The old namespace is referenced through the `x:Class` attribute, which defines which class type is the code-behind for the XAML. This entry isn't just the namespace, but the namespace with the type. Change the `x:Class` value to `WinUINotes.Views.NotePage`:

```
XAML

x:Class="WinUINotes.Views.NotePage"
```

Fix the namespace reference in MainWindow

In the previous step, you created the note page and updated `MainWindow.xaml` to navigate to it. Remember that it was mapped with the `local:` namespace mapping. It's common practice to map the name `local` to the root namespace of your project, and the Visual Studio project template already does this for you (`xmlns:local="using:WinUINotes"`). Now that the page has moved to a new namespace, the type mapping in the XAML is now invalid.

Fortunately, you can add your own namespace mappings as needed. You need to do this to access items in different folders you create in your project. This new XAML namespace will map to the namespace of `WinUINotes.Views`, so name it `views`. The declaration should look like the following attribute: `xmlns:views="using:WinUINotes.Views"`.

1. In the **Solution Explorer** pane, double-click on the **MainWindow.xaml** entry to open it in the XAML editor.

2. Add this new namespace mapping on the line below the mapping for `local`:

```
XAML

xmlns:views="using:WinUINotes.Views"
```

3. The `local` XAML namespace was used to set the `Frame.SourcePageType` property, so change it to `views` there. Your XAML should now look like this:

```
XAML

<Window
    x:Class="WinUINotes.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:WinUINotes"
    xmlns:views="using:WinUINotes.Views"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
```

```
    mc:Ignorable="d"
    Title="WinUI Notes">

    <!-- ... Unchanged XAML not shown. -->

        <Frame x:Name="rootFrame" Grid.Row="1"
               SourcePageType="views:NotePage"/>

    <!-- ... Unchanged XAML not shown. -->

</Window>
```

4. Build and run the app. The app should run without any compiler errors, and everything should still work as before.

Define the model

Currently, the model (the data) is embedded in the note view. You'll create a new class to represent a note page's data:

1. In the **Solution Explorer** pane, right-click on the **Models** folder and select **Add** > **Class...**.
2. Name the class *Note.cs* and press **Add**. The **Note.cs** file will open in the code editor.
3. Replace the code in the **Note.cs** file with this code, which makes the class `public` and adds properties and methods for handling a note:

```
C#
```

```
using System;
using System.Threading.Tasks;
using Windows.Storage;

namespace WinUINotes.Models
{
    public class Note
    {
        private StorageFolder storageFolder =
ApplicationData.Current.LocalFolder;
        public string Filename { get; set; } = string.Empty;
        public string Text { get; set; } = string.Empty;
        public DateTime Date { get; set; } = DateTime.Now;

        public Note()
        {
            Filename = "notes" + DateTime.Now.ToBinary().ToString() + ".txt";
        }

        public async Task SaveAsync()
        {
```

```
            // Save the note to a file.
            StorageFile noteFile = (StorageFile)await
storageFolder.TryGetItemAsync(Filename);
            if (noteFile is null)
            {
                noteFile = await storageFolder.CreateFileAsync(Filename,
CreationCollisionOption.ReplaceExisting);
            }
            await FileIO.WriteTextAsync(noteFile, Text);
        }

        public async Task DeleteAsync()
        {
            // Delete the note from the file system.
            StorageFile noteFile = (StorageFile)await
storageFolder.TryGetItemAsync(Filename);
            if (noteFile is not null)
            {
                await noteFile.DeleteAsync();
            }
        }
    }
}
```

4. Save the file.

You'll notice that this code is very similar to the code in **NotePage.xaml.cs**, with a few changes and additions.

`Filename` and `Text` have been changed to `public` properties, and a new `Date` property has been added.

The code to save and delete the files has been placed in `public` methods. It is mostly identical to the code you used in the button `Click` event handlers in `NotePage`, but extra code to update the view after the file is deleted has been removed. It's not needed here because you'll be using data binding to keep the model and view synchronized.

These async method signatures return Task instead of `void`. The `Task` class represents a single asynchronous operation that does not return a value. Unless the method signature requires `void`, as is the case for the `Click` event handlers, `async` methods should return a `Task`.

You also won't be keeping a reference to the `StorageFile` that holds the note anymore. You just try to get the file when you need it to save or delete.

In `NotePage`, you used a placeholder for the file name: `note.txt`. Now that the app supports more than one note, file names for saved notes need to be different and unique. To do this, set the `Filename` property in the constructor. You can use the DateTime.ToBinary method to create

a part of the file name based on the current time and make the file names unique. The generated file name looks like this: `notes-8584626598945870392.txt`.

Update the note page

Now you can update the `NotePage` view to use the `Note` data model and delete code that was moved to the `Note` model.

1. Open the **Views\NotePage.xaml.cs** file if it's not already open in the editor.
2. After the last `using` statement at the top of the page, add a new `using` statement to give your code access to the classes in the `Models` folder and namespace.

```
C#

using WinUINotes.Models;
```

3. Delete these lines from the class:

```
C#

private StorageFolder storageFolder = ApplicationData.Current.LocalFolder;
private StorageFile? noteFile = null;
private string fileName = "note.txt";
```

4. Instead, add a `Note` object named `noteModel` in their place. This represents the note data that `NotePage` provides a view of.

```
C#

private Note? noteModel;
```

5. You also don't need the `NotePage_Loaded` event handler anymore. You won't be reading text directly from the text file into the TextBox. Instead, the note text will be read into `Note` objects. You'll add the code for this when you add the `AllNotesPage` in a later step. Delete these lines.

```
C#

Loaded += NotePage_Loaded;

...

private async void NotePage_Loaded(object sender, RoutedEventArgs e)
```

```
{
    noteFile = (StorageFile)await storageFolder.TryGetItemAsync(fileName);
    if (noteFile is not null)
    {
      NoteEditor.Text = await FileIO.ReadTextAsync(noteFile);
    }
}
```

6. Replace the code in the `SaveButton_Click` method with this:

```
C#

if (noteModel is not null)
{
    await noteModel.SaveAsync();
}
```

7. Replace the code in the `DeleteButton_Click` method with this:

```
C#

if (noteModel is not null)
{
    await noteModel.DeleteAsync();
}
```

Now you can update the XAML file to use the `Note` model. Previously, you read the text directly from the text file into the `TextBox.Text` property in the code-behind file. Now, you use data binding for the `Text` property.

1. Open the **Views\NotePage.xaml** file if it's not already open in the editor.
2. Add a `Text` attribute to the `TextBox` control. Bind it to the `Text` property of `noteModel`: `Text="{x:Bind noteModel.Text, Mode=TwoWay}"`.
3. Update the `Header` to bind to the `Date` property of `noteModel`: `Header="{x:Bind noteModel.Date.ToString()}"`.

```
C#

<TextBox x:Name="NoteEditor"
         <!-- ↓ Add this line. ↓ -->
         Text="{x:Bind noteModel.Text, Mode=TwoWay}"
         AcceptsReturn="True"
         TextWrapping="Wrap"
         PlaceholderText="Enter your note"
         <!-- ↓ Update this line. ↓ -->
```

```
            Header="{x:Bind noteModel.Date.ToString()}"
            ScrollViewer.VerticalScrollBarVisibility="Auto"
            Width="400"
            Grid.Column="1"/>
```

Data binding is a way for your app's UI to display data, and optionally to stay in sync with that data. The `Mode=TwoWay` setting on the binding means that the `TextBox.Text` and `noteModel.Text` properties are automatically synchronized. When the text is updated in the `TextBox`, the changes are reflected in the `Text` property of the `noteModel`, and if `noteModel.Text` is changed, the updates are reflected in the `TextBox`.

The `Header` property uses the default `Mode` of `OneTime` because the `noteModel.Date` property doesn't change after the file is created. This code also demonstrates a powerful feature of `x:Bind` called function binding, which lets you use a function like `ToString` as a step in the binding path.

> **ⓘ Important**
>
> It's important to choose the correct BindingMode; otherwise, your data binding might not work as expected. (A common mistake with `{x:Bind}` is to forget to change the default `BindingMode` when `OneWay` or `TwoWay` is needed.)

Expand table

Name	Description
`OneTime`	Updates the target property only when the binding is created. Default for `{x:Bind}`.
`OneWay`	Updates the target property when the binding is created. Changes to the source object can also propagate to the target. Default for `{Binding}`.
`TwoWay`	Updates either the target or the source object when either changes. When the binding is created, the target property is updated from the source.

Data binding supports the separation of your data and UI, and that results in a simpler conceptual model as well as better readability, testability, and maintainability of your app.

In WinUI, there are two kinds of binding you can choose from:

- The `{x:Bind}` markup extension is processed at compile-time. Some of its benefits are improved performance and compile-time validation of your binding expressions. It's recommended for binding in WinUI apps.

- The `{Binding}` markup extension is processed at run-time and uses general-purpose runtime object inspection.

 Learn more in the docs:

- Data binding in Windows apps
- {x:Bind}, {Binding}

Data binding and MVVM

Model-View-ViewModel (MVVM) is a UI architectural design pattern for decoupling UI and non-UI code that is popular with .NET developers. You'll probably see and hear it mentioned as you learn more about creating WinUI apps. Separating the views and models, as you've done here, is the first step towards a full MVVM implementation of the app, but it's as far as you'll go in this tutorial.

> **Note**
>
> We've used the term "model" to refer to the data model in this tutorial, but it's important to note that this model is more closely aligned with the *ViewModel* in a full MVVM implementation, while also incorporating aspects of the *Model*.

To learn more about MVVM, see these resources:

- Windows data binding and MVVM
- Introduction to the MVVM Toolkit
- MVVM Building Blocks for WinUI and WPF Development on YouTube.

Continue to step 4 - Add a view and model for all notes

Add a view and model for all notes

This portion of the tutorial adds a new page to the app, a view that displays all of the notes previously created.

Multiple notes and navigation

Currently the **note** view displays a single note. To display all your saved notes, create a new view and model: **AllNotes**.

1. In the **Solution Explorer** pane, right-click on the **Views** folder and select **Add** > **New Item...**
2. In the **Add New Item** dialog, select **WinUI** in the template list on the left-side of the window. Next, select the **Blank Page (WinUI)** template. Name the file *AllNotesPage.xaml* and press **Add**.
3. In the **Solution Explorer** pane, right-click on the **Models** folder and select **Add** > **Class...**
4. Name the class *AllNotes.cs* and press **Add**.

> **Tip**
>
> You can download or view the code for this tutorial from the GitHub repo. To see the code as it is in this step, see this commit: all notes view and model.

Code the AllNotes model

The new data model represents the data required to display multiple notes. Here, you'll get all the notes from the app's local storage and create a collection of `Note` objects that you'll display in the `AllNotesPage`.

1. In the **Solution Explorer** pane, open the **Models\AllNotes.cs** file.

2. Replace the code in the **AllNotes.cs** file with this code:

```
C#
```

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Threading.Tasks;
using Windows.Storage;
```

```
namespace WinUINotes.Models
{
    public class AllNotes
    {
        public ObservableCollection<Note> Notes { get; set; } =
                                    new ObservableCollection<Note>();

        public AllNotes()
        {
            LoadNotes();
        }

        public async void LoadNotes()
        {
            Notes.Clear();
            // Get the folder where the notes are stored.
            StorageFolder storageFolder =
                        ApplicationData.Current.LocalFolder;
            await GetFilesInFolderAsync(storageFolder);
        }

        private async Task GetFilesInFolderAsync(StorageFolder folder)
        {
            // Each StorageItem can be either a folder or a file.
            IReadOnlyList<IStorageItem> storageItems =
                                        await folder.GetItemsAsync();
            foreach (IStorageItem item in storageItems)
            {
                if (item.IsOfType(StorageItemTypes.Folder))
                {
                    // Recursively get items from subfolders.
                    await GetFilesInFolderAsync((StorageFolder)item);
                }
                else if (item.IsOfType(StorageItemTypes.File))
                {
                    StorageFile file = (StorageFile)item ;
                    Note note = new Note()
                    {
                        Filename = file.Name,
                        Text = await FileIO.ReadTextAsync(file),
                        Date = file.DateCreated.DateTime
                    };
                    Notes.Add(note);
                }
            }
        }
    }
}
```

The previous code declares a collection of `Note` items, named `Notes`, and uses the `LoadNotes` method to load notes from the app's local storage.

> **Note**
>
> The `LoadNotes` method uses `async void` instead of `async Task` because it's called from the constructor, which cannot be `async`. This fire-and-forget pattern is acceptable here since the method is an event-like initialization routine.

The `Notes` collection uses an ObservableCollection, which is a specialized collection that works well with data binding. When a control that lists multiple items, such as an ItemsView, is bound to an `ObservableCollection`, the two work together to automatically keep the list of items in sync with the collection. If an item is added to the collection, the control is automatically updated with the new item. If an item is added to the list, the collection is updated.

 Learn more in the docs:

- StorageFolder class, StorageFile class, IStorageItem.IsOfType method
- Access files and folders with Windows App SDK and WinRT APIs

Now that the `AllNotes` model is ready to provide data for the view, you need to create an instance of the model in `AllNotesPage` so the view can access the model.

1. In the **Solution Explorer** pane, open the **Views\AllNotesPage.xaml.cs** file.
2. In the `AllNotesPage` class, add this code to create an `AllNotes` model named *notesModel*:

   > **Note**
   >
   > You'll need to add `using WinUINotes.Models;` at the top of the file to reference the `AllNotes` class.

   C#

   ```
   public sealed partial class AllNotesPage : Page
   {
       // ↓ Add this. ↓
       private AllNotes notesModel = new AllNotes();
       // ↑ Add this. ↑

       public AllNotesPage()
       {
           this.InitializeComponent();
       }
   }
   ```

Design the AllNotes page

Next, you need to design the view to support the `AllNotes` model.

1. In the **Solution Explorer** pane, open the **Views\AllNotesPage.xaml** file.

2. Replace the `<Grid> ... </Grid>` element with the following markup:

XAML

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*"/>
    </Grid.RowDefinitions>

    <CommandBar DefaultLabelPosition="Right">
        <AppBarButton Icon="Add" Label="New note"/>
        <CommandBar.Content>
            <TextBlock Text="Quick notes" Margin="16,8"
                   Style="{ThemeResource SubtitleTextBlockStyle}"/>
        </CommandBar.Content>
    </CommandBar>

    <ItemsView ItemsSource="{x:Bind notesModel.Notes}"
           Grid.Row="1" Padding="16" >
        <ItemsView.Layout>
            <UniformGridLayout MinItemWidth="200"
                               MinColumnSpacing="12"
                               MinRowSpacing="12"
                               ItemsJustification="Start"/>
        </ItemsView.Layout>
    </ItemsView>
</Grid>
```

The previous XAML introduces a few new concepts:

- The CommandBar control contains an AppBarButton. This button has a `Label` and `Icon`, and is influenced by the `CommandBar` that contains it. For example, this `CommandBar` sets the label position of its buttons to `Right`. Command bars are usually displayed at the top of the app, along with the page title.
- The ItemsView control displays a collection of items, and in this case, is bound to the model's `Notes` property. The way items are presented by the items view is set through the `ItemsView.Layout` property. Here, you use a UniformGridLayout.

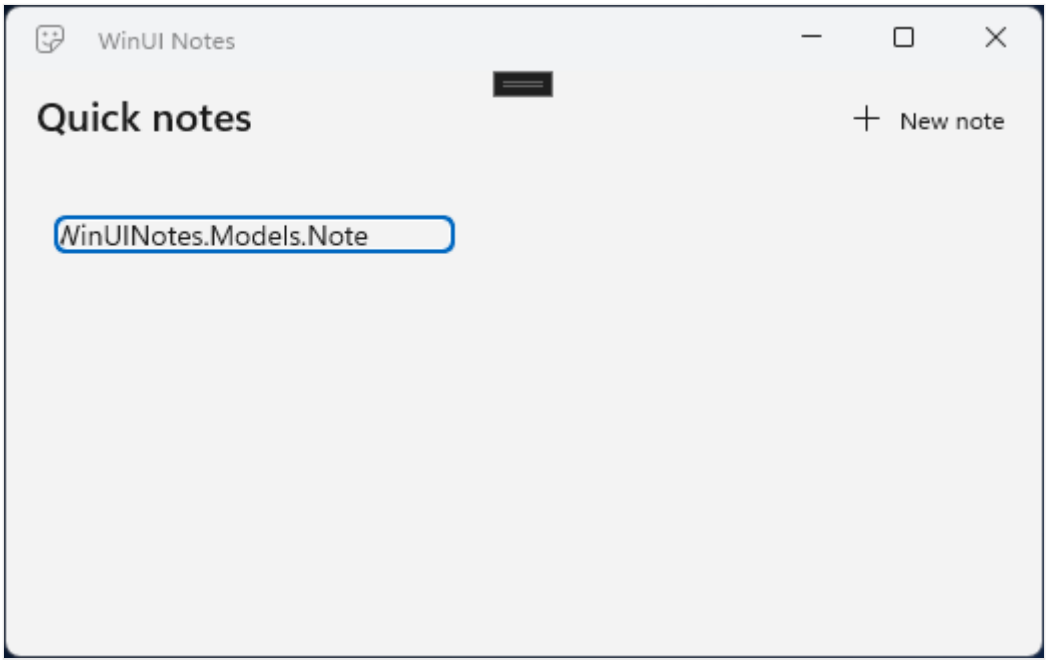
Now that you've created `AllNotesPage`, you need to update `MainWindow.xaml` one last time so that it loads `AllNotesPage` instead of an individual `NotePage`.

1. In the **Solution Explorer** pane, open the **MainWindow.xaml** file.

2. Update the `rootFrame` element so that the `SourcePageType` points to `views.AllNotesPage`, like this:

```
XAML
<Frame x:Name="rootFrame" Grid.Row="1"
       SourcePageType="views:AllNotesPage"/>
```

If you run the app now, you'll see that the note you created previously is loaded into the `ItemsView` control. However, it's just shown as the string representation of the object. The `ItemsView` doesn't know how this item should be displayed. You'll correct this in the next section.



Add a data template

You need to specify a DataTemplate to tell the `ItemsView` how your data item should be shown. The `DataTemplate` is assigned to the ItemTemplate property of the `ItemsView`. For each item in the collection, the `ItemsView.ItemTemplate` generates the declared XAML.

1. In the **Solution Explorer** pane, double-click on the **AllNotesPage.xaml** entry to open it in the XAML editor.

2. Add this new namespace mapping on the line below the mapping for `local`:

```
XAML
xmlns:models="using:WinUINotes.Models"
```

3. Add a `<Page.Resources>` element after the opening `<Page...>` tag. This gets the ResourceDictionary from the `Page`'s Resources property so that you can add XAML resources to it.

XAML

```
<Page
    x:Class="WinUINotes.Views.AllNotesPage"
    ... >
<!-- ↓ Add this. ↓ -->
<Page.Resources>

</Page.Resources>
```

4. Inside the `<Page.Resources>` element, add the `DataTemplate` that describes how to display a `Note` item.

XAML

```
<Page.Resources>
    <!-- ↓ Add this. ↓ -->
    <DataTemplate x:Key="NoteItemTemplate"
                  x:DataType="models:Note">
        <ItemContainer>
            <Grid Background="LightGray">
                <Grid.RowDefinitions>
                    <RowDefinition Height="120"/>
                    <RowDefinition Height="Auto"/>
                </Grid.RowDefinitions>
                <TextBlock Text="{x:Bind Text}" Margin="12,8"
                           TextWrapping="Wrap"
                           TextTrimming="WordEllipsis"/>
                <Border Grid.Row="1" Padding="8,6,0,6"
                        Background="Gray">
                    <TextBlock Text="{x:Bind Date}"
                               Foreground="White"/>
                </Border>
            </Grid>
        </ItemContainer>
    </DataTemplate>
    <!-- ↑ Add this. ↑ -->
</Page.Resources>
```

5. In the XAML for `ItemsView`, change `Padding="16"` to `Margin="24"` and assign the `ItemTemplate` property to the data template you just created:

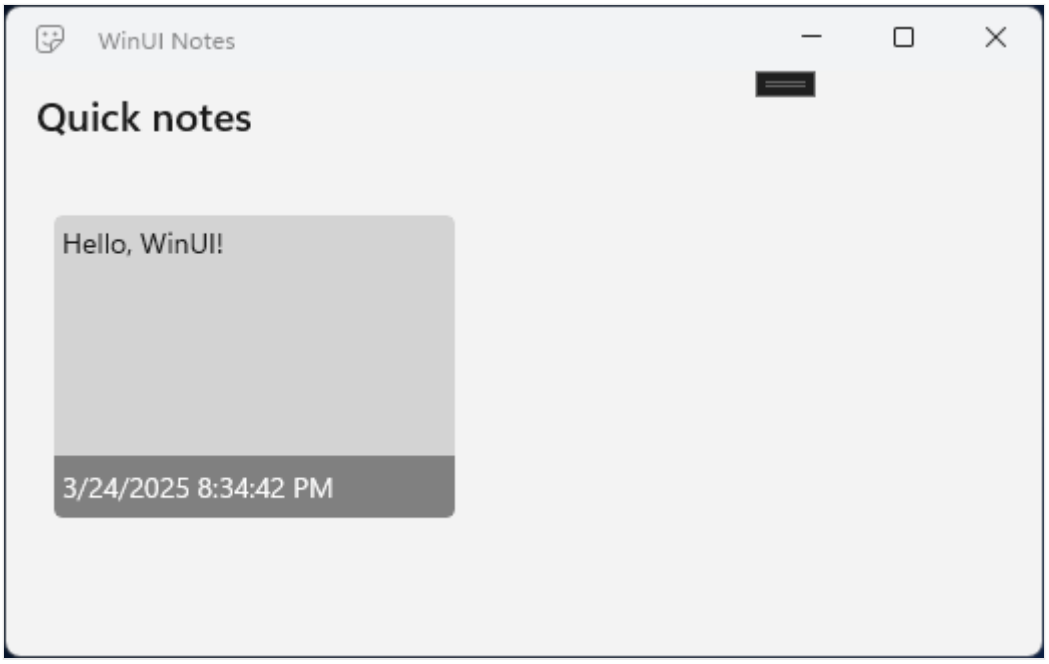
XAML

```
<ItemsView ItemsSource="{x:Bind notesModel.Notes}"
           Grid.Row="1" Margin="24"
           <!-- ↓ Add this. ↓ -->
           ItemTemplate="{StaticResource NoteItemTemplate}">
```

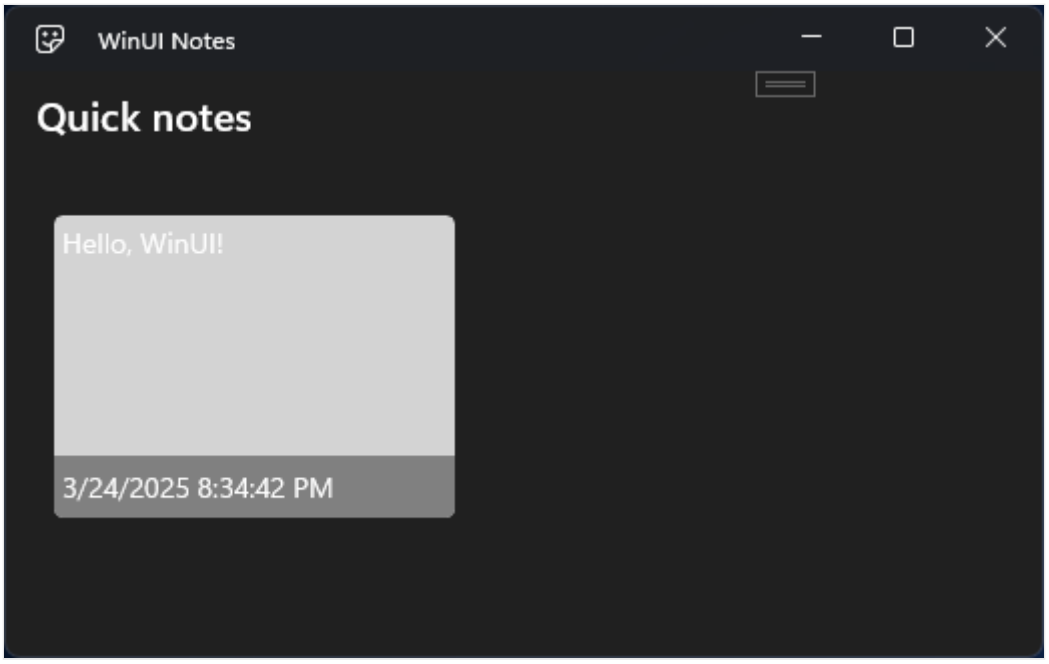
6. Build and run the app.

When you use the `x:Bind` markup extension in a `DataTemplate`, you have to specify the `x:DataType` on the `DataTemplate`. In this case, that's an individual `Note` (so you have to add a XAML namespace reference for `Models`). The template for the note uses two `TextBlock` controls, which are bound to the note's `Text` and `Date` properties. The Grid element is used for layout and to provide a background color. A Border element is used for the background of the date. (The XAML `Border` element can provide both an outline and background.)

When you run the app, the data template is applied to your `Note` items and looks like this if your Windows Personalization > Colors settings use the Light mode:



However, if your Windows Personalization > Colors settings use the Dark mode, it will look like this:



This is not the intended look for the app. It happened because there are hard-coded color values in the data template for the note. By default, WinUI 3 elements adapt to the user's Dark or Light color preference. When you define you own elements, like a data template, you need to be careful to do the same.

When you define a resource in a XAML `ResourceDictionary`, you have to assign an `x:Key` value to identify the resource. Then, you can use that `x:Key` to retrieve the resource in XAML using the `{StaticResource}` markup extension or `{ThemeResource}` markup extension.

- A `{StaticResource}` is the same regardless of the color theme, so it's used for things like `Font` or `Style` settings.
- A `{ThemeResource}` changes based on the selected color theme, so it's used for `Foreground`, `Background`, and other color-related properties.

WinUI includes a variety of built-in resources that you can use to make your app follow Fluent style guidelines, as well as accessibility guidelines. You'll replace the hard-coded colors in the data template with built-in theme resources, and apply a few other resources to match the Fluent Design guidelines.

1. In the data template you added previously, update the sections indicated here to use built-in resources:

XAML

```
<DataTemplate x:Key="NoteItemTemplate"
              x:DataType="models:Note">

<!-- ↓ Update this. ↓ -->
```

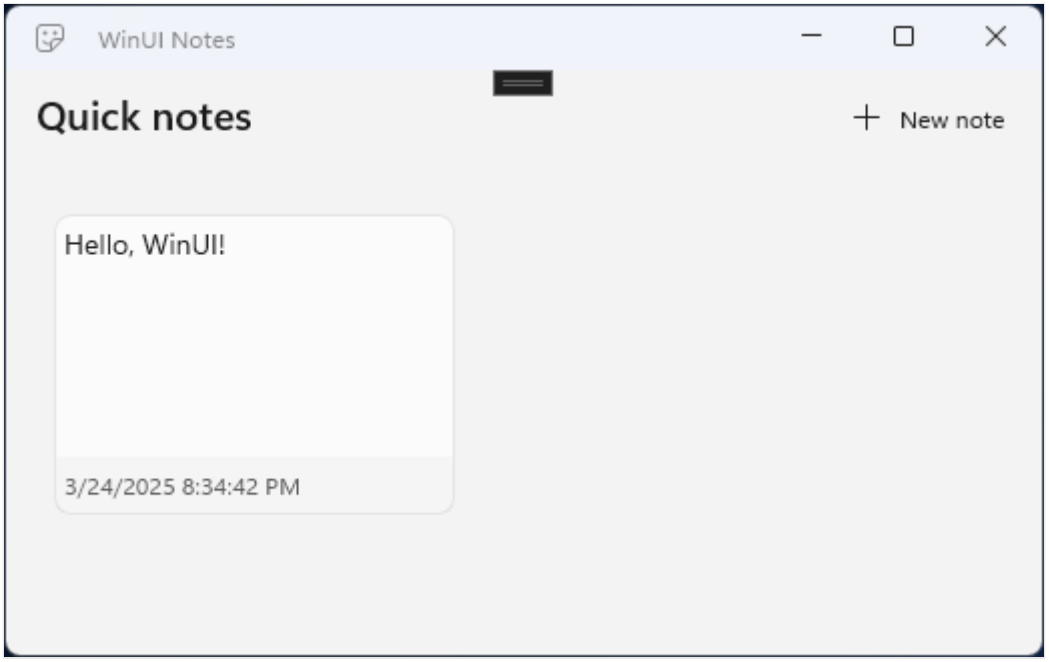
```
    <ItemContainer CornerRadius="{StaticResource OverlayCornerRadius}">
        <Grid Background="{ThemeResource CardBackgroundFillColorDefaultBrush}"
              BorderThickness="1"
              BorderBrush="{ThemeResource CardStrokeColorDefaultBrush}"
              CornerRadius="{StaticResource OverlayCornerRadius}">
<!-- ↑ Update this. ↑ -->

            <Grid.RowDefinitions>
                <RowDefinition Height="120"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>
            <TextBlock Text="{x:Bind Text}" Margin="12,8"
                       TextWrapping="Wrap"
                       TextTrimming="WordEllipsis"/>

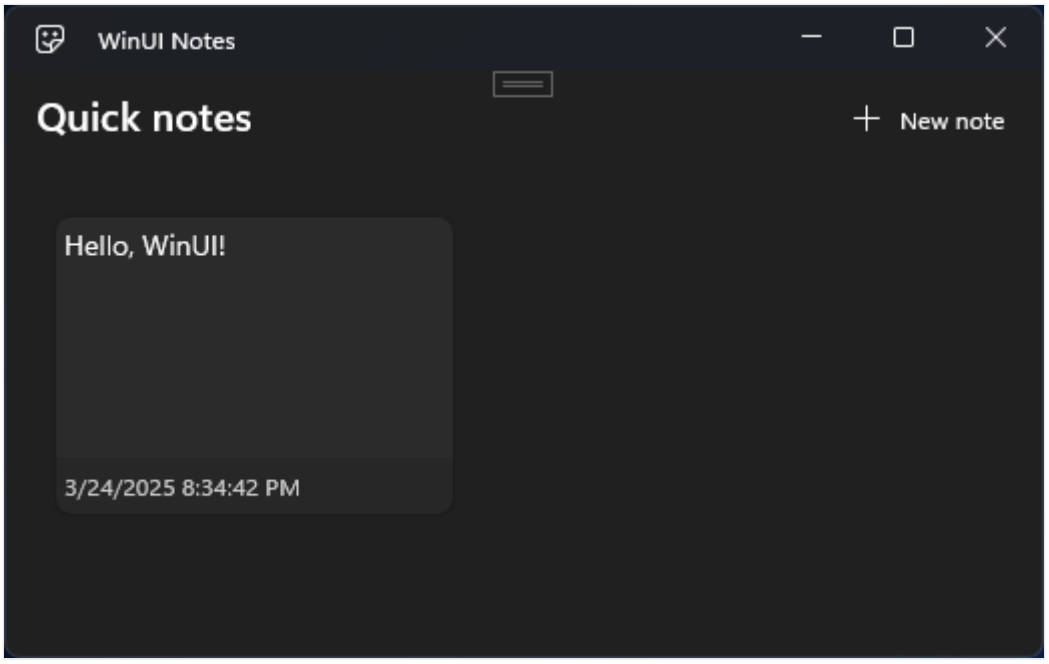
<!-- ↓ Update this. ↓ -->
            <Border Grid.Row="1" Padding="8,6,0,6"
                    Background="{ThemeResource SubtleFillColorSecondaryBrush}">
                <TextBlock Text="{x:Bind Date}"
                    Style="{StaticResource CaptionTextBlockStyle}"
                    Foreground="{ThemeResource TextFillColorSecondaryBrush}"/>
<!-- ↑ Update this. ↑ -->

            </Border>
        </Grid>
    </ItemContainer>
</DataTemplate>
```

Now when you run the app with a Light color setting, it will look like this:



And when you run the app with a Dark color setting, it will look like this:



Learn more in the docs:

- Item containers and templates
- ResourceDictionary and XAML resource references

> **Tip**
>
> The WinUI 3 Gallery app is a great way to learn about different WinUI controls and design guidelines. To see the theme resources used in the data template, open the WinUI 3 Gallery app to the Color guidance. From there, you can see what the resources look like and copy the values you need directly from the app.
>
> You can also open the Typography page and Geometry page to see other built-in resources used in this data template.

Expand table

The **WinUI 3 Gallery** app includes interactive examples of WinUI controls and features. Get the app from the Microsoft Store or browse the source code on GitHub.

Continue to step 5 - Add navigation between pages

Last updated on 04/09/2026

Navigate between pages

This portion of the tutorial adds the final piece to the app, navigation between the *all notes* page and the individual *note* page.

Before writing any code to handle in-app navigation, let's describe the expected navigation behavior.

In `AllNotesPage`, there's the collection of existing notes and a **New note** button.

- Clicking an existing note should navigate to the note page and load the note that was clicked.
- Clicking the **New note** button should navigate to the note page and load an empty, unsaved note.

On the note page, you'll add a *back* button, and there are **Save** and **Delete** buttons .

- Clicking the back button should navigate back to the all notes page, discarding any changes made to the note.
- Clicking the **Delete** button should delete the note and then navigate back.

New note

First, you'll handle navigation for a new note.

> **Tip**
>
> You can download or view the code for this tutorial from the GitHub repo. To see the code as it is in this step, see this commit: navigation - new note.

1. In **AllNotesPage.xaml**, find the `AppBarButton` for a new note.
2. Add a `Click` event handler and rename it to `NewNoteButton_Click`. (See *Add event handlers* in the note page step if you need a reminder how to do this.)

```
XAML
<AppBarButton Icon="Add" Label="New note"
              Click="NewNoteButton_Click"/>
```

3. In the `NewNoteButton_Click` method (in **AllNotesPage.xaml.cs**), add this code:

```
C#
private void NewNoteButton_Click(object sender, RoutedEventArgs e)
{
    // ↓ Add this. ↓
    Frame.Navigate(typeof(NotePage));
}
```

Each Page has a Frame property that provides a reference to the Frame instance that the `Page` belongs to (if any). That's the `Frame` that you call the Navigate method on here. The `Navigate` method takes the Type of the page that you want to navigate to. You get that `Type` in C# by using the `typeof` operator.

If you run the app now, you can click the **New note** button to navigate to the note page, and you can type into the note editor. However, if you try to save the note, nothing will happen. This is because an instance of the `Note` model hasn't been created in the note page yet. You'll do that now.

1. Open **NotePage.xaml.cs**.

2. Add code to override the page's OnNavigatedTo method.

```
C#
public NotePage()
{
    this.InitializeComponent();
}
//  ↓ Add this. ↓
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);

    noteModel = new Note();
}
```

Now, when you navigate to `NotePage`, a new instance of the `Note` model is created.

Existing notes

Now you'll add navigation for existing notes. Currently, when you click the note in the `ItemsView`, the note is selected, but nothing happens. The default behavior for most collection controls is

single selection, which means one item is selected at a time. You'll update the `ItemsView` so that instead of selecting it, you can click an item like a button.

> **Tip**
>
> You can download or view the code for this tutorial from the GitHub repo. To see the code as it is in this step, see this commit: navigation - final app.

1. Open **AllNotesPage.xaml**.
2. Update the XAML for the `ItemsView` with SelectionMode = None and IsItemInvokedEnabled = `True`.
3. Add a handler for the ItemInvoked event.

XAML

```
<ItemsView ItemsSource="{x:Bind notesModel.Notes}"
           Grid.Row="1" Margin="24"
           ItemTemplate="{StaticResource NoteItemTemplate}"
           <!-- ↓ Add this. ↓ -->
           SelectionMode="None"
           IsItemInvokedEnabled="True"
           ItemInvoked="ItemsView_ItemInvoked">
```

4. In **AllNotesPage.xaml.cs**, find the `ItemsView_ItemInvoked` method. (If Visual Studio didn't create it for you, which could happen if you copy and paste the code, add it in the next step.)
5. In the `ItemsView_ItemInvoked` method, add code to navigate to `NotePage`. This time, you'll use an overload of the Navigate method that lets you pass an object to the other page. Pass the invoked `Note` as the second navigation parameter.

C#

```
private void ItemsView_ItemInvoked(ItemsView sender,
ItemsViewItemInvokedEventArgs args)
{
    Frame.Navigate(typeof(NotePage), args.InvokedItem);
}
```

6. Open **NotePage.xaml.cs**.

7. Update the `OnNavigatedTo` method override to handle the `Note` that's passed by the call to `Navigate`.

C#

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);

    //  ↓ Update this. ↓
    if (e.Parameter is Note note)
    {
        noteModel = note;
    }
    else
    {
        noteModel = new Note();
    }
   // ↑ Update this. ↑
}
```

In this code, you first check to see if the passed *parameter* is a `Note` object. If it is, you assign it as the `Note` model for the page. If it's `null` or not a `Note`, create a new `Note` as before.

Back navigation

Lastly, you need to update the app so that you can navigate back from an individual note to the all notes page.

The WinUI 3 TitleBar control includes a back button that meets all the Fluent Design guidelines for placement and appearance. You'll use this built-in button for back navigation.

1. Open **MainWindow.xaml**.
2. Update the XAML for the `TitleBar` with IsBackButtonVisible = `True` and IsBackButtonEnabled bound to the Frame.CanGoBack property.
3. Add a handler for the BackRequested event. Your XAML should look like this:

XAML

```
<TitleBar x:Name="AppTitleBar"
          Title="WinUI Notes"
          IsBackButtonVisible="True"
```

```
            IsBackButtonEnabled="{x:Bind rootFrame.CanGoBack, Mode=OneWay}"
            BackRequested="AppTitleBar_BackRequested">
```

Here, the IsBackButtonVisible property is set to `true`. This makes the back button appear in the upper-left corner of the app window.

Then, the IsBackButtonEnabled property is bound to the Frame.CanGoBack property, so the back button is enabled only if the frame can navigate back.

Finally, the BackRequested event handler is added. This is where you put the code to navigate back.

4. In **MainWindow.xaml.cs**, add this code to the `AppTitleBar_BackRequested` method:

C#

```
private void AppTitleBar_BackRequested(TitleBar sender, object args)
{
    // ↓ Add this. ↓
    if (rootFrame.CanGoBack == true)
    {
        rootFrame.GoBack();
    }
    // ↑ Add this. ↑
}
```

The `Frame` class keeps track of navigation in its BackStack, so you can navigate back to previous pages simply by calling the GoBack method. It's a best practice to always check the CanGoBack property before calling `GoBack`.

Next, you need to update the code in `NotePage` to navigate back after the note is deleted.

1. Open **NotePage.xaml.cs**.

2. Update the `DeleteButton_Click` event handler method with a call to the `Frame.CanGoBack` method after the note is deleted:

C#

```
private async void DeleteButton_Click(object sender, RoutedEventArgs e)
{
    if (noteModel is not null)
    {
        await noteModel.DeleteAsync();
    }
    // ↓ Add this. ↓
    if (Frame.CanGoBack == true)
```

```
    {
        Frame.GoBack();
    }
    // ↑ Add this. ↑
}
```

> **Tip**
>
> You might have noticed that in `NotePage`, you call `Frame.GoBack`, while in `MainWindow` you called `rootFrame.GoBack`. This is because the **Page** class has a **Frame** property that gets the `Frame` that is hosting the `Page`, if any. In this case, it gets a reference to `rootFrame`.

Now you can run your app. Try adding new notes, navigating back and forth between notes, and deleting notes.

 Learn more in the docs:

- Implement navigation between two pages
- Navigation history and backwards navigation
- Frame class, Page class

Next steps

You've completed the *Build a WinUI 3 notes app* tutorial.

In order to keep things simple and introduce some foundational concepts, this tutorial focused on simplicity over efficiency, and introduced features at the most basic level. So while the app works, there are some things that can be improved.

We recommend that you continue with one of these follow-on tutorials to learn more:

- WinUI Notes part 2 – Explains more features related to navigation and data binding. You'll update the app to work more efficiently while keeping the same basic architecture.

  OR

- Data binding, dependency injection, and unit testing in WinUI – Shows how to implement data binding, dependency injection, and unit testing with the Model-View-ViewModel (MVVM) design pattern. You'll change the app and view models to leverage the MVVM Toolkit.

Last updated on 09/10/2026

WinUI Notes part 2 - navigation and data binding

This tutorial demonstrates how to use navigation and data binding in a WinUI 3 app using XAML and C#.

In this tutorial, you learn how to:

- ✓ Enable `NavigationCacheMode` to keep the same instance of a `Page` when navigating.
- ✓ Implement the `INotifyPropertyChanged` interface to let a data binding know that data has been updated.
- ✓ Pass objects between pages when navigating.

This tutorial improves on the *WinUI Notes* sample app from the Build a WinUI 3 notes app tutorial. If you've completed that tutorial, you can continue working with the same code. Or, you can download the completed code for that tutorial from the GitHub repo and use it as a starting point for this tutorial.

In either case, you should be familiar with concepts presented and the code created in the Build a WinUI 3 notes app tutorial.

> **Tip**
>
> You can also download or view the completed code for this tutorial from the **GitHub repo at WinUI Notes part 2**. To see the differences between the start and end points for the project, see this commit: **updates for part 2**.

Background

In order to keep things simple and introduce some foundational concepts, the Build a WinUI 3 notes app tutorial focused on simplicity over efficiency. So while the app works, there are some things that can be improved.

The key issue has to do with navigating between pages in the app. By default, Page instances are not saved when navigating, so each time you navigate to a `Page`, a new instance is created. In the WinUI Notes app, the `notesModel` object is created in the `AllNotesPage` constructor, and is populated by reading all the notes from the file system.

C#

```
public sealed partial class AllNotesPage : Page
{
    private AllNotes notesModel = new AllNotes();
    ...
}
```

Each time you navigate from `NotePage` back to `AllNotesPage`, the `notesModel` is recreated and all the notes are re-read from the file system. This inefficiency typically isn't noticeable in a small sample app without much data, but it would be unacceptable in, for example, a photos app that was reading thousands of large images from the file system.

To fix this issue, this tutorial will cover these steps:

- First, ensure that the `Page` instance is cached so that on navigation it's re-used and not re-created.
- Update the `Note` class so that bound properties are notified when there are any changes to the note text.
- Ensure that the `notesModel` is properly updated with saved or deleted notes, since it's not being re-created with each navigation.

> **Tip**
>
> You'll frequently refer to API reference docs and conceptual docs while building Windows apps. In this tutorial, you'll see links inline in the text, and in groups labeled, "Learn more in the docs:". These links are optional; you don't need to follow them to complete the tutorial. They're provided in case you want to make note of where to find the information you'll need when you start to create your own apps.

Continue to step 1 - Navigation cache and change notification

Last updated on 09/10/2026

Navigation cache and change notification

The first step is to do the basic setup for the changes you need to make:

- enable the navigation cache.
- implement property change notifications for the `Note.Text` property.

One that's done, you'll adapt other parts of the app to work with these changes.

> **Tip**
>
> You can download or view the completed code for this tutorial from the GitHub repo at WinUI Notes part 2. To see the differences between the start and end points for the project, see this commit: updates for part 2.

Enable NavigationCacheMode

By default, a new Page instance is created with its default values every time navigation occurs. In the WinUI Notes app, this is also where the `notesModel`, which stores all the `Note` instances, is created.

In `AllNotesPage.xaml`, set NavigationCacheMode to Enabled (`NavigationCacheMode="Enabled"`). With `NavigationCacheMode` enabled, the same page instance is kept around, so a new `Page` instance isn't created on each navigation, and `notesModel` is not re-created.

XAML

```
<Page
    x:Class="WinUI_Notes.Views.AllNotesPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:WinUI_Notes.Views"
    xmlns:models="using:WinUI_Notes.Models"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    Background="{ThemeResource ApplicationPageBackgroundThemeBrush}"
    //  ↓ Add this. ↓
    NavigationCacheMode="Enabled">
```

Run the app now and you'll notice some side effects of this change.

1. When you edit an existing note, your changes aren't reflected in the all-notes page when you navigate back.
2. When you create and save a new note, it doesn't appear in the all-notes list when you navigate back.
3. When you delete an existing note, it's not removed from the all-notes list when you navigate back.

You'll fix these issues next.

 Learn more in the docs:

- Implement navigation between two pages
- Page.NavigationCacheMode class, NavigationCacheMode enum

Implement INotifyPropertyChanged

When you edit and save an existing note, your change is saved to the file system, but the change isn't propagated to the all-notes list. This is because the `Note` class doesn't notify the data binding, which connects the `TextBox` to the `Note` text, that an update has occurred. To make this notification happen, the `Note` class needs to implement the `INotifyPropertyChanged` interface for its `Text` property.

> ⓘ **Note**
>
> WinUI includes the Microsoft.UI.Xaml.Data.INotifyPropertyChanged interface. This is used only by C++ apps, which don't use .NET.
>
> C# apps created with .NET use the System.ComponentModel.INotifyPropertyChanged interface instead.

The implementation of `INotifyPropertyChanged` follows a set pattern.

1. Add `using` statements for the required namespaces.

   C#

   ```
   // ↓ Add this. ↓
   using System.ComponentModel
   using System.Runtime.CompilerServices
   ```

2. Implement `INotifyPropertyChanged`. The `Note` class now implements this interface.

C#

```
// ↓ Update this. ↓
public class Note : INotifyPropertyChanged
```

3. Create a backing field (`_text`) for the `Text` property.

C#

```
// ↓ Delete this. ↓
// public string Text { get; set; } = string.Empty;

// ↓ Add this. ↓
private string _text = string.Empty;
```

4. Modify the `Text` property to use a getter/setter pattern with property change notification.

C#

```
// ↓ Add this. ↓
public string Text
{
    get => _text;
    set
    {
        if (_text != value)
        {
            _text = value;
            OnPropertyChanged();
        }
    }
}
```

5. Add the PropertyChanged event required by the `INotifyPropertyChanged` interface.

C#

```
// ↓ Add this. ↓
public event PropertyChangedEventHandler? PropertyChanged;
```

6. Add the `OnPropertyChanged` method. This helper method raises the `PropertyChanged` event using the CallerMemberName attribute for automatic property name detection.

C#

```
// ↓ Add this. ↓
protected void OnPropertyChanged([CallerMemberName] string? propertyName = null)
{
```

```
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
```

> **Tip**
>
> You can use GitHub Copilot to quickly implement `INotifyPropertyChanged` in your app. These code changes were generated with the prompt: "Implement INotifyPropertyChanged for the Note.Text property".

Binding mode

Now the `Text` property will notify any UI elements bound to it whenever its value changes, so your UI can be updated automatically. However, in order for the bound UI elements to react to the update notification, you have to ensure the correct BindingMode is used.

> **Important**
>
> It's important to choose the correct BindingMode; otherwise, your data binding might not work as expected. (A common mistake with `{x:Bind}` is to forget to change the default `BindingMode` when `OneWay` or `TwoWay` is needed.)

Expand table

Name	Description
`OneTime`	Updates the target property only when the binding is created. Default for `{x:Bind}`.
`OneWay`	Updates the target property when the binding is created. Changes to the source object can also propagate to the target.
`TwoWay`	Updates either the target or the source object when either changes. When the binding is created, the target property is updated from the source.

In `AllNotesPage.xaml`, find the `NoteItemTemplate` in `Page.Resources`. Then, in the template, find the `TextBlock` that's bound to the `Text` property. Update the binding to use the `OneWay` binding mode.

C#

```
// ↓ Update this. ↓                ↓    ↓
<TextBlock Text="{x:Bind Text, Mode=OneWay}"
```

```
            Margin="4" TextWrapping="Wrap"
            TextTrimming="WordEllipsis"/>
```

Since the user can't update the text in the `TextBlock`, only a `OneWay` binding is needed, from the source (`Note.Text`) to the target (`TextBlock.Text`).

 Learn more in the docs:

- Notes tutorial: Add a view and model for the note
- Windows data binding in depth
- INotifyPropertyChanged.PropertyChanged event
- CallerMemberName attribute

Continue to step 2 - Track note state

Last updated on 09/10/2026

Track note state

In the previous step, you fixed the first side effect of navigation caching by implementing `INotifyPropertyChanged` so that edits are reflected in the bound text control. The other side effect of caching the page on navigation is that the notes collection isn't updated when a new note is added or deleted. That's because previously, the note was saved, and then the collection was recreated by re-reading all the saved notes. You'll fix these issues now by tracking the state of a note, then using the state to determine whether the note needs to be added or deleted.

> **Tip**
>
> You can download or view the completed code for this tutorial from the GitHub repo at WinUI Notes part 2. To see the differences between the start and end points for the project, see this commit: updates for part 2.

Update the collection

First, you need to add code to update the collection when a note is added or deleted. In `AllNotes.cs`, add the `AddNote` and `RemoveNote` methods as shown here.

C#

```
    public class AllNotes
    {
        public ObservableCollection<Note> Notes { get; set; } = new
ObservableCollection<Note>();
        // ...

        // ↓ Add this. ↓
        public void AddNote(Note note)
        {
            // Insert the note at the beginning of the collection.
            Notes.Insert(0, note);
        }

        public void RemoveNote(Note note)
        {
            Notes.Remove(note);
        }
    }
```

> ⓘ Note
>
> `Notes.Add` would add the note at the end of the collection. Instead, `Insert` it at the beginning so new notes are shown first.

 Learn more in the docs:

- Notes tutorial: Add a view and model for all notes
- ObservableCollection

Add State to the note

Notes are added or deleted in `NotePage`. But the collection of notes is maintained in `AllNotesPage`, so you still need a way to let `AllNotesPage` know about new notes and deleted notes. For this, you'll add a new `State` property to the `Note` class. Then, in step 3, you'll modify the navigation between the pages to pass new or deleted notes as the navigation parameter.

In `Note.cs`, add a new enum called `NoteState`. (Add it below the `Note` class, but inside of the namespace brackets.)

C#

```
// ↓ Add this. ↓
public enum NoteState
{
    Unset = 0, Unsaved, Saved, Deleted
}
```

Add a new `State` property to the `Note` class, and set it as appropriate:

- `Unset`: New note
- `Unsaved`: Text has been changed, but not saved.
- `Saved`: Text is changed and saved to the file system.
- `Deleted`: Note has been deleted from the file system.

C#

```
// ↓ Add this. ↓
public NoteState State { get; set; } = NoteState.Unset;

// ↓ Update these. ↓
public string Text
{
```

```
    get => _text;
    set
    {
        if (_text != value)
        {
            _text = value;
            // ↓ Add this. ↓
            State = NoteState.Unsaved;
            // ↑ Add this. ↑
            OnPropertyChanged();
        }
    }
}

public async Task SaveAsync()
{
    // Save the note to a file.
    StorageFile noteFile = (StorageFile)await storageFolder.TryGetItemAsync(Filename);
    if (noteFile is null)
    {
        noteFile = await storageFolder.CreateFileAsync(Filename,
CreationCollisionOption.ReplaceExisting);
    }
    await FileIO.WriteTextAsync(noteFile, Text);
    // ↓ Add this. ↓
    State = NoteState.Saved;
    // ↑ Add this. ↑
}

public async Task DeleteAsync()
{
    // Delete the note from the file system.
    StorageFile noteFile = (StorageFile)await storageFolder.TryGetItemAsync(Filename);
    if (noteFile is not null)
    {
        await noteFile.DeleteAsync();
    }
    Filename = string.Empty;
    // ↓ Add this. ↓
    State = NoteState.Deleted;
    // ↑ Add this. ↑
}
```

The note `State` also needs to be set when the notes are initially loaded from the file system. By default, the `State` is `Unset` when a new note is created in the editor but hasn't been saved. However, when a previously saved note is loaded from the file system, it should have an initial `State` of `Saved`.

In `AllNotes.cs`, find the `GetFilesInFolderAsync` method. Then update the code to create a new `Note` object with an initial `State` of `Saved`.

C#

```
Note note = new Note()
{
    Filename = file.Name,
    Text = await FileIO.ReadTextAsync(file),
    Date = file.DateCreated.DateTime, // << Add a comma here.
    // ↓ Add this. ↓
    State = NoteState.Saved
    // ↑ Add this. ↑
};
```

Continue to step 3 - Pass navigation parameters

Last updated on 09/10/2026

Pass the note as a navigation parameter

Now that the `Note` has a `State`, update the navigation to pass it back to `AllNotesPage`, where you can handle it as appropriate based on it's `State`. This also gives you the opportunity to improve the user experience related to navigation.

> **Tip**
>
> If needed, review **Notes tutorial: Navigate between pages**. It's helpful to understand how the navigation is set up before making these changes.

Currently, all navigation from `NotePage` back to `AllNotesPage` is done with a simple call to Frame.GoBack. However, the `GoBack` method doesn't allow you to pass a navigation parameter. In order to pass a `Note` as a parameter, you'll need to replace the back navigation with a forward navigation (Frame.Navigate). `GoBack` also doesn't add an entry to the navigation stack like a forward navigation does, so you'll need to manage the backstack to prevent this forward navigation from being added.

> **Tip**
>
> You can download or view the completed code for this tutorial from the **GitHub repo at WinUI Notes part 2**. To see the differences between the start and end points for the project, see this commit: **updates for part 2**.

OnNavigatingFrom

In `NotePage.cs`, override the OnNavigatingFrom method. This is called after the user presses the back button and GoBack is called. It lets you intercept the navigation, check the note `State`, and cancel the navigation if needed.

Here, if the note is not saved, you cancel the back navigation and show a dialog that asks whether the user wants to save the note.

- If the user saves the note, call `SaveAsync`, then replace the back navigation with a call to `Navigate` that passes the note to `AllNotesPage`.

- If the user doesn't save their changes, use TextBox.Undo to undo any edits, reset the note state, and restart the back navigation.

C#

```
// ↓ Add this. ↓
protected async override void OnNavigatingFrom(NavigatingCancelEventArgs e)
{
    if (noteModel?.State == NoteState.Unsaved)
    {
        e.Cancel = true;
        ContentDialog dialog = new ContentDialog();

        // XamlRoot must be set for the ContentDialog.
        dialog.XamlRoot = this.XamlRoot;
        dialog.Title = "Save your work?";
        dialog.PrimaryButtonText = "Save";
        dialog.SecondaryButtonText = "Don't Save";
        dialog.CloseButtonText = "Cancel";
        dialog.DefaultButton = ContentDialogButton.Primary;

        ContentDialogResult result = await dialog.ShowAsync();

        if (result == ContentDialogResult.Primary)
        {
            await noteModel.SaveAsync();
            Frame.Navigate(typeof(AllNotesPage), noteModel);
        }
        else if (result == ContentDialogResult.Secondary)
        {
            while (NoteEditor.CanUndo)
            {
                NoteEditor.Undo();
            }
            NoteEditor.Focus(FocusState.Programmatic);
            noteModel.State = NoteState.Saved;
            Frame.Navigate(typeof(AllNotesPage), noteModel);
        }
    }
}
```

Delete

Next, modify the code for the delete button `Click` event in `NotePage.xaml.cs`. Instead of simply deleting the note and navigating back, you'll check the note state.

- If the state is `Unset` – which means the note is newly created, doesn't have any edits, and hasn't been saved – you just navigate back. You don't need to pass the note as a parameter.

- Otherwise, delete the note file from the file system and pass the `Note` object back as the navigation parameter. Then `AllNotesPage` will receive the `Note` with it's `Deleted` state and know to delete it from the `Notes` collection.

C#

```
private async void DeleteButton_Click(object sender, RoutedEventArgs e)
{
    if (noteModel is not null)
    {
        if (noteModel.State == NoteState.Unset)
        {
            // If the note is new, doesn't have any edits,
            // and hasn't been saved, just call GoBack.
            // There's no need to pass back the noteModel.
            if (Frame.CanGoBack == true)
            {
                Frame.GoBack();
            }
        }
        else
        {
            // If the note has been saved before, then delete it
            // and navigate back to the AllNotesPage passing the
            // noteModel with its Deleted state.
            await noteModel.DeleteAsync();
            Frame.Navigate(typeof(AllNotesPage), noteModel);
        }
    }
}
```

Save and close

Currently, the user has to click to save a note, and then click the back arrow to close the note page and go back to the notes collection. This experience can be improved by letting the user save and close the note with one click. To do this, you'll replace the "Save" button with a "Save and close" SplitButton with a drop-down option to just save.

XAML

```
<!-- ↓ Delete this. ↓ -->
<!--<Button Content="Save" Click="SaveButton_Click"/>-->

<!-- ↓ Add this. ↓ -->
<SplitButton Content="Save &amp;amp; close" Click="SaveCloseButton_Click"
             Height="32">
    <SplitButton.Flyout>
        <MenuFlyout>
            <MenuFlyoutItem Text="Save" Click="SaveButton_Click"/>
```

```
        </MenuFlyout>
    </SplitButton.Flyout>
</SplitButton>
```

Add a new event handler for the "Save and close" button `Click` event. Here, you save the note and then pass it back to `AllNotesPage` as the navigation parameter.

C#

```
// ↓ Add this. ↓
private async void SaveCloseButton_Click(SplitButton sender,
SplitButtonClickEventArgs args)
{
    if (noteModel is not null)
    {
        await noteModel.SaveAsync();
        Frame.Navigate(typeof(AllNotesPage), noteModel);
    }
}
```

Handle the navigation parameter in AllNotesPage

In `AllNotesPage`, you need to handle the incoming navigation parameter (`Note`) and add it to or remove it from the `Notes` collection as needed.

To handle the incoming navigation parameter, override the OnNavigatedTo method, as shown here.

C#

```
// ↓ Add this. ↓
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    if (e.Parameter is Note note)
    {
        if (note.State == NoteState.Deleted)
        {
            notesModel.RemoveNote(note);
        }
        else if (!notesModel.Notes.Contains(note))
        {
            notesModel.AddNote(note);
        }
        // This navigation should be treated like a
        // back navigation, so clear the backstack.
        Frame.BackStack.Clear();
    }
}
```

Now you can run your app to see how these changes work. Try adding new notes, navigating back and forth between notes, and deleting notes.

 Learn more in the docs:

- Implement navigation between two pages
- Navigation history and backwards navigation
- Frame class, Page class

Next steps

Congratulations! You've completed the *WinUI Notes part 2* tutorial!

The following links provide more information about creating apps with WinUI and the Windows App SDK:

- Samples and resources
- Design for Windows apps
- Develop Windows desktop apps
- Controls for Windows apps

Last updated on 09/10/2026

WinUI 3



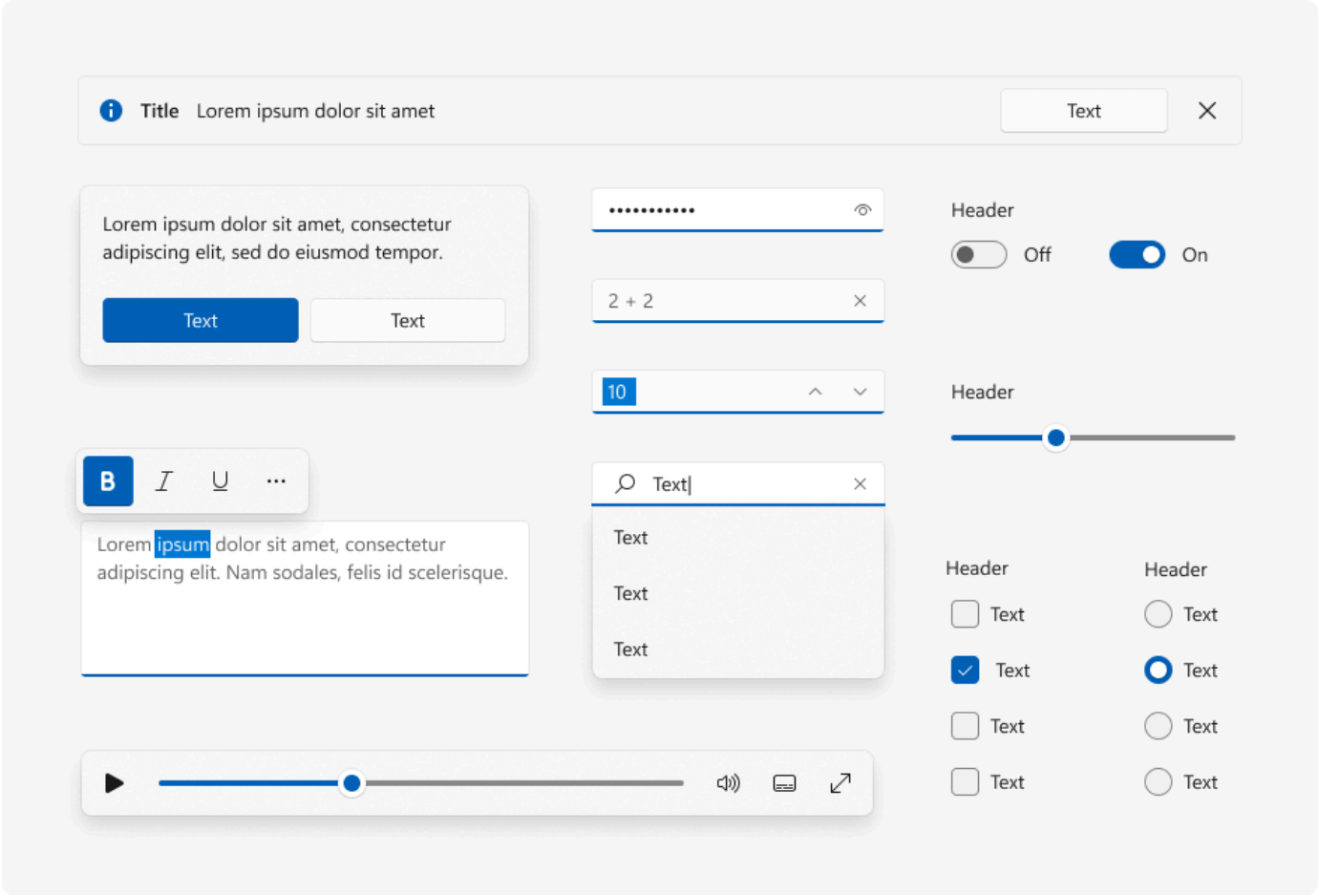
WinUI 3 is the recommended native UI framework for building new Windows desktop applications. It brings the Fluent Design System, high-performance rendering, and a powerful XAML-based programming model to both C# and C++ developers.

Delivered as part of the Windows App SDK, WinUI enables you to create modern, polished, and responsive desktop experiences that run on Windows 10, version 1809 (build 17763) and later, including Windows 11.

WinUI is built in the open, and contributions or discussions are welcome. Visit the repository for details:

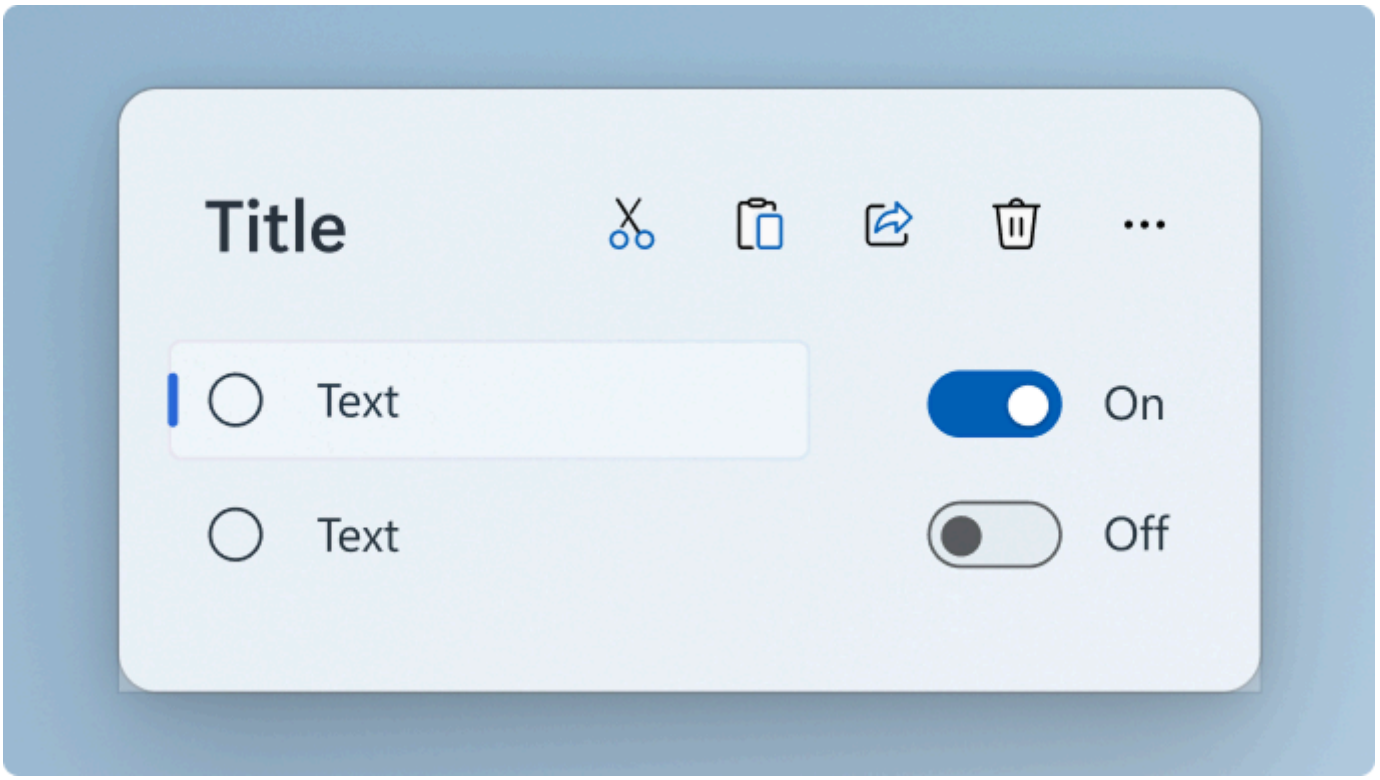
View on GitHub

To stay up to date with what's new with WinUI, check out the latest Windows App SDK release notes.



Key benefits of WinUI

WinUI combines the flexibility of the Win32 app model with the richness of modern Windows UX. It's designed for developers who want to build production-grade desktop apps using the same design language and system features as built-in Windows experiences.



Modern UI

WinUI includes a comprehensive set of modern UI controls and styles, designed for usability, adaptability, and a polished, Fluent look and feel.

Explore Fluent Design



Code in your preferred language

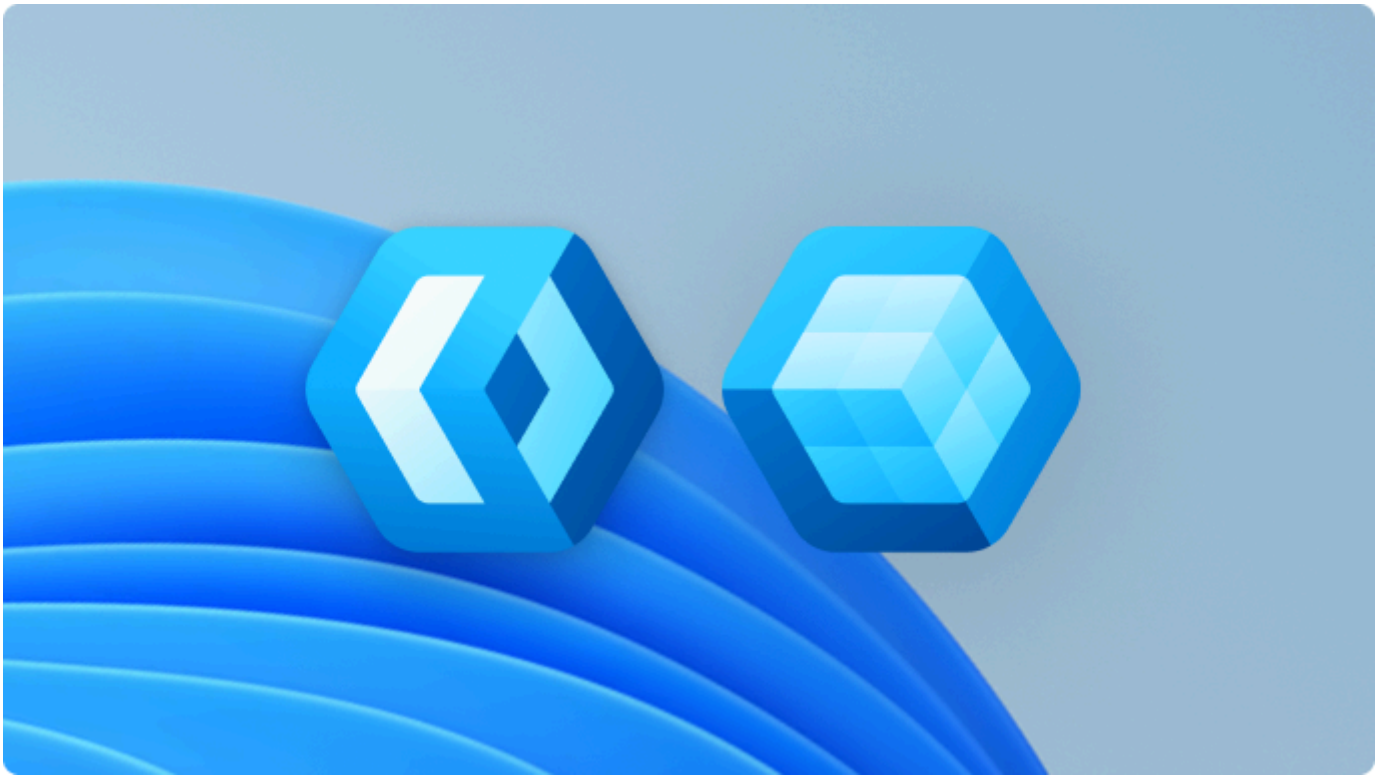
Use .NET (C#) or C++ with XAML markup to build apps that run seamlessly across x86, x64, and

ARM devices.



Optimized for modern experiences and hardware

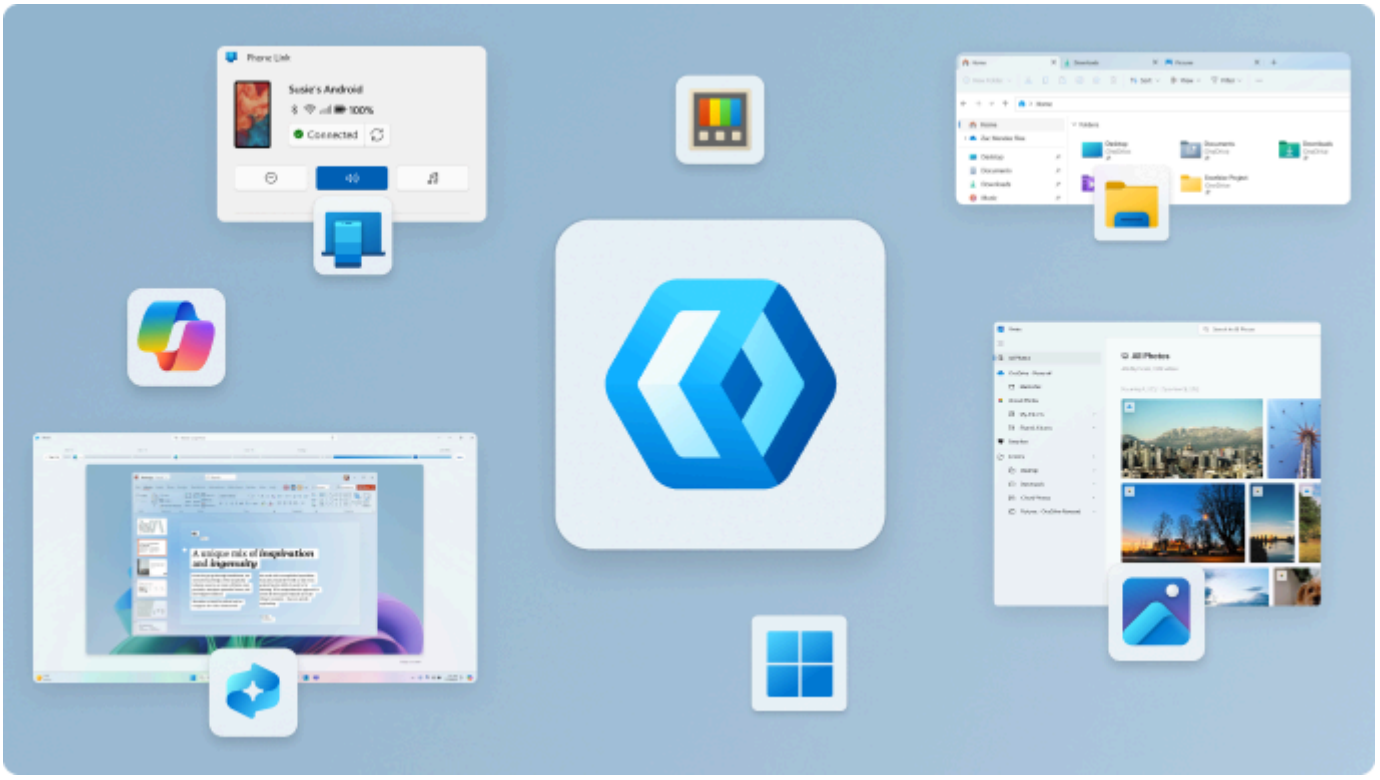
Create rich, high-performance apps with advanced pixel rendering, high-DPI visuals, and smooth animations, optimized for modern hardware and input methods such as touch, pen, keyboard, and mouse.



Built on the Windows App SDK

WinUI is part of the Windows App SDK, giving you access to modern Windows APIs, consistent app behaviors, and tools that simplify development and deployment.

Learn more about the Windows App SDK



Powers key experiences in Windows

Parts of the Windows shell and its built-in apps are built with WinUI. Open-source projects like PowerToys are also built with WinUI.

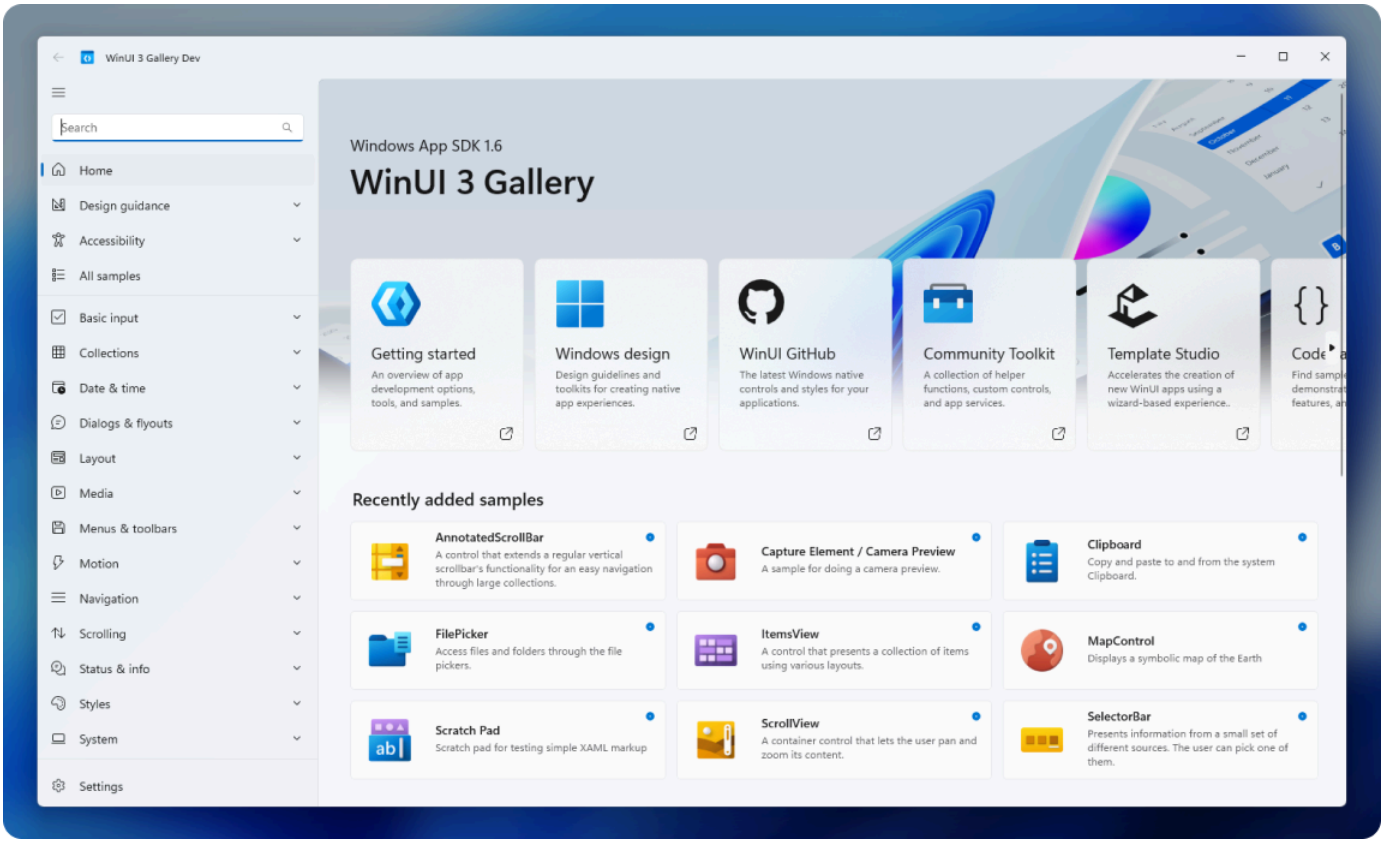
Get started with WinUI

To get started with WinUI, try out these resources:



Create and run a WinUI 3 project

Setup your development environment and build your first WinUI app.



WinUI 3 Gallery

Explore interactive examples of WinUI controls, features, and functionality.



Tools and samples

Tools, samples, and AI-powered resources to help you apps faster.

Last updated on 10/02/2026

Windows App SDK



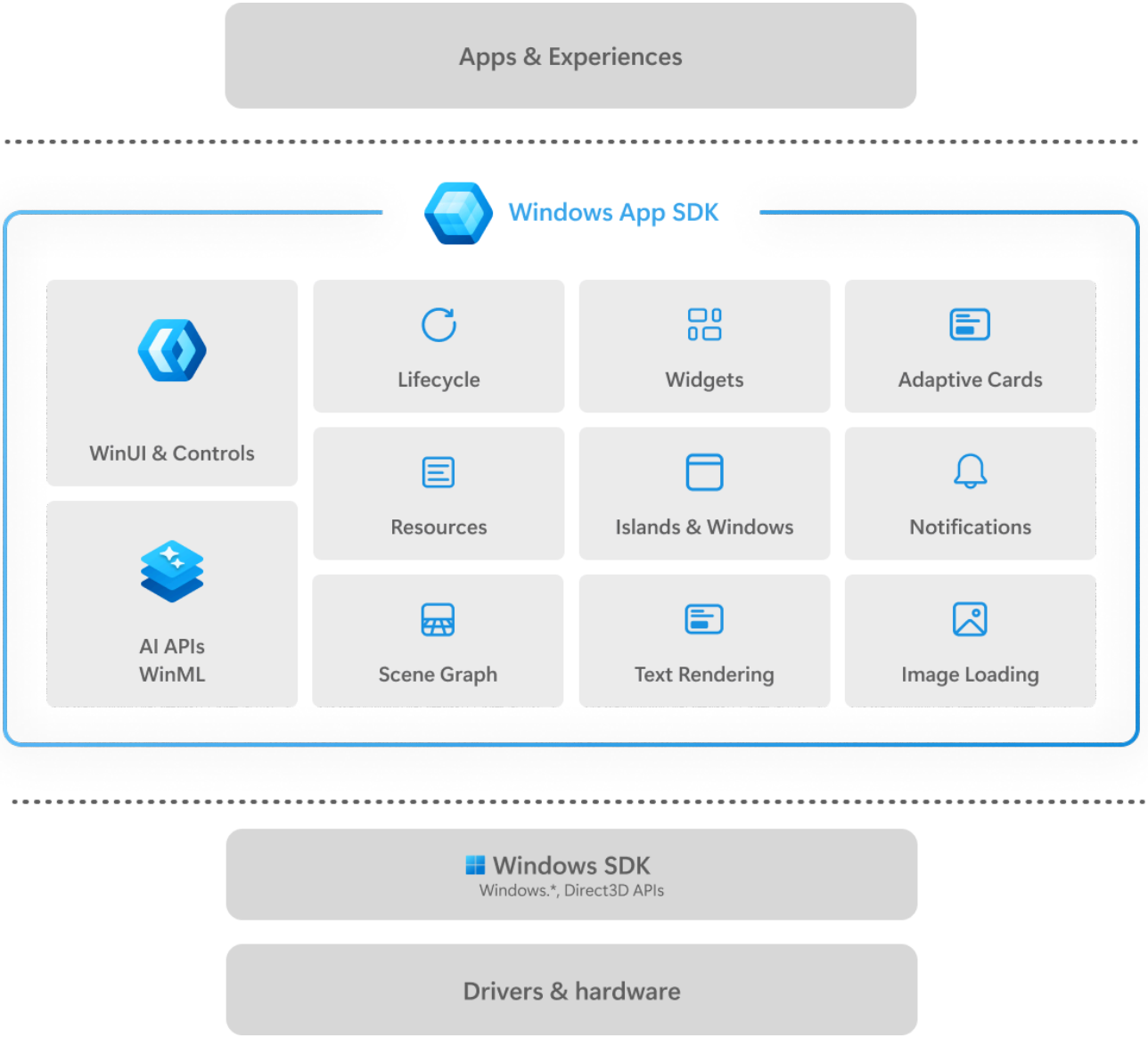
The Windows App SDK is the recommended development platform for building new Windows desktop applications. It provides a unified set of APIs and tools that bring the latest Windows platform features to both WinUI 3 and existing desktop app frameworks such as WPF, Windows Forms, or Win32.

Windows App SDK is built in the open, and contributions or discussions are welcome. Visit the repository for details:

View on GitHub

What you can do with the Windows App SDK

Whether you are building a new app with WinUI or enhancing an existing one with modern capabilities, the Windows App SDK helps you take advantage of the latest Windows APIs.



Here's a breakdown of the major features that Windows App SDK provides:

Expand table

Feature	Description
WinUI	The modern native UI (user interface) framework for Windows apps, supporting both .NET (C#) and C++ projects. WinUI enables consistent, accessible, and beautiful user experiences that align with the Fluent Design system.
Windows AI APIs	Bring powerful, hardware-accelerated artificial intelligence (AI) capabilities to your apps, running local models seamlessly and efficiently on Copilot+ PCs.
Windows ML	Run ONNX AI models locally on Windows, automatically optimizing performance across available hardware such as CPUs, GPUs, and NPUs for fast, efficient inference.
Modern text rendering	Use a device-independent text layout and rendering system with ClearType subpixel rendering, hardware acceleration, and broad language support for precise, high-quality text.

Feature	Description
Resource management	Manage app resources such as strings and images in multiple languages, scales, and contrast variants independently of your app's logic.
App lifecycle	Manage key aspects of your app's runtime behavior, including instance management, rich activation (file, protocol, restart, and more), and power management. Gain control over how your app starts, runs, and conserves system resources.
Windowing	Create, position, and customize app windows with modern windowing APIs.
Notifications	Send local or cloud-based notifications to engage users and keep your app connected, including both app notifications and push notifications.
Widgets	Bring personalized, glanceable information to the Windows widget board, allowing users to stay connected to your app's live content right from their desktop.
XAML Islands	Embed modern Windows UI controls inside existing desktop apps to refresh your interface and add new capabilities without a full rewrite.
Deployment	Deploy the Windows App SDK runtime with your app, whether it is packaged or unpackaged, to ensure consistent and reliable operation across devices.

Benefits of using the Windows App SDK

The Windows App SDK provides a modern foundation for building Windows apps with a unified set of APIs delivered through NuGet. It works alongside the Windows SDK and gives developers access to new Windows features on a faster release cycle.

- **Modern UI out of the box:** The Windows App SDK includes WinUI, a powerful native UI framework for creating modern, high-performance interfaces that align with the Fluent Design system. You can use it to build new desktop apps or refresh existing experiences with modern visuals and controls.
- **Works with your existing app and development stack:** The Windows App SDK can be added to existing apps built with WPF, Windows Forms, Win32, or other application frameworks.
- **Modular SDK design:** Employs a metapackage structure that lets apps reference the full SDK or only specific components, for example WinUI, AI, or text rendering. This enables incremental adoption and reduces overall package size.
- **Consistent across Windows versions:** Windows App SDK APIs run on Windows 11 and earlier versions starting from Windows 10, version 1809. This allows you to use new features

as soon as they are released without depending on operating system updates or writing version adaptive code.

- **Faster release cadence:** Because the Windows App SDK is released independently from the operating system, new APIs and improvements become available several times per year. This faster cadence gives developers earlier access to the latest Windows features without waiting for major OS updates.

Windows App SDK release channels

The Windows App SDK is available through multiple release channels that let you choose the right balance between stability and early access.

Expand table

Release channel	Description
Stable	Intended for production apps and includes only stable, supported APIs. This is the default channel used throughout the Windows App SDK documentation.
Preview	Offers an early look at what's coming in the next stable release. API changes can occur between a preview release and its corresponding stable version.
Experimental	Contains features that are early in development and may change or be removed before future releases.

For more details about the release channels of the Windows App SDK, see Windows App SDK release channels.

Get started with the Windows App SDK

- For new apps, explore WinUI and create and run a WinUI 3 project.
- To integrate the Windows App SDK into an existing WPF, Windows Forms, Win32, or cross-platform project, see use the Windows App SDK in an existing app guidance.
- For version-specific details, visit Release channels and Downloads.

> **Important**
>
> If you're working on a UWP app, then refer to Migrate from UWP to the Windows App SDK.

Give feedback and contribute

We are building the Windows App SDK as an open source project. We have a lot more information on our GitHub page about how we're building the Windows App SDK, and how you can be a part of the development process. Check out our contributor guide to ask questions, start discussions, or make feature proposals. We want to make sure that the Windows App SDK brings the biggest benefits to developers like you.

Related topics

- Release channels and release notes
- Create and run a WinUI 3 project

Last updated on 09/10/2026

Windows App SDK release channels

The latest version of the Windows App SDK ships via three release channels: Experimental, Preview, and Stable. The following table provides an overview of these release channels.

Expand table

	Channel	Description	Release cadence	Supported?	Latest release
✅	**Stable**	Production-ready channel intended for apps in market. Includes only stable, supported APIs suitable for long-term use.	Major releases no more than every six months *(plus minor/patch servicing updates as needed)*	Yes	**2.5.1** Released: 09/16/2026 Release notes
✳️	**Preview**	Early look at the next Stable release. May introduce breaking API changes before final stabilization.	At least one preview for each Stable release	No	**2.0 Preview2 (2.0.0-preview2)** Released: 03/31/2026 Release notes
🔄	**Experimental**	Early-stage features under active development. APIs may change, be removed, or never ship. Intended for exploration and feedback only.	Published as needed to gather feedback on prototypes and early designs	No	**2.5 Experimental (2.5.4-experimental)** Released: 09/29/2026 Release notes

For a comprehensive list of all current and previous releases of the Windows App SDK, including download locations, see Downloads for the Windows App SDK.

Features available by release channel

The following table shows which features are currently available in each release channel.

Expand table

Feature	✅ Stable	✳️ Preview	🔄 Experimental
Deployment guide for framework-dependent packaged apps	✔ Available	✔ Available	✔ Available

Feature	Stable	Preview	Experimental
Deployment guide for framework-dependent apps packaged with external location or unpackaged	✔ Available	✔ Available	✔ Available
WinUI	✔ Available	✔ Available	✔ Available
Text rendering	✔ Available	✔ Available	✔ Available
Manage resources	✔ Available	✔ Available	✔ Available
App lifecycle: App instancing	✔ Available	✔ Available	✔ Available
App lifecycle: Rich activation	✔ Available	✔ Available	✔ Available
App lifecycle: Power management	✔ Available	✔ Available	✔ Available
Manage app windows	✔ Available	✔ Available	✔ Available
Push notifications	✔ Available	✔ Available	✔ Available
App notifications	✔ Available	✔ Available	✔ Available
Windows Widgets	✔ Available	✔ Available	✔ Available
App Content Search	✔ Available	✘ Not available	✔ Available

> **Note**
>
> App Content Search entered the Stable channel in Windows App SDK 2.5.1 and is a Limited Access Feature. Apps must obtain a token before calling the APIs. The experimental channel release does not require a token.

Windows App SDK release servicing policy

The following policies define the servicing you can expect when you use a given Windows App SDK release.

Release lifecycle

The Windows App SDK has a lifecycle. A lifecycle begins when a version or service is released and ends when it's no longer supported. Knowing key dates in this release lifecycle helps you make informed decisions about when to upgrade or make other changes to your software.

Expand table

Windows App SDK version	Original release date	Latest patch version	Patch release date	Support level	End of servicing
2.0	04/29/2026	2.5.1	09/16/2026	Current	04/29/2027
1.8	09/09/2025	1.8.260921001	09/24/2026	Maintenance	09/24/2026
1.7	03/18/2025	1.7.260224002	03/10/2026	Out of Support	03/18/2026
1.6	09/04/2024	1.6.250602001	06/10/2025	Out of Support	09/04/2025
1.5	02/29/2024	1.5.250108004	01/15/2025	Out of Support	02/28/2025
1.4	08/29/2023	1.4.240802001	08/13/2024	Out of Support	08/29/2024
1.3	04/12/2023	1.3.230724000	07/25/2023	Out of Support	04/12/2024
1.2	11/10/2022	1.2.230313.1	03/15/2023	Out of Support	11/10/2023
1.1	05/24/2022	1.1.5	09/14/2022	Out of Support	05/24/2023
1.0	11/16/2021	1.0.4	06/14/2022	Out of Support	11/16/2022
0.8	06/24/2021	0.8.12	08/03/2022	Out of Support	06/24/2022
0.5	03/29/2021	0.5.9	08/10/2021	Out of Support	11/01/2021

Servicing

Customers can choose *Current* releases or *Maintenance* releases. Both types of releases receive critical fixes throughout their lifecycle, for security, compatibility, and reliability. You must stay up to date with the latest patches to qualify for support.

The quality of all releases is the same. The only difference is the servicing time frame.

Servicing for Current releases

Current releases are the latest stable release and they receive fixes more frequently. During the servicing period, Windows App SDK current release is updated to improve functional capabilities and mitigate security vulnerabilities.

Functional improvements are typically very targeted, and may address the following:

- Resolve reported crashes.
- Resolve severe performance issues.
- Resolve functional bugs in key scenarios.
- Add support for a new operating system versions.

Servicing for Maintenance releases

Maintenance releases are no longer the latest stable release, but they will still receive critical fixes. The bar for fixes in Maintenance releases will be higher than the bar for fixes to the Current release.

After the maintenance period ends, the release is out of support.

Microsoft support for the Windows App SDK

The Windows App SDK is governed by the Microsoft Modern Lifecycle.

Although the Windows App SDK is currently backward compatible to Windows 10 version 1809, the Windows App SDK is supported by Microsoft only on the Windows releases still in support.

Support has two key benefits:

- Patches are provided for free, as required for functional or security issues.
- You can contact Microsoft support to request help, potentially at a cost.

Support is conditional on using the latest Windows App SDK patch update and a supported operating system.

End of support

End of support refers to the date when Microsoft no longer provides fixes, updates, or online technical assistance. End of support may also be referred to as 'end of life' or abbreviated 'EOL'. This is the time to make sure you have the latest available update installed.

Updates are cumulative, with each update built upon all of the updates that preceded it. A device needs to install the latest update to remain supported. Updates may include new features, fixes (security and/or non-security), or a combination of both. Not all features in an update will work on all devices. Update availability may vary, for example by country, region, network connectivity, or hardware capabilities (including, for example, free disk space).

Your use of out-of-support Windows App SDK versions may put your applications at risk. You are strongly recommended to not use out-of-support software.

How to get help

- The Windows App SDK uses GitHub Issues to track bugs and feature requests. Search the existing issues before filing new issues to avoid duplicates. For new issues, file your bug or feature request as a new issue.
- For help and questions about using the Windows App SDK, search for existing questions or post a new question on our GitHub Discussions page.
- Technical support for the use of the Windows App SDK may be available from Microsoft Customer Support Services (CSS). If you're a Premier or Unified Support customer, reach out to your account manager for further assistance. Otherwise, visit the Support For Business site to open a new support case for the Windows App SDK.

Related topics

- Install tools for preview and experimental channels of the Windows App SDK
- Install tools for the Windows App SDK
- Create and run a WinUI 3 project

Last updated on 09/29/2026

Windows App SDK and supported Windows releases

This topic identifies the versions of the Windows client and Windows Server operating systems (OS) supported by the Windows App SDK, along with the various servicing categories, channels, and SKUs for each OS.

The following table lists the Windows client and server releases supported by the Windows App SDK (the subsequent sections provide details on the various servicing categories, channels, and SKUs for each OS).

Expand table

Windows App SDK	Release date	Windows client	Windows Server
2	2026-04-29	Win11 26H2 Win11 25H2 Win11 24H2 Win11 23H2 Win11 22H2 Win11 21H2 Win10 23H2 Win10 22H2 Win10 21H2 Win10 1809	Server 2022 Server 2019
1.8	2025-09-09	Win11 26H2 Win11 25H2 Win11 24H2 Win11 23H2 Win11 22H2 Win11 21H2 Win10 23H2 Win10 22H2 Win10 21H2 Win10 1809	Server 2022 Server 2019
1.7	2025-03-18	Win11 25H2 Win11 24H2 Win11 23H2 Win11 22H2 Win11 21H2 Win10 23H2 Win10 22H2	Server 2022 Server 2019

Windows App SDK	Release date	Windows client	Windows Server
		Win10 21H2 Win10 1809	
1.6	2024-09-04	Win11 25H2 Win11 24H2 Win11 23H2 Win11 22H2 Win11 21H2 Win10 23H2 Win10 22H2 Win10 21H2 Win10 1809	Server 2022 Server 2019

Windows client SKUs and channels

The Windows client OS runs on personal devices such as workstations, desktop PCs, laptops, and tablets.

There are four servicing categories that apply to various combinations of Windows client SKUs and servicing channels.

Expand table

Servicing channel		SKU	SKU
		Home, Pro	Enterprise
GA*		Shortest servicing	Short servicing
LTSC**	Mainstream	N/A	Long servicing
	Extended	N/A	Longest servicing

* The General Availability (GA) servicing channel includes the majority of Windows devices.

** The Long Term Servicing Channel (LTSC) includes devices that need to continue running older versions of Windows.

Several versions of Windows client are still in service for at least one servicing category. Each version will continue to be supported by the Windows App SDK until the latest applicable Windows servicing end-date for that version. The following table shows the end-date for each servicing category.

Expand table

Version	Build	Servicing end-dates				Max end-dates
		GA Home, Pro	GA Enterprise	LTSC Mainstream	LTSC Extended	
Win11 25H2	26200	2027-10-12	2028-10-10	N/A	N/A	2028-10-10
Win11 24H2	26100	2026-10-13	2027-10-12	2029-10-09	2034-10-10	2034-10-10
Win11 23H2	22631	2025-11-11	2026-11-10	N/A	N/A	2026-11-10
Win10 21H2	19044	Past	2024-06-11	2027-01-12	2032-01-13 (IoT Enterprise only)	2032-01-13
Win10 1809	17763	Past	Past	Past	2029-01-09	2029-01-09

Windows Server SKUs and channels

Windows Server is the platform for building an infrastructure of connected applications, networks, and web services, from the workgroup to the data center. Windows Server offers services to other devices, has more features and capacities than a client OS, and can support more network connections.

The following table shows the end-date for each servicing category.

Expand table

Version	Build	Servicing end-dates		Max end-dates
		LTSC Mainstream	LTSC Extended	
Server 2022	20348	2026-10-13	2031-10-14	2032-01-13
Server 2019	17763	Past	2029-01-09	2029-01-09

Related articles

- Check for installed versions of the Windows App SDK runtime

Last updated on 09/24/2026

Windows App SDK 2.0 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 2.0 Preview 2 (2.0.0-Preview2)

Released: **March 31, 2026**

▼ Windows ML

> The Microsoft.WindowsAppSDK.ML NuGet package has been refactored, adding a new base dependency. The existing ML NuGet package's functionality remains the same, but the core Windows ML features have been refactored into a base dependent package called Microsoft.Windows.AI.MachineLearning. This new package contains a minimal set of dependencies, but on C#/WinRT or C++/WinRT it only supports down to Windows 10 18362. If you need Windows 10 17763 support, use the existing Microsoft.WindowsAppSDK.ML package.
>
> For new transitive dependencies, we recommend using tooling to upgrade the package (nuget.exe or VS). See also: Windows App SDK 1.6 release notes.

▼ Updated ONNX Runtime

> The version of ONNX Runtime has been updated to 1.24.4. See ONNX Runtime versions for more info.

▼ Windows AI APIs

> Enforced Phi Silica APIs to be part of a Limited Access Feature.
>
> Added checks for various network and Windows Update errors during AI model installation, flagging durable failures so users can understand why model packages failed to install.
>
> Improved diagnosability for Text Intelligence APIs used in Windows AI scenarios.

▼ App Content Search

> App Content Search has been removed from 2.0 preview while we work to improve fundamentals and ensure future compatibility as the underlying AI models change.In the meantime, please continue to use the experimental channel to try the latest changes.

▼ WebView2 (WinUI 3)

> Enabled drag support for WebView2 content hosted in WinUI 3 applications. This capability was previously unsupported and is now available without introducing new public APIs. Note: Drop into WebView2 from external sources is already supported.
>
> ## Supported scenarios
>
> - **Standard drag-and-drop content** Dragging text, HTML, images, and URLs is supported. Based on current behavior, dragging images outside the app results in a default file name (for example, `download.jpg`) when dropped into File Explorer. *Example scenario:* A user selects text or an image inside WebView2 and drags it either within the app or to another destination such as File Explorer.
> - **Drag cancellation** On-demand cancellation of an active drag operation is supported. *Example scenario:* An app conditionally blocks drag operations for certain content types (such as restricted or ephemeral content) by cancelling the drag after it has been initiated.
> - **Custom drag visuals** Custom drag UI, such as icons or previews, is supported to help users clearly identify the content being dragged. *Example scenario:* An app displays a thumbnail preview or icon while a user drags media content, making it clear which item is currently being moved.
> - **Customizable drag data** Editing and customizing drag data is supported, enabling app-specific scenarios such as dragging messages within the app (for example, as an alternative interaction for message forwarding). *Example scenario:* An app customizes the drag payload to include contextual metadata, such as the originating application name. When the content is dropped, the destination can display or process information like "From <app name>" to indicate the source of the dragged content.
>
> ## Limitations

- Support for additional drag data types is limited. The following formats are not currently supported:
  - `DownloadURL`

Requirements

- **Minimum WebView2 Runtime version (Edge Beta channel):** 144.0.3719.11

▼ Custom XAML Conditionals (IXamlCondition)

The `IXamlCondition` interface enables developers to define custom conditions that integrate with XAML's conditional namespace syntax and are evaluated at XAML parse time. This is a rename and graduation of the experimental `IXamlPredicate` interface. Custom conditions enable conditional XAML scenarios based on application-specific factors such as feature flags, device capabilities, business logic, configuration settings, and other runtime conditions.

▼ Bug fixes

- Fixed MSB8027 and LNK4042 build warnings caused by duplicate ClCompile items in Windows App SDK NuGet .targets files by moving preprocessor definitions from `<Target>` blocks to `<ItemDefinitionGroup>`. An opt-out workaround (`WindowsAppSDK_Arm64EcCompilerWorkaround`) is included for ARM64EC+LTCG builds to avoid a known MSVC internal compiler error.
- Fixed a ListView crash that could occur during keyboard navigation (Tab/Shift+Tab) after the items list was updated.
- Fixed an issue where WinUI 3 could crash if focus was moved to the CoreWebView2Controller while the controller was not visible.

▼ New or updated APIs

This release includes the following new and modified APIs compared to 2.0-Preview1:

```
Microsoft.UI.Xaml.Markup

    IXamlCondition
```

Version 2.0 Preview 1 (2.0.0-Preview1)

Released: **February 13, 2026**

▼ Windows ML

> - The version of ONNX Runtime in Windows ML has been updated to 1.24 RC
> - Fixed a bug where if RegisterCertifiedAsync is called again in the same process, it incorrectly returns 0 execution providers (EP)

▼ Microsoft.UI.Content

> - New `InputFocusController.ShouldShowKeyboardCues` property to guide developers on whether to show keyboard cues right after the creation of a `ContentIsland`.
> - New convenience API `PointerPoint.GetCurrentPoint`, to allow developers to get the active `PointerPoint` data from the provided `pointerId`.

▼ App Content Search

> - Improved `DeleteIndex` reliability. Sometimes `DeleteIndex` would fail with ERROR_SHARING_VIOLATION.
> - Fix for OCR Bounding boxes returning negative values in some edge cases.
> - App Content Search are part of a separate `Microsoft.Windows.Search` package instead of being part of `Microsoft.Windows.AI package`.
> - Fix for prefix search not working with short query strings.
> - New APIs and renames based on official API review.
>   - AppContentIndexer
>   - GetContentItemsRequiringReindexing
>   - Remove -> RemoveContentItem
>   - RemoveMultiple -> RemoveContentItems
>   - RemoveAllContentItems
>   - GetContentIndexingStatus -> GetContentItemStatus
>   - GetMultipleContentIndexingStatus -> GetContentItemStatuses
>   - GetContentItems
>   - AppContentIndexListener
>   - IndexingStatusChanged -> ContentItemStatusChanged
>   - AppIndexTextQuerySession

    - UpdateQuery -> UpdateQueryPhrase
    - MostRecentResult -> GetResult
    - MostRecentResultChanged -> ResultChanged
  - AppIndexImageQuerySession
    - UpdateQuery -> UpdateQueryPhrase
    - MostRecentResult -> GetResult
    - MostRecentResultChanged -> ResultChanged
  - AppManagedImageQueryMatch
    - Subregion -> RegionOfInterest
  - ContentItemStatusResult
    - ReindexingStatus * ContentItemReindexingStatus
  - TextQuerySessionResult
    - IsValid
  - ImageQuerySessionResult
    - IsValid
  - QueryContentItemsFilterFlags

▼ Bug Fixes

- Fixed an issue where the WindowsAppSDK installer showed no progress during installation, making it appear stalled. The installer now provides clearer progress feedback.
- Improved error handling of scenarios where WindowsAppSDKSelfContained is enabled for class libraries.

▼ New or updated APIs

This release includes the following new and modified experimental APIs compared to 1.8.5:

```
Microsoft.Graphics.Imaging

    ImageBufferPixelFormat
        Bgr8
```

```
Microsoft.UI.Content

    DesktopPopupSiteBridge
```

```
        AnchoringBehavior
        AnchoringPixelAlignment

    PopupAnchor
```

```
Microsoft.UI.Input

    InputFocusController
        ShouldShowKeyboardCues

    PointerPoint
        GetCurrentPoint
```

```
Microsoft.UI.Xaml.Automation.Peers

    SplitMenuFlyoutItemAutomationPeer
```

```
Microsoft.UI.Xaml.Controls

    SplitMenuFlyoutItem
    SystemBackdropElement
```

```
Microsoft.Windows.Management.Deployment

    AddPackageOptions
        GetValidationEventSourceForUri
        IsPackageValidationSupported
        PackageValidators

    IPackageValidator
    PackageCertificateEkuValidator
    PackageFamilyNameValidator
    PackageMinimumVersionValidator
    PackageValidationEventArgs
    PackageValidationEventSource
    PackageValidationHandler
    PackageVolume
        AddAsync
        GetAvailableSpaceAsync
        GetDefault
        GetPackageVolumeByName
        GetPackageVolumeByPath
```

```
        IsFeatureSupported
        IsOffline
        RemoveAsync
        SetDefault
        SetOfflineAsync
        SetOnlineAsync

    PackageVolumeFeature
    StagePackageOptions
        GetValidationEventSourceForUri
        IsPackageValidationSupported
        PackageValidators
```

```
Microsoft.Windows.Search.AppContentIndex

    AppContentIndexContract
    AppContentIndexer
    AppContentIndexListener
    AppIndexContentRegion
    AppIndexImageQuery
    AppIndexImageQuerySession
    AppIndexQueryMatch
    AppIndexTextQuery
    AppIndexTextQuerySession
    AppIndexTextStreamEncoding
    AppManagedImageQueryMatch
    AppManagedIndexableAppContent
    AppManagedTextQueryMatch
    ContentItemErrorDetail
    ContentItemReader
    ContentItemReindexingStatus
    ContentItemStatus
    ContentItemStatusResult
    ContentRegionTextOptions
    DeleteIndexResult
    DeleteIndexStatus
    DeleteIndexWhileInUseBehavior
    GetOrCreateIndexOptions
    GetOrCreateIndexResult
    GetOrCreateIndexStatus
    ImageQueryMatch
    ImageQueryOptions
    ImageQuerySessionResult
    IndexableAppContent
    IndexCapabilities
    IndexCapabilitiesOfCurrentSystem
    IndexCapability
    IndexCapabilityInitializationStatus
    IndexCapabilityLanguageStatus
    IndexCapabilityOfCurrentSystemStatus
    IndexCapabilityRequirement
    IndexCapabilityState
```

```
    IndexStatistics
    QueryContentItemsFilterFlags
    QueryMatchContentKind
    QueryMatchScope
    RegionContentKind
    TextLexicalMatchType
    TextQueryMatch
    TextQueryOptions
    TextQuerySessionResult
```

```
Microsoft.Windows.SemanticSearch

    EmbeddingVector
    SemanticSearchContract
```

```
Microsoft.Windows.Storage.Pickers

    FileOpenPicker
        FileTypeChoices
        InitialFileTypeIndex
        SettingsIdentifier
        SuggestedFolder
        SuggestedStartFolder
        Title

    FileSavePicker
        InitialFileTypeIndex
        SettingsIdentifier
        ShowOverwritePrompt
        SuggestedStartFolder
        Title

    FolderPicker
        PickMultipleFoldersAsync
        SettingsIdentifier
        SuggestedFolder
        SuggestedStartFolder
        Title
```

```
Microsoft.Windows.Vision

    ScreenRegionBoundingBox
    ScreenRegionDetectionContract
    ScreenRegionLabel
```

Last updated on 09/29/2026

Windows App SDK 1.8 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.8.12 (1.8.260921001)

Released: **September 24, 2026**

▼ Bug fixes

> Fixed an issue where ExecutionProviderCatalog.FindAllProviders could return an execution provider in the NotPresent state on certain Arm64 devices even though ExecutionProvider.EnsureReadyAsync could not acquire it. The unavailable provider is no longer returned on affected devices. (RuntimeCompatibilityChange: N/A, Windows ML provider availability fix)

Version 1.8.11 (1.8.260804001)

Released: **August 13, 2026**

▼ Bug fixes

> - Fixed an issue where the Storage Pickers (`FileOpenPicker`, `FileSavePicker`, and `FolderPicker`) left the calling WinUI app window without keyboard focus after the file dialog closed. The app now restores its previous focus after the dialog closes. For more info, see GitHub issue #6505. (RuntimeCompatibilityChange: StoragePickers_RestoreFocusAfterDialogCloses)
> - Fixed a regression where `MrmGetFilePathFromName` returned `ERROR_FILE_NOT_FOUND` when the PRI file didn't exist, instead of returning the documented fallback path. For more info, see GitHub issue #5940. (RuntimeCompatibilityChange: ResourceManager_RestoreDefaultFallbackPath)
> - Fixed an issue where `MICROSOFT_WINDOWSAPPRUNTIME_BASE_DIRECTORY` could be inherited by child processes, disrupting MRT Core and catalog lookups. For more info, see GitHub

issue #5987. (RuntimeCompatibilityChange: WindowsAppRuntime_BaseDirectoryIsolation)

Version 1.8.10 (1.8.260710003)

Released: **July 14, 2026**

▼ Bug fixes

- Fixed an issue where `Microsoft.UI.System.ThemeSettings` could crash an app if it was destroyed on a background thread, for example when destroyed by the .NET garbage collector. (RuntimeCompatibilityChange: ThemeSettings_OffThreadDestructorFix)

Version 1.8.9 (1.8.260529003)

Released: **June 9, 2026**

▼ Bug fixes

- Fixed a memory leak in `ItemsRepeater` where recycled elements were never garbage collected due to a reference cycle through the `RecyclePool`, which could also cause crashes in `InvalidateChildrenMeasure`. (RuntimeCompatibilityChange: ItemTemplateWrapper_RecyclePoolLeak)
- Fixed a potential crash when releasing touch contacts in a `ScrollViewer` before scrolling. (RuntimeCompatibilityChange: InputPointerSource_TouchUpScrollCrashFix)

Version 1.8.8 (1.8.260508005)

Released: **May 12, 2026**

▼ Bug fixes

- Fixed an issue where windowed popup content opened in a XAML Island did not respect `OverrideScale`, causing content to appear oversized and clipped. For more info, see GitHub issue #11000. (RuntimeCompatibilityChange: Popup_WindowedPopupOverrideScale)
- Fixed an `ACCESS_VIOLATION` crash in `RenderTargetBitmap.RenderAsync` that occurred when the target element left the visual tree (for example, a popup closing) before the

capture completed. (RuntimeCompatibilityChange: RTB_CancelRenderOnTreeLeave)

- Fixed a potential crash in package management when a package was uninstalled prior to being processed. (RuntimeCompatibilityChange: PackageManager_FixCrash)
- Fixed an issue where `GetReadyState` could return incorrect error codes when required Windows AI packages were not yet deployed. The API now correctly reports `NotReady` in this scenario, improving diagnostic clarity for developers and reducing false-positive failure signals in telemetry. (RuntimeCompatibilityChange: ModelInitialization_GetReadyStateAvailabilityGuard)
- Improved internal performance diagnostics for `LanguageModel.GenerateResponseAsync` to better identify sources of latency before the first token is returned. (RuntimeCompatibilityChange: LanguageModelInsights_GetPartialResultLatency)
- Fixed a fail-fast crash caused by re-entrant dispatch during cross-apartment COM release operations in `UIAffinityReleaseQueue::DoCleanup`. (RuntimeCompatibilityChange: UIAffinityReleaseQueue_PauseDispatchDuringCleanup)
- Fixed an issue where `Microsoft.Windows.Workloads.dll` failed to load on Windows builds prior to 22000 due to static imports of Dynamic Dependencies APIs unavailable on those OS versions. The functions are now resolved dynamically; failures on unsupported OS versions surface as a normal `HRESULT` instead of a loader error dialog. (RuntimeCompatibilityChange: PackageDependency_ResolveDynamically)
- Fixed an issue where XAML compiler errors were silently lost when using `dotnet build`, showing only `MSB3073: exited with code 1` instead of the actual error messages. For more info, see GitHub issue #9813. (RuntimeCompatibilityChange: N/A, build-time tooling change)
- Fixed a design issue in the Windows App SDK NuGet `.targets` files where `%(PreprocessorDefinitions)` metadata references inside MSBuild `<Target>` blocks triggered task batching, causing the same auto-initializer source file to be added to `ClCompile` multiple times and producing duplicate `.obj` outputs. This resulted in MSB8027 ("Two or more files with the same name will produce outputs to the same location") and LNK4042 ("object specified more than once") warnings. Preprocessor definitions are now set via `<ItemDefinitionGroup>`, which correctly applies definitions without duplication. An opt-out property (`WindowsAppSDK_Arm64EcCompilerWorkaround`) is included for ARM64EC+LTCG builds to work around a known MSVC compiler internal error. For more info, see GitHub issue #5395. (RuntimeCompatibilityChange: N/A, build .targets change)

Version 1.8.7 (1.8.260416003)

Released: **April 21, 2026**

▼ New CoPilot+ PC capability check

> A new **AICapabilities.HasAICapability** API enables third-party applications to determine whether the device is a Copilot+ PC.

▼ New AIFeatureReadyResult.PackageInstallationFailed API

> The `AIFeatureReadyResult.PackageInstallationFailed` API enables apps to detect when an `EnsureReadyAsync` failure was caused by a known or expected install condition (such as a network failure, Windows Update error, or low disk space). Apps can use this signal to handle expected install failures differently from genuine errors, rather than treating them all as fatal.

▼ Expanded NPU hardware support

> Windows ML NPU detection has been centralized to support new and upcoming NPU hardware types without requiring client application updates.

▼ Updated ONNX Runtime

> The version of ONNX Runtime has been updated to 1.23.5. See ONNX Runtime versions for more info.

▼ Bug fixes

> - Fixed a crash in Windows AI **LanguageModel** APIs that could occur when an exception was encountered during input processing. (RuntimeCompatibilityChange: LanguageModel_ProcessingFix)
> - Fixed `ImageDescription.DescribeAsync` failing with `InternalError` when called concurrently from multiple threads. (RuntimeCompatibilityChange: ImageDescription_ConcurrencyFix)
> - Fixed an issue where content moderation error codes from **LanguageModel** APIs could be silently overwritten by subsequent errors, preventing callers from receiving the correct content safety signals. (RuntimeCompatibilityChange: LanguageModel_ContentErrorHandlingFix)
> - Fixed an issue where the touch keyboard would not appear in XAML island scenarios (such as File Explorer) after using touch elsewhere in the app.

(RuntimeCompatibilityChange: InputPointerSource_TouchKeyboardInvocationFix)

- Fixed a ListView crash (`E_BOUNDS` in `Selector::OnGettingFocus`) that could occur during keyboard navigation (Tab/Shift+Tab) after the items list was updated. (RuntimeCompatibilityChange: ListViewSelector_FixFocusedIndex)
- Fixed a crash that may occur on certain devices when active touch contacts are canceled by input hardware, such as when pen input overrides them. (RuntimeCompatibilityChange: InputPointerSource_PointerCancelCrashFix)
- Improved diagnosability for Windows AI model initialization telemetry. (RuntimeCompatibilityChange: ModelInitialization_Activities)
- Improved diagnosability for Windows ML APIs. (RuntimeCompatibilityChange: WindowsML_DiagnosabilityFix)

▼ New or updated APIs

This release includes the following new APIs compared to the 1.8.6 release:

```
Microsoft.Windows.AI

    AICapabilities
    AICapabilityCategory
    AIFeatureReadyResult
        PackageInstallationFailed
```

Version 1.8.6 (1.8.260317003)

Released: **March 18, 2026**

▼ New SplitMenuFlyoutItem Control

The new **SplitMenuFlyoutItem** control is designed to provide a split button experience within a menu flyout. This control will enable developers to expose a default primary action while also offering additional options through a submenu, ideal for condensing complex functionality into a smaller footprint and saving overall menu length. For more information, see GitHub spec 10862. (RuntimeCompatibilityChange: SplitMenuFlyoutItem_Available)

▼ Windows ML CMake support

Windows ML can now be used from C++ projects using CMake. See the Get started page to learn more.

▼ Updated ONNX Runtime

> The version of ONNX Runtime has been updated to 1.23.4. See ONNX Runtime versions for more info.

▼ Bug fixes

> - Fixed a pointer arithmetic issue in the app-instance activation-redirection shared-memory queue that could cause incorrect behavior when using `AppInstance.RedirectActivationTo`. (RuntimeCompatibilityChange: AppLifecycle_SharedMemoryRedirectionQueueFix)
> - Fixed an issue in the deployment manager where activity data was being incorrectly reset during package installation and license operations. (RuntimeCompatibilityChange: DeploymentManager_ActivityCollectionFix)
> - Improved diagnosability for Text Intelligence APIs used in Windows AI scenarios. (RuntimeCompatibilityChange: TextIntelligence_Insights)
> - Fixed a visual alignment issue with `AppBarToggleButton` in `CommandBarFlyout` secondary commands. (RuntimeCompatibilityChange: N/A, XAML control template fix)

▼ New or updated APIs

> This release includes the following new APIs compared to the 1.8.5 release:
>
> ```
> Microsoft.UI.Xaml.Automation.Peers
>
>     SplitMenuFlyoutItemAutomationPeer
> ```
>
> ```
> Microsoft.UI.Xaml.Controls
>
>     SplitMenuFlyoutItem
> ```

Version 1.8.5 (1.8.260209005)

Released: **February 10, 2026**

▼ Enhanced Windows ML reliability and deployment flexibility

- The version of ONNX Runtime in Windows ML has been updated to ~1.23.3. Benefits from the latest bug fixes, security updates, and performance improvements across all execution providers (CPU, DirectML, OpenVINO, TensorRT)
- Updated Windows ML License agreement to clarify legal complexities.

▼ Bug fixes

- Fixed a package downgrade init failure and added 1.8.5 runtime support—ensuring reliable deployment and correct app recognition across configurations. (RuntimeCompatibilityChange: DeploymentManager_PackageDowngradeFix).
- Fixed an issue where `CompositionIsland.StateChanged` event handlers were not triggered when the user modified the size of a window by dragging it to the top of the screen or using the edges of the screen and the window has a non-default `TitleBar` with an `IconSource` set. Closing #10374 (RuntimeCompatibilityChange: InputNonClientPointerSource_PropagateWindowPosChangedMessage).
- Fixed a crash during app shutdown for certain custom input handling scenarios inside of a ScrollViewer. (RuntimeCompatibilityChange: DwmCoreI_ShutdownManipulationCrash).
- Improved diagnosability for Windows AI model initialization and Text Intelligence APIs. (RuntimeCompatibilityChange: ModelInitialization_Insights).
- Fixed an issue in the Windows AI API `GetReadyState` which erroneously returns `AIFeatureReadyState::NotReady` on unsupported Windows OS versions. The API now returns `AIFeatureReadyState::NotSupportedOnCurrentSystem` (RuntimeCompatibilityChange: ModelInitialization_KnownExceptions).
- Fixed a critical issue where WebView2 crashes in nested or windowless Xaml Islands, blocking React Native for Windows scenarios. (RuntimeCompatibilityChange: XamlRoot_FixGetHostWindowInNestedIslands).

Version 1.8.4 (1.8.260101001)

Released: **January 13, 2026**

▼ Windows AI Text Rewriter: New Custom tones

The **TextRewriter** supports new custom tones based on user-provided instructions. The new **RewriteCustomAsync** API lets you provide an input string that guides Phi Silica in rewriting selected text. You can experiment with new custom tones based on user-provided instructions to transform your content as desired. Try out changes like "Rewrite as Shakespeare" or "Rewrite in Sci fi".

▼ Windows ML size reduction

> Reduced the size of `Microsoft.Windows.AI.MachineLearning.dll` by approximately 160 KB.

▼ Bug fixes

> - Fixed "Class not registered" errors when using Windows ML in self-contained deployments. Developers using self-contained deployment no longer need to register all the Foundation package activatable classes that were used internally.
> - Fixed a crash that occurs during process shutdown after using Windows ML.
> - Improved `FileOpenPicker`/`FileSavePicker` behavior:
>   - Filter names display correctly when extensions are hidden. For more info, see GitHub issue #5837. (RuntimeCompatibilityChange: StoragePickers_DisplayFileTypeFilterNames)
>   - Existing files are not truncated on save unless overwritten. For more info, see GitHub issue #5976. (RuntimeCompatibilityChange: StoragePickers_DoNotTruncateExistingFileOnSave)
>   - File type choices preserve insertion order. For more info, see GitHub issue #5827. (RuntimeCompatibilityChange: StoragePickers_PreserveFileTypeChoicesInsertionOrder)
> - Fixed an issue that prevented Image Super Resolution from being available for some applications.
> - Fixed problem with apps not launching when using `PublishSingleFile` support with component packages. For more info, see GitHub issue #5969. RuntimeCompatibilityChange: N/A, build .targets change)
> - Fixed an issue where IconElements created using IconSource.CreateIconElement were not rendered on the screen. (RuntimeCompatibilityChange: FixCreateIconElementRendering)
> - Fixed issue with incremental builds rebuilding too much when using WinAppSDKSelfContained. RuntimeCompatibilityChange: N/A, build .targets change)
> - Fixed build failure when referencing DWrite component package with WinAppSDKSelfContained. RuntimeCompatibilityChange: N/A, build .targets change)

▼ New or updated APIs

> This release includes the following new APIs compared to the 1.8.3 release:
>
> ```
> 
> ```

```
Microsoft.Windows.AI.Text

    TextRewriter
        RewriteCustomAsync
```

Version 1.8.3 (1.8.251106002)

Released: **December 8, 2025**

▼ Windows ML Updates

> **Expanded OS Support** Windows ML now supports Windows 10 (version 1809 and later) and Windows Server 2019 and later for CPU and GPU workloads with select execution providers. Learn more: Supported Execution Providers

> **AMD MiGraphX Execution Provider** Added support for the AMD MiGraphX execution provider, enabling ML workloads on AMD GPUs for the latest Ryzen AI 300-series processors. This provider is currently being flighted in Windows Insider Program channels and is targeted for retail availability by the end of the month. Learn more: Supported Execution Providers

> **Windows ML Model Catalog APIs** These APIs enable your app or library to dynamically discover and download large AI model files from your own online model catalogs, and share them across apps on the PC—without bundling those large files directly with your app or library. See the docs to learn how to use these APIs!

▼ New APIs for 1.8.3

> This release includes the following new APIs compared to the previous 1.8 release:

```
Microsoft.Windows.AI.MachineLearning

    CatalogModelInfo
    CatalogModelInstance
    CatalogModelInstanceResult
    CatalogModelInstanceStatus
    CatalogModelStatus
    ModelCatalog
    ModelCatalogSource
```

▼ Bug fixes

> - Fixed a potential crash if OrientedVirtualizingPanel hits an overflow when computing bounds. (RuntimeCompatibilityChange: OrientedVirtualizingPanel_FixBoundsOverflow).

Version 1.8.2 (1.8.251003001)

Released: **October 14, 2025**

▼ Updated ONNX Runtime

> Updated the `onnxruntime.dll` to 1.23.1 introducing several enhancements to ONNX Runtime's Python and C++ APIs, focusing on improved device and memory information handling, synchronization stream support, and tensor copy functionality. It adds new Python bindings for device/memory types, exposes more detailed session input/output metadata, and provides a Python-accessible tensor copy API. The changes also refactor and extend the C++ API for better stream and memory info management.
>
> **Key enhancements include:**
>
> - Python bindings for `OrtMemoryInfoDeviceType`, `OrtDeviceMemoryType`, and expanded `OrtDevice` to expose the memory type via a new `mem_type` method. The `OrtMemoryInfo` Python class now supports both legacy and new V2 constructors and exposes additional properties such as device memory type and Vendor ID.
> - Extended the Python `InferenceSession` object to provide access to input/output `OrtMemoryInfo` and `OrtEpDevice` objects through new properties and methods
> - Introduced Python bindings for `OrtSyncStream`, including creation via `OrtEpDevice.create_sync_stream()` and retrieval of device-specific `OrtMemoryInfo` via `OrtEpDevice.memory_info()`.
> - Refactored the C++ API to generalize `SyncStream` handling, allowing for unowned streams and improved type safety.
> - Added a new Python-level `copy_tensors` function and corresponding C++ binding, enabling efficient copying of tensor data between OrtValue objects, optionally using a synchronization stream.
> - Changed the return type of the `OrtValue.data_ptr` method in the Python binding from `int64_t` to `uintptr_t` for better cross-platform compatibility.

- Minor improvements to error messages and device type handling in the Python API (e.g., for OrtDevice).
- Addressed edge cases in memory information handling
- Resolved minor issues to improve stability and reliability

▼ Bug fixes

- Fixed deployment handler code to report the actual failure HRESULT for increased clarity when troubleshooting.

Version 1.8.1 (1.8.250916003)

Released: **September 23, 2025**

▼ LanguageModel text generation

LanguageModel is now available using **Phi Silica** to generate text responses to broad user prompts with built in content moderation. Phi Silica, Microsoft's most powerful NPU-tuned local language model, is optimized for efficiency and performance on Windows Copilot+ PCs devices while still offering many of the capabilities found in Large Language Models (LLMs).

See Get started with Phi Silica in the Windows App SDK and API ref for Phi Silica in the Windows App SDK for more information.

▼ Microsoft Windows ML

Windows ML enables developers to run ONNX AI models locally on Windows PCs on a shared system-wide copy of the ONNX Runtime using dynamically-installed hardware-specific execution providers.

Key benefits:

- **Dynamically get latest EPs** - Automatically downloads and manages the latest hardware-specific execution providers
- **Shared ONNX Runtime** - Uses system-wide runtime instead of bundling your own, reducing app size
- **Smaller downloads/installs** - No need to carry large EPs and the ONNX Runtime in your app

- **Broad hardware support** - Runs on all Windows 11 PCs (x64 and ARM64) with any hardware configuration

▼ New APIs for 1.8.1

This release includes the following new APIs compared to the previous 1.8 release:

```
Microsoft.Windows.AI.MachineLearning

    ExecutionProvider
    ExecutionProviderCatalog
    ExecutionProviderCertification
    ExecutionProviderReadyResult
    ExecutionProviderReadyResultState
    ExecutionProviderReadyState
    MachineLearningContract
```

```
Microsoft.Windows.AI.Text

    LanguageModel
        CreateContext
        GenerateEmbeddingVectors
        GenerateResponseAsync
        GenerateResponseFromEmbeddingsAsync
        GetUsablePromptLength
        GetVectorSpaceId
```

▼ Known issues

- C# developers must manually reference the System.Numerics.Tensors version 9.0.0 or greater NuGet package in order to use the `Microsoft.ML.OnnxRuntime.Tensors`. Without this NuGet package reference, you will experience the following runtime error when calling the `Microsoft.ML.OnnxRuntime.Tensors` APIs: `Could not load file or assembly 'System.Numerics.Tensors, Version=9.0.0.0`.

Version 1.8.0 (1.8.250907003)

Released: **September 9, 2025**

▼ Windows AI APIs

> The Windows App SDK now includes a suite of artificial intelligence (AI) APIs that can be used with a local language model to perform a variety of tasks on Copilot+ PCs. Your apps can now intelligently respond to prompts, recognize text within images, describe the content of images, remove objects from images, and more.
>
> For information on responsible development practices utilized during the creation of the Windows AI APIs, which can also be applied when creating AI-assisted features, consult the Developing Responsible Generative AI Applications and Features on Windows guidance.

▼ Windows AI Prompt Size Limit Reporting

> Allows applications to determine if an input exceeds the allowable size for a Text Summarizer call. If the input is too large, the API returns an index indicating the current limit, enabling developers to adjust the input accordingly. This limit is based on token count rather than byte or character length, and it can vary over time due to multiple factors. Therefore, applications should treat the limit as dynamic and subject to change.

▼ Windows AI Text Rewriter Tone

> Enables text rewriting with specific tones. The Casual option rephrases content to sound more informal and conversational, using natural, spontaneous phrasing while preserving meaning and format. The Formal option transforms text into a polished, professional version, maintaining the original structure and details with precise language suitable for formal context. The General option retains the original tone and intent, ensuring the meaning remains unchanged.

▼ Text Intelligence - Conversation Summary

> Phi Silica now has a Summarize Conversation feature that allows you to summarize what people have said over an email, chat, or thread. See Phi Silica for more details.

▼ Conversation Summary Options

> Enables developers to specify the desired output language for conversation summarization. This allows applications to generate summaries in a targeted language, enhancing localization, and user experience.

▼ Windows AI Object Erase

> Object Erase can be used to remove objects from images. The model takes both an image and a greyscale mask indicating the object to be removed, erases the masked area from the image, and replaces the erased area with the image background.

▼ Decimal DataType

> The new `Decimal` support offers a high-precision base-10 numeric data type that is invaluable for financial and scientific calculations, avoiding imprecision and rounding errors inherent to floating-point data types. It is structured as a 96-bit (12-byte) unsigned integer, scaled by a variable power of 10, allowing for precise representation of decimal values. This enables decimal support for programming languages lacking decimal data types and provides interoperability with languages that do support decimal (e.g. C#, Python).

▼ NuGet Metapackage

> The Windows App SDK NuGet package has been converted to a NuGet metapackage. Each component contributing to the Windows App SDK is now a component NuGet package and is listed as a dependency by the metapackage. This allows developers to choose either the metapackage or select specific component packages for their applications. The use of individual component packages enables developers to include only the APIs and functionalities that are necessary for their apps. The default experience behaves as if `WindowsAppSDKSelfContained` had been set as True, but the `Microsoft.WindowsAppSDK.Runtime` package can be referenced to use framework package deployment.

▼ Microsoft.Windows.SDK.BuildTools.MSIX Refactor

> The MSIX publishing support has been factored into a standalone nuget package, which can be independently maintained and consumed by Windows App SDK and other projects. In addition, several feature gaps with Single-Project solutions have been addressed including generation of MSIX bundles and MSIX upload packages.

▼ Storage Pickers

> The Microsoft.Windows.Storage.Pickers API in the Windows App SDK provides a modernized file and folder picker experience for desktop applications. This API is based on the existing Windows.Storage.Pickers API design, but with key improvements for desktop scenarios. The

new Microsoft.Windows.Storage.Pickers API addresses two critical limitations of the UWP file and folder pickers on Apps developed with Windows App SDK/WinUI:

- Elevated Process Support: The existing Windows.Storage.Pickers APIs do not work when the application is running as an administrator. The new API enables file and folder selection in elevated mode.
- Simplified Usage in WinUI 3: Using the existing UWP pickers in WinUI 3 requires initializing a window handle for window association. The new pickers eliminate this requirement by accepting a WindowId directly in the constructor, making them easier to use.

▼ Other notable changes

- Prior to Windows App SDK 1.8, packaged apps running in the AppContainer did not require the packageManagement capability, due to a DeploymentManager auto-initialization issue. That issue has now been resolved, and in turn, the packageManagement capability is now required for AppContainer-based apps.
- The experimental WinML APIs have been removed from this release and will be included in a future release.

▼ Bug fixes

- Fixed an issue where the hover effects of other windows for the app could flicker when at least one window had ExtendsContentIntoTitleBar set to true.
- NavigationView: Fixed a bug where setting SelectedItem as null did not correctly clear the selection state in collapsed mode.
- TabView: Fixed an issue where closing a tab would move keyboard focus to the "Add tab" button instead of the newly selected tab.
- SplitButton: Fixed UI inconsistency where the SplitButton control appeared shorter than standard Button controls
- TabView: Fixed issue TabView spacing in WinUI, When setting the TabWidthMode property of a TabView to SizeToContent, the padding between the header text and the left/right edges of the tab becomes uneven

▼ New APIs for 1.8.0

```
Microsoft.Windows.AI.Foundation
```

```
    AIFoundationContract
    EmbeddingVector
```

```

```

```
Microsoft.Windows.AI.Imaging

    ImageObjectRemover
    ImageObjectRemoverContract
```

```

```

```
Microsoft.Windows.AI.Text

    ConversationItem
    ConversationSummaryOptions
    InputKind
    LanguageModelEmbeddingVectorResult
    TextRewriter
        RewriteAsync

    TextRewriteTone
    TextSummarizer
        IsPromptLargerThanContext
        SummarizeConversationAsync
```

```

```

```
Microsoft.Windows.Foundation

    DecimalContract
    DecimalHelper
    DecimalValue
```

```

```

```
Microsoft.Windows.Storage.Pickers

    FileOpenPicker
    FileSavePicker
    FolderPicker
    PickerLocationId
    PickerViewMode
    PickFileResult
    PickFolderResult
    StoragePickersContract
```

```

```

```
Microsoft.Windows.Widgets.Feeds.Providers

    FeedManager
        TryRemoveAnnouncementById

    IFeedManager3
```

▼ New APIs compared to 1.8-Preview1

```

Microsoft.Windows.AI.Text

    TextRewriteTone
        Concise
```

```

Microsoft.Windows.Foundation

    DecimalContract
    DecimalHelper
    DecimalValue
```

Last updated on 09/24/2026

Windows App SDK 1.7 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.7.6 (1.7.251107005)

> Released on **November 11, 2025**.

▼ Bug fixes

> - Fixed a potential crash if OrientedVirtualizingPanel hits an overflow when computing bounds. (RuntimeCompatibilityChange: OrientedVirtualizingPanel_FixBoundsOverflow).
> - Fixed a potential crash on app launch if the computer has not been rebooted in a long time. (RuntimeCompatibilityChange: ContentIsland_StartupCrashWhenLongRunning).

▼ Windows AI Object Erase

> Object Erase can be used to remove objects from images. The model takes both an image and a greyscale mask indicating the object to be removed, erases the masked area from the image, and replaces the erased area with the image background.

▼ New APIs for 1.7.6

> This release includes the following new APIs compared to the 1.7.5 release:

```
Microsoft.Windows.AI.Imaging

    ImageObjectRemover
    ImageObjectRemoverContract
```

Version 1.7.5 (1.7.251014001)

▼ Bug fixes

- Fixed issue where mouse wheel scrolling doesn't work when the Windows setting "Scroll inactive windows when hovering over them" is off. For more info, see GitHub issue #10091. (RuntimeCompatibilityChange: InputPointerSource_FixScrollForInactiveWindowSetting)
- Fixed an issue that caused a false positive warning about a missing mspdbcmf.exe. For more info, see GitHub issue #5262. (RuntimeCompatibilityChange: N/A, build .targets change)

Version 1.7.4 (1.7.250909003)

▼ Bug fixes

- Fixed an issue where the hover effects of other windows for the app could flicker when at least one window had ExtendsContentIntoTitleBar set to true. (RuntimeCompatibilityChange: XamlIsland_ExitNonClientArea). For more information, see GitHub issue #10572.
- Fixed an issue where UnhandledExceptions were not being propagated to managed code. (RuntimeCompatibilityChange: DispatcherQueue_PropagateException). For more information, see GitHub issue #10447.
- Fixed an occasional crash in `DirectUI::PointerRoutedEventArgs::GetCurrentPointImpl`. (RuntimeCompatibilityChange: GetIntermediatePointsImpl_Crash)
- Apps with an incorrect activation manifest no longer crash in certain situations but instead return an error. (RuntimeCompatibilityChange: DesktopSiteBridge_ActivationErrorCrash)
- Fixed an issue where setting `NavigationView.SelectedItem` as null did not correctly clear the selection state in collapsed mode. (RuntimeCompatibilityChange: NavigationView_ClearSelectedItem)
- Fixed certain corrupted upgrade scenarios from 1.6 to 1.7 by ensuring version numbers on certain binaries are consistently increasing. (RuntimeCompatibilityChange: N/A, file version change)
- Fixed an issue where resizing the window would fire an AppWindow.Changed event with `AppWindowChangedEventArgs.DidPresenterChange` set to true. (RuntimeCompatibilityChange: AppWindow_NoDidPresenterChangeOnRestore)
- Fixed a potential crash on window close if an InputPointerSource is disposed during certain reentrant scenarios. (RuntimeCompatibilityChange: InputPointerSource_ReentrantDisposedCrash)
- Fixed an issue where the installer failed to update the Singleton package if an older version was running in the background. The fix ensures AddPackage is always called with ForceTargetAppShutdown for the Singleton and restarts the

NotificationLongRunningPlatform to complete the update.
(RuntimeCompatibilityChange: N/A, installer change)

Version 1.7.3 (1.7.250606001)

▼ Windows AI APIs

> **ⓘ Important**
>
> The underlying ML models required for these APIs currently require your device to be running the latest Windows 11 Insider Preview Build on the Dev Channel. Additionally, these APIs require your device to be a Copilot+ PC. See **Copilot+ PCs Developer Guide** to learn more about these devices. APIs will throw an exception when called on devices lacking the necessary support.
>
> The Windows App SDK now includes a suite of artificial intelligence (AI) APIs that can be used with a local language model to perform a variety of tasks on Copilot+ PCs. Your apps can now intelligently respond to prompts, recognize text within images, describe the content of images, remove objects from images, and more.
>
> For information on responsible development practices utilized during the creation of the Windows AI APIs, which can also be applied when creating AI-assisted features, consult the **Developing Responsible Generative AI Applications and Features on Windows** guidance.

▼ New Rank property for Widgets

Added a new `Rank` property to Widgets. Rank may be used by the platform's recommendation engine to sort Widgets from a same application package identity. Should multiple widgets from the same provider be recommended for a UI surface, the Rank property will determine the order in which they appear. The Rank property does not change how a Widget is placed compared to other provider's Widgets, nor does it affect the chance a Widget will be recommended.

▼ Bug fixes

- Added the following sentence to section 1a of the .nupkg license: When building Generative AI applications follow the guidelines in Developing Responsible Generative AI Applications and Features on Windows.
- Fixed a potential crash in ApplicationDataProvider::GetStateFolderUris caused by reentrancy. For more info, see GitHub issue #10513. (RuntimeCompatibilityChange:

ApplicationDataProvider_ReentrancyProtection)

- Fixed a potential crash in WindowChrome::SetTitleBar when closing a window. For more info, see GitHub issue #9203. (RuntimeCompatibilityChange: WindowChrome_SetTitleBarCrash)
- Fixed a potential crash in PointerInputObserverWinRT::FlushCoalescedInput_Callback when there is reentrancy while processing input. (RuntimeCompatibilityChange: InputPointerSource_FlushReentrancyCrash)

▼ New APIs for 1.7.3

This release includes the following new APIs compared to the previous 1.7 release:

```
Microsoft.Windows.Widgets.Providers

    WidgetInfo
        Rank

    WidgetUpdateRequestOptions
        Rank
```

Version 1.7.2 (1.7.250513003)

▼ Windows AI APIs

> **ⓘ Important**
>
> The underlying ML models required for these APIs currently require your device to be running the latest Windows 11 Insider Preview Build on the Dev Channel. Additionally, these APIs require your device to be a Copilot+ PC. See Copilot+ PCs Developer Guide to learn more about these devices. APIs will throw an exception when called on devices lacking the necessary support.
>
> The Windows App SDK now includes a suite of artificial intelligence (AI) APIs that can be used with a local language model to perform a variety of tasks on Copilot+ PCs. Your apps can now intelligently respond to prompts, recognize text within images, describe the content of images, remove objects from images, and more.
>
> For information on responsible development practices utilized during the creation of the Windows AI APIs, which can also be applied when creating AI-assisted features, consult the Developing Responsible Generative AI Applications and Features on Windows guidance.

Phi Silica Text Intelligence

With Phi Silica, Microsoft's most powerful NPU-tuned local language model, you can specify it to perform common tasks like summarizing a piece of text, rewriting a piece of text for clarity, and converting text to a table format. Phi Silica is optimized for efficiency and performance on Windows Copilot+ PCs devices while still offering many of the capabilities found in Large Language Models (LLMs).

See Get started with Phi Silica in the Windows App SDK and API ref for Phi Silica in the Windows App SDK for more information.

Image Description

The Image Description APIs enable the generation of textual descriptions of images. The length and type of these descriptions can be configured to meet accessibility requirements, ranging from short captions to long descriptions.

For additional details, see What can I do with Image Description? and API ref for AI imaging features in the Windows App SDK.

Text Recognition

Text recognition, also known as optical character recognition (OCR), detects and extracts text within images, converting it into machine-readable character streams. These APIs identify characters, words, lines, polygonal text boundaries, and provide confidence levels for each match. Benefiting from NPU-assisted acceleration, the Windows AI AI-assisted APIs perform faster and more accurately than the legacy Windows.Media.Ocr.OcrEngine APIs.

For additional details, see Get Started with Text Recognition (OCR) in the Windows App SDK and API ref for AI-backed Text Recognition (OCR) in the Windows App SDK.

Image Super Resolution

The 'ImageScaler' APIs can increase the sharpness and clarity of an image and upscale the image by up to 8x its original resolution.

For additional details, see What can I do with Image Super Resolution? and API ref for AI imaging features in the Windows App SDK.

Image Segmentation

The Image Segmentation APIs allow for the identification of specific objects within an image. By inputting an image and a "hints" object, the model returns a mask of the identified object.

For additional details, see **What can I do with Image Segmentation?** and **API ref for AI imaging features in the Windows App SDK**.

▼ ApplicationData.MachinePath folder creation support

> ApplicationData.MachineFolder is now easier to use on Windows >=10.0.26100.0 (Ge). Windows will create the Machine folder when a package manifesting opt-in support is added to a system if Windows App SDK 1.7.2 is present on the system. For more details see the ApplicationData spec.

▼ Bugfixes

> - Fixed PackageDeploymentManager telemetry to properly capture completion status. For more info, see GitHub issue #5296. (RuntimeCompatibilityChange: N/A)
> - Fixed a crash when using pen input on an x86 app. (RuntimeCompatibilityChange: InputStateManager_PenInputCrashX86)
> - Fixed a potential crash if the window is already destroyed when WinUI is attempting to initialize for scrolling. (RuntimeCompatibilityChange: ActivateDirectManipulationManager_CheckCanInit)
> - Fixed the WINDOWSAPPSDK_RELEASE_PATCH define and Microsoft::WindowsAppSDK::Release::Patch values in WindowsAppSDK-VersionInfo.h to not always be 0. The define is now the yymmdd date of the build, and the Patch value is the mmdd date. This change provides better runtime information on the version being used without changing any variable sizes or the version scheme. (RuntimeCompatibilityChange: N/A, header change)
> - Fixed a potential issue in the Bootstrapper if it is used to load a 1.6 or earlier version of Windows App SDK. For more info, see GitHub issue #5349. (RuntimeCompatibilityChange: N/A)
> - Fixed an issue where using MSBuild to build a single-project app could incorrectly fail with a build error if it didn't have a correct launchSettings.json. (RuntimeCompatibilityChange: N/A, build .targets change)
> - Improved the performance of rendering the first frame on application launch. (RuntimeCompatibilityChange: DwmCoreI_OptimizeFirstFrameLatency)

▼ New APIs for 1.7.2

> This release includes the following new APIs compared to the previous 1.7 release:

```
Microsoft.Graphics.Imaging

    ImageBuffer
    ImageBufferContract
    ImageBufferPixelFormat
```

```
Microsoft.Windows.AI

    AIFeatureReadyContract
    AIFeatureReadyResult
    AIFeatureReadyResultState
    AIFeatureReadyState
```

```
Microsoft.Windows.AI.ContentSafety

    ContentFilterOptions
    ContentSafetyContract
    ImageContentFilterSeverity
    SeverityLevel
    TextContentFilterSeverity
```

```
Microsoft.Windows.AI.Imaging

    ImageDescriptionContract
    ImageDescriptionGenerator
    ImageDescriptionKind
    ImageDescriptionResult
    ImageDescriptionResultStatus
    ImageObjectExtractor
    ImageObjectExtractorContract
    ImageObjectExtractorHint
    ImageScaler
    ImageScalerContract
    RecognizedLine
    RecognizedLineStyle
    RecognizedText
    RecognizedTextBoundingBox
    RecognizedWord
    TextRecognitionContract
    TextRecognizer
```

```
Microsoft.Windows.AI.Text

    LanguageModel
    LanguageModelContext
    LanguageModelContract
    LanguageModelOptions
    LanguageModelResponseResult
    LanguageModelResponseStatus
    TextIntelligenceContract
    TextRewriter
    TextSummarizer
    TextToTableConverter
    TextToTableResponseResult
    TextToTableRow
```

```
Microsoft.Windows.Workloads

    WorkloadPriority
    WorkloadsContract
```

Version 1.7.1 (1.7.250401001)

▼ Bugfixes

- Improved the telemetry for failure scenarios in WindowsAppRuntimeInstall-<arch>.exe. For more info, see GitHub issue #5289. (RuntimeCompatibilityChange: N/A, installer change).
- Fixed an issue where pointer input would stop working when using arrow keys at the same time. For more info, see GitHub issue #10126. (RuntimeCompatibilityChange: FixStuckPointerInputQueue).
- Fixed an issue where apps in remote desktop stop responding to pointer input. For more info, see GitHub issue #10009. (This is the same fix as the pointer input plus arrow keys fix, due to remote desktop automatically sending some key input during the switch away and back.) (RuntimeCompatibilityChange: FixStuckPointerInputQueue).
- Fixed a potential crash trying to restore focus if a window activation event is delivered for a window which is closing. (RuntimeCompatibilityChange: FixWindowCloseFocusCrash)
- Fixed a performance regression introduced in Windows App SDK 1.6 due to WinUI binaries missing some linker optimizations. (RuntimeCompatibilityChange: N/A, changed linker options)

- Fixed a potential crash if ProgressBar::SetProgressBarIndicatorWidth is called on a ProgressBar which is not in the tree. (RuntimeCompatibilityChange: FixSetProgressBarIndicatorWidthCrash)
- Fixed a potential crash caused by CPopup::EnsureBridgeClosed sometimes triggering reentrancy. (RuntimeCompatibilityChange: FixPopupClosingReentrancyCrash)
- Fixed a potential crash when closing a popup due to CUIElement::FlushPendingKeepVisibleOperations using a null children collection. (RuntimeCompatibilityChange: FixPopupUnloadingCrash)
- Fixed PackageDeploymentManager.EnsurePackage*Ready to ensure version supersedence. For more info, see GitHub issue #5191. (RuntimeCompatibilityChange: EnsurePackageReadyVersionSupercedence)
- Fixed a potential crash caused by WebView2::UpdateCoreWebViewVisibility sometimes triggering reentrancy. For more info, see GitHub issue #10305. (RuntimeCompatibilityChange: FixWebViewVisibilityReentrancyCrash)
- Fixed an issue where app UI sometimes permanently freezes and can stop rendering due to the DispatcherQueue getting stuck. (RuntimeCompatibilityChange: FixRandomUIFreezeInDispatcher)

Version 1.7.0 (1.7.250310001)

▼ New Badge Notifications

> The notification badge conveys a summary or status information specific to an app. This can be numeric (1-99) or a glyph from one of the system-provided glyphs. This new functionality provides an easy way for apps to show status, such as number of unread mails in a mail app or number of new posts in a social media app.
>
> For more info, see GitHub #4926.

▼ New CameraCaptureUI API

> Developers have encountered challenges in the desktop environments due to WinRT CameraCaptureUI being dependent on CoreWindows, and lack of InitializeWithWindow support. The team has released this new `Microsoft.Windows.Media.Capture.CameraCaptureUI` API to Windows App SDK to provide a streamlined solution with feature parity, now supporting WindowID in the constructor for enhanced desktop compatibility.
>
> For more info, see GitHub issue #4721.

▼ New Authentication API

A new `OAuth2Manager` API provides a streamlined solution for web authentication, offering OAuth 2.0 capabilities with full feature parity across all Windows platforms supported by Windows App SDK. This new Authentication Manager is different from the public WebAuthentication Broker API, as it better aligns with OAuth best practices.

For more info, see GitHub issue #4772.

▼ New Background Task support

Background tasks are app components that run in the background without a user interface, performing actions like download files, syncing data, sending notifications or updating files. The new `BackgroundTaskBuilder` API provides Windows App SDK dependent apps the ability to directly register the full trust COM components with background tasks, removing the need to implement a workaround.

For more info, see GitHub #4831.

▼ New TitleBar control

A new `TitleBar` control makes it much easier to create a great, customizable titlebar for your app. Configure properties such as the titlebar icon, Title, and Subtitle, include an integrated back button, or even add a custom control like a search box! The control includes robust titlebar capabilities like empty-space draggable regions, theme responsiveness, caption buttons, and built-in accessibility support so you can focus on your personalized design and still get the same reliable titlebar as the default experience.

For more info, see GitHub #10056.

▼ Support for MathML

`RichEditBox` now supports MathML, via `RichEditTextDocument.SetMathMode` and `RichEditTextDocument.SetMathML`.

For more info, see GitHub #4196.

▼ Enhanced runtime

- Windows App SDK's Dynamic Dependencies APIs delegate all calls to Windows 11's implementation when running on >= Windows 11 24H2 (10.0.26100.0) providing improved performance and robustness. This holds true for all C/C++ (Mdd*()) and WinRT (namespace Microsoft.Windows.ApplicationModel.DynamicDependency) APIs.
  - Packaged processes calling Windows App SDK's Dynamic Dependencies APIs is now supported on >= Windows 11 24H2 (10.0.26100.0). This is still unsupported

on older systems (the Windows App SDK implementation doesn't support packaged apps).
    - This has no impact to the developer experience. Callers can continue using the Bootstrapper API to add the Windows App SDK framework package to the calling process' package graph.
    - For more info, see GitHub PR #4949.
- Undocked Registration-free WinRT (URFW) is not enabled on >= Windows 11 24H2 (10.0.26100.0). The OS' implementation handles all Registration-free WinRT activity on these systems providing improved performance and robustness. For more info, see GitHub PR #4949.
- Detours is not used on >= Windows 11 24H2 (10.0.26100.0). Detours was only used by Windows App SDK's implementations of Dynamic Dependencies and Registration-free WinRT, but as those features are now handled by the OS' implementations there's no need for them to initialize or otherwise wire up Detours. This provides a small performance gain when loading Microsoft.WindowsAppRuntime.dll. For more info, see GitHub PR #4949.

▼ New AppWindow APIs

New `AppWindow` APIs make it easier to control your app windows to create a great experience. New capabilities include using `SetTaskBarIcon` and `SetTitleBarIcon` to independently set the taskbar and titlebar icons, using `AppWindowTitleBar.PreferredTheme` to set the light/dark theme of the titlebar, and using new properties like `OverlappedPresenter.PreferredMinimumWidth` and `OverlappedPresenter.PreferredMaximumHeight` to set a minimum or maximum width or height for the window.

▼ New Island APIs

The updates in the Microsoft.UI.Content namespace introduce several significant enhancements and new features aimed at improving the functionality and interoperability of the ContentIsland APIs. These changes are designed to support new hosting scenarios, enhance rendering capabilities, and ensure better synchronization of input and accessibility states. Key updates include:

1. New primitives for hosting ContentIslands:
    - `DesktopPopupSiteBridge`: Enables hosting a `ContentIsland` in the environment of a Win32 window with WS_POPUP style, facilitating scenarios where applications use popup windows for dialog boxes and message boxes.
    - `ChildSiteLink`: Allows a parent `ContentIsland` to host a nested child `ContentIsland`, providing a seamless partitioning of the rendering surface

without user experience seams.
    - `DesktopAttachedSiteBridge`: Attaches to an existing Win32 window instead of creating a new one, designed to host a `ContentIsland` with Windows.UI.Composition.Visuals at the root of the Win32 window hierarchy, ensuring full control over Win32-based input processing and accessibility.
2. Enhanced rendering and input synchronization:
    - The `LocalToParentTransformMatrix` and `ActualSize` properties of a `ChildSiteLink` are updated relative to the parent `ContentIsland` before rendering, avoiding latency and ensuring synchronized input and accessibility states.
3. ContentIslands with Windows.UI.Composition.Visuals:
    - `ContentIsland` can use Windows.UI.Composition.Visuals for rendering and Win32 window APIs for input processing, enabling interoperability with applications that use legacy UX frameworks. This allows for a gradual adoption of newer UX frameworks layered on top of the Windows App SDK Scene Graph, such as WinUI and React Native for Windows on Fabric. These updates collectively enhance the flexibility, performance, and interoperability of the ContentIsland APIs, enabling developers to create more sophisticated and responsive applications.

Additionally, the updates in the Microsoft.UI.Xaml namespace introduce a new `XamlIsland` API, which allows for the hosting of Xaml content within a SiteBridge or a `ChildSiteLink`. The `XamlIsland` offers greater flexibility compared to the `DesktopWindowXamlSource` API. While `DesktopWindowXamlSource` requires hosting within an existing Win32 window, the `XamlIsland` exposes a `ContentIsland`, enabling more options for hosting Xaml content.

▼ Other notable changes

- New `RuntimeCompatibilityOptions` support will allow more control over how servicing changes affect apps. For more info, see GitHub #4966.
- A new `ReleaseInfo` API provides easy access to the version of the Windows App SDK Runtime in use. For more info, see GitHub #2893.
- Note: Windows AI APIs are not included this release. To experiment with these APIs, please continue to use the 1.7-experimental3 release and share your feedback!

▼ New APIs for 1.7.0

This release includes the following new APIs compared to the stable 1.6 release:

```
Microsoft.Security.Authentication.OAuth

    AuthFailure
    AuthRequestParams
    AuthRequestResult
    AuthResponse
    ClientAuthentication
    CodeChallengeMethodKind
    OAuth2Manager
    TokenFailure
    TokenFailureKind
    TokenRequestParams
    TokenRequestResult
    TokenResponse
```

```
Microsoft.UI.Content

    ChildSiteLink
    ContentAutomationOptions
    ContentEnvironmentStateChangedEventArgs
        DidDisplayScaleChange

   ContentIsland
        AutomationOption
        Children
        CreateForSystemVisual
        FindAllForSystemCompositor
       FragmentRootAutomationProvider
        GetBySystemVisual
        LocalToClientTransformMatrix
        LocalToParentTransformMatrix
        NextSiblingAutomationProvider
        ParentAutomationProvider
        Popups
        PreviousSiblingAutomationProvider
        ProcessesKeyboardInput
        ProcessesPointerInput

    ContentIslandEnvironment
        DisplayScale

    ContentIslandStateChangedEventArgs
        DidLocalToClientTransformMatrixChange
        DidLocalToParentTransformMatrixChange

    ContentSite
        LocalToClientTransformMatrix
        LocalToParentTransformMatrix
        ProcessesKeyboardInput
        ProcessesPointerInput
```

```
    ContentSiteAutomationProviderRequestedEventArgs
    ContentSiteEnvironment
        DisplayScale

    ContentSiteEnvironmentView
        DisplayScale

    ContentSiteView
        AutomationOption
        LocalToClientTransformMatrix
        LocalToParentTransformMatrix
        ProcessesKeyboardInput
        ProcessesPointerInput

    DesktopAttachedSiteBridge
    DesktopChildSiteBridge
        CreateWithDispatcherQueue

    DesktopPopupSiteBridge
    IContentSiteAutomation
    IContentSiteInput
    IContentSiteLink
```

```
Microsoft.UI.Input

    InputFocusNavigationHost
        GetForSiteLink
```

```
Microsoft.UI.Text

    RichEditTextDocument
        GetMathML
        GetMathMode
        SetMathML
        SetMathMode
```

```
Microsoft.UI.Windowing

    AppWindow
        SetTaskbarIcon
        SetTitleBarIcon

    AppWindowTitleBar
        PreferredTheme

    OverlappedPresenter
```

```
        PreferredMaximumHeight
        PreferredMaximumWidth
        PreferredMinimumHeight
        PreferredMinimumWidth

    TitleBarTheme
```

```
Microsoft.UI.Xaml

    XamlIsland
    XamlRoot
        ContentIsland
```

```
Microsoft.UI.Xaml.Controls

    TitleBar
    TitleBarAutomationPeer
    TitleBarTemplateSettings
```

```
Microsoft.Windows.ApplicationModel.Background

    BackgroundTaskBuilder
```

```
Microsoft.Windows.ApplicationModel.Background.UniversalBGTask

    Task
```

```
Microsoft.Windows.ApplicationModel.WindowsAppRuntime

    ReleaseInfo
    RuntimeCompatibilityChange
    RuntimeCompatibilityOptions
    RuntimeInfo
    WindowsAppRuntimeVersion
```

```
Microsoft.Windows.BadgeNotifications
```

```
    BadgeNotificationGlyph
    BadgeNotificationManager
```

```
Microsoft.Windows.Media.Capture

    CameraCaptureUI
    CameraCaptureUIMaxPhotoResolution
    CameraCaptureUIMaxVideoResolution
    CameraCaptureUIMode
    CameraCaptureUIPhotoCaptureSettings
    CameraCaptureUIPhotoFormat
    CameraCaptureUIVideoCaptureSettings
    CameraCaptureUIVideoFormat
```

▼ Bug fixes

- Changed `SplitButton` so touch input now matches the behavior of mouse input. For more info, see GitHub issue #178.
- Changed cascading menus so sub menus now open immediately if clicked. For more info, see GitHub issue #939.
- Fixed an issue where opening a `ComboBox` which is in a flyout closes all flyouts. For more info, see GitHub issue #1467.
- Fixed an issue where `SwipeControl` would randomly crash in a `ListView`. For more info, see GitHub issue #2527.
- Fixed an issue where drag-and-drop only a `ListViewItem` would leave it in the wrong visual state. For more info, see GitHub issue #3458.
- Fixed an issue in `StackLayout` so that it respects the ItemsRepeater.HorizontalAlignment and ItemsRepeater.VerticalAlignment properties (when StackLayout.Orientation is Vertical and Horizontal respectively). The old layout behaved as if the ItemsRepeater alignment was Stretch. With the fix, the layout results in items aligned to the right when the Right alignment is used, for example. For more info, see GitHub issue #3842.
- Fixed a potential crash when using a resource which contains an `x:Bind`. For more info, see GitHub issue #5786.
- Fixed an issue where deleting items in the `ItemsRepeater`'s source would not generate items which moved up into view. For more info, see GitHub issue #6661.
- Fixed an issue where the right Alt key would not show keytips for Access Keys. For more info, see GitHub issue #8447. **Note:** This may result in key events for the right Alt key no longer being delivered to handles in the app or controls.
- Fixed an issue where using a ResourceDictionary containing only a single resource would fail to find that resource and likely cause a crash. For more info, see GitHub issue #8832.

- Fixed a crash where `UniformGridLayout` would sometimes pick a wrong layout anchor and cause infinite layout passes when scrolling backwards. For more info, see GitHub issue #9199.
- Fixed an issue where setting `NavigationFailedEventArgs.Handled` to True would still throw an exception. For more info, see GitHub issue #9632.
- Fixed an issue where `TabView` would not apply any specified `CornerRadius`. For more info, see GitHub issue #9846.
- Fixed a potential layout cycle crash in `StackLayout`. For more info, see GitHub issue #9852.
- Fixed a potential crash in `ItemsView` when removing items. For more info, see GitHub issue #9868.
- Fixed an issue in 1.7-preview1 where popups no longer correctly moved with their parent window. For more info, see GitHub issue #10386.
- Based on feedback from 1.7-preview1, renamed some properties on the new `TitleBar` control.

Last updated on 12/20/2025

Windows App SDK 1.6 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.6.9 (1.6.250602001)

▼ Bug Fixes

> - Fixed a potential crash in WindowChrome::SetTitleBar when closing a window. For more info, see GitHub issue #9203.

Version 1.6.8 (1.6.250430001)

▼ ApplicationData.MachinePath folder creation support

> ApplicationData.MachineFolder is now easier to use on Windows >=10.0.26100.0 (Ge). Windows will create the Machine folder when a package manifesting opt-in support is added to a system if Windows App SDK 1.6.8 is present on the system. For more details see the ApplicationData spec.

▼ Bug Fixes

> - Fixed PackageDeploymentManager telemetry to properly capture when completion status. For more info, see GitHub issue #5297.
> - Fixed a crash when using pen input on an x86 app.
> - Fixed a potential crash if the window is already destroyed when WinUI is attempting to initialize for scrolling.
> - Fixed the WINDOWSAPPSDK_RELEASE_PATCH define and Microsoft::WindowsAppSDK::Release::Patch values in WindowsAppSDK-VersionInfo.h to not always be 0. The define is now the yymmdd date of the build, and the Patch value is the mmdd date. This change provides better runtime information on the version being used without changing any variable sizes or the version scheme.

Version 1.6.7 (1.6.250402001)

▼ Bug Fixes

- Improved the telemetry for failure scenarios in `WindowsAppRuntimeInstall-<arch>.exe`. For more info, see GitHub issue #5291.
- Fixed an issue where pointer input would stop working when using arrow keys at the same time. For more info, see GitHub issue #10126.
- Fixed an issue where apps in remote desktop stop responding to pointer input. For more info, see GitHub issue #10009. (This is the same fix as the pointer input plus arrow keys fix, due to remote desktop automatically sending some key input during the switch away and back.)
- Fixed a potential crash trying to restore focus if a window activation event is delivered for a window which is closing.
- Fixed a performance regression introduced in Windows App SDK 1.6 due to WinUI binaries missing some linker optimizations.
- Fixed a small performance issue when creating multiple WinUI windows/islands.
- Fixed a potential crash if `ProgressBar::SetProgressBarIndicatorWidth` is called on a ProgressBar which is not in the tree.
- Fixed a potential crash caused by `CPopup::EnsureBridgeClosed` sometimes triggering reentrancy.
- Fixed a potential crash when closing a popup due to `CUIElement::FlushPendingKeepVisibleOperations` using a null children collection.
- Fixed `PackageDeploymentManager.EnsurePackage*Ready` to ensure version supersedence. For more info, see GitHub issue #5225.

Version 1.6.6 (1.6.250228001)

▼ Bug Fixes

This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.6 release.

- Fixed an issue where a child window posting WM_NCMOUSELEAVE to the parent window would result in a loop that blocks new mouse input events.
- Fixed a crash which would occur on the next AppWindow.Changed event after a WebView2 process failure.
- Fixed a potential crash when using an Accessibility tool and closing a window.
- Fixed an issue where a textbox would not accept key input if given focus by clicking in the area of the clear button of the textbox. For more info, see GitHub issue #7703.
- Fixed an issue where a tooltip is not shown for the Minimize button in the titlebar when using `ExtendsContentIntoTitleBar=true`. For more info, see GitHub issue #9149.

This release includes the following new APIs:

A new `IsPlaceholderContent` property on `WidgetInfo` and `WidgetUpdateRequestOptions` enables a Widget provider to indicate that it would display placeholder content if rendered. For example, a Widget that shows weather information should set IsPlaceholderContent to true if the user has not yet specified a weather location and the Widget is merely showing weather information for a default location like Seattle. When a Widget is marked as placeholder, certain hosts may decide to hide the Widget or prioritize other Widgets.

```
Microsoft.Windows.Widgets.Providers

    WidgetInfo
        IsPlaceholderContent

    WidgetUpdateRequestOptions
        IsPlaceholderContent
```

Version 1.6.5 (1.6.250205002)

▼ Bug Fixes
- Fixed several memory leak issues.

Version 1.6.4 (1.6.250108002)

▼ Bug Fixes
- Fixed an issue with text selection highlighting in a multi-line TextBox. For more info, see GitHub issue [#9965](https://github.com/microsoft/microsoft-ui-xaml/issues/9965). - Fixed an issue where the DDLM package would sometimes not install, preventing launch of unpackaged apps. For more info, see GitHub issue [#3855](https://github.com/microsoft/WindowsAppSDK/issues/3855). - Fixed a potential crash in Detours in some scenarios. For more info, see GitHub issue [#4937](https://github.com/microsoft/WindowsAppSDK/pull/4937). - Fixed another potential issue where a menu off a CommandBar may incorrectly open up instead of down when the CommandBar is at the bottom of the window. - Fixed a potential crash when running on older graphics hardware. - Fixed a potential crash in pointer event handling when closing a window. - Fixed a potential crash caused by `CUIAWindow::InitIds` sometimes triggering reentrancy. - Fixed a potential crash when using `CompositionCapabilities.Changed` event. - Fixed an issue with some Unicode characters displaying as squares in TextBox/RichEditBox. - Fixed `PackageDeploymentManager.EnsurePackage*Async()` handling of `options.RegisterNewerIfAvailable`. For more info, see GitHub issue [#4864](https://github.com/microsoft/WindowsAppSDK/issues/4864).

Version 1.6.3 (1.6.241114003)

▼ Bug Fixes

This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.6 release. - Fixed an issue where reading the `AppWindow.ExtendsContentIntoTitleBar` property turns on custom titlebar rendering. For more info, see GitHub issue [#9988] (https://github.com/microsoft/microsoft-ui-xaml/issues/9988). - Fixed a potential crash during destruction of a `TextBox`/`RichEditBox`. For more info, see GitHub issue [#9070] (https://github.com/microsoft/microsoft-ui-xaml/issues/9070). - Fixed an issue where `PackageDeploymentManager.IsPackageReadyOrNewerAvailable()` failed. For more info, see GitHub issue [#4817](https://github.com/microsoft/WindowsAppSDK/issues/4817). - Fixed an issue where `ScrollViewer` would leak. - Added detection for a rare scenario where the app stops rendering and never recovers. - Fixed an issue where `PackageDeploymentManager.RegisterPackageSetAsync()` requires URI when it should be optional to register by PackageFamilyName. - Fixed an issue that prevented apps from being installed or uninstalled. For more info, see GitHub issue [#4881] (https://github.com/microsoft/WindowsAppSDK/issues/4881).

This release includes the following new APIs which allow for providers of Widgets to incorporate web content in their Widgets:

C#

```
Microsoft.Windows.Widgets.Providers
  IWidgetManager2
  IWidgetProviderMessage
  IWidgetResourceProvider
  WidgetManager
     SendMessageToContent

  WidgetMessageReceivedArgs
  WidgetResourceRequest
  WidgetResourceRequestedArgs
  WidgetResourceResponse
```

Version 1.6.1 (1.6.240923002)

▼ Bugfixes

- Fixed a crash when using FocusVisualKind.Reveal(). For more info, see GitHub issue #9966.
- Fixed noisy C++ exceptions from Bcp47Langs.dll. For more info, see GitHub issue #4691. Note that this fix removes the synchronization with `Windows.Globalization.ApplicationLanguages.PrimaryLanguageOverride`.

- Fixed an issue where an extra `Unloaded` event was raised immediately after showing a `ContentDialog`. For more info, see GitHub issue #8402.
- Fixed an issue where a CommandBar menu might have incorrectly opened up even when there was room for it to open down.
- Fixed some issues where input to `InputNonClientPointerSource` regions was not handled correctly when the top-level window was running in right-to-left mode.
- Fixed the compile-time check for the Windows SDK framework version to handle the slightly different framework name used for .NET 9.

Version 1.6

▼ C++ project changes

When updating a C++ project to 1.6, you'll need to add a project reference to the `Microsoft.Web.WebView2` package. If you update via NuGet Package Manager in Visual Studio, this dependency will be added for you.

▼ C# project changes

In 1.6, Windows App SDK managed apps require Microsoft.Windows.SDK.NET.Ref `*.*.*.38` or later, which can be specified via WindowsSdkPackageVersion in your `csproj` file. For example:

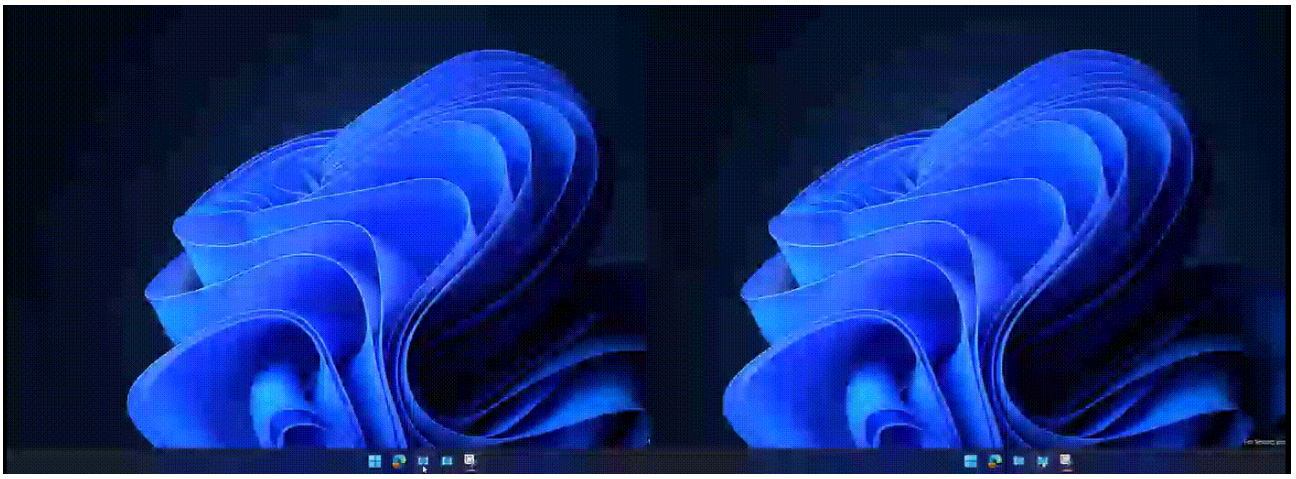
XML

```
<Project Sdk="Microsoft.NET.Sdk">
   <PropertyGroup>
       <OutputType>WinExe</OutputType>
       <TargetFramework>net8.0-windows10.0.22621.0</TargetFramework>
       <TargetPlatformMinVersion>10.0.17763.0</TargetPlatformMinVersion>
       <WindowsSdkPackageVersion>10.0.22621.38</WindowsSdkPackageVersion>
   <PropertyGroup>
   ...
```

In addition, Windows App SDK managed apps should update to Microsoft.Windows.CsWinRT `2.1.1` (or later).

> **Note**
>
> These manual references will no longer be needed once the next .NET SDK servicing update is released.

▼ Native AOT support



The .NET `PublishAot` project property is now supported for native Ahead-Of-Time compilation. For details on Native AOT, see Native AOT Deployment. Because AOT builds on Trimming support, much of the following trimming-related guidance applies to AOT as well.

For `PublishAot` support and trimming support, in addition to the C# project changes described in the previous section you'll also need a package reference to Microsoft.Windows.CsWinRT `2.1.1` (or later) to enable the source generator from that package until the next .NET SDK servicing update is released when it will no longer be required.

For more info, see the CsWinRT Trimming / AOT support doc and the CsWinRT 2.1.1 Release Notes.

Because the Windows App SDK invokes publishing targets when F5 deploying, we recommend enabling `PublishAot` at NuGet restore time by adding this to your `csproj` file:

XML

```
<PublishAot>true</PublishAot>
```

▼ Resolving AOT Issues

In this release, the developer is responsible for ensuring that all types are properly rooted to avoid trimming (such as with reflection-based `{Binding}` targets). Later releases will enhance both C#/WinRT and the XAML Compiler to automate rooting where possible, alert developers to trimming risks, and provide mechanisms to resolve.

Partial Classes

C#/WinRT also includes `PublishAot` support in version 2.1.1. To enable a class for AOT publishing with C#/WinRT, it must first be marked `partial`. This allows the C#/WinRT AOT source analyzer to attribute the classes for static analysis. Only classes (which contain methods, the targets of trimming) require this attribute.

Unsafe Code Error

The CsWinRT source generator might generate code that makes use of `unsafe`. If you hit such an error during compilation or a diagnostic warning for it (CS0227 for "Unsafe code may only appear if compiling with /unsafe"), you should set EnableUnsafeBlocks to true. For more info, see GitHub issue CsWinRT #1721.

WebView2 not yet AOT compatible

The WebView2 projections in `Microsoft.Web.WebView2` package version 1.0.2651.64 are not yet AOT compatible. This will be fixed in an upcoming release of the `Microsoft.Web.WebView2` package, which you can then reference in your project.

Reflection-Free Techniques

To enable AOT compatibility, reflection-based techniques should be replaced with statically typed serialization, AppContext.BaseDirectory, typeof(), etc. For details, see Introduction to trim warnings.

Rooting Types

Until full support for `{Binding}` is implemented, types may be preserved from trimming as follows:

Given project `P` consuming assembly `A` with type `T` in namespace `N`, which is only dynamically referenced (so normally trimmed), `T` can be preserved via:

`P.csproj`:

XML

```xml
<ItemGroup>
    <TrimmerRootDescriptor Include="ILLink.Descriptors.xml" />
</ItemGroup>
```

`ILLink.Descriptors.xml`:

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<linker>
```

```
    <assembly fullname="A">
        <type fullname="N.T" preserve="all" />
    </assembly>
</linker>
```

For complete root descriptor XML expression syntax, see Root Descriptors.

> **Note**
>
> Dependency packages that have not yet adopted AOT support may exhibit runtime issues.

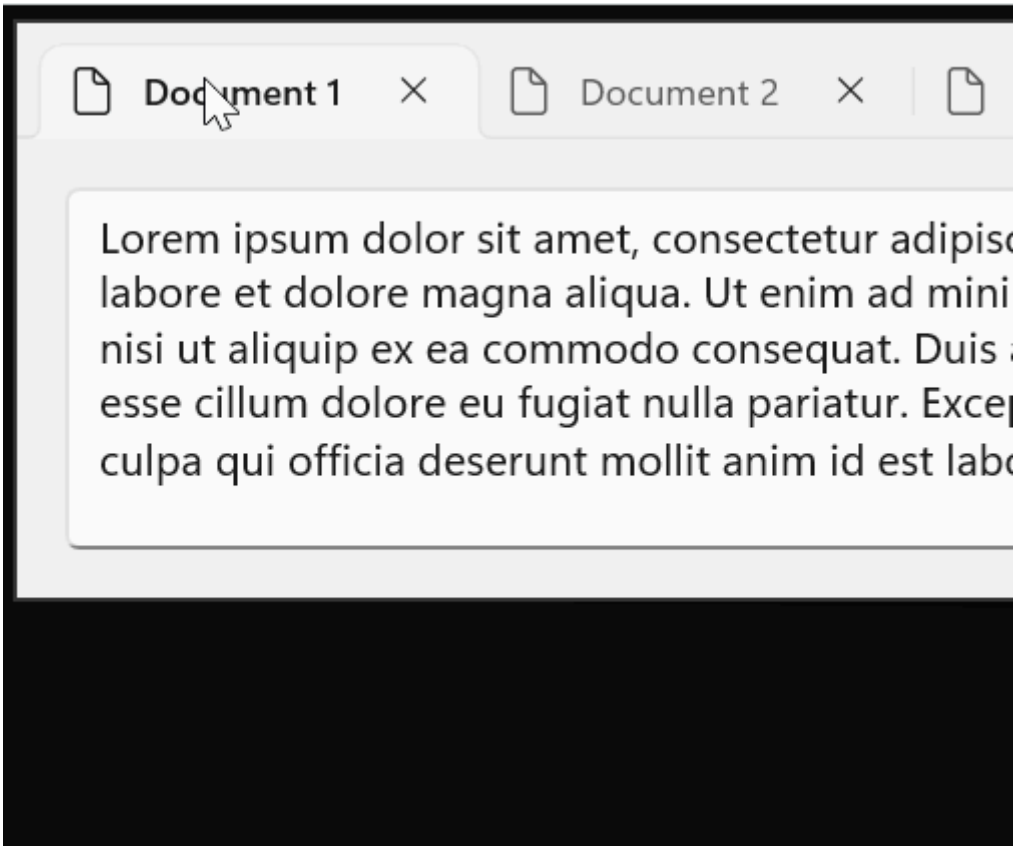
▼ Decoupled WebView2 versioning

The Windows App SDK now consumes the Edge WebView2 SDK as a NuGet reference rather than embedding a hardcoded version of the Edge WebView2 SDK. The new model allows apps to choose a newer version of the `Microsoft.Web.WebView2` package instead of being limited to the version with which the Windows App SDK was built. The new model also allows apps to reference NuGet packages which also reference the Edge WebView2 SDK. For more info, see GitHub issue #5689.

▼ New Package Deployment APIs

The Package Management API has received several enhancements including Is*ReadyOrNewerAvailable*(), EnsureReadyOptions.RegisterNewerIfAvailable, Is*Provisioned*(), IsPackageRegistrationPending(), and several bug fixes. See PackageManagement.md and Pull Request #4453 for more details.

▼ Improved TabView tab tear-out



`TabView` supports a new `CanTearOutTabs` mode which provides an enhanced experience for dragging tabs and dragging out to a new window. When this new option is enabled, tab dragging is very much like the tab drag experience in Edge and Chrome where a new window is immediately created during the drag, allowing the user to drag it to the edge of the screen to maximize or snap the window in one smooth motion. This implementation also doesn't use drag-and-drop APIs, so it isn't impacted by any limitations in those APIs. Notably, tab tear-out is supported in processes running elevated as Administrator.

▼ Other notable changes

- Added a new `ColorHelper.ToDisplayName()` API, filling that gap from UWP.
- Added a new `Microsoft.Windows.Globalization.ApplicationLanguages` class, which notably includes a new `PrimaryLanguageOverride` feature. For more info, see GitHub issue #4523.
- Unsealed `ItemsWrapGrid`. This should be a backward-compatible change.
- `PipsPager` supports a new mode where it can wrap between the first and last items.



- `RatingControl` is now more customizable, by moving some hard-coded style properties to theme resources. This allows apps to override these values to better customize the appearance of RatingControl.



- WinUI 3 has changed to the typographic model for font selection rather than the legacy weight/stretch/style model. The typographic model is required for some newer fonts, including Segoe UI Variable, and enables enhanced font capabilities. Some older fonts which rely on the weight/stretch/style model for selection may not be found with the typographic model.

▼ Known issues

- If the debugger is set to break on all C++ exceptions, it will break on some noisy exceptions on start-up in the BCP47 (Windows Globalization) code. For more info, see GitHub issue #4691.
- Component library packages which reference the Windows App SDK 1.6 package will not correctly get the referenced WebView2 package contents. For more info, see WebView2Feedback #4743. A workaround is to add a direct reference to the `Microsoft.Web.WebView2` package where needed.
- Apps compiled with Native AOT might sometimes experience a hanging issue after page navigation due to a race condition in the .NET runtime's GC thread. For more info, see .NET issue #104582.
- The initial release of 1.6.0 introduced an issue with one of our dependencies that we expect will be resolved in an upcoming release of the .NET SDK. If you experience an error with the version of your Microsoft.Windows.SDK.NET reference, you'll need to explicitly reference the version of the .NET SDK that is specified by your error message. For example, if the error says you need version 10.0.19041.38, add the following to your `.csproj` file:
  - `<WindowsSdkPackageVersion>10.0.19041.38</WindowsSdkPackageVersion>`.

▼ Bug fixes

- Fixed a crash when setting `InfoBar.IsOpen` in .xaml. For more info, see GitHub issue #8391.
- Fixed an issue where HTML elements would lose pointer capture when the mouse moved outside of the `WebView2` bounds. For more info, see GitHub issue #8677.
- Fixed an issue where drag and drop into a flyout with `ShouldConstrainToRootBounds=false` did not work. For more info, see GitHub issue #9276.

- Fixed an issue where `ms-appx://` references did not work when `PublishSingleFile` is enabled. For more info, see GitHub issue #9468.
- Fixed an issue where debugger symbols weren't working correctly for some binaries. For more info, see GitHub issue #4633.
- Fixed a potential crash when subclassing `NavigationView`.
- Fixed an issue where table borders in a `RichEditBox` would not correctly erase when scrolling or reducing the size of the table.
- Fixed an issue where flyouts from `MediaTransportControls` had a fully transparent background.
- Fixed an issue where dragging into a WebView2 would fail or drop in the wrong location on display scale factors other than 100% or when system text scaling is enabled.
- Fixed an issue where `TextBox`/`RichEditBox` would not announce to Accessibility tools when input is blocked due to being at the `MaxLength` limit.
- Fixed a few issues around handling of custom titlebar scenarios. For more info, see GitHub issues #7629, #9670, #9709 and #8431.
- Fixed an issue where `InfoBadge` icon was not visible. For more info, see GitHub issue #8176.
- Fixed an issue with icons sometimes showing in the wrong position in `CommandBarFlyout`. For more info, see GitHub issue #9409.
- Fixed an issue with keyboard focus in menus when opening or closing a sub menu. For more info, see GitHub issue #9519.
- Fixed an issue with `TreeView` using the incorrect `IsExpanded` state when recycling items. For more info, see GitHub issue #9549.
- Fixed an issue when using an ElementName binding in an `ItemsRepeater.ItemTemplate`. For more info, see GitHub issue #9715.
- Fixed an issue with the first item in an `ItemsRepeater` sometimes having an incorrect position. For more info, see GitHub issue #9743.
- Fixed an issue with `InputNonClientPointerSource` sometimes breaking input to the min/max/close buttons. For more info, see GitHub issue #9749.
- Fixed a compile error when using Microsoft.UI.Interop.h with clang-cl. For more info, see GitHub issue #9771.
- Fixed an issue where the `CharacterReceived` event was not working in `ComboBox`/`TextBox`. For more info, see GitHub issue #9786.
- Fixed an issue where duplicate `KeyUp` events were raised for arrow and tab keys. For more info, see GitHub issue #9399.
- Fixed an issue where the `PowerManager.SystemSuspendStatusChanged` event was unusable to get the `SystemSuspendStatus`. For more info, see GitHub issue #2833.

- Fixed an issue where initial keyboard focus was not correctly given to a `WebView2` when that was the only control in the window.
- Fixed an issue when using `ExtendsContentIntoTitleBar=true` where the Min/Max/Close buttons did not correctly appear in the UI Automation, which prevented Voice Access from showing numbers for those buttons.
- Fixed an issue where an app might crash in a lock check due to unexpected reentrancy.
- Fixed an issue where `Hyperlink` colors did not correctly update when switching into a high contrast theme.
- Fixed an issue where changing the collection of a `ListView` in a background window may incorrectly move that window to the foreground and take focus.
- Fixed an issue where calling `ItemsRepeater.StartBringIntoView` could sometimes cause items to disappear.
- Fixed an issue where touching and dragging on a `Button` in a `ScrollViewer` would leave it in a pressed state.
- Updated IntelliSense, which was missing information for many newer types and members.
- Fixed an issue where clicking in an empty area of a `ScrollViewer` would always move focus to the first focusable control in the `ScrollViewer` and scroll that control into view. For more info, see GitHub issue #597.
- Fixed an issue where the `Window.Activated` event sometimes fired multiple times. For more info, see GitHub issue #7343.
- Fixed an issue where setting the `NavigationViewItem.IsSelected` property to `true` prevented its children from showing when expanded. For more info, see GitHub issue #7930.
- Fixed an issue where `MediaPlayerElement` would not properly display captions with `None` or `DropShadow` edge effects. For more info, see GitHub issue #7981.
- Fixed an issue where the `Flyout.ShowMode` property was not used when showing the flyout. For more info, see GitHub issue #7987.
- Fixed an issue where `NumberBox` would sometimes have rounding errors. For more info, see GitHub issue #8780.
- Fixed an issue where using a library compiled against an older version of Windows App SDK could hit an error trying to find a type or property. For more info, see GitHub issue #8810.
- Fixed an issue where initial keyboard focus was not set when launching a window. For more info, see GitHub issue #8816.
- Fixed an issue where `FlyoutShowMode.TransientWithDismissOnPointerMoveAway` didn't work after the first time it was shown. For more info, see GitHub issue #8896.

- Fixed an issue where some controls did not correctly template bind `Foreground` and `Background` properties. For more info, see GitHub issue #7070, #9020, #9029, #9083 and #9102.
- Fixed an issue where `ThemeResource`s used in `VisualStateManager` setters wouldn't update on theme change. This commonly affected controls in flyouts. For more info, see GitHub issue #9198.
- Fixed an issue where `WebView` would lose key focus, resulting in extra blur/focus events and other issues. For more info, see GitHub issue #9288.
- Fixed an issue where `NavigationView` could show a binding error in debug output. For more info, see GitHub issue #9384.
- Fixed an issue where SVG files defining a negative viewbox no longer rendered. For more info, see GitHub issue #9415.
- Fixed an issue where changing `ItemsView.Layout` orientation caused an item to be removed. For more info, see GitHub issue #9422.
- Fixed an issue where scrolling a `ScrollView` generated a lot of debug output. For more info, see GitHub issue #9434.
- Fixed an issue where `MapContorl.InteractiveControlsVisible` did not work properly. For more info, see GitHub issue #9486.
- Fixed an issue where `MapControl.MapElementClick` event didn't properly fire. For more info, see GitHub issue #9487.
- Fixed an issue where x:Bind didn't check for null before using a weak reference, which could result in a crash. For more info, see GitHub issue #9551.
- Fixed an issue where changing the `TeachingTip.Target` property didn't correctly update its position. For more info, see GitHub issue #9553.
- Fixed an issue where dropdowns did not respond in WebView2. For more info, see GitHub issue #9566.
- Fixed a memory leak when using `GeometryGroup`. For more info, see GitHub issue #9578.
- Fixed an issue where scrolling through a very large number of items from an `ItemRepeater` in a `ScrollView` could cause blank render frames. For more info, see GitHub issue #9643.
- Fixed an issue where `SceneVisual` wasn't working.

▼ New APIs

Version 1.6.0 includes the following new APIs compared to the stable 1.5 release:

C#

```
Microsoft.UI
```

```
    ColorHelper
        ToDisplayName
```

C#

```
Microsoft.UI.Input

    EnteredMoveSizeEventArgs
    EnteringMoveSizeEventArgs
    ExitedMoveSizeEventArgs
    InputNonClientPointerSource
        EnteredMoveSize
        EnteringMoveSize
        ExitedMoveSize
        WindowRectChanged
        WindowRectChanging

    MoveSizeOperation
    WindowRectChangedEventArgs
    WindowRectChangingEventArgs
```

C#

```
Microsoft.UI.Xaml

    XamlRoot
        CoordinateConverter
```

C#

```
Microsoft.UI.Xaml.Automation.Peers

    ScrollPresenterAutomationPeer
```

C#

```
Microsoft.UI.Xaml.Controls

    PipsPager
        WrapMode
        WrapModeProperty

    PipsPagerWrapMode
    TabView
        CanTearOutTabs
        CanTearOutTabsProperty
        ExternalTornOutTabsDropped
        ExternalTornOutTabsDropping
        TabTearOutRequested
        TabTearOutWindowRequested
```

```
    TabViewExternalTornOutTabsDroppedEventArgs
    TabViewExternalTornOutTabsDroppingEventArgs
    TabViewTabTearOutRequestedEventArgs
    TabViewTabTearOutWindowRequestedEventArgs
```

C#

```
Microsoft.Windows.Globalization

    ApplicationLanguages
```

C#

```
Microsoft.Windows.Management.Deployment

    EnsureReadyOptions
        RegisterNewerIfAvailable

    PackageDeploymentFeature
    PackageDeploymentManager
        IsPackageDeploymentFeatureSupported
        IsPackageProvisioned
        IsPackageProvisionedByUri
        IsPackageReadyOrNewerAvailable
        IsPackageReadyOrNewerAvailableByUri
        IsPackageSetProvisioned
        IsPackageSetReadyOrNewerAvailable

    PackageReadyOrNewerAvailableStatus
```

C#

```
Microsoft.Windows.Storage

    ApplicationData
    ApplicationDataContainer
    ApplicationDataContract
    ApplicationDataCreateDisposition
    ApplicationDataLocality
```

Last updated on 12/20/2025

Windows App SDK 1.5 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.5.9 (1.5.250108004)

▼ Bugfixes

> This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.5 release.
>
> - Fixed another potential issue where a menu off a CommandBar may incorrectly open up instead of down when the CommandBar is at the bottom of the window.
> - Fixed a potential crash when running on older graphics hardware.

Version 1.5.8 (1.5.241107002)

▼ Bugfixes
> > This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.5 release. > > - Fixed a potential crash when the input queue on the thread appears to be in a bad state. > - Fixed some potential crashes when using `{x:Bind}` by checking every component for null. Apps wanting this fix need to recompile with this release. >

Version 1.5.7 (1.5.241001000)

▼ Bugfixes

> This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.5 release.
>
> - Added detection for a rare scenario where the app stops rendering and never recovers.
> - Fixed a potential crash when a UI thread uninitializes.
> - Fixed a potential issue where a menu off a CommandBar might have incorrectly opened up instead of down when the CommandBar was at the bottom of the window.

Version 1.5.6 (1.5.240802000)

▼ Bugfixes

> This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.5 release.
>
> - Fixed an issue with `InputNonClientPointerSource` sometimes breaking input to the Min/Max/Close buttons. For more info, see GitHub issue #9749.
> - Fixed a potential crash when closing a window while using a touchpad.

Version 1.5.5 (1.5.240627000)

▼ Bugfixes

> This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.5 release.
>
> - Fixed a potential crash during image loading when the image source is changed before loading completes.
> - Fixed a potential crash when using `InputNonClientPointerSource`.
> - Fixed an issue where a window containing only a WebView2 control doesn't correctly set initial keyboard focus into the WebView2, leaving it unusable by keyboard and accessibility tools. For more info, see GitHub issue WebView2Feedback#2330.

Version 1.5.4 (1.5.240607001)

▼ Bugfixes

> - Fixed an issue with the fix for GitHub issue #8857 to properly merge a library's resources.pri into the app's resources.pri.
> - Fixed an issue where WebView2 sent extra `blur` and `focus` events on every click. For more info, see GitHub issue #9288. This also fixes problems with dropdowns not responding, as seen in GitHub issue #9566.
> - Fixed an issue where creating a new window or popup canceled any in-progress drag operation. For more info, see GitHub issue #9360.
> - Fixed an issue where SVG files defining a negative viewbox no longer rendered. For more info, see GitHub issue #9415.
> - Fixed an issue where `x:Bind` didn't check for **null** before using a weak reference, which could result in a crash. For more info, see GitHub issue #9551.

- Fixed an issue where a resize cursor was incorrectly shown at titlebar edges when a window was maximized, which also caused incorrect behavior when clicking or dragging in those areas. For more info, see GitHub issue #8665.

Version 1.5.3 (1.5.240428000)

▼ Bugfixes

- Fixed an issue where `ItemsRepeater` sometimes caused its window to be brought to the front.
- Fixed a potential crash when the transparency state of an `AcrylicBrush` was updated.
- Fixed an issue where `ItemsRepeater` would not generate enough items if it was in a `ShouldConstrainToRootBounds="false"` popup that was taller or wider than the main window.
- Fixed an issue where error **MSB4086** could occur at build time if the project file didn't specify a *TargetPlatformVersion*. This error was hit as part of GitHub issue #9531.

Version 1.5.2 (1.5.240404000)

▼ Widget feed provider updates

- Feed providers are now able to show announcements in the Widgets Dashboard:
  - The new `FeedManager.TryShowAnnouncement` API allows a Feed Provider to request that the Widget Dashboard show their announcements. To show your announcement, create a `FeedAnnouncement` object and call the `FeedManager.TryShowAnnouncement` API.
  - Feed announcements will have their own badges displayed in the taskbar that can be customized using the `FeedAnnouncement` API.
- Feed providers can now opt in to receive Analytics and Error Reports information about their provider as a user interacts with it. To opt in, implement the `IFeedProviderAnalytics` and/or `IFeedProviderErrors` interfaces by your provider.

The feed provider updates includes the following new APIs:

C#

```
Microsoft.Windows.Widgets.Feeds.Providers

    FeedAnalyticsInfoReportedArgs
    FeedErrorInfoReportedArgs
    FeedManager
        SendMessageToContent
        TryShowAnnouncement
```

```
    FeedMessageReceivedArgs
    FeedResourceRequest
    FeedResourceRequestedArgs
    FeedResourceResponse
    IFeedAnnouncementInvokedTarget
    IFeedManager2
    IFeedProviderAnalytics
    IFeedProviderErrors
    IFeedProviderMessage
    IFeedResourceProvider
```

C#

```
Microsoft.Windows.Widgets.Notifications

    AnnouncementActionKind
    AnnouncementTextColor
    FeedAnnouncement
    FeedAnnouncementInvokedArgs
```

▼ Bug fixes

- Fixed an issue where `DispatcherTimer` would not fire at the correct time if the primary display wasn't running at 60 Hz.
- Fixed an issue where a textbox input method editor (IME) window would be incorrectly positioned when running at a scale factor other than 100%. For more info, see GitHub issue #9382.
- Fixed an issue where a drag-and-drop operation started from another app might not allow the correct Copy/Move/Link drop operations.
- Fixed a potential crash when a ListView's `ItemsSource` is reset and given focus right away.
- Fixed a problem where `ShouldConstrainToRootBounds="false"` popups/flyouts don't apply any inherited scale transform. For more info, see GitHub issues #9118 and #9433.
- Fixed a potential crash when a Composition object is destroyed with an animation still running.
- Fixed a potential crash on shutdown in `NavigationView::RevokeNavigationViewItemRevokers`.

Version 1.5.1 (1.5.240311000)

▼ Bugfixes

This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.5 release.

- Fixed an issue where self-contained apps may fail to build due to not being able to copy "map.html".
- Fixed an issue where `MapControl` would fail to initialize due to rejecting valid tokens. For more info, see GitHub issue #9324.
- Fixed an issue where `MapControl` loaded with a blue background. For more info, see GitHub issue #9377.
- Fixed an issue where clicking on the chevron of a `NavigationViewItem` did not correctly expand or collapse on a single click. This also caused menus to show as blank when clicking the chevron in `PaneDisplayMode="Top"` mode. For more info, see GitHub issues #9423 and #9426.
- Fixed an issue where tapping on a `NavigationViewItem` with touch or pen would prevent the item from responding to any future input. For more info, see GitHub issue #9429.
- Fixed a crash when clicking on an item in the `NavigationView.PaneFooter` area. For more info, see GitHub issue #9396.
- Fixed an issue where icons in menus were sometimes showing in the wrong place. For more info, see GitHub issue #9409.
- Fixed an issue where acrylic does not show on menus until switching away from the window and back. For more info, see GitHub issue #9406.
- Fixed a crash which could occur during `TextBox`/`RichEditBox` initialization. For more info, see GitHub issue #9216.
- Fixed some noisy exceptions which `NavigationView` threw and caught on destruction.
- Fixed an issue where a "pinch to zoom" gesture would sometimes show up as a pan or tap due to an incorrect `PointerCaptureLost` message firing.

Version 1.5

▼ XAML Islands runtime and shutdown updates

There is a behavioral difference between Windows App SDK 1.4 and Windows App SDK 1.5 for XAML Islands-based apps when the last XAML Window on any thread is closed.

- In Windows App SDK 1.4, the XAML runtime always exits the thread's event loop when the last XAML window on a thread is closed.
- In Windows App SDK 1.5:
  - If your app is a WinUI Desktop app, the default behavior is still the same as in Windows App SDK 1.4.
  - If you're using XAML for the DesktopWindowXamlSource ("XAML Islands") API, the default behavior is now that XAML does not automatically exit the thread's event loop.

    - In both modes, you can change this behavior by setting the `Application.DispatcherShutdownMode` property.

For more information, see the documentation for the `Application.DispatcherShutdownMode` property when available. This completes GitHub proposal #8492.

There is a behavioral difference between Windows App SDK 1.4 and Windows App SDK 1.5 for XAML Islands-based apps in the lifetime of the XAML runtime:

- In Windows App SDK 1.4, the XAML runtime shuts down on a thread if either all `WindowsXamlManager` and `DesktopWindowXamlSource` objects on a given thread are closed or shut down, or the `DispatcherQueue` running on that thread is shut down (in this case, the XAML runtime shuts down during the `DispatcherQueue.FrameworkShutdownStarting` stage).
- In Windows App SDK 1.5, the XAML runtime shuts down on a thread only when the DispatcherQueue running on that thread is shut down (the Xaml runtime always shuts down during the `DispatcherQueue.FrameworkShutdownStarting` stage).

For more information, see the documentation for the `WindowsXamlManager` class when available.

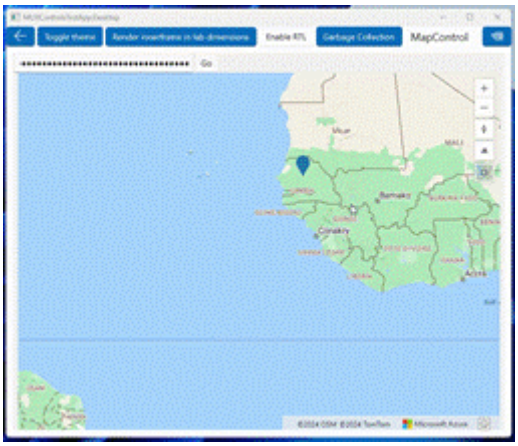
There is a behavioral difference in `WindowsXamlManager.InitializeForCurrentThread()`:

- In Windows App SDK 1.4, `WindowsXamlManager.InitializeForCurrentThread()` returns a unique instance of a `WindowsXamlManager` object with each call.
- In Windows App SDK 1.5, `WindowsXamlManager.InitializeForCurrentThread()` returns an existing instance if one already exists on the thread. `Close/Dispose()` is now ignored.

▼ WinUI Maps control

The WinUI `Maps` control is now available! This control is powered by WebView2 and Azure Maps, providing the following features:

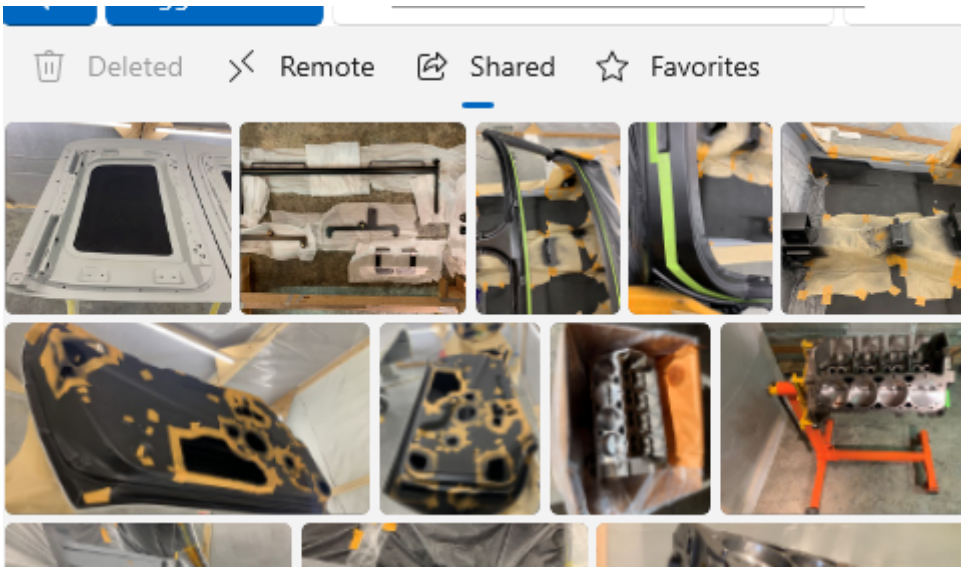
- Panning and zooming with either the map buttons or touch.
- Changing the style of the map to satellite, terrain, or street view.
- Programmatically adding interactable pins with developer-customizable icons to the map.
- Developer customization for where the map is centered on initial load.
- Control for developers over hiding or showing the buttons for panning, zooming, and map styles.



> **Note**
>
> To use the `Maps` control, you'll need an Azure Maps key. To create the key, see the Azure Maps documentation page for creating a web app.
>
> The `Maps` control is entirely new and we welcome your feedback to evaluate its future direction!

▼ New SelectorBar control

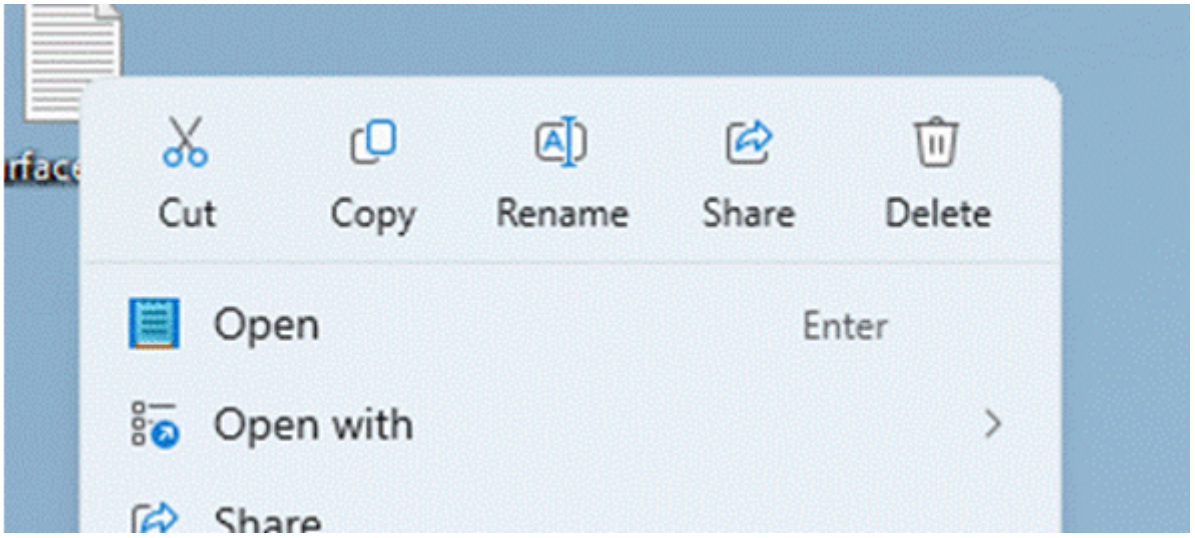
In 1.5, we've added a new `SelectorBar` control for enabling users to switch between multiple views of data. This control was previously known as "SegmentedControl" on our 1.5 roadmap.



▼ Labels in the CommandBarFlyout primary commands

The visuals of the `CommandBarFlyout` have been updated to display a text label for the items in the primary commands area if the `Label` property has been set on the `AppBarButton`.

Previously, the primary commands in the `CommandBarFlyout` area only displayed an icon, but now they can show both an icon and a label for improved usability.



▼ WebView2 support for custom environment/options

The WinUI `WebView2` control now exposes the ability to customize the underlying `CoreWebView2` object with a custom `CoreWebView2Environment` and `CoreWebView2ControllerOptions`. This enables the app author to specify an alternate path from which to load the WebView2Runtime, choose to use a different *UserDataFolder*, or set options such as *IsPrivateModeEnabled* and *ScriptLocale*.

▼ Support for .NET 8

We added support for .NET 8 in a recent 1.4 servicing release, but that still kept the warning for the use of platform-specific RIDs. In 1.5, we completed that work so that the warning is no longer present.

▼ Improved debugging and source availability

We now inject Github source server information for code in the microsoft-ui-xaml repo into our public symbols, allowing debuggers to automatically download source code. We also made other fixes and improvements to our symbols across the entire Windows App SDK to improve the debugging experience.

▼ Improved functionality for debugging layout cycles

Debugging layout cycles in a WinUI app can be a challenge, so in 1.5 the `DebugSettings` object now exposes options to enable improved logging and breakpoints for the layout process to make it easier to debug and fix layout cycles in the app.

▼ Other new features from across the Windows App SDK

> - Added support for the PublishSingleFile deployment model. For more info about PublishSingleFile, see the Single-file deployment documentation.
> - Added improvements to screen reader support, text scaling support, and other accessibility features.
> - Various stability and performance improvements based on our prioritized GitHub bug backlog.

▼ New features being released separately

> New versions of the Windows App SDK Visual Studio Templates for C# and C++ are being released through the Visual Studio Marketplace and they will appear a few weeks after the release of 1.5. With the new version, the templates may now be released independently of Windows App SDK releases, giving us much more flexibility in getting updates to customers.

▼ Other previously planned features
>

> In 1.5, we made progress on the following features that we announced on our roadmap, but did not complete them. They will continue into the 1.6 timeframe.
>
> - Tabbed windows
> - Drag-n-drop support for WebView2
>   - To clarify, in 1.5 dragging *into* WebView2 is supported, such as dragging a PNG from File Explorer into Bing for Visual Search. Work is ongoing in 1.6 to support dragging *out* of WebView2.
> - Investigations into the table view and ink controls
>
> Dynamic lighting has been removed from the roadmap for the time being.

▼ Known issues

> - When using libraries which contain resources such as .xaml files, you may hit an error message at runtime indicating that those resources cannot be found. In this case, it might be necessary to insert `<ShouldComputeInputPris>true</ShouldComputeInputPris>` in the project file to ensure those resources get included.
> - Clicking on the chevron of a `NavigationViewItem` no longer correctly expands or collapses on a single click. Double-clicking still works, as does clicking elsewhere on the `NavigationViewItem`.

▼ Bug fixes

- Fixed an issue where `StackPanel` applied spacing to collapsed items. For more info, see GitHub issue #916.
- Fixed problems with scrolling controls no longer working after closing another app window. Fore more info, see GitHub issues #9292 and #9355.
- Fixed a crash when setting `DebugSettings.EnableFrameRateCounter` to *true* before the first frame rendered. For more info, see GitHub issue #2835.
- Fixed a potential compile error for C++ where some headers did not include necessary dependencies. Note that the change of `#include` order might impact some apps, such as possibly causing a compile error for `IInspectable` if the app is using a version of C++/WinRT older than 2023. For more info, see GitHub issue #9014.
- Fixed an issue where `ElementName` bindings didn't work inside the `ItemsRepeater` `DataTemplate`. For more info, see GitHub issue #560.
- Fixed crashes when running an app under Visual Studio with the in-app toolbar enabled. Visual Studio 17.8 Preview 2 or later is required to fully get the fixes. For more info, see GitHub issue #8806.
- Fixed an issue where `AnnotatedScrollbar` could sometimes crash when quickly scrolling.
- Fixed an issue where menu text would sometimes get truncated.
- Fixed an issue where teaching tips did not receive proper focus. For more info, see GitHub issue #3257.
- Fixed an issue that crashed the application when setting the `TailVisibility` of a `TeachingTip` to *Collapsed* on startup. For more info, see GitHub issue #8731.
- Fixed an issue with how PRI files were handled when using libraries. For more info, see GitHub issue #8857.
- Fixed an issue from the 1.5-experimental2 release where the projection DLL was not generated. For more info, see GitHub issue #4152.
- Fixed an issue where the ellipsis button on the text formatting popup of the `RichEditBox` was not displaying the list of actions properly. For more info, see GitHub issue #9140.
- Fixed an issue where `ListView` didn't handle keyboard accelerators properly. For more info, see GitHub issue #8063.
- Fixed an access violation issue with using `AccessKey` to close a window. For more info, see GitHub issue #8648.
- Fixed a crash when using an `AccessKey` to close a window. For more info, see GitHub issue #9002.
- Fixed an issue affecting text alignment in a `MenuFlyoutItem` within a `MenuBar`. For more info, see GitHub issue #8755.

- Fixed an issue where highlighted text would not remain highlighted upon right-click. For more info, see GitHub issue #1801.
- Fixed an issue causing inactive windows to crash the app when closed. For more info, see GitHub issue #8913.
- Fixed an issue that could hang applications when scrolling with the middle mouse button and left-clicking immediately afterwards. For more info, see GitHub issue #9233.
- Fixed an issue causing apps to crash on startup when using a custom `NavigationViewItem`. For more info, see GitHub issue #8814.
- Fixed a `NavigationView` issue where the ellipsis button would incorrectly generate an error. For more info, see GitHub issue #8380.
- Fixed an issue where a `SystemBackdrop` would not render properly in a multi-window app. For more info, see GitHub issue #8423.
- Fixed a duplication issue when inserting into the beginning of an `ObservableCollection`. For more info, see GitHub issue #8370.

Last updated on 12/20/2025

Windows App SDK 1.4 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.4.7 (1.4.240802001)

▼ Bug fixes

- Fixed an issue where `ItemsRepeater` would not generate enough items if it was in a `ShouldConstrainToRootBounds="false"` popup which was taller or wider than the main window.
- Fixed a potential crash on shutdown if an `AnnotatedScrollBar` label update was pending.

Version 1.4.6 (1.4.240512000)

▼ Bug fixes

- Fixed a potential crash when processing input.
- Fixed an issue where a drag-and-drop operation that started from another app might not allow the correct Copy/Move/Link drop operations.
- Fixed WinUI source server information for debugging to properly point to the microsoft-ui-xaml GitHub repo.
- Fixed an issue with the fix for GitHub issue #8857 to properly merge a library's `resources.pri` into the app's `resources.pri`.

Version 1.4.5 (1.4.240211001)

▼ Bug fixes

- Fixed an issue that could hang applications when clicking a mouse button while scrolling with the mouse wheel. For more info, see GitHub issue #9233.
- Fixed an issue with duplicate assets when referencing a chain of NuGet packages. For more info, see GitHub issue #8857.
- Fixed several `BreadcrumbBar` issues including a memory leak, a crash when the ellipsis menu is empty, and the ellipsis menu being incorrectly constrained within the window.

- Fixed a potential crash on shutdown when releasing graphics resources.

Version 1.4.4 (1.4.231219000)

▼ Bug fixes

- Fixed a WinUI 3 diagnostics security issue.
- Fixed an input issue where the password box didn't show the on-screen keyboard when activated via touch. For more info, see GitHub issue #8946.
- Fixed an issue that caused the `Microsoft.UI.Xaml.Controls.dll` file size to grow unexpectedly.
- Fixed a `CommandBarFlyout` issue that could cause crashes when setting focus.
- Updated Windows App SDK support for .NET 8 RID-specific asset handling.
- Fixed an issue causing some swapchains to be positioned or stretched incorrectly.

Version 1.4.3 (1.4.231115000)

▼ Bug fixes

- Fixed an issue where a menu could appear without a background for a short period of time.
- Fixed a crash that may occur in specific multi-monitor scenarios.
- Fixed an issue where a context menu could appear off-screen.
- Fixed an issue with Window styles and maximizing behavior. For more info, see GitHub issue #8996.
- Fixed an issue with Islands where focus could be unexpectedly grabbed from another control.
- Fixed an issue with tab order on `NavigationView`.
- Fixed a rendering issue where a white bar might be visible at the top of the titlebar. For more info, see GitHub issue #8947.
- Various performance fixes.

Version 1.4.2 (1.4.231008000)

▼ Bug fixes

- Fixed a crashing issue in explorer.exe caused by excessive memory and object allocation.
- Fixed a titlebar interaction issue that prevented the back button from working properly.

- Fixed an issue that caused a warning to be generated for a source file being included multiple times.
- Fixed an issue impacting context menu performance.
- Fixed a .lnk shortcut issue that made the target .exe always point to the same location for packages in the WindowsApps folder.
- Fixed a DWriteCore issue affecting proper rendering of Indic text in certain fonts.
- Fixed an issue in a List View that prevented proper keyboard navigation to and from nested selected items with *Tab/Shift + Tab*.
- Fixed an issue that broke scrolling ComboBox items by touch after expanding the ComboBox a second time. For more info, see GitHub issue #8831.
- Fixed an issue where Windows App SDK packages did not include WinUI's localized resources for some languages.
- Fixed an inconsistency between how File Explorer and XAML display a user's preferred language.
- Fixed a craftsmanship issue in File Explorer causing a thin line to show under the active tab.
- Fixed an issue where some framework-provided keyboard accelerators were not properly localized. For more info, see GitHub issue #2023.
- Fixed an issue with RepeatButton controls that were repeatedly scrolling when tapped.
- Fixed the Windows App SDK installer .exe to have proper resource version info.

Version 1.4.1 (1.4.230913002)

▼ Bug fixes

- Fixed performance issues to improve the time to first frame.
- Fixed an issue where menus didn't respect `RequestedTheme`. For example, it was possible for this issue to lead to white text on a white background. For more info, see GitHub issue #8756.
- Fixed an issue that caused acrylic backgrounds to sometimes become fully transparent in some menus.
- Fixed an issue where XAML sometimes caused Windows to unnecessarily repaint the desktop wallpaper.
- Fixed support for `TabNavigation = Local` and `TabNavigation = Cycle` for `ListView` and `GridView`, which now enables navigating between headers and items with TAB in addition to arrow keys.
- Fixed some noisy exceptions when dismissing a tooltip. For more info, see GitHub issue #8699.

Version 1.4

▼ Custom titlebar + AppWindow titlebar merger

> The WinUI 3 custom titlebar uses the AppWindow titlebar implementation, along with the NonClientInputPointerSource APIs, under the hood in the Windows App SDK 1.4. As a result, both titlebar implementations now behave the same way with the same features and limitations. This is fully backwards compatible in all supported cases - any app with a custom-defined titlebar will behave as before. But, it's now easier for WinUI 3 developers who might be new to custom titlebars to understand and use them by taking advantage of these new features:
>
> - A better default scenario where the developer doesn't define a titlebar element specifically (replacing the fallback titlebar from WinUI 2)
> - Distinct drag regions in the titlebar, enabling you to create multiple drag regions and place clickable controls on any part of the non-client area (titlebar area)
> - App-wide draggable regions that can be put anywhere in the app or make the whole app draggable
> - Better theming support that replaces resource-based theming
>   - Since drag regions are transparent, they follow the app theme every time
> - More customization: hide the min, max, and close buttons; place system icons in the titlebar; or have different regions act as caption buttons that receive NCHITTEST responses
> - More developer freedom that enables you to mix and match with AppWindow titlebar APIs, such as using higher-level WinUI 3 APIs for most scenarios but with AppWindow APIs mixed in for lower-level control

▼ Widgets updates

> Three new interfaces have been added for Widget Providers to implement: `IWidgetProvider2`, `IWidgetProviderAnalytics`, and `IWidgetProviderErrors`. `IWidgetProvider2` allows providers to respond to the *Customize* action invoked by the user, which is identical to what is available for 1st party Widgets. The `IWidgetProviderAnalytics` and `IWidgetProviderErrors` interfaces are used by providers to gather telemetry for their widgets; analytics and failure events about widgets are communicated to the respective widget providers. The `WidgetCustomizationRequestedArgs`, `WidgetAnalyticsInfoReportedArgs`, and `WidgetErrorInfoReportedArgs` classes are used to communicate relevant information to support new functionalities.

▼ XAML Islands no longer experimental

XAML Islands and the underlying ContentIslands platform are no longer experimental.

- Currently XAML Islands are only tested for use in C++ apps. This release does not include any convenient wrapper elements for use in WPF or WinForms.
- `DesktopWindowXamlSource` and related types have been added in the Microsoft.UI.Xaml.Hosting namespace for XAML Islands. `XamlRoot.ContentIslandEnvironment` was added to help access the underlying Island information for an element.
- Many new types have been introduced in the Microsoft.UI.Content namespace and the Microsoft.UI.Input namespace as the underlying support for XAML Islands or for using this ContentIslands functionality without XAML.
- A new `DragDropManager` (plus related types) has been added in the Microsoft.UI.Input.DragDrop namespace for Island scenarios.

▼ ItemsView

We're introducing a new list control called the `ItemsView` and a corresponding concrete `ItemContainer` class. `ItemContainer` is a lightweight container with built-in selection states and visuals, which can easily wrap desired content and be used with `ItemsView` for a collection control scenario.

- The new `ItemsView` control displays a data collection. `ItemsView` is similar to the `ListView` and `GridView` controls, but is built using the `ItemsRepeater`, `ScrollView`, `ItemContainer` and `ItemCollectionTransitionProvider` components. It offers the unique ability to plug in custom `Layout` or `ItemCollectionTransitionProvider` implementations. Another key advantage is the ability to switch the layout on the fly while preserving items selection. The inner `ScrollView` control also offers features unavailable in `ListView`/`GridView`'s `ScrollViewer` control such as the ability to control the animation during programmatic scrolls.
  - A new `ItemTransitionProvider` property on `ItemsRepeater` (and the new `ItemsView` control) lets you specify an `ItemCollectionTransitionProvider` object to control transition animations on that control. A `CreateDefaultItemTransitionProvider` method has also been added to `Layout`, which enables a layout object to provide a fallback transition to accompany it if you do not provide one explicitly on the `ItemsView` control.
  - A new `IndexBasedLayoutOrientation` property on `Layout` where the layout orientation, if any, of items is based on their index in the source collection. The default value is `IndexBasedLayoutOrientation.None`. Custom layouts set this property by calling the new (protected) `SetIndexBasedLayoutOrientation` method.
  - A new `VisibleRect` property on `VirtualizingLayoutContext` gets the visible viewport rectangle within the `FrameworkElement` associated with the `Layout`. The

protected virtual `VirtualizingLayoutContext.VisibleRectCore` method can be overridden to provide the value that will be returned from the `VisibleRect` property.

- The new `LinedFlowLayout` class is typically used to lay out the items of the `ItemsView` collection control. It is particularly useful for displaying collection of pictures. It does so by laying them out from left to right, and top to bottom, in lines of equal height. The pictures fill a horizontal line and then wrap to a next line. Pictures may be cropped at the left and right edges to fit into a line. They may also be expanded horizontally and cropped at the top and bottom edges to fill a line when the stretching mode is employed.

▼ New features from across the Windows App SDK

- A new `ThemeSettings` class that allows Win32 WinRT apps to detect when the system's High Contrast setting has changed, similar to UWP's AccessibilitySettings class. See the ThemeSettings API spec on GitHub for more information.
- `AccessKeyManager.EnterDisplayMode` is a new method to display access keys for the current focused element of a provided root. Access keys are in "display mode" when showing a key tip to invoke a command, such as pressing the Alt key in Paint to show what keys correspond to what controls. This method allows for programmatically entering display mode.
- `Application.ResourceManagerRequested` provides a mechanism to provide a different `IResourceManager` to resolve resource URIs for scenarios when the default `ResourceManager` won't work. For more information, see the Application.ResourceManagerRequested API spec on GitHub.
- The version of the WebView2 SDK was updated from 1661.34 to 1823.32.
- `Popup/FlyoutBase.IsConstrainedToRootBounds = false` is now supported, allowing a popup/flyout to extend outside the bounds of the parent window. A `SystemBackdrop` property has been added to these types to support having acrylic in these unconstrained popups. Menus by default use this to have acrylic.
- `Closed`, `FrameworkClosed`, and `IsClosed` have been added to `DesktopAcrylicController` and `MicaController` to improve handling during object/thread shutdown.
- `DesktopAcrylicController.Kind` can now be set to choose between some standard acrylic appearances.
- `DispatcherQueue` has some new events and helpers to facilitate better organized shutdown and for apps using Islands to easily run a standard supported event loop.
- `InputNonClientPointerSource` in the Microsoft.UI.Input namespace can be used for custom titlebar scenarios to define non-client area regions. Code can register for corresponding events like hover and click events on these regions.
- `AppWindow` has some new helpers to get and associate with a `DispatcherQueue`.

- The new `TreeView.SelectionChanged` event allows developers to respond when the user or code-behind changes the set of selected nodes in the `TreeView` control.
- The new `ScrollView` control provides a new alternative to `ScrollViewer`. This new control is highly aligned in behavior and API with the existing `ScrollViewer` control, but is based on `InteractionTracker`, has new features such as animation-driven view changes, and is also designed to ensure full functionality of `ItemsRepeater`. See A more flexible ScrollViewer · Issue #108 · microsoft/microsoft-ui-xaml (github.com) for more details. Various new types, including `ScrollPresenter`, are part of the overall `ScrollView` model.
- The new `AnnotatedScrollBar` control extends a regular scrollbar's functionality by providing an easy way to navigate through a large collection of items. This is achieved through a clickable rail with labels that act as markers. It also allows for a more granular understanding of the scrollable content by displaying a tooltip when hovering over the clickable rail.

▼ Known issues

- When using `ExtendsContentIntoTitleBar = true`, clicks at the top-left corner of the window by default always show the system window menu (Minimize/Close/etc.) rather than letting the pointer input through to the content of the window. This, for example, means that a Back button in that area of the titlebar will not work. A workaround for this issue is to set `AppWindow.TitleBar.IconShowOptions = Microsoft.UI.Windowing.IconShowOptions.HideIconAndSystemMenu` on the Window's AppWindow.
- There are some new continuable exceptions when hiding `ShouldConstrainToRootBounds="False"` popups/flyouts. That includes hiding tooltips, as reported here: Dismissing a tooltip throws 4 native exceptions · Issue #8699 · microsoft/microsoft-ui-xaml (github.com)
- In 1.4, the min/max/close caption buttons for `ExtendsContentIntoTitleBar = true` are now drawn by AppWindow rather than XAML. This is by design, but it can impact apps that were overriding XAML's internal styles to hide or do extra customization of these buttons, such as in this report: Cannot hide caption button on titlebar · Issue #8705 · microsoft/microsoft-ui-xaml (github.com)
- There was a breaking change in .NET 8 to how it handles the runtime identifier graph: [Breaking change]: Projects targeting .NET 8 and higher will by default use a smaller, portable RID graph. · Issue #36527 · dotnet/docs (github.com) . Because of this issue and because .NET 8 has not officially released yet, the Windows App SDK 1.4 does not officially support .NET 8. However, if you would still like to target the pre-release version of .NET 8 with this version of the App SDK, we recommend the following steps:

    - Setting UseRidGraph to true is recommended. You'll also need to update the `<RuntimeIdentifiers>` property in the `.csproj` file to `<RuntimeIdentifiers>win-x86;win-x64;win-arm64</RuntimeIdentifiers>`, as well as update each `Propeties\*pubxml` file to switch from `win10` to `win` in the `<RuntimeIdentifier>` property (for example, `<RuntimeIdentifier>win-x86</RuntimeIdentifier>`).
- With Windows App SDK 1.4, the target `GenerateDeploymentManagerCS` in `Microsoft.WindowsAppSDK.DeploymentManager.CS.targets` was renamed to `GenerateBootstrapCS`.
- `MenuFlyout` background doesn't use the application's requested theme:
    - MenuFlyoutItem text doesn't sync with system theme. · Issue #8678 · microsoft/microsoft-ui-xaml (github.com)
    - 1.4 preview: Flyout backdrop theme is derived from system theme and not from its associated element · Issue #8756 · microsoft/microsoft-ui-xaml (github.com)

▼ Bug fixes

- Fixed an issue where calling the `Microsoft.Windows.AppLifecycle.AppInstance.Restart("")` API caused unpackaged apps to crash. For more info, see GitHub issue #2792.
- Fixed an installer crashing issue that was introduced in 1.4-experimental1. For more info, see GitHub issue #3760.
- Fixed an issue where text strikethrough was not removed properly in a TextBlock. For more info, see GitHub issue #1093.
- Fixed an issue causing incorrect Shift + Tab navigation in a Panel with *TabFocusNavigation* set to "Once." For more info, see GitHub issue #1363.
- Fixed an issue in C++/WinRT that prevented `{x:Bind}` from working properly with a property of a named XAML control. For more info, see GitHub issue #2721.
- Fixed a runtime AccessViolation issue in WinUI Desktop apps caused by setting `DebugSettings.EnableFrameRateCounter = true`. For more info, see GitHub issue #2835.
- Fixed an issue where `XamlTypeInfo.g.cpp` did not include needed headers. For more info, see GitHub issue #4907.
- Fixed a crashing issue caused by simultaneous multi-touch and mouse input. For more info, see GitHub issue #7622.
- Fixed an issue that prevented an active WinUI 3 app window from scrolling when the system setting to disable scrolling of inactive windows on mouse over was in effect. For more info, see GitHub issue #8764.
- Fixed a crash when trying to subclass `MediaPlayerElement`.
- Fixed some crash and memory leak issues in `TreeView`.
- Fixed an app hang issue that could happen when using keyboard to navigate in `RadioButtons`.

- Fixed a crash when using the keyboard to navigate in a `PipsPager`.
- Fixed WebView2 content to scale with the "Text size" Accessibility setting in Windows Settings.
- Fixed a crash that could occur when animations were running when the display turned off.
- Fixed a performance issue introduced in 1.3 that added a ~10% overhead to first layout/render.

Last updated on 12/20/2025

Windows App SDK 1.3 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.3.3 (1.3.230724000)

▼ Bug fixes

- Fixed an issue where the mouse would sometimes stop working when a dialog box was closed.
- Fixed a deployment issue that prevented apps from installing due to a mismatch of package versions on the system. For more information, see GitHub issue #3740.
- Fixed an issue affecting context menu positioning in Windows App SDK 1.3.
- Fixed an issue causing some WinUI3 apps, in some situations, to crash when the app was closed because XAML shut itself down too early.
- Fixed an issue where font icons were not mirroring properly in right-to-left languages. For more information, see GitHub issue #7661.
- Fixed an issue causing an app to crash on shutdown when resources were torn down in a bad order. For more information, see GitHub issue #7924.

Version 1.3.2 (1.3.230602002)

▼ Bug fixes

- Fixed a crash when setting a Protected Cursor.
- Fixed a performance issue in XamlMetadataProvider during app startup. For more information, see GitHub issue #8281.
- Fixed an issue with hyperlinks and touch in a RichTextBlock. For more information, see GitHub issue #6513.
- Fixed an issue with scrolling and touchpads in WebView2. For more information, see GitHub issue #7772.
- Fixed an issue where an update of Windows App SDK sometimes required a restart of Visual Studio. For more information, see GitHub issue #3554.
- Fixed a noisy exception on shutdown when running in a debugger.

Version 1.3.1 (1.3.230502000)

▼ Bug fixes

- Fixed issue causing apps to crash when setting the SystemBackdrop if the Content was null. For more information, see GitHub issue #8416.
- Fixed issue causing apps to crash when setting the Window Title in XAML, a new capability added in 1.3.0. For more information, see GitHub issue #3689.
- Fixed issue where a window incorrectly took focus when its content changed.
- Fixed an issue with creating C++ projects with the Windows App SDK 1.3 project templates.
- Updated templates on Visual Studio Marketplace

Version 1.3

▼ XAML Backdrop APIs

With properties built in to the XAML Window, Mica & Background Acrylic backdrops are now easier to use in your WinUI 3 app. See the System Backdrop and Mica Backdrop API docs for more information about the Xaml Backdrop properties.

C#

```
public MainWindow()
{
    this.InitializeComponent();

    this.SystemBackdrop = new MicaBackdrop();
}
```

▼ Window.AppWindow

By replacing several lines of boilerplate code, you're now able to use AppWindow APIs directly from a **Window** through Window.AppWindow.

▼ New features from across Windows App SDK

- `ApplicationModel.DynamicDependency`: `PackageDependency.PackageGraphRevisionId` that replaces the deprecated MddGetGenerationId.
- Environment Manager: EnvironmentManager.AreChangesTracked to inform you whether changes to the environment manager are able to be tracked in your application.
- A new event, DebugSettings.XamlResourceReferenceFailed is now raised when a referenced Static/ThemeResource lookup can't be resolved. This event gives access to

a trace that details where the framework searched for that key in order to better enable you to debug Static & ThemeResource lookup failures. For more information, see the Tracing XAML resource reference lookup failures API spec on GitHub.

▼ Other updates

- See our Windows App SDK 1.3 milestone on the Windows App SDK GitHub for additional issues addressed in this release.
- See our WinUI 3 in Windows App SDK 1.3 milestone on the microsoft-ui-xaml GitHub for additional issues addressed in this release.
- With the latest experimental VSIX, you're now able to convert your app between unpackaged and packaged through the Visual Studio menu instead of in your project file.

▼ Known issue

Due to a recent change to the xaml compiler, an existing project that upgrades to 1.3 may experience a build error like the following within Visual Studio:

Console

```
C:\Users\user\\.nuget\packages\microsoft.windowsappsdk\\**1.3.230331000**\buildT
ransitive\Microsoft.UI.Xaml.Markup.Compiler.interop.targets(537,17): error
MSB4064: The "PrecompiledHeaderFile" parameter is not supported by the
"CompileXaml" task loaded from assembly: Microsoft.UI.Xaml.Markup.Compiler,
Version=1.0.0.0, Culture=neutral, PublicKeyToken=de31ebe4ad15742b from the path:
C:\Users\user\\.nuget\packages\microsoft.windowsappsdk\\**1.2.230118.102**\tools
\net472\Microsoft.UI.Xaml.Markup.Compiler.dll. Verify that the parameter exists
on the task, the <UsingTask> points to the correct assembly, and it is a
settable public instance property.
```

This is caused by Visual Studio using a cached xaml compiler task dll from 1.2, but driving it with incompatible MSBuild logic from 1.3, as seen in the error text above. The workaround is to shut down Visual Studio, restart it, and reload the solution.

Last updated on 12/20/2025

Windows App SDK 1.2 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.2.5 (1.2.230313.1)

▼ Bug fixes

- Fixed issue causing apps to crash during Composition shutdown.
- Fixed issue causing apps to continue running animations even when the screen is off.
- Fixed issue causing mouse and touch input to fail in WebView2 when mouse and keyboard input occurred simultaneously. For more information, see GitHub issue #3266.

Version 1.2.4 (1.2.230217.4)

▼ Bug fixes

- Fixed issue causing self-contained apps to not be able to set UAC Settings. For more information, see GitHub issue #3376.
- Fixed issue causing push notifications to return an inaccurate Expiration time with `PushNotificationChannel::ExpirationTime`. For more information, see GitHub issue #3300.
- Fixed issue causing negative numbers to be considered "invalid" when passing a double as a parameter into an x:Bind function.
- Several fixes to update the WinUI VSIX. These updates included simplifying the project template dipAwareness in app.manifest, removing the UWP templates, updating localized resource files, adding the phone id to unblock store submission, and removing the copyright notice and license. For more info see GitHub issues #5659, #3205, #3323, #3322, #3143.

Version 1.2.3 (1.2.230118.102)

▼ Bug fixes

- Fixed issue causing WinUI 3 apps to crash when multiple windows are closed.

- Fixed issue causing a crash on app close when two or more references to the ThreadPoolTimer interface are called. For more information, see GitHub issues #7260 and #7239.
- Fixed issue causing all Single-project MSIX apps to run as full trust. For more information, see GitHub issue #7766.

Version 1.2.2 (1.2.221209.1)

▼ Bug fixes

- Fixed issue that caused the Store and side-load packages (e.g. from installer, NuGet, and bootstrapper) to fail to install if the other is already installed. For more information, see GitHub issue #3168.
- Fixed issue causing missing elasticity effects and animation curves when scrolling with a touchpad. For more information, see GitHub issue #7874.
- Fixed issue in ListView causing memory leaks.
- Fixed issue causing the Button template to not respect the Foreground property after mouse hover. For more information, see GitHub issue #7208.
- Fixed issue causing an unneeded exception when there is no MediaPlaybackItem in a MediaElement.
- Fixed issue causing a white frame to appear in MediaPlayerElement on content transitions.
- Fixed additional issues causing App.UnhandledException to not catch exceptions from other threads. For more information, see GitHub issues #1259 and #5221.

Version 1.2.1 (1.2.221116.1)

▼ Bug fixes

- Fixed issue that caused a crash on startup in C++ WinUI 3 apps when adding a WebView2 or TextBox control. For more information see GitHub issues #7911 & #3117.

Version 1.2

▼ Third-party Widgets in Windows

The widgets board was first introduced in Windows 11 and was limited to displaying built-in widgets. Widgets are small UI containers that display text and graphics on the widgets board, and are associated with an app installed on the device. With Windows App SDK, as

third party developers you can now create widgets for your packaged Win32 apps and test them locally on the Windows 11 widgets board.

For more information about widgets, check out Widgets Overview.

To get started developing widgets for your app, check out the Widget service providers development docs and Widgets design fundamentals for prerequisites, guidance and best practices.

Prerequisites for this release include:

- Developer mode enabled on the development machine.
- The development machine is running a version of Windows from the Dev Channel of Windows Insider Preview (WIP) that is greater than or equal to 25217 with widgets board version 521.20060.1205.0 or above.

Known limitations when developing Widgets:

- Third-party Widgets can only be tested locally on devices enrolled in WIP for this preview release.
- Widgets can only be created for packaged, Win32 apps. Widgets for Progressive Web Apps (PWA) are planned to be supported as part of Microsoft Edge 108.

▼ DisplayInformation

Windows desktop apps can now support High Dynamic Range (HDR) and Auto Color Management (ACM) through the DisplayInformation class in Windows App SDK. The DisplayInformation class enables you to monitor display-related information for an application view. This includes events to allow clients to monitor for changes in the application view affecting which display(s) the view resides on, as well as changes in displays that can affect the application view.

▼ WinUI 3

WinUI 3 apps can play audio and video with the **MediaPlayerElement** and **MediaTransportControls** media playback controls. For more info on how and when to use media controls, see Media players.

WinUI 3 has been updated with the latest controls, styles, and behaviors from WinUI 2.8. These updates include the addition of the **InfoBadge** control, improvements to accessibility and high contrast mode, as well as bug fixes across controls. For more details, see the release notes for WinUI 2.7 and WinUI 2.8.

Fixed issues:

- Acrylic backdrop material with DesktopAcrylicController is now supported in Windows 10 apps. For more information, check out issue 7112 on GitHub.
- Fixed various issues that caused routing of App.UnhandledException to fail. For more information, check out issue 5221 on GitHub. Regarding the remaining issues, workarounds are documented at the following GitHub issues and will be resolved in a future 1.2 release:
  - App_UnhandledException's UnhandledExceptionEventArgs.Exception can only be fetched once - should be cached
  - Do_Abi_* event/callback handlers should wrap invocations with RoReportUnhandledError
- Fixed issue causing ListView styles to regress and change from Windows App SDK 1.1. For more information, check out issue 7666 on GitHub.
- Fixed issue causing the incorrect Mica fallback background color to appear when the app is inactive. For more information, check out issue 7801 on GitHub.

Known limitations:

- When creating a new WinUI 3 project with Visual Studio 2022 17.4.0, it will reference a preview version of the Windows App SDK. Use NuGet Package Manager to update the reference to this release.
- Setting MediaPlayerElement.Source to relative URI (ms-appx/ms-resource) fails in unpackaged apps. The recommended workaround is to convert the relative ms-appx:/// URI to a fully resolved file:/// URI.

▼ Trimming for apps developed with .NET

.NET developers can now publish trimmed Windows App SDK apps. With CsWinRT 2.0, the C#/WinRT projections distributed in Windows App SDK are now trimmable. Publishing your app trimmed can reduce the disk footprint of your app by removing any unused code from trimmable binaries. Apps may also see a startup performance improvement. With a basic Hello World app, we have seen a ~80% disk footprint improvement and a ~7% startup performance improvement when published trimmed. With WinUI gallery, we have seen a ~45% disk footprint improvement.

For more details on how to enable trimming, trimming limitations (such as reflection against trimmable types), and trim warnings, see Trim self-contained deployments and executables. Developers should thoroughly test their apps after trimming to ensure everything works as expected. For more information, check out issue 2478 on GitHub.

▼ Support for Visual Studio Arm64

As early as Project Reunion (now Windows App SDK) 0.5, apps developed with Windows App SDK were able to run on Arm64. Starting with Visual Studio 17.3 Preview 2, you can

develop native applications with Windows App SDKpp SDK on Arm64 devices.

To get started developing on an Arm64 device, see Windows on Arm and Arm64 Visual Studio.

▼ Notifications

AppNotificationBuilder introduced as an alternative to XML payload for creating and defining App Notifications.

For usage information, see the AppNotificationBuilder spec on GitHub.

Also see Quickstart: App notifications in the Windows App SDK for an example of how to create a desktop Windows application that sends and receives local app notifications.

Breaking change:

For push notifications, when making a channel request call, apps will need to use the Azure Object ID instead of the Azure App ID. See Quickstart: Push notification in the Windows App SDK for details on finding your Azure Object ID.

Fixed issue:

PushNotificationManager.IsSupported will perform a check for elevated mode. It will return `false` if the app is elevated.

Known limitations (Notifications):

- In **AppNotificationScenario**, `Urgent` is only supported for Windows builds 19041 and later. You can use **AppNotificationBuilder.IsUrgentScenarioSupported** to check whether the feature is available at runtime.
- In **AppNotificationButton**, `hint-toolTip` and `hint-buttonStyle` are only supported for builds 19041 and later. You can use **IsButtonStyleSupported** and **IsToolTipSupported** to check whether the feature is available at runtime.
- In **MediaPlayerElement**, when used in XAML markup for an unpackaged app, the **Source** property cannot be set with an ms-appx or ms-resource URI. As an alternative, set the Source using a file URI, or set from code.

▼ Windowing

Full title bar customization is now available on Windows 10, version 1809 and later through the **AppWindowTitleBar** class. You can set **AppWindowTitleBar.ExtendsContentIntoTitleBar** to `true` to extend content into the title bar area, and **SetDragRectangles** to define drag regions (in addition to other customization options).

> If you've been using the **AppWindowTitleBar.IsCustomizationSupported** property to check whether you can call the **AppWindowTitleBar** APIs, it now returns `true` on supported Windows App SDK Windows 10 versions (1809 and later).
>
> **Known limitations (Windowing):**
>
> Basic title bar customizations are not supported on Windows 10. These include **BackgroundColor**, **InactiveBackgroundColor**, **ForegroundColor**, **InactiveForegroundColor** and **IconShowOptions**. If you call these properties, they will be ignored silently. All other **AppWindowTitleBar** APIs work in Windows 10, version 1809 and later. For the caption button color APIs (among others) and **Height**, **ExtendsContentIntoTitleBar** must be set to `true`, otherwise they will also be ignored silently.

▼ Access control

> Introduced security.accesscontrol.h with the **GetSecurityDescriptorForAppContainerNames** function to ease and streamline named object sharing between packaged processes and general Win32 APIs. This method takes a list of Package Family Names (PFNs) and access masks, and returns a security descriptor. For more information, see the GetSecurityDescriptorForAppContainerNames spec on GitHub.

▼ Other limitations and known issues

> **ⓘ Important**
>
> When you reference Windows App SDK 1.2 from a project you might see an error similar to: "*Detected package downgrade: Microsoft.Windows.SDK.BuildTools from 10.0.22621.1 to 10.0.22000.194.*", which is caused by incompatible references to the package from the app project and the Windows App SDK package. To resolve this you can update the reference in the project to a more recent and compatible version of Microsoft.Windows.SDK.BuildTools.
>
> - Unit tests may fail with a `REGDB_E_CLASSNOTREG` error in the Tests output pane in Visual Studio. As a workaround, you can add `<WindowsAppContainer>true</WindowsAppContainer>` to your project file.
> - .NET PublishSingleFile isn't supported.
> - Bootstrapper and Undocked RegFree WinRT auto-initializer defaults is (now) only set for projects that produce an executable (OutputType=Exe or WinExe). This prevents adding auto-initializers into class library DLLs and other non-executables by default.

    - If you need an auto-initializer in a non-executable (e.g. a test DLL loaded by a generic executable that doesn't initialize the Bootstrapper) you can explicitly enable an auto-initializer in your project via `<WindowsAppSdkBootstrapInitialize>true</WindowsAppSdkBootstrapInitialize>` or `<WindowsAppSdkUndockedRegFreeWinRTInitialize>true</WindowsAppSdkUndockedRegFreeWinRTInitialize>`.
- Microsoft.WindowsAppRuntime.Release.Net.dll is always the Arm64 binary and does not work for x86 and x64 apps. When explicitly calling the Bootstrap API do not use the Microsoft.WindowsAppRuntime.Release.Net.dll assembly. As a workaround you can include version constants in this source file distributed with the NuGet package: '..\include\WindowsAppSDK-VersionInfo.cs' or use the auto-initializer.

Last updated on 12/20/2025

Windows App SDK 1.1 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.1.5

▼ Bug fixes

> - Fixed issue where Acrylic does not work if Mica is enabled. For more information, see issue 7200 on GitHub.
> - Fixed issue causing apps that depend on the WindowsAppRuntime installer (e.g. unpackaged apps) to fail to run on Windows 10 ARM64 machines. For more information, see issue 2564 on GitHub.

Version 1.1.4

▼ Bug fixes

> - Fixed regression from 1.0.x causing ListView, TreeView, and other 'List' controls to crash when scrolling with many items. For more information, see issue 7230 on GitHub.
> - Fixed issue with DispatcherQueue causing enqueued callbacks to no longer be invoked.
> - Fixed issue causing an app crash when calling `DeploymentManager.Initialize` multiple times in same app session.
> - Fixed issue causing C# apps to fail to build on Arm64 Visual Studio. For more information, see issue 7140 on GitHub.
> - Fixed intermittent crash in XAML imaging code due to incorrect failure handling.
> - Fixed memory leak issue when attaching an event handler in ItemsRepeater with a parent UserControl. For more information, see issue 6123 on GitHub.
> - Fixed issue causing a build failure in Visual Studio 17.3 when an app project is configured to enable automatic updates of its package when it's sideloaded (i.e. .appinstaller). For more information, see issue 2773.

- Fixed issue causing Store-distributed packaged apps that call Initialize (necessary e.g. for Push) to call it redundantly as DeploymentManager::GetStatus returned `Package Install Needed` when main and singleton packages are already installed. This caused a perf degradation on app launch.
- Fixed issue causing an exception in single instance apps when the cleanup event was intended to be ignored if it couldn't be opened. For more information, see the PR on GitHub.

Version 1.1.3

▼ Bug fixes

- Fixed related set of issues where XAML crashes when including a ProgressBar, ProgressRing, PipsPager, PersonPicture, or Expander control in the first page of your app. For more information see issue 7164 on GitHub.
- Fixed issue causing the x64 installer to fail to install the Windows App SDK runtime. For more information see issue 2713 on GitHub.
- Fixed issue causing the WindowsAppRuntime to fail to install if a higher version of the runtime is installed. For more information see discussion 2708 on GitHub.

Version 1.1.2

▼ Bug fixes

- Fixed issue where XAML crashes when closing a window while a dialog is open. For more information see issue 1032 on GitHub.
- Added `<auto-generated>` tag in C# files to prevent StyleCop warnings. For more information see issue 4526 on GitHub.
- Fixed issue causing an access violation error and crash when calling MddBootstrapInitialize when the matching framework package isn't installed. For more information see issue 2592 on GitHub.
- Fixed issue where the C# WinUI 3 item templates were missing in Visual Studio. For more information see issue 7148 on GitHub.
- Fixed issue where the WindowsAppRuntime installer fails when run as System user. For more information see issue 2546 on GitHub.

Version 1.1.1

▼ Bug fixes

> - Fixed issue causing apps to sometimes crash during a drag and drop operation. For more information see issue 7002 on GitHub.
> - Fixed issue causing the title bar to disappear when switching AppWindowPresenterKind from FullScreen to Default.
> - Fixed issue where Bootstrapper APIs like `ApiInformation.IsPropertyPresent` and `ApiInformation.IsMethodPresent` would cause unhandled exceptions in apps that aren't packaged. For more information see issue 2382 on GitHub.
> - Fixed issue causing app freeze when maximizing application with pen input.

Version 1.1

▼ App Lifecycle & Restart

> Apps are now able to initiate an explicit restart with specific arguments & state building off of the existing RegisterApplicationRestart API to register with the OS to restart in update, hang & reboot scenarios.
>
> **New features:**
>
> - Any packaged or unpackaged desktop app can terminate and restart itself on command, and have access to an arbitrary command-line string for the restarted instance using the `AppInstance.Restart()` API.
>   - This is a lifted and synchronous version of the UWP `RequestRestartAsync()` API which enables restarting with arguments and returns an `AppRestartFailureReason` if the restart is unsuccessful.
>   - Check out the Restart API docs on GitHub for usage & reference information.

▼ WinUI 3

> WinUI 3 is the native user experience (UX) framework for Windows App SDK. This release includes new features from Windows App SDK 1.0 as well as several stability improvements from 1.0 & 1.1 preview releases.
>
> **New features:**

- Mica and Background Acrylic are now available for WinUI applications.
  - For more information about these materials, check out Materials in Windows 11. Check out our sample code for applying Mica in C++ applications at Apply Mica or Acrylic materials in desktop apps for Windows 11 and in C# applications on GitHub as part of the WinUI 3 Gallery.
- First introduced in 1.0.1, we have stabilized and enabled the creation of **multiple windows on the same thread** in WinUI applications. See issue 5918 for more information.

Fixed bugs:

- Fixed issue when using Mica where the app would crash when a window is divided equally by two screens. See issue 7079 on GitHub for more info.
- Fixed issue causing C# apps with WebView2 to crash on launch when the C/C++ Runtime (CRT) isn't installed by upgrading the WebView2 SDK from 1020.46 to 1185.39.
- Fixed issue causing some rounded corners to show a gradient when they should be a solid color. For more information see issue 6076 & issue 6194 on GitHub.
- Fixed issue where updated styles were missing from generic.xaml.
- Fixed layout cycle issue causing an app to crash when scrolling to the end of a ListView. For more information see issue 6218 on GitHub.
- Fixed issue where users are unable to drop an element when drag-and-drop is enabled. For more information see issue 7008 on GitHub.

Known limitations:

- When using a custom title bar, the caption controls do not change color on theme change.
- XAML crashes when a user closes a window while a dialog is open.

▼ Deployment

New features:

- Packaged apps can now force deploy the Windows App SDK runtime packages using the **DeploymentManager.Initialize(DeploymentInitializeOptions) API** or using the --force option with the Windows App Runtime installer.
- There are additional functional extension categories, UnlockedDEHs, available for packaged apps. Check out the 1.1 Preview 3 release notes for more details. These require the Windows App SDK framework package to be installed. See Latest Windows App SDK downloads to install the runtime.

- Self-contained deployment is supported. Check out the Windows App SDK deployment overview for the differences between framework-dependent and self-contained deployment, and how to get started.
- The Bootstrapper API required for apps that don't deploy with MSIX include new options for improved usability and troubleshooting. Please view our documentation for C# apps, Bootstrapper C# APIs and for C++ apps, mddbootstrapheader.h header. For more details, see Use the Windows App SDK runtime for apps packaged with external location or unpackaged.

Known limitations:

- Running the Windows App Runtime installer (WindowsAppRuntimeInstall.exe) requires sideloading to be enabled. See issue 2469 on GitHub for more information.
- Creating an MSIX package through the Visual Studio Project menus can crash Visual Studio in some scenarios. This issue will be fixed in Visual Studio version 17.3 Preview 2 and serviced to 17.2. If you encounter this issue, you can work around it by generating an MSIX from the command line, switching to an unpackaged project, or reverting back to Windows App SDK 1.0.
- Self-contained applications packaged with MSIX are unsupported on 1809 causing app crash on launch.

▼ Elevation

Apps are now able to run with elevated privilege.

Known limitations:

- Elevated support requires the following OS servicing update:
  - Win11 - May 10, 2022—KB5013943 (OS Build 22000.675)
  - Win10 - May 10, 2022—KB5013942 (OS Builds 19042.1706, 19043.1706, and 19044.1706)
- App and Push Notifications for an elevated unpackaged app is not supported.
- Elevated WinUI apps crash when dragging an element during a drag-and-drop interaction.

▼ Environment Variable Manager

Environment Variable Manager is a new API introduced in Windows App SDK 1.1. Environment Variable Manager allows developers to access, and modify environment variables in the process, user, and machine scope from one API surface.

If Environment Variable Manager is used from a packaged application, all environment variable operations are recorded. When the package is removed all environment variable operations are reverted.

New features:

- Get and set environment variables in the process, user, and machine scope.
- Automatic environment variable revert when a package that uses environment variable manager is removed.
- Includes specific APIs for PATH and PATHEXT.

Known limitations:

- Only available on Windows 11

▼ MRT Core

MRT Core is a streamlined version of the modern Windows Resource Management System that is distributed as part of the Windows App SDK.

Fixed issues:

- An issue causing resources not to be indexed by default when a resource file is added using the VS UI is fixed in .NET SDK 6.0.300. If using an older .NET SDK version, please continue to use the workaround documented in 1.0's release notes. See issue 1786 on GitHub for additional information.
- An issue causing the resource URI to not be built correctly in unpackaged C++ WinUI apps was fixed in Visual Studio 2022 17.2. If using an older Visual Studio version, please update Visual Studio to 17.2 to receive this fix.

Known limitations:

- In .NET projects, resource files copy-pasted into the project folder aren't indexed on F5 if the app was already built. As a workaround, rebuild the app. See issue 1503 on GitHub for more info.

For more information, see Manage resources with MRT Core.

▼ Notifications

Developers of packaged (including packaged with external location) and unpackaged apps can now send Windows notifications.

New features:

- Support for app notifications for packaged and unpackaged apps.
  - Developers can send app notifications, also known as toast notifications, locally or from their own cloud service. See App notifications overview.
- Support for push notifications for packaged and unpackaged apps.
  - Developers can send raw notifications and app notifications from their own cloud service. See Push notifications overview.

Known limitations:

- Sending notifications from an elevated app is not supported. `PushNotificationManager::IsSupported()` will not perform a check for elevated mode.

▼ Windowing

For easier programming access to functionality that's implemented in USER32.dll (see Windows and messages), this release surfaces more of that functionality in AppWindow itself.

New features:

- Apps with existing windows have more control over how a window is shown, by calling `AppWindow.ShowOnceWithRequestedStartupState`—the equivalent of `ShowWindow(SW_SHOWDEFAULT)`.
- Apps can show, minimize, or restore a window while specifying whether the window should be activated or not at the time the call is made.
- Apps can now determine specific dimensions for their window's client area size in Win32 coordinates without needing to calculate the non-client area sizing to get a specific client area size.
- Additional WinRT APIs are available to support z-order management of windows based off of SetWindowPos's hWndInsertAfter functionality.
- Apps drawing custom title bars with AppWindowTitleBar.ExtendsContentIntoTitleBar can set a `PreferredTitleBarHeight` option. You now have a choice between a standard height title bar, or a tall title bar that provides more room for interactive content. See Title bar in the Fluent design guidelines for advice about when to use a tall title bar.

Fixed issues:

- When full screen presenter is invoked the first time, the window now fits on the entire screen correctly. See issue 1853 on GitHub for more info.

- Windows created with `AppWindow::GetFromWindowId` have the `OverlappedPresenter` as the default presenter but does not have restrictions in terms of changes to window styles coming from other APIs. Windows created with AppWindow::Create will have the default Presenter guardrails in place from the start. See issue 2049 on GitHub for more info.
- Using the `OverlappedPresenter.SetBorderAndTitlebar` API to hide caption buttons and borders would result in a 1px top border when maximized. This has been resolved. See issue 1693 on GitHub for more info.

Known limitations:

- When using the **AppWindowTitlebar** API to customize the colors of the standard title bar, the icon and text is misaligned compared to the standard title bar. For more info, see GitHub issue 2459.

- When solving GitHub issue 2049 (seen above), we introduced the following bug: if you apply any **AppWindowPresenter** to an **AppWindow** that you've retrieved from **GetFromWindowId**, then change a window style that's being tracked by that Presenter through calling USER32 APIs, and then try to revert back to the window's previous state by re-applying the default Presenter, the result is a window that has no title bar. If you rely on any Presenter in your app, and use calls to USER32 for changing window styles at the time that a non-default Presenter is applied, then you might need to add a workaround to ensure correct window behavior until this bug is serviced. You can use the following code snippet as a template for how to work around the issue:

C#

```
AppWindow m_appWindow;
OverlappedPresenter m_defaultPresenter;

private void EnterFullScreen_Click(object sender, RoutedEventArgs e)
{
    // Capture the default presenter.
    m_defaultPresenter = m_appWindow.Presenter as OverlappedPresenter;

    // Opt in the default overlapped presenter so it can control various
aspects of the AppWindow.
    m_defaultPresenter.IsAlwaysOnTop = m_defaultPresenter.IsAlwaysOnTop;
    m_defaultPresenter.IsResizable = m_defaultPresenter.IsResizable;
    m_defaultPresenter.IsMinimizable = m_defaultPresenter.IsMinimizable;
    m_defaultPresenter.IsMaximizable = m_defaultPresenter.IsMaximizable;
    m_defaultPresenter.SetBorderAndTitleBar(m_defaultPresenter.HasBorder,
m_defaultPresenter.HasTitleBar);

    m_appWindow.SetPresenter(AppWindowPresenterKind.FullScreen);
```

```
}

private void ExitFullScreen_Click(object sender, RoutedEventArgs e)
{
    m_appWindow.SetPresenter(AppWindowPresenterKind.Default);
}
```

▼ C#/WinRT

> C# Windows Runtime Components, including WinUI custom controls, are now supported. This enables component authors to distribute C#-authored runtime components to any WinRT compatible language (e.g., C++/WinRT). See Walkthrough—Create a C# component with WinUI 3 controls, and consume it from a C++/WinRT app that uses the Windows App SDK and the sample on GitHub to get started.

▼ Other limitations and known issues

> - Apps that reference a package that depends on WebView2 (like Microsoft.Identity.Client) fail to build. This is caused by conflicting binaries at build time. See issue 2492 on GitHub for more information.
> - Using `dotnet build` with a Windows App SDK C# class library project may see a build error "Microsoft.Build.Packaging.Pri.Tasks.ExpandPriContent task could not be loaded". To resolve this issue set `<EnableMsixTooling>true</EnableMsixTooling>` in your project file.
> - The default Windows App SDK templates note that the MaxVersionTested="10.0.19041.0" when it should be "10.0.22000.0". For full support of some features, notably UnlockedDEHs, update the MaxVersionTested to "10.0.22000.0" in your project file.

Last updated on 08/21/2026

Windows App SDK 1.0 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 1.0.4

▼ Bug fixes

- Fixed issue causing AppBars, when used as Page.TopAppBar or Page.BottomAppBar to not render on screen.
- Fixed issue where apps with a package name of 12 characters or less that use a WinUI control from MUXControls.dll will immediately crash. For more information, see issue 6360 on GitHub.
- Fixed touch input issues causing problems with keyboard shortcuts and other scenarios. For more information, see issue 6291 on GitHub.
- Fixed issue causing apps packaged with MSIX or deployed as self-contained to fail to deploy.
- Fixed issue causing apps to sometimes crash during a drag and drop operation. For more information see issue 7002 on GitHub.

Version 1.0.3

▼ Bug fixes

- Fixed issue causing C# apps with WebView2 to crash on launch when the C/C++ Runtime (CRT) isn't installed.
- Fixed touch input issues causing problems with keyboard shortcuts and other scenarios. For more information, see issue 6291 on GitHub.

Note: We don't usually add functionality in a servicing release, but this release's WebView2 fix required us to update to the latest version of the WebView2 SDK (1020.46 to 1185.39). See Release Notes for the WebView2 SDK for additional information on WebView2 1.0.1185.39 and Distribute your app and the WebView2 Runtime for additional information on the WebView2 Runtime.

Version 1.0.2

This is a servicing release of the Windows App SDK that includes critical bug fixes for the 1.0 release.

▼ Bug fixes

- Fixed layout cycle issue causing an app to crash when scrolling to the end of a ListView. For more information see issue 6218 on GitHub.
- Fixed issue causing C# apps to crash on launch when the C/C++ Runtime (CRT) isn't installed. However, the CRT is still required for C# apps using WebView2. For more information see issue 2117 on GitHub.
- Fixed issue where applications with Single-project MSIX did not generate a .appinstaller file. For more information see issue 1821 on GitHub.
- Fixed issue where WinUI applications did not support .NET 6 `dotnet build`.

Version 1.0.1

This is a servicing release of the Windows App SDK that includes critical bug fixes and multi-window support for the 1.0 release.

▼ Bug fixes

- Fixed issue causing the MddBootstrapAutoinitializer to not compile with enabled ImplicitUsings. For more information see issue 1686 on GitHub.
- Fixed issue where focus in WebView2 would be unexpectedly lost causing input and selection issues. For more information, see issue 5615 & issue 5570 on GitHub.
- Fixed issue causing the in-app toolbar in Visual Studio to be unclickable when using a custom title bar in a WinUI 3 app.
- Fixed issue causing Snap Layout to not appear when using a custom title bar in a WinUI 3 app. For more information, see issue 6333 & issue 6246 on GitHub.
- Fixed issue causing an exception when setting Window.ExtendsContentIntoTitleBar property when Window.SetTitlebar has been called with a still-loading UIElement.
- Fixed issue where Single-project MSIX apps did not support `dotnet build`.
- Fixed issue causing unpackaged apps to not install after installing a packaged app. For more information, see issue 1871 on GitHub.
- Fixed issue reducing performance during mouse drag operations.
- Fixed crash when calling GetWindowIdFromWindow() in unpackaged apps. For more information, see discussion 1891 on GitHub.

The limitations and known issues for version 1.0 also apply to version 1.0.1.

Additionally, for apps with custom title bars, we have made changes in this release (and fixed numerous issues) that include fixes to the glass window used for drag&drop operations. The recommendation is to use the default values and behaviors (give them a

try!). If your title bar used margins so that the default caption buttons were interactive, we recommend visualizing your drag region by setting the background of your title bar to red and then adjusting the margins to extend the drag region to the caption controls.

▼ New features

We have stabilized and enabled the creation of **multiple windows on the same thread** in WinUI 3 applications. See issue 5918 for more information.

Version 1.0

▼ WinUI 3

WinUI 3 is the native user experience (UX) framework for Windows App SDK. In this release we've added multiple new features from Windows App SDK 0.8 and stabilized issues from 1.0 Preview releases.

New features and updates:

- We've added new controls (PipsPager, Expander, BreadcrumbBar) and updated existing controls to reflect the latest Windows styles from WinUI 2.6.
- Single-project MSIX packaging is supported in WinUI by creating a new application using the "Blank App, Packaged..." template.
- We now support deploying WinUI 3 apps that aren't packaged on Windows versions 1809 and above. Please view Create your first WinUI 3 (Windows App SDK) project for additional information.
- WinUI 3 projects can now set their target version down to Windows 10, version 1809. Previously, they could only be set as low as version 1903.
- In-app toolbar, Hot Reload, & Live Visual Tree for WinUI packaged apps are supported in Visual Studio 2022 Preview 5 and GA.

Important limitations:

- Known issues for **both packaged and unpackaged WinUI applications**:
  - *Run-time error in C++ or C# apps that reference a C++ Windows Runtime Component:*
    - To resolve, add the below target to the end of the Windows Runtime Component's .vcxproj:

XML

```
<Target Name="GetPriIndexName">
<PropertyGroup>
    <!-- Winmd library targets use the default root namespace of the
```

```
project for the App package name -->
    <PriIndexName Condition="'$(RootNamespace)' != ''">$(RootNamespace)
</PriIndexName>
    <!-- If RootNamespace is empty fall back to TargetName -->
    <PriIndexName Condition="$(PriIndexName) == ''">$(TargetName)
</PriIndexName>
</PropertyGroup>
</Target>
```

    - The expected error will be similar to *WinRT originate error - 0x80004005 : 'Cannot locate resource from 'ms-appx:///BlankPage.xaml'.'.*

- Known issues for **WinUI applications with Single-project MSIX** (Blank App, Packaged template):
    - *Missing Package & Publish menu item until you restart Visual Studio:* When creating a new app with Single-project MSIX in both Visual Studio 2019 and Visual Studio 2022 using the Blank App, Packaged (WinUI 3 in Desktop) project template, the command to publish the project doesn't appear in the menu until you close and re-open Visual Studio.
    - A C# app with Single-project MSIX will not compile without the "C++ (v14x) Universal Windows Platform Tools" optional component installed. See Install tools for the Windows App SDK for additional information.
    - *Potential run-time error in an app with Single-project MSIX that consumes types defined in a referenced Windows Runtime Component:* To resolve, manually add activatable class entries to the appxmanifest.xml.
        - The expected error in C# applications is "COMException: Class not registered (0x80040154 (REGDB_E_CLASSNOTREG)).
        - The expected error in C++/WinRT applications is "winrt::hresult_class_not_registered".

- Known issues for **WinUI 3 apps that aren't packaged** (unpackaged apps):
    - Some APIs require package identity, and aren't supported in unpackaged apps, such as:
        - ApplicationData
        - StorageFile.GetFileFromApplicationUriAsync
        - StorageFile.CreateStreamedFileFromUriAsync
        - ApiInformation (not supported on Windows 10)
        - Package.Current
        - Any API in the Windows.ApplicationModel.Resources namespace
        - Any API in the Microsoft.Windows.ApplicationModel.Resources namespace

- Known issues for **packaging and deploying WinUI applications**:
    - The `Package` command is not supported in WinUI apps with Single-project MSIX (Blank App, Packaged template). Instead, use the `Package & Publish` command to

create an MSIX package.
    - To create a NuGet package from a C# Class Library with the `Pack` command, ensure the active `Configuration` is `Release`.
    - The `Pack` command is not supported in C++ Windows Runtime Components to create a NuGet package.

For more info, or to get started developing with WinUI, see:

- WinUI
- Create your first WinUI 3 (Windows App SDK) project

▼ Windowing

The Windows App SDK provides an AppWindow class that evolves the previous easy-to-use Windows.UI.WindowManagement.AppWindow preview class and makes it available to all Windows apps, including Win32, WPF, and WinForms.

New Features:

- AppWindow is a high-level windowing API that allows for easy-to-use windowing scenarios that integrates well with the Windows user experience and with other apps. Represents a high-level abstraction of a system-managed container of the content of an app. This is the container in which your content is hosted, and represents the entity that users interact with when they resize and move your app on screen. For developers familiar with Win32, the AppWindow can be seen as a high-level abstraction of the HWND.
- DisplayArea represents a high-level abstraction of a HMONITOR, follows the same principles as AppWindow.
- DisplayAreaWatcher allows a developer to observe changes in the display topology and enumerate DisplayAreas currently defined in the system.

For more info, see Manage app windows (Windows App SDK).

▼ Input

These are the input APIs that support WinUI and provide a lower level API surface for developers to achieve more advanced input interactions.

New Features:

- Pointer APIs: PointerPoint, PointerPointProperties, and PointerEventArgs to support retrieving pointer event information with XAML input APIs.
- InputPointerSource API: Represents an object that is registered to report pointer input, and provides pointer cursor and input event handling for XAML's

SwapChainPanel API.
- Cursor API: Allows developers to change the cursor bitmap.
- GestureRecognizer API: Allows developers to recognize certain gestures such as drag, hold, and click when given pointer information.

Important limitations:

- All PointerPoint static factory functions have been removed: **GetCurrentPoint**, **GetCurrentPointTransformed**, **GetIntermediatePoints**, and **GetIntermediatePointsTransformed**.
- The Windows App SDK doesn't support retrieving **PointerPoint** objects with pointer IDs. Instead, you can use the **PointerPoint** member function **GetTransformedPoint** to retrieve a transformed version of an existing **PointerPoint** object. For intermediate points, you can use the **PointerEventArgs** member functions **GetIntermediatePoints** and **GetTransformedIntermediatePoints**.
- Direct use of the platform SDK API **Windows.UI.Core.CoreDragOperation** will not work with WinUI applications.
- **PointerPoint** properties **RawPosition** and **ContactRectRaw** were removed because they referred to non-predicted values, which were the same as the normal values in the OS. Use **Position** and **ContactRect** instead. Pointer prediction is now handled with the **Microsoft.UI.Input.PointerPredictor** API object.

▼ App Lifecycle

Most of the App Lifecycle features already exist in the UWP platform, and have been brought into the Windows App SDK for use by desktop app types, especially unpackaged Console apps, Win32 apps, Windows Forms apps, and WPF apps. The Windows App SDK implementation of these features cannot be used in UWP apps, since there are equivalent features in the UWP platform itself.

Non-UWP apps can also be packaged into MSIX packages. While these apps can use some of the Windows App SDK App Lifecycle features, they must use the manifest approach where this is available. For example, they cannot use the Windows App SDK **RegisterForXXXActivation** APIs and must instead register for rich activation via the manifest.

All the constraints for packaged apps also apply to WinUI apps, which are packaged, and there are additional considerations as described below.

Important considerations:

- Rich activation: GetActivatedEventArgs
  - *Unpackaged apps*: Fully usable.

    - *Packaged apps*: Usable, but these apps can also use the platform `GetActivatedEventArgs`. Note that the platform defines Windows.ApplicationModel.AppInstance whereas the Windows App SDK defines Microsoft.Windows.AppLifecycle.AppInstance. And while UWP apps can use the `ActivatedEventArgs` classes, such as `FileActivatedEventArgs` and `LaunchActivatedEventArgs`, apps that use the Windows App SDK AppLifecycle feature must use the interfaces not the classes (e.g, `IFileActivatedEventArgs`, `ILaunchActivatedEventArgs`, and so on).
    - *WinUi apps*: WinUI's App.OnLaunched is given a Microsoft.UI.Xaml.LaunchActivatedEventArgs, whereas the platform `GetActivatedEventArgs` returns a Windows.ApplicationModel.IActivatedEventArgs, and the WindowsAppSDK `GetActivatedEventArgs` returns a Microsoft.Windows.AppLifecycle.AppActivationArguments object which can represent a platform `LaunchActivatedEventArgs`.
    - For more info, see Rich activation with the app lifecycle API.

- Register/Unregister for rich activation
    - *Unpackaged apps*: Fully usable.
    - *Packaged apps*: Not usable use the app's MSIX manifest instead.
    - For more info, see Rich activation with the app lifecycle API.

- Single/Multi-instancing
    - *Unpackaged apps*: Fully usable.
    - *Packaged apps*: Fully usable.
    - *WinUI apps*: If an app wants to detect other instances and redirect an activation, it must do so as early as possible, and before initializing any windows, etc. To enable this, the app must define DISABLE_XAML_GENERATED_MAIN, and write a custom Main (C#) or WinMain (C++) where it can do the detection and redirection.
    - RedirectActivationToAsync is an async call, and you should not wait on an async call if your app is running in an STA. For Windows Forms and C# WinUI apps, you can declare Main to be async, if necessary. For C++ WinUI and C# WPF apps, you cannot declare Main to be async, so instead you need to move the redirect call to another thread to ensure you don't block the STA.
    - For more info, see App instancing with the app lifecycle API.

- Power/State notifications
    - *Unpackaged apps*: Fully usable.
    - *Packaged apps*: Fully usable.
    - For more info, see Power management with the app lifecycle API.

Known issue:

- File Type associations incorrectly encode %1 to be %251 when setting the Verb handler's command line template, which crashes unpackaged Win32 apps. You can manually edit the Registry value to be %1 instead as a partial workaround. If the target file path has a space in it, then it will still fail and there is no workaround for that scenario.
- These Single/Multi-instancing bugs will be fixed in an upcoming servicing patch:
  - AppInstance redirection doesn't work when compiled for x86
  - Registering a key, unregistering it, and re-registering it causes the app to crash

▼ DWriteCore

DWriteCore is the Windows App SDK implementation of DirectWrite, which is the DirectX API for high-quality text rendering, resolution-independent outline fonts, and full Unicode text and layout support. DWriteCore is a form of DirectWrite that runs on versions of Windows down to Windows 10, version 1809 (10.0; Build 17763), and opens the door for you to use it cross-platform.

Features:

DWriteCore contains all of the features of DirectWrite, with a few exceptions.

Important limitations:

- DWriteCore does not contain the following DirectWrite features:
  - Per-session fonts
  - End-user defined character (EUDC) fonts
  - Font-streaming APIs
- Low-level rendering API support is partial.
- DWriteCore doesn't interoperate with Direct2D, but you can use IDWriteGlyphRunAnalysis and IDWriteBitmapRenderTarget.

For more information, see DWriteCore overview.

▼ MRT Core

MRT Core is a streamlined version of the modern Windows Resource Management System that is distributed as part of the Windows App SDK.

Important limitations:

- In .NET projects, resource files copy-pasted into the project folder aren't indexed on F5 if the app was already built. As a workaround, rebuild the app. See issue 1503 for more info.

- In .NET projects, when a resource file is added to the project using the Visual Studio UI, the files may not be indexed by default. See issue 1786 for more info. To work around this issue, please remove the entries below in the CSPROJ file:

  XML

  ```xml
  <ItemGroup>
      <Content Remove="<image file name>" />
  </ItemGroup>
  <ItemGroup>
      <PRIResource Remove="<resw file name>" />
  </ItemGroup>
  ```

- For unpackaged C++ WinUI apps, the resource URI is not built correctly. To work around this issue, add the following in the vcxproj:

  XML

  ```xml
  <!-- Add the following after <Import
  Project="$(VCTargetsPath)\Microsoft.Cpp.props" /> -->

  <PropertyGroup>
      <AppxPriInitialPath></AppxPriInitialPath>
  </PropertyGroup>
  ```

For more information, see Manage resources with MRT Core.

▼ Deployment

New Features and updates:

- You can auto-initialize the Windows App SDK through the `WindowsPackageType` `project` property to load the Windows App SDK runtime and call the Windows App SDK APIs. See Create your first WinUI 3 (Windows App SDK) project for instructions.
- Unpackaged apps can deploy Windows App SDK by integrating in the standalone Windows App SDK `.exe` installer into your existing MSI or setup program. For more info, see Windows App SDK deployment guide for framework-dependent apps packaged with external location or unpackaged.
- Unpackaged .NET apps can also use .NET wrapper for the bootstrapper API to dynamically take a dependency on the Windows App SDK framework package at run time. For more info about the .NET wrapper, see .NET wrapper library.
- Packaged apps can use the deployment API to verify and ensure that all required packages are installed on the machine. For more info about how the deployment API works, see Windows App SDK deployment guide for framework-dependent packaged apps.

Important limitations:

- The .NET wrapper for the bootstrapper API is only intended for use by unpackaged .NET applications to simplify access to the Windows App SDK.
- Only MSIX packaged apps that are full trust or have the packageManagement restricted capability have the permission to use the deployment API to install the main and singleton package dependencies. Support for partial-trust packaged apps will be coming in later releases.
- When F5 testing an x86 app which uses the DeploymentManager.Initialize method on an x64 system, ensure that the x64 framework is first installed by running the WindowsAppRuntimeInstall.exe. Otherwise, you will encounter a **NOT_FOUND** error due to Visual Studio not deploying the x64 framework, which normally occurs through Store deployment or sideloading.

▼ Other limitations and known issues

- **No support for Any CPU build configuration**: When adding the Windows App SDK to an existing .NET application or component that supports **Any CPU**, you must specify the desired architecture: `x86`, `x64` or `arm64`.

- **Upgrading from .NET 5 to .NET 6**: When upgrading in the Visual Studio UI, you might run into build errors. As a workaround, manually update your project file's `TargetFrameworkPackage` to the below:

  XML

  ```
  <TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
  ```

- **C# Single-project MSIX app doesn't compile if C++ UWP Tools aren't installed.** If you have a C# Single-project MSIX project, then you'll need to install the **C++ (v14x) Universal Windows Platform Tools** optional component.

- **Subsequent language VSIX fails to install into Visual Studio 2019 when multiple versions of Visual Studio 2019 are installed.** If you have multiple versions of Visual Studio 2019 installed (e.g. Release and Preview) and then install the Windows App SDK VSIX for both C++ *and* C#, the second installation will fail. To resolve, uninstall the Single-project MSIX Packaging Tools for Visual Studio 2019 after the first language VSIX. View this feedback for additional information about this issue.

- An alternative to **DispatcherQueue.TryEnqueue** (for resuming execution on the dispatcher queue thread) is to use the resume_foreground helper function in the Windows Implementation Library (WIL):

1. Add a reference to your project to the Microsoft.Windows.ImplementationLibrary NuGet package.
2. Add `#include <wil/cppwinrt_helpers.h>` to your `pch.h`.
3. Add `#include <winrt/Microsoft.UI.Dispatching.h>` to your `pch.h`.
4. Now `co_await wil::resume_foreground(your_dispatcherqueue);`.

Last updated on 12/20/2025

Windows App SDK 0.8 release notes

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 0.8.12

This is a servicing release of the Windows App SDK that includes critical bug fixes for the 0.8.0 release.

▼ Bug fixes

> - Fixed issue where apps with SwapChainPanel or WebView2 would unpredictably crash due to an access violation.

Version 0.8.11

▼ Bug fixes

> - Fixed regression causing the lost focus event to fire when selecting text using mouse.

Version 0.8.10

▼ Bug fixes

> - Fixed issues causing apps to sometimes crash during a drag and drop operation.

> **Note**
>
> Windows App SDK 0.8.9 was not released. The version released directly after 0.8.8 is 0.8.10.

Version 0.8.8

This is a servicing release of the Windows App SDK that includes critical bug fixes for the 0.8.0 release.

[!NOTE] For C# developers, one of the following .NET SDK versions (or later) is required: 6.0.202, 6.0.104, 5.0.407, 5.0.213. To update your .NET SDK version, visit .NET Downloads or

update to the latest version of Visual Studio. Without the required .NET SDK version, when updating your NuGet package you will see an error like: "*This version of WindowsAppSDK requires WinRT.Runtime.dll version 1.6 or greater.*".

▼ Bug fixes

> - Fixed touch input issues in TextBox regarding soft keyboard and general interaction. These issues also affected keyboard shortcuts. For more information, see issue 6291 on GitHub.
> - Fixed issue where an app window would sometimes show as inactive when active.
> - Fixed performance issue caused by UIA (UI Automation) running in external processes.
> - Fixed app stability issue with pen input.
> - Fixed issue where the render of png icons in a Menu are dramatically delayed because of UIA.

Version 0.8.7

This is a servicing release of the Windows App SDK that includes several performance updates for C#/.NET applications. To update to this version, you'll need to reference the latest Windows SDK package version. To do that, add the property `<WindowsSdkPackageVersion>10.0.<sdk_version>.24</WindowsSdkPackageVersion>` to your `.csproj` file with the SDK version your app is targeting from the `TargetFramework` property. For example:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <OutputType>WinExe</OutputType>
        <TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
        <TargetPlatformMinVersion>10.0.17763.0</TargetPlatformMinVersion>
        <WindowsSdkPackageVersion>10.0.19041.24</WindowsSdkPackageVersion>
    <PropertyGroup>
    ...
```

This version of the Windows SDK projection will be available in an upcoming .NET 6 servicing release. After that .NET SDK update is available, you should remove the `<WindowsSdkPackageVersion>` property from your project file.

If you don't set this property, then you'll see an error like: `"Error: This version of Project Reunion requires WinRT.Runtime.dll version 1.6 or greater."`

Version 0.8.6

This is a servicing release of the Windows App SDK that includes several performance improvements for C#/.NET applications for the 0.8.0 release.

To update to this version of Windows App SDK, you will need to have the latest .NET SDK December update installed (see Download .NET and .NET 5 will reach End of Support on May 10, 2022). If you don't have the minimum required version of the .NET SDK installed, then you'll see an error like `"Error: This version of Project Reunion requires WinRT.Runtime.dll version 1.4 or greater."`

▼ Bug fixes

> For a detailed list of the performance improvements, see the C#/WinRT 1.4.1 release notes.

Version 0.8.5

This is a servicing release of the Windows App SDK that includes more critical bug fixes for the 0.8.0 release.

▼ Bug fixes

> - Fixed issue that was causing WinUI apps using pointer input to crash.
> - Fixed issue causing the title bar buttons (min, max, close) to not have rounded corners on Windows 11.
> - Fixed issue causing the resizing layout options to not appear when hovering over maximize/restore button on Windows 11.
> - Fixed issue causing a crashing exception where creating a PointCollection object. For more information, see issue 971 on Github.

Version 0.8.4

This is a servicing release of the Windows App SDK that includes more critical bug fixes for the 0.8.0 release.

▼ Bug fixes

> - Fixes for custom title bars so that ContentDialog doesn't cover it up, and the title bar buttons are rounded.
> - Fix for a crash in image processing when the display scale is changed.
> - Fixes clipping bugs where UI disappear or clipped incorrectly

Version 0.8.3

This is a servicing release of the Windows App SDK that includes more critical bug fixes for the 0.8.0 release.

▼ Bug fixes

> - Keyboard focus was being lost when a window was minimized and then restored, requiring a mouse click to restore focus.

Version 0.8.2

This is a servicing release of the Windows App SDK that includes more critical bug fixes for the 0.8.0 release.

▼ Bug fixes

> - Windows App SDK and WinUI 3 are now supported in Visual Studio 2022 Preview 2 and later.
> - For .NET apps, you may receive the following error when passing in an array of enums: `Object contains non-primitive or non-blittable data.`
> - Writing using the HandWriting Panel inside a textbox causes a crash
> - Icons/images always load at their 100% scale value rather than based on the monitor scale value
> - Garbage collection of **EventSource<T>** causes subsequent failure to unsubscribe handlers (see GitHub issue for more details)
> - Security fix – see CVE-2021-34533 for more details.
> - SwapChainPanel.CompositionScaleChanged sometimes returning incorrect CompositionScale values after changing display scale

Version 0.8.1

This is a servicing release of the Windows App SDK that includes a few critical bug fixes for the 0.8.0 release.

▼ Bug fixes

> - Windows App SDK cannot run on the latest Windows Insider build
> - Crash in EditableComboBox when entering a value that does not appear in dropdown
> - WebView2 doesn't allow user to tab out once focused has been received
> - Fully qualify Windows.Foundation.Metadata.DefaultOverload namespace in WinUI generated code to avoid namespace ambiguity
>   - This fixes bug #5108.
> - Security fix – see CVE-2021-34489 for more details.

Version 0.8.0 Stable

This release supports all stable channel features.

▼ WinUI 3

> > This release includes many bug fixes and improved stabilization across WinUI 3. These are all of the new changes in WinUI 3 since the release of WinUI 3 - Project Reunion 0.5: > > - The Pivot control has been added back in and can now be used in any WinUI 3 app. > - All bug fixes from Project Reunion v0.5.5, v0.5.6, and v0.5.7 are included with this release. > - New bug fixes, including: > - Mouse right-click in TextBox crashes the application > - NavigationView causes crash in UWP, Reunion 0.5 Preview > - ProgressBar doesn't show difference between Paused and Error option > - Crash in RichEditBox when copying/pasting/changing text style > - Window caption buttons are misplaced when SetTitleBar is not set or null > > **For the full list of bugs addressed in this release, see [our GitHub repo](https://aka.ms/winui3/0.8/bugs-fixed).** > > - The `ColorHelper.ToDisplayName` API is no longer available. > - The following types have been removed: > - `Microsoft.Graphics.IGeometrySource2D` > - `Microsoft.Graphics.IGeometrySource2DInterop` > > Use `Windows.Graphics.IGeometrySource2D` and `Windows.Graphics.IGeometrySource2DInterop` instead. > > - All types in the `Microsoft.System` namespace have been moved to the `Microsoft.UI.Dispatching` namespace, including the [DispatcherQueue class](/windows/windows-app-sdk/api/winrt/microsoft.ui.dispatching.dispatcherqueue). > > - The `AcrylicBrush.BackgroundSource` property has been removed, since `HostBackdrop` is not supported as a `BackgroundSource` in WinUI 3. > > For more information on WinUI, see [WinUI](../../winui/index.md). > > To see WinUI 3 controls and features in action, you can clone and build the WinUI 3 Gallery app [from GitHub](https://github.com/microsoft/WinUI-Gallery/tree/main), or download the app [from the Microsoft Store](https://apps.microsoft.com/detail/9P3JFPWWDZRC). > > To get started developing with WinUI, check out the following articles: > > - [WinUI 3 templates in Visual Studio](../../winui/winui3/winui-project-templates-in-visual-studio.md) > - [Create your first WinUI 3 (Windows App SDK) project](../../winui/winui3/create-your-first-winui3-app.md) > - [Build a C# .NET app with WinUI 3 and Win32 interop](../../winui/winui3/desktop-winui3-app-with-basic-interop.md) > - [WinUI 3 API Reference](/windows/winui/api)

▼ DWriteCore

> > This release of DWriteCore includes the following new and updated features. DWriteCore is introduced and described in the [DWriteCore overview](/windows/win32/directwrite/dwritecore-overview). > > - DWriteCore now has support for underline—see [**IDWriteTextLayout::GetUnderline**](/windows/win32/api/dwrite/nf-dwrite-idwritetextlayout-getunderline) and [**IDWriteTextLayout::SetUnderline**](/windows/win32/api/dwrite/nf-dwrite-idwritetextlayout-setunderline). > - Support for strikethrough—see [**IDWriteTextLayout::GetStrikethrough**](/windows/win32/api/dwrite/nf-dwrite-idwritetextlayout-getstrikethrough) and [**IDWriteTextLayout::SetStrikethrough**]

(/windows/win32/api/dwrite/nf-dwrite-idwritetextlayout-setstrikethrough). > - Support for vertical text via [**IDWriteTextLayout**](/windows/win32/api/dwrite/nn-dwrite-idwritetextlayout)—see [Vertical text](/windows/win32directwrite/vertical-text). > - All of the methods of the [**IDWriteTextAnalyzer**](/windows/win32/api/dwrite/nn-dwrite-idwritetextanalyzer) and [**IDWriteTextAnalyzer1**](/windows/win32/api/dwrite_1/nn-dwrite_1-idwritetextanalyzer1) interfaces are implemented. > - The [**DWriteCoreCreateFactory**](/windows/windows-app-sdk/api/win32/dwrite_core/nf-dwrite_core-dwritecorecreatefactory) free function creates a factory object that is used for subsequent creation of individual DWriteCore objects. > > [!NOTE] > [**DWriteCoreCreateFactory**](/windows/windows-app-sdk/api/win32/dwrite_core/nf-dwrite_core-dwritecorecreatefactory) is functionally the same as the [**DWriteCreateFactory**](/windows/win32/api/dwrite/nf-dwrite-dwritecreatefactory) function exported by the system version of DirectWrite. The DWriteCore function has a different name to avoid ambiguity in the event that you link both `DWriteCore.lib` and `DWrite.lib`. > > For DWriteCore and DirectWrite API reference, see [DWriteCore API Reference](/windows/windows-app-sdk/api/win32/_dwritecore/) and [DirectWrite API Reference](/windows/win32/directwrite/reference).

▼ MRTCore

> - The **Build Action** for resources is automatically set when you add the resource to your project, reducing the need for manual project configuration.

▼ Limitations

> - This release is not currently supported on the Dev Channel of the Windows Insider Program. **This is fixed in version 0.8.1**.
> - Desktop apps (C# or C++ desktop): This release is supported for use only in desktop apps (C++ or C#) that are packaged using MSIX. To use the Windows App SDK in unpackaged desktop apps, you must use the experimental release channel.

▼ Known issues

> - WinUI 3 tooling such as Live Visual Tree, Live Property Explorer, and Hot Reload in version 0.8 and later requires Visual Studio 2019 16.11 Preview 3 and later.
> - Apps currently using WinUI 3 and the Windows App SDK 0.8 cannot use class libraries that use Project Reunion 0.5. Update the class libraries to use the Windows App SDK 0.8.
> - .NET apps must target build 18362 or later: Your TFM must be set to net6.0-windows10.0.18362 or later, and your packaging project's must be set to 18362 or later. For more info, see GitHub issue #921.

- You may encounter a crash when switching frequently between light and dark mode.
- For .NET apps, you may receive the following error when passing in an array of enums: `Object contains non-primitive or non-blittable data.` **This is fixed in version 0.8.2**.
- For .NET apps, there is currently no way to opt out of an image getting indexed as an app resource using the Visual Studio UI. To work around this, add a Directory.Build.targets (see Customize your build - Visual Studio for instructions) to the project and remove the image(s) as follows:
  - To remove specific images (note that the relative path is needed):

    XML

    ```
    <Project>
    <ItemGroup>
        <Content Remove="..\Bitmap1.bmp" />
    </ItemGroup>
    </Project>
    ```

  - To remove images based on metadata:

    XML

    ```
    <Project>
    <ItemGroup>
        <Content Remove="@(None->WithMetadataValue('Pack','true'))" />
    </ItemGroup>
    </Project>
    ```

    A fix for this issue is planned for an upcoming release - at that point, the above workarounds will no longer be needed.

Last updated on 12/20/2025

What's new in Windows App SDK 0.5

You can choose between the **Stable**, **Preview**, and **Experimental** channels depending on your development needs — from reliable production builds to early access to upcoming features. Learn more about Release channels.

For the updated runtime and MSIX, see Downloads for the Windows App SDK.

Version 0.5

▼ New features and updates

> This release supports all stable channel features.

▼ Known issues and limitations

> ### Known issues and limitations
>
> This release has the following limitations and known issues:
>
> - **Desktop apps (C# or C++ desktop)**: This release is supported for use only in desktop apps (C++ or C#) that are packaged using MSIX. To use the Windows App SDK in unpackaged desktop apps, you must use the experimental release channel.
> - **.NET apps must target build 18362 or later**: Your TFM must be set to `net6.0-windows10.0.18362` or later, and your packaging project's `<TargetPlatformVersion>` must be set to 18362 or later. For more info, see the known issue on GitHub.

Last updated on 12/20/2025

Overview and release notes: WinUI 3 - Project Reunion 0.8 Preview (May 2021)

WinUI 3 is a native user experience (UX) framework for building modern Windows apps. It ships independently from the Windows operating system as a part of Project Reunion (now called the **Windows App SDK**). The 0.8 Preview release provides WinUI 3 templates in Visual Studio to help you start building apps with a WinUI 3-based user interface.

WinUI 3 - Project Reunion 0.8 Preview is a pre-release version of WinUI 3 that includes bug fixes, general improvements, and experimental features - some of which will be stabilized for the Windows App SDK 0.8 stable release in June 2021.

> ⓘ **Important**
>
> This preview release is intended for early evaluation and to gather feedback from the developer community. It should **NOT** be used for production apps.
>
> If you're looking to ship a production app using Project Reunion and WinUI 3, see **Overview and release notes: WinUI 3 - Project Reunion 0.5**.
>
> Please use the **WinUI GitHub repo** to provide feedback and log suggestions and issues.

> ⓘ **Note**
>
> **Project Reunion** is the previous code name for the **Windows App SDK**. This documentation still uses **Project Reunion** when referring to previous releases that used this code name.

Install WinUI 3 - Project Reunion 0.8 Preview

This version of WinUI 3 is available as part of the Project Reunion 0.8 Preview. To install, see:

Install tools for preview and experimental channels of the Windows App SDK

Since WinUI ships as a part of Project Reunion, you'll download the Project Reunion Visual Studio Extension (VSIX) to get started, which includes a set of developer tools and components. For more on the Project Reunion package, see Deploy apps that use the Windows App SDK. The

Windows App SDK VSIX includes WinUI 3 templates in Visual Studio that you'll use to build your WinUI app.

> **Note**
>
> To see WinUI 3 controls and features in action, you can clone and build the WinUI 3 Gallery from GitHub, or download it from the **Microsoft Store**.

Once you've set up your development environment, see WinUI 3 templates in Visual Studio to familiarize yourself with the available Visual Studio project and item templates.

For more information about getting started with building a WinUI app, see the following articles:

- Create your first WinUI 3 project
- Build a C# .NET app with WinUI 3 and Win32 interop

Aside from the limitations and known issues, building an app using the WinUI projects is similar to building a UWP app with XAML and WinUI 2. Therefore, most of the guidance documentation for UWP apps and the **Windows.UI** WinRT namespaces in the Windows SDK is applicable.

WinUI 3 API reference documentation is available here: WinUI 3 API Reference

WebView2

To use WebView2 with this WinUI 3 release, please download the Evergreen Bootstrapper or Evergreen Standalone Installer found on this page if you don't already have the WebView2 Runtime installed.

Windows Community Toolkit

If you're using the Windows Community Toolkit, download the latest version.

Visual Studio Support

In order to take advantage of the latest tooling features added into WinUI 3 like Hot Reload, Live Visual Tree, and Live Property Explorer, you must use a preview version of Visual Studio 2019 16.10. Please note that Visual Studio preview releases are pre-release products, so you may run into bugs and limitations when using preview versions of Visual Studio to build WinUI apps.

The table below shows the compatibility of Visual Studio 2019 versions with WinUI 3 - Project Reunion 0.5.

Expand table

VS Version	WinUI 3 - Project Reunion 0.5
16.8	No
16.9	Yes, but with no Hot Reload, Live Visual Tree, or Live Property Explorer
16.10 Previews	Yes, with all WinUI 3 tooling (in preview)

Updating your existing WinUI app

If you created an app with an earlier preview or stable version of WinUI 3, you can update the project to use WinUI 3 - Project Reunion 0.8 Preview. For instructions, see Update existing projects to the latest release of Project Reunion.

Major changes introduced in this release

- Pivot control is now available to use in this release.
- Most of the critical bug fixes from the Project Reunion v0.5.5, v0.5.6, and v0.5.7 servicing releases are included in this release as well. For servicing fixes that didn't make it into this preview, see Known Issues.

List of bugs fixed in WinUI 3 - Project Reunion 0.8 Preview

- x:Bind does not work in a custom MenuFlyoutItem (more generally, x:Bind in ControlTemplate doesn't work if the parent namescope also uses x:Bind)
  - Thanks to @Ihak for filing this issue on GitHub!
- Hide windowed popups when the top-level window (or island) moves
- StandardUICommand page in WinUI 3 Gallery not showing everything
- Mouse right-click in TextBox crashes the application
  - Thanks to @Herdubreid for filing this issue on GitHub!
- Context menu displayed at the wrong location for EditableComboBox
- Changing RichTextBlock Selection with touch makes selection indicator go away

- Microsoft C++ exception: winrt::hresult_error at memory location 0x... when NavigationView is being used
  - Thanks to @LeftTwixWand for filing this issue on GitHub!
- WinUI 3 error message needs rewording: "Cannot resolve 'Windows.metadata'. Please install the Windows Software Development Kit. The Windows SDK is installed with Visual Studio."
- VSM Setter quirk for raising exception has backwards logic
  - Thanks to @HppZ for filling this issue on GitHub!
- Move focus off the WebView2 (if necessary) when browser process crashes
- ProgressBar doesn't show difference between Paused and Error option
  - Thanks to @j0shuams for filing this issue on GitHub!
- PointerReplay constantly running
- NavigationView causes crash in UWP, Reunion 0.5 Preview
  - Thanks to @kalin-todorov for filing this issue on GitHub!
- AutoSuggestBox, ComboBox, and CommandBarFlyout aren't setting ShouldConstrainToRootBounds="false" on their popup
- WinUI 3 - Project Reunion 0.5 generating C++ exceptions for a C# managed app
  - Thanks to @Noemata for filing this issue on GitHub!
- WebView2 initial tab focus lost
- WebView2 crashes when DPI changes after Close()
- Change in Appearance of AppBarButton with Flyout on CommandBar in WinUI3
  - Thanks to @eleanorleffler for filing this issue on GitHub!

Features and capabilities introduced in past WinUI 3 Previews

The following features and capabilities were introduced in past WinUI 3 preview releases and continue to be supported in WinUI 3 - Project Reunion 0.8 Preview.

> **Note**

Some of the following features will continue to be a part of WinUI 3 previews, but may not be a part of the next supported release. These features are marked as experimental and will throw a warning when used in an app. APIs that are a part of the WinUI for UWP 2.6 pre-release are also marked as experimental in this release.

- Ability to create desktop apps with WinUI, including .NET for Win32 apps
- Preview-level support for building UWP apps
- RadialGradientBrush
- TabView updates
- Dark theme updates
- Improvements and updates to WebView2
  - Support for High DPI
  - Support for window resizing and moving
  - Updated to target more recent version of Edge
  - No longer necessary to reference a WebView2-specific Nuget package
- SwapChainPanel
- MRT Core Support
  - This makes apps faster and lighter on startup and provides quicker resource lookup.
- Arm64 Support
- Drag and drop inside and outside of apps
- RenderTargetBitmap (currently only XAML content - no SwapChainPanel content)
- Custom cursor support
- Off-thread input
- Improvements to our tooling/developer experience:
  - Live Visual Tree, Hot Reload, Live Property Explorer and similar tools
  - Intellisense for WinUI 3
- Improvements required for open source migration
- Custom titlebar capabilities: new Window.ExtendsContentIntoTitleBar and Window.SetTitleBar APIs that allow developers to create custom title bars in desktop apps.
- VirtualSurfaceImageSource support
- In-app acrylic
- Multi-window support in desktop apps
- Input validation

Provide feedback and suggestions

We welcome your feedback in the WinUI GitHub repo.

Limitations and known issues

The WinUI 3 - Project Reunion 0.8 Preview is just that, a preview. Please expect bugs, limitations, and other issues.

The following items are some of the known issues with WinUI 3 - Project Reunion 0.8. If you find an issue that isn't listed below, please let us know by contributing to an existing issue or filing a new issue through the WinUI GitHub repo.

Platform and OS support

WinUI 3 - Project Reunion 0.8 Preview is compatible with PCs running the Windows 10 October 2018 Update (version 1809 - build 17763) and newer.

Developer tools

- Only C# and C++/WinRT apps are supported.
- Desktop apps support .NET 6 (and later) and C# 9, and must be packaged in an MSIX app.
- No XAML Designer support.
- New C++/CX apps are not supported, however, your existing apps will continue to function (please move to C++/WinRT as soon as possible).
- Unpackaged desktop deployment is not supported.
- When running a desktop app using F5, make sure that you are running the packaging project. Hitting F5 on the app project will run an unpackaged app, which WinUI 3 does not yet support.

Missing platform features

- Xbox support
- HoloLens support
- Windowed popups
  - More specifically, the `ShouldConstrainToRootBounds` property always acts as if it's set to `true`, regardless of the property value.
- Inking support, including:
  - InkCanvas
  - HandwritingView
  - InkPresenter

- Background acrylic
- MediaElement and MediaPlayerElement
- MapControl
- SwapChainPanel does not support transparency
- AcrylicBrush and other effects using a CompositionBackdropBrush can't sample from a SwapChainPanel or WebView2.
- Global Reveal uses fallback behavior, a solid brush
- XAML Islands is not supported in this release
- Using WinUI 3 directly in an existing non-WinUI desktop app has the following limitation: The currently available path for migrating an existing app is to add a **new** WinUI 3 project to your solution, and adjust or refactor your logic as needed.
- Application.Suspending is not called in desktop apps. See API reference documentation on the Application.Suspending Event for more details.
- The UISettings.ColorValuesChanged Event and AccessibilitySettings.HighContrastChanged Event are no longer supported in desktop apps. This may cause issues if you are using it to detect changes in Windows themes.
- Previously, to get a CompositionCapabilities instance you would call CompositionCapabilites.GetForCurrentView(). However, the capabilities returned from this call were *not* dependent on the view. To address and reflect this, we've deleted the GetForCurrentView() static in this release, so now you can create a CompositionCapabilties object directly.
- CoreWindow, ApplicationView, CoreApplicationView, CoreDispatcher and their dependencies are not supported in desktop apps (see below)

CoreWindow, ApplicationView, CoreApplicationView, and CoreDispatcher in desktop apps

New in WinUI 3 Preview 4 and standard going forward, CoreWindow, ApplicationView, CoreApplicationView CoreDispatcher, and their dependencies are not available in desktop apps. For example, the Window.Dispatcher property is always **null**, but the **Window.DispatcherQueue** property can be used as an alternative.

These APIs only work in UWP apps. In past previews they've partially worked in desktop apps as well, but since Preview 4 they've been fully disabled. These APIs are designed for the UWP case where there is only one window per thread, and one of the features of WinUI 3 is to enable multiple in the future.

There are APIs that internally depend on existence of these APIs, which consequently aren't supported in a desktop app. These APIs generally have a static `GetForCurrentView` method. For example UIViewSettings.GetForCurrentView.

For more information on affected APIs as well as workarounds and replacements for these APIs, please see Support for WinRT APIs in desktop apps.

Known issues

- You may receive a build error due to mismatched versions of the .NET SDK and the winrt.runtime.dll. As a workaround, you can try the following:

  Explicitly set your .NET SDK to the correct version. To determine the correct version for your app, locate the `<TargetFramework>` tag in your project file. Using the Windows SDK build number that your app is targeting in the `<TargetFramework>` tag (such as 18362 or 19041), add the following item to your project file, then save your project:

  XML

  ```
  <ItemGroup>
      <FrameworkReference Update="Microsoft.Windows.SDK.NET.Ref"
  RuntimeFrameworkVersion="10.0.{Target Windows SDK Build Number}.16" />
      <FrameworkReference Update="Microsoft.Windows.SDK.NET.Ref"
  TargetingPackVersion="10.0.{Target Windows SDK Build Number}.16" />
  </ItemGroup>
  ```

  Note this workaround is required for .NET SDK 5.0.203 and earlier, but will not be required for .NET SDK 5.0.204 or 5.0.300.

- When using Visual Studio 2019 16.10 Preview 2, Live Visual Tree may cause a crash. To avoid this, update to the latest Visual Studio 2019 16.10 Preview.

- Window caption buttons may be misplaced when SetTitleBar is not set or null

WinUI 3 Gallery snapshot

The `winui3` and `winui3preview` branches are no longer available, and the XAML Controls Gallery repository has been renamed to microsoft/WinUI-Gallery. The sample app as updated for Project Reunion 0.8 Preview is preserved in a historical commit. This snapshot uses Project Reunion 0.8.0-preview and doesn't reflect the current WinUI 3 Gallery.

To download the historical snapshot, clone the repository and check out the preserved commit:

Console

```
git clone https://github.com/microsoft/WinUI-Gallery.git
cd WinUI-Gallery
git fetch origin refs/pull/698/head
git checkout 77d9e0a68bb67edb2b79fd1535fb18cf438b4265
```

See also

- Windows App SDK
- Windows App SDK Samples

Last updated on 10/02/2026

WinUI 3 - Project Reunion 0.5 Preview (March 2021) release notes

WinUI 3 is a native user experience (UX) platform for building modern Windows apps. This preview of WinUI 3 works with both desktop/Win32 and UWP apps, and includes Visual Studio project templates to help get started building apps with a WinUI-based user interface as well as a NuGet package that contains the WinUI libraries.

WinUI 3 - Project Reunion 0.5 Preview is the first release of WinUI 3 where it is provided as a part of the Project Reunion package (now called the **Windows App SDK**). Alongside that change, this preview release contains critical bug fixes, increased stability, and a few other general improvements (see **Capabilities introduced in WinUI 3 - Project Reunion 0.5 Preview**).

> **Important**
>
> This WinUI 3 preview is intended for early evaluation and to gather feedback from the developer community. It should **NOT** be used for production apps.
>
> Please use the WinUI GitHub repo to provide feedback and log suggestions and issues.

> **Note**
>
> **Project Reunion** is the previous code name for the Windows App SDK. This documentation still uses **Project Reunion** when referring to previous releases that used this code name.

Install WinUI 3 - Project Reunion 0.5 Preview

This version of WinUI 3 is available as part of the Project Reunion 0.5 Preview.

To install, follow the directions found in Install tools for preview and experimental channels of the Windows App SDK.

In contrast to past preview versions of WinUI 3, you'll download a Project Reunion VSIX package instead of a WinUI VSIX package. The Project Reunion VSIX includes WinUI 3 templates in Visual Studio that you'll use to build your WinUI app. Once you've completed your installation, the experience of developing a WinUI app should not change.

> **Note**
>
> You can also clone and build the WinUI 3 Preview version of the XAML Controls Gallery.

> **Note**
>
> To use WinUI 3 tooling such as Live Visual Tree, Hot Reload, and Live Property Explorer, you must enable WinUI 3 tooling with Visual Studio Preview Features as described in the instructions here.

Once you've set up your development environment, see WinUI 3 templates in Visual Studio to familiarize yourself with the available Visual Studio Project and Item templates.

For more information about getting started with the WinUI project templates, see the following articles:

- Create your first WinUI 3 project

Aside from the limitations and known issues, building an app using the WinUI projects is similar to building a UWP app with XAML and WinUI 2. Therefore, most of the guidance documentation for UWP apps and the **Windows.UI** WinRT namespaces in the Windows SDK is applicable.

WinUI 3 API reference documentation is available here: WinUI 3 API Reference

If you created a project using WinUI Preview 4, you can upgrade your project to use Project Reunion 0.5 Preview.

WebView2

To use WebView2 with this WinUI 3 preview, please download the Evergreen Bootstrapper or Evergreen Standalone Installer found on this page if you don't already have the WebView2 Runtime installed.

Windows Community Toolkit

If you're using the Windows Community Toolkit, download the latest version.

Visual Studio Support

In order to take advantage of the latest tooling features added into WinUI 3 like Hot Reload, Live Visual Tree, and Live Property Explorer, you must use the latest **preview** version of Visual Studio with the latest WinUI 3 preview and be sure to enable WinUI tooling in Visual Studio Preview Features, as described in the instructions here. The table below shows the compatibility of future versions with WinUI 3 - Project Reunion 0.5 Preview:

Expand table

VS Version	WinUI 3 - Project Reunion 0.5 Preview
16.8 RTM	No
16.9 Previews	Yes, with tooling
16.9 RTM	Yes, but no Hot Reload, Live Visual Tree, or Live Property Explorer
16.10 Previews	Yes, with tooling

Major changes introduced in this release

- WinUI 3 now ships as a part of the Project Reunion package, which will also be the mechanism for how our future supported releases will ship.
- In-app acrylic is now supported.
- The Pivot control is no longer supported, and has been deprecated in WinUI 3. We recommend using the NavigationView control for your in-app navigation scenarios.
- WinUI 3 and Project Reunion will only be supported down-level to Windows 10 version 1809 - it requires build 17763 or later.
- Preview features are now marked as experimental.
  - A preview feature is anything that will continue to be a part of WinUI 3 previews, but will not be a part of the next WinUI 3 supported release.
  - Preview features also include any experimental APIs that are a part of the WinUI for UWP 2.6 preview.
  - When building an app that uses a preview feature, your app will throw a warning.

List of bugs fixed in WinUI 3 - Project Reunion 0.5 Preview

Below is a list of user-facing bugs that the team has fixed since Preview 3. There has also been a lot of work going on surrounding stabilization and improving our testing.

- WinUI 3 error message needs rewording: "Cannot resolve 'Windows.metadata'. Please install the Windows Software Development Kit. The Windows SDK is installed with Visual Studio."
- App does not respond to theme changes in Windows when selecting Windows Default theme until restarted
- Exception on XamlDirect.CreateInstance call
  - Thanks to @BorzillaR for filing this issue on GitHub!
- ProgressBar doesn't show difference between Paused and Error option
- StandardUICommand list view item flyout shows in wrong position.
- Crash in desktop XamlControlsGallery when trying to reorder ListView items with touch
  - Thanks to @j0shuams for filing this issue on GitHub!
- Press and hold on RichTextBlock puts flyout in wrong place
- Left/Right arrows aren't moving focus through RadioButtons
  - Thanks to @vmadurga for filing this issue on Github!
- Narrator remains silent when user hits the down/up arrow keys to select the next/previous month/year/date in DatePicker
- NavigationView light-dismiss doesn't work in WinUI 3

New features and capabilities introduced in past WinUI 3 Previews

The following features and capabilities were introduced in WinUI 3 Previews 1-4 and continue to be supported in WinUI 3 - Project Reunion 0.5 Preview.

- Ability to create desktop apps with WinUI, including .NET for Win32 apps
- RadialGradientBrush
- TabView updates
- Dark theme updates
- Improvements and updates to WebView2
  - Support for High DPI
  - Support for window resizing and moving

    - Updated to target more recent version of Edge
    - No longer necessary to reference a WebView2-specific Nuget package
- SwapChainPanel
- MRT Core Support
    - This makes apps faster and lighter on startup and provides quicker resource lookup.
- Arm64 Support
- Drag and drop inside and outside of apps
- RenderTargetBitmap (currently only XAML content - no SwapChainPanel content)
- Custom cursor support
- Off-thread input
- Improvements to our tooling/developer experience:
    - Live Visual Tree, Hot Reload, Live Property Explorer and similar tools
    - Intellisense for WinUI 3
- Improvements required for open source migration
- Custom titlebar capabilities: new Window.ExtendsContentIntoTitleBar and Window.SetTitleBar APIs that allow developers to create custom title bars in desktop apps.
- VirtualSurfaceImageSource support

Provide feedback and suggestions

We welcome your feedback in the WinUI GitHub repo.

Limitations and known issues

The WinUI 3 - Project Reunion 0.5 Preview is just that, a preview. The scenarios around desktop apps are especially new. Please expect bugs, limitations, and other issues.

The following items are some of the known issues with WinUI 3 - Project Reunion 0.5 Preview. If you find an issue that isn't listed below, please let us know by contributing to an existing issue or filing a new issue through the WinUI GitHub repo.

Platform and OS support

WinUI 3 - Project Reunion 0.5 Preview is compatible with PCs running the Windows 10 October 2018 Update (version 1809 - build 17763) and newer.

Developer tools

- Only C# and C++/WinRT apps are supported

- Desktop apps support .NET 6 (and later) and C# 9, and must be packaged in an MSIX app
- UWP apps support .NET Native and C# 7.3
- Developer tools and Intellisense may not work properly in Visual Studio.
- No XAML Designer support
- New C++/CX apps are not supported, however, your existing apps will continue to function (please move to C++/WinRT as soon as possible)
- Support for multiple windows in desktop apps is in progress, but not yet complete and stable.
    - Please file a bug on our repo if you find new issues or regressions with multi-window behavior.
- Unpackaged desktop deployment is not supported
- When running a desktop app using F5, make sure that you are running the packaging project. Hitting F5 on the app project will run an unpackaged app, which WinUI 3 does not yet support.

Missing Platform Features

- Xbox support
- HoloLens support
- Windowed popups
    - More specifically, the `ShouldConstrainToRootBounds` property always acts as if it's set to `true`, regardless of the property value.
- Inking support
- Acrylic
- MediaElement and MediaPlayerElement
- MapControl
- RenderTargetBitmap for SwapChainPanel and non-XAML content
- SwapChainPanel does not support transparency
- Global Reveal uses fallback behavior, a solid brush
- XAML Islands is not supported in this release
- 3rd party ecosystem libraries will not fully function
- IMEs do not work
- CoreWindow, ApplicationView, CoreApplicationView, CoreDispatcher and their dependencies are not supported in desktop apps (see below)

CoreWindow, ApplicationView, CoreApplicationView, and CoreDispatcher in desktop apps

New in Preview 4 and standard going forward, CoreWindow, ApplicationView, CoreApplicationView CoreDispatcher, and their dependencies are not available in desktop apps. For example the Window.Dispatcher property is always null, but the Window.DispatcherQueue property can be used as an alternative.

These APIs only work in UWP apps. In past previews they've partially worked in desktop apps as well, but in Preview4 they've been fully disabled. These APIs are designed for the UWP case where there is only one window per thread, and one of the features of WinUI3 is to enable multiple.

There are APIs that internally depend on existence of these APIs, which consequently aren't supported in a desktop app. These APIs generally have a static `GetForCurrentView` method. For example UIViewSettings.GetForCurrentView.

For more information on affected APIs as well as workarounds and replacements for these APIs, please see WinRT API changes for desktop apps

Known issues

- There's an issue that's causing UWP apps to fail to launch on Windows 10 version 1809. The team is actively working on fixing this bug for the next preview release.
- Alt+F4 does not close desktop app windows.
- The UISettings.ColorValuesChanged Event and AccessibilitySettings.HighContrastChanged Event are no longer supported in desktop apps. This may cause issues if you are using it to detect changes in Windows themes.
- This release includes some experimental APIs, which will throw build warnings when used. These have not been thoroughly tested by the team and may have unknown issues. Please file a bug on our repo if you encounter any issues.
- Previously, to get a CompositionCapabilities instance you would call CompositionCapabilites.GetForCurrentView(). However, the capabilities returned from this call were *not* dependent on the view. To address and reflect this, we've deleted the GetForCurrentView() static in this release, so now you can create a CompositionCapabilties object directly.
- For C# UWP apps:

The WinUI 3 framework is a set of WinRT components which can be used from C++ (using C++/WinRT) or C#. When using C#, there are two versions of .NET, depending on the app model: when using WinUI in a UWP app, you're using .NET Native; when using in a desktop app, you're using .NET 6 or later (and C#/WinRT).

When using C# for a WinUI app in UWP, there are a few API namespace differences compared to C# in a WinUI desktop app or a C# WinUI for UWP app: some types are in a `Microsoft` namespace rather than a `System` namespace. For example, rather than the `INotifyPropertyChanged` interface being in the `System.ComponentModel` namespace, it's in the `Microsoft.UI.Xaml.Data` namespace.

This applies to:

- `INotifyPropertyChanged` (and related types)
- `INotifyCollectionChanged`
- `ICommand`

The `System` namespace versions still exist, but cannot be used with WinUI 3. This means that `ObservableCollection` doesn't work as-is in WinUI 3 C# UWP apps. For a workaround, see the CollectionsInterop sample in the preserved XAML Controls Gallery snapshot.

XAML Controls Gallery snapshot for WinUI 3 Preview

The `winui3preview` branch is no longer available, and the XAML Controls Gallery repository has been renamed to microsoft/WinUI-Gallery. The sample app as it existed for WinUI 3 - Project Reunion 0.5 Preview is preserved in a historical commit. This snapshot doesn't reflect the current WinUI 3 Gallery.

To download the historical snapshot, clone the repository and check out the preserved commit:

Console

```
git clone https://github.com/microsoft/WinUI-Gallery.git
cd WinUI-Gallery
git checkout 136c81578c6f30750efb268a4a8e2da871f182ef
```

Last updated on 08/21/2026

Overview and release notes: WinUI 3 - Project Reunion 0.5

WinUI 3 is a native user experience (UX) framework for building modern Windows apps. It ships independently from the Windows operating system as a part of Project Reunion (now called the **Windows App SDK**).

WinUI 3 - Project Reunion 0.5 is the first stable, supported version of WinUI 3 that can be used to create production apps that can be published to the Microsoft Store. This release consists of the stability updates and general improvements that allow WinUI 3 to be forward-compatible and production-ready.

> ⓘ **Note**
>
> **Project Reunion** is the previous code name for the Windows App SDK. This documentation still uses **Project Reunion** when referring to previous releases that used this code name.

Install WinUI 3 - Project Reunion 0.5

This new version of WinUI 3 is available as part of Project Reunion 0.5. To install, see:

Install tools for preview and experimental channels of the Windows App SDK

Now that WinUI ships as a part of Project Reunion, you'll download the Project Reunion Visual Studio Extension (VSIX) to get started, which includes a set of developer tools and components. For more on the Project Reunion package, see Deploy apps that use the Windows App SDK. The Windows App SDK VSIX includes WinUI 3 templates in Visual Studio that you'll use to build your WinUI app.

> ⓘ **Note**
>
> To see WinUI 3 controls and features in action, you can clone and build the WinUI 3 version of the XAML Controls Gallery from GitHub.

Once you've set up your development environment, see WinUI 3 templates in Visual Studio to familiarize yourself with the available Visual Studio Project and Item templates.

For more information about getting started with building a WinUI app, see the following articles:

- Create your first WinUI 3 project
- Build a C# .NET app with WinUI 3 and Win32 interop

Aside from the limitations and known issues, building an app using the WinUI projects is similar to building a UWP app with XAML and WinUI 2. Therefore, most of the guidance documentation for UWP apps and the **Windows.UI** WinRT namespaces in the Windows SDK is applicable.

WinUI 3 API reference documentation is available here: WinUI 3 API Reference

WebView2

To use WebView2 with this WinUI 3 release, please download the Evergreen Bootstrapper or Evergreen Standalone Installer found on this page if you don't already have the WebView2 Runtime installed.

Windows Community Toolkit

If you're using the Windows Community Toolkit, download the latest version.

Visual Studio Support

In order to take advantage of the latest tooling features added into WinUI 3 like Hot Reload, Live Visual Tree, and Live Property Explorer, you must use a preview version of Visual Studio 2019 16.10. Please note that Visual Studio preview releases are pre-release products, so you may run into bugs and limitations when using preview versions of Visual Studio to build WinUI apps.

The table below shows the compatibility of Visual Studio 2019 versions with WinUI 3 - Project Reunion 0.5.

Expand table

VS Version	WinUI 3 - Project Reunion 0.5
16.8	No
16.9	Yes, but with no Hot Reload, Live Visual Tree, or Live Property Explorer
16.10 Previews	Yes, with all WinUI 3 tooling (in preview)

Update your existing WinUI app

If you created an app with an earlier preview or release version of WinUI 3, you can update the project to use the latest release of WinUI 3 - Project Reunion 0.5. For instructions, see Update existing projects to the latest release of the Windows App SDK.

Major changes introduced in this release

Stable features

This release provides the stability and support to make WinUI 3 suitable for production apps that can ship to the Microsoft Store. It includes support and forward compatibility for most features introduced in past previews:

- Ability to create desktop apps with WinUI, including .NET for Win32 apps
- RadialGradientBrush
- TabView updates
- Dark theme updates
- Improvements and updates to WebView2
  - Support for High DPI
  - Support for window resizing and moving
  - Updated to target more recent version of Edge
  - No longer necessary to reference a WebView2-specific Nuget package
- SwapChainPanel
- MRT Core Support
  - This makes apps faster and lighter on startup and provides quicker resource lookup.
- Arm64 Support
- Drag and drop inside and outside of apps
- RenderTargetBitmap (currently only XAML content - no SwapChainPanel content)
- Custom cursor support
- Off-thread input
- Improvements to our tooling/developer experience:
  - Live Visual Tree, Hot Reload, Live Property Explorer and similar tools
  - Intellisense for WinUI 3
- Improvements required for open source migration
- Custom titlebar capabilities: new Window.ExtendsContentIntoTitleBar and Window.SetTitleBar APIs that allow developers to create custom title bars in desktop apps.
- VirtualSurfaceImageSource support
- In-app acrylic

Preview features

As this is a stable release of WinUI 3, preview features have been removed. You can still access preview features by using the latest preview version of WinUI 3 (see WinUI 3 - Project Reunion 0.8 Preview). Please note the following key features are still in preview, and work to stabilize them is ongoing:

- UWP support
  - This means you cannot build or run a UWP app using the WinUI 3 - Project Reunion 0.5 VSIX.
- Multi-window support in desktop apps
- Input validation

Provide feedback and suggestions

We welcome your feedback in the WinUI GitHub repo.

Limitations and known issues

The following items are some of the known issues with WinUI 3 - Project Reunion 0.5. If you find an issue that isn't listed below, please let us know by contributing to an existing issue or filing a new issue through the WinUI GitHub repo.

Platform and OS support

WinUI 3 - Project Reunion 0.5 is compatible with PCs running the Windows 10 October 2018 Update (version 1809 - build 17763) and newer.

Developer tools

- Only C# and C++/WinRT apps are supported
- Desktop apps support .NET 6 (and later) and C# 9, and must be packaged in an MSIX app
- No XAML Designer support
- New C++/CX apps are not supported, however, your existing apps will continue to function (please move to C++/WinRT as soon as possible)
- Unpackaged desktop deployment is not supported
- When running a desktop app using F5, make sure that you are running the packaging project. Hitting F5 on the app project will run an unpackaged app, which WinUI 3 does not

yet support.

Missing Platform Features

- Xbox support
- HoloLens support
- Windowed popups
  - More specifically, the `ShouldConstrainToRootBounds` property always acts as if it's set to `true`, regardless of the property value.
- Inking support, including:
  - InkCanvas
  - HandwritingView
  - InkPresenter
- Background acrylic
- MediaElement and MediaPlayerElement
- MapControl
- SwapChainPanel does not support transparency
- AcrylicBrush and other effects using a CompositionBackdropBrush can't sample from a SwapChainPanel or WebView2.
- Global Reveal uses fallback behavior, a solid brush
- XAML Islands is not supported in this release
- Using WinUI 3 directly in an existing non-WinUI desktop app has the following limitation: The currently available path for migrating an existing app is to add a **new** WinUI 3 project to your solution, and adjust or refactor your logic as needed.
- Application.Suspending is not called in desktop apps. See API reference documentation on the Application.Suspending Event for more details.
- CoreWindow, ApplicationView, CoreApplicationView, CoreDispatcher and their dependencies are not supported in desktop apps (see below)

CoreWindow, ApplicationView, CoreApplicationView, and CoreDispatcher in desktop apps

New in WinUI 3 Preview 4 and standard going forward, CoreWindow, ApplicationView, CoreApplicationView CoreDispatcher, and their dependencies are not available in desktop apps. For example, the Window.Dispatcher property is always **null**, but the **Window.DispatcherQueue** property can be used as an alternative.

These APIs only work in UWP apps. In past previews they've partially worked in desktop apps as well, but since Preview 4 they've been fully disabled. These APIs are designed for the UWP case where there is only one window per thread, and one of the features of WinUI 3 is to enable multiple in the future.

There are APIs that internally depend on existence of these APIs, which consequently aren't supported in a desktop app. These APIs generally have a static `GetForCurrentView` method. For example UIViewSettings.GetForCurrentView.

For more information on affected APIs as well as workarounds and replacements for these APIs, please see WinRT API changes for desktop apps

Known issues

- The UISettings.ColorValuesChanged Event and AccessibilitySettings.HighContrastChanged Event are no longer supported in desktop apps. This may cause issues if you are using it to detect changes in Windows themes.
- Previously, to get a CompositionCapabilities instance you would call CompositionCapabilites.GetForCurrentView(). However, the capabilities returned from this call were *not* dependent on the view. To address and reflect this, we've deleted the GetForCurrentView() static in this release, so now you can create a CompositionCapabilties object directly.
- You may receive a build error due to mismatched versions of the .NET SDK and the winrt.runtime.dll. As a workaround, you can try the following:

  Explicitly set your .NET SDK to the correct version. To determine the correct version for your app, locate the `<TargetFramework>` tag in your project file. Using the Windows SDK build number that your app is targeting in the `<TargetFramework>` tag (such as 18362 or 19041), add the following item to your project file, then save your project:

  ```
  ```

XML

```
<ItemGroup>
    <FrameworkReference Update="Microsoft.Windows.SDK.NET.Ref"
RuntimeFrameworkVersion="10.0.{Target Windows SDK Build Number}.16" />
    <FrameworkReference Update="Microsoft.Windows.SDK.NET.Ref"
TargetingPackVersion="10.0.{Target Windows SDK Build Number}.16" />
</ItemGroup>
```

Note this workaround is required for .NET SDK 5.0.203 and earlier, but will not be required for .NET SDK 5.0.204 or 5.0.300.

- When using Visual Studio 2019 16.10 Preview 2, Live Visual Tree may cause a crash. To avoid this, update to the latest Visual Studio 2019 16.10 Preview.

WinUI 3 Gallery snapshot

The `winui3` branch is no longer available, and the XAML Controls Gallery repository has been renamed to microsoft/WinUI-Gallery. The sample app as updated for Project Reunion 0.5.5 is preserved in a historical commit. This snapshot doesn't reflect the current WinUI 3 Gallery.

To download the historical snapshot, clone the repository and check out the preserved commit:

Console

```
git clone https://github.com/microsoft/WinUI-Gallery.git
cd WinUI-Gallery
git checkout e8d1cb2534f9f13d22dbe63171597a162ca01e00
```

Last updated on 08/21/2026

Latest Windows App SDK downloads

This page provides links to the *latest* downloads of the Windows App SDK for all three release channels (stable, preview, and experimental). See Windows App SDK release channels for more details on the Windows App SDK release channels.

Stable release

Windows App SDK 2.0

Expand table

Version	Runtime downloads
2.5.1 09/16/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.4.0 08/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.3.1 07/16/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.2.0 06/09/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.1.3 05/21/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0.1 04/29/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)

For older releases, see the Archive of Windows App SDK downloads.

Preview release

Windows App SDK 2.0 Preview

Expand table

Version	Runtime downloads
2.0 Preview2 (2.0.0-preview2) 03/31/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Preview1 (2.0.0-preview1) 02/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)

For older releases, see the Archive of Windows App SDK downloads.

Experimental release

Windows App SDK 2.0 Experimental

Expand table

Version	Runtime downloads
2.5 Experimental (2.5.4-experimental) 09/29/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.4 Experimental (2.4.1-experimental) 08/25/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.3 Experimental A (2.3.2-experimentalA) 07/30/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)

Version	Runtime downloads
2.2 Experimental 9 (2.2.2-experimental9) 06/09/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.1 Experimental 8 (2.1.4-experimental8) 05/21/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 7 (2.0.0-experimental7) 04/21/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 6 (2.0.0-experimental6) 03/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 5 (2.0.0-experimental5) 02/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 4 (2.0.0-experimental4) 01/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 3 (2.0.0-experimental3) 11/11/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 2 (2.0.0-experimental2) 11/05/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
2.0 Experimental 1 (2.0.250930001-experimental1) 10/02/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)

For older releases, see the Archive of Windows App SDK downloads.

Other downloads

Depending on your development scenario, you might also require the following.

SDK downloads

Expand table

Tool	Description
NuGet package	The **Microsoft.WindowsAppSDK** NuGet package provides access to APIs provided by the Windows App SDK. The NuGet package is included with the Visual Studio extension project templates. If you have an existing project in which you want to use the Windows App SDK, you can install the latest version of the Windows App SDK NuGet package in your project directly from Visual Studio. For setup instructions, see Use the Windows App SDK in an existing project.

Runtime downloads

Apps packaged with external location, and unpackaged apps, that use the Windows App SDK can use the standalone `.exe` runtime installer or MSIX packages to deploy the Windows App SDK package dependencies with them. For setup instructions, see Windows App SDK deployment guide for framework-dependent apps packaged with external location or unpackaged.

Expand table

Tool	Description
Installer	The standalone .exe installer, **WindowsAppRuntimeInstall.exe**, is available as a separate download beginning with Windows App SDK 1.0.1. It installs the Windows App SDK Runtime which includes the Framework, Main, Singleton and DDLM packages.
Redistributable	The Windows App Runtime Redistributable (**Microsoft.WindowsAppRuntime.Redist**) is a zip file that includes the installer and MSIX packages for all architectures (x64, x86, and ARM64).

Last updated on 09/29/2026

Archive of Windows App SDK downloads

This page provides download links to *older* releases of the Windows App SDK for the three release channels (stable, preview, and experimental). For the *latest* releases, see Latest downloads for the Windows App SDK.

SDK downloads:

Expand table

Tool	Description
Visual Studio extension	The Windows App SDK Visual Studio extension (VSIX) provides project and item templates to get started. You can choose from three versions of the extension: stable, preview, and experimental. See Set up your development environment for more details on how to install the extension.
NuGet package	The **Microsoft.WindowsAppSDK** NuGet package provides access to APIs provided by the Windows App SDK. The NuGet package is included with the Visual Studio extension project templates. If you have an existing project in which you want to use the Windows App SDK, you can install the latest version of the Windows App SDK NuGet package in your project directly from Visual Studio. For setup instructions, see Use the Windows App SDK in an existing project.

Runtime downloads:

Apps packaged with external location, and unpackaged apps, that use the Windows App SDK can use the standalone `.exe` runtime installer or MSIX packages to deploy the Windows App SDK package dependencies with them. For setup instructions, see Windows App SDK deployment guide for framework-dependent apps packaged with external location or unpackaged.

Expand table

Tool	Description
Installer	The standalone .exe installer, **WindowsAppRuntimeInstall.exe**, is available as a separate download beginning with Windows App SDK 1.0.1. It installs the Windows App SDK Runtime which includes the Framework, Main, Singleton and DDLM packages.
Redistributable	The Windows App Runtime Redistributable (**Microsoft.WindowsAppRuntime.Redist**) is a zip file that includes the installer and MSIX packages for all architectures (x64, x86, and ARM64).

To learn more, see the Windows App SDK release channels and the release notes provided below for each version.

Supported stable releases

Windows App SDK 1.8

Expand table

Version	Runtime downloads
1.8.12 (1.8.260921001) 09/24/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.11 (1.8.260804001) 08/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.10 (1.8.260710003) 07/14/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.9 (1.8.260529003) 06/09/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.8 (1.8.260508005) 05/12/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.7 (1.8.260416003) 04/21/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.6 (1.8.260317003) 03/18/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.5 (1.8.260209005) 02/10/2026	Installer (x64) Installer (x86)

Version	Runtime downloads
Release notes Source	Installer (arm64) Redistributable (ZIP)
1.8.4 (1.8.260101001) 01/13/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.3 (1.8.251106002) 11/11/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.2 (1.8.251003001) 10/14/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.1 (1.8.250916003) 09/22/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.8.0 (1.8.250907003) 09/09/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)

Out-of-support stable releases

Windows App SDK 1.7

Expand table

Version	Runtime downloads
1.7.9 (1.7.260224002) 03/10/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.8 (1.7.260208005) 02/10/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Version	Runtime downloads
1.7.7 (1.7.260114001) 01/14/2026 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.6 (1.7.251107005) 11/11/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.5 (1.7.251014001) 10/16/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.4 (1.7.250909003) 09/16/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.3 (1.7.250606001) 06/10/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.2 (1.7.250513003) 05/19/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.1 (1.7.250401001) 04/08/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7.0 (1.7.250310001) 03/18/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Windows App SDK 1.6

Expand table

Version	Runtime downloads
1.6.9 (1.6.250602001) 06/10/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.8 (1.6.250430001) 05/13/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.7 (1.6.250402001) 04/08/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.6 (1.6.250228001) 03/11/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.5 (1.6.250205002) 02/11/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.4 (1.6.250108002) 01/15/2025 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.3 (1.6.241114003) 11/18/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.1 (1.6.240923002) 10/01/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6.0 (1.6.240829007) 09/04/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Windows App SDK 1.5

Expand table

Version	Runtime downloads
1.5.9 (1.5.250108004) 11/12/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.8 (1.5.241107002) 11/12/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.7 (1.5.241001000) 10/08/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.6 (1.5.240802000) 08/13/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.5 (1.5.240627000) 07/09/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.4 (1.5.240607001) 06/11/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.3 (1.5.240428000) 05/01/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.2 (1.5.240404000) 04/09/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.1 (1.5.240311000) 03/12/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5.0 (1.5.240227000) 02/29/2024	Installer (x64) Installer (x86)

Version	Runtime downloads
Release notes Source	Installer (arm64) Redistributable

Windows App SDK 1.4

Expand table

Version	Runtime downloads
1.4.7 (1.4.240802001) 08/13/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.6 (1.4.240512000) 05/14/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.5 (1.4.240211001) 02/13/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.4 (1.4.231219000) 01/09/2024 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.3 (1.4.231115000) 11/16/2023 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.2 (1.4.231008000) 10/10/2023 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.1 (1.4.230913002) 09/19/2023 Release notes Source	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4.0 (1.4.230822000) 08/29/2023 Release notes	Installer (x64) Installer (x86)

Version	Runtime downloads
	Installer (arm64) Redistributable

Windows App SDK 1.3

Expand table

Version	Runtime downloads
1.3.3 (1.3.230724000) 07/25/2023 Release notes	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.3.2 (1.3.230602002) 06/13/2023 Release notes	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.3.1 (1.3.230502000) 05/09/2023 Release notes	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.3.0 (1.3.230331000) 04/12/2023 Release notes	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Windows App SDK 1.2

Expand table

Version	SDK downloads	Runtime downloads
1.2.5 (1.2.230313.1) 03/15/2023 Release notes		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.2.4 (1.2.230217.4) 02/22/2023 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Version	SDK downloads	Runtime downloads
1.2.3 (1.2.230118.102) 01/25/2023 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.2.2 (1.2.221209.1) 12/14/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.2.0 11/10/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Windows App SDK 1.1

Expand table

Version	SDK downloads	Runtime downloads
1.1.5 09/14/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.1.4 08/11/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.1.3 07/20/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.1.2 06/28/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.1.1 06/14/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Version	SDK downloads	Runtime downloads
1.1.0 05/24/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable

Windows App SDK 1.0

Expand table

Version	SDK downloads	Runtime downloads
Version 1.0.4 06/14/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.0.3 04/18/2022 Release notes	No new Visual Studio extensions.	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.0.2 04/05/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.0.1 03/15/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.0.0 11/16/2021 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension (no longer available) C++ Visual Studio 2019 extension(no longer available)	Redistributable

Windows App SDK 0.x

Expand table

Version	SDK downloads	Runtime downloads
0.8 06/24/2021 Release notes	Visual Studio extension(no longer available)	Redistributable

Version	SDK downloads	Runtime downloads
0.5 03/29/2021		
0.1 12/11/2020		

Preview releases

Expand table

Version	SDK downloads	Runtime downloads
1.8 Preview (1.8-preview) 08/19/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable (ZIP)
1.6 Preview2 (1.6.240821007-preview2) 08/22/2024 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6 Preview1 (1.6.240807006-preview1) 08/13/2024 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5 Preview1 (1.5.240205001-preview1) 02/06/2024 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.4 Preview 2 (1.4.0-preview2) 08/15/2023 Release notes		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.4 Preview1 (1.4.230628000-preview1) 07/06/2023 Release notes		Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.3 Preview 1 (1.3.0-preview1) 03/07/2023 Release notes		Installer (x64) Installer (x86) Installer (arm64) Redistributable

Version	SDK downloads	Runtime downloads
1.2 Preview2 10/04/2022 Release notes	No new Visual Studio extensions.	Installer (x64 Preview2) Installer (x86 Preview2) Installer (arm64 Preview2) Redistributable (Preview2)
1.2 Preview1 09/06/2022 Release notes	No new Visual Studio extensions.	Installer (x64 Preview1) Installer (x86 Preview1) Installer (arm64 Preview1) Redistributable (Preview1)
1.1.0 Preview 3 05/03/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.1.0 Preview 2 04/19/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.1.0 Preview 1 03/29/2022 Release notes	No new Visual Studio extensions.	Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.0 Preview 3 (1.0.0-preview3) 10/27/2021 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Redistributable
1.0 Preview 2 10/5/2021 Release notes		
1.0 Preview 1 09/17/2021 Release notes		

Experimental releases

Expand table

Version	SDK downloads	Runtime downloads
1.8 Experimental4 (1.8.250702007-experimental4) 07/08/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.8 Experimental3 (1.8.250610002-experimental3) 06/12/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.8 Experimental2 (1.8.250515001-experimental2) 05/19/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.8 Experimental1 (1.8.250410001-experimental1) 04/15/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7 Experimental3 (1.7.250127003-experimental3) 04/15/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7 Experimental2 (1.7.250109001-experimental2) 04/15/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.7 Experimental1 (1.7.241114004-experimental1) 04/15/2025 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.6 Experimental2 (1.6.240701003-experimental2) 07/02/2024 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable

Version	SDK downloads	Runtime downloads
1.6 Experimental1 (1.6.240531000-experimental1) 06/04/2024 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5 Experimental2 (1.5.240124002-experimental2) 01/25/2024 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.5 Experimental1 (1.5.231202003-experimental1) 12/06/2023 Release notes Source		Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.4 Experimental (1.4.0-experimental1) 05/23/2023 Release notes		Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.3 Experimental (1.3.0-experimental1) 02/06/2023 Release notes		Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.2 Experimental (1.2.0-experimental2) 09/21/2022 Release notes	C# Visual Studio 2022 extension C++ Visual Studio 2022 extension C# Visual Studio 2019 extension C++ Visual Studio 2019 extension	Installer (x64) Installer (x86) Installer (arm64) Redistributable
Version 1.2 Experimental (1.2.0-experimental1) 08/15/2022 Release notes	No new Visual Studio extensions.	Installer (x64) Installer (x86) Installer (arm64) Redistributable
1.0.0 Experimental 08/09/2021 Release notes		Redistributable
0.8 Experimental 05/27/2021 Release notes	Visual Studio extension	Redistributable

Last updated on 09/23/2026

Windows SDK



The Windows SDK (10.0.28000) for Windows 11 contains the latest platform headers, libraries, WinRT metadata, and build tools required to develop Windows applications. It supports both UWP and Win32 development and can target Windows 11, version 26H1, in addition to previous Windows releases. By using this SDK, developers gain access to the full set of Windows APIs and capabilities needed to build native and performant applications.

Additionally, the Windows App SDK builds on top of the Windows SDK by offering a consistent set of modern Windows APIs for desktop apps, allowing developers to adopt new capabilities without depending on a specific Windows version.

Download Windows SDK

To see what's new, check out the release notes.

Release channels

The Windows SDK is available in the following channels:

Expand table

Release channel	Description	Includes experimental APIs
Stable	Default SDK release intended for production use. Includes all current stable APIs and a limited set of experimental APIs.	✅
Preview	Preview SDK built on Windows Insider Preview releases. Provides access to newer and upcoming APIs, including experimental ones.	✅

Support and servicing

Expand table

SDK Version	Status	EOS Date	Notes
28000+	Supported		

SDK Version	Status	EOS Date	Notes
26100+	Supported		
22621	Out of support	2025-11-12	Aligned with 22621 Enterprise OS EOS
22000	Out of support		
20348	Out of support		Patched version shipped
19041	Out of support	2025-10-14	
18362 and before	Out of support		

System requirements

The Windows SDK has the following minimum system requirements:

Supported operating systems

- Windows 11, version 21h2 or higher: Home, Pro, Education, and Enterprise (LTSC is not supported for UWP)
- Windows 10, version 1507 or higher: Home, Pro, Education, and Enterprise (LTSB/LTSC and S mode are not supported for UWP)
- Windows Server 2022, Windows Server 2019, Windows Server 2016, and Windows Server 2012 R2 (Command line only)
- Windows 8.1
- Windows 7 SP1

(Not all tools are supported on earlier operating systems)

Hardware requirements

- 1.6 GHz or faster processor
- 1 GB of RAM
- 4 GB of available hard disk space

Additional SDK requirements

Installation on Windows 8.1 and earlier operating systems requires an Update for Universal C Runtime in Windows. To install through Windows Update, make sure you install the latest recommended updates and patches from Microsoft Update before you install the Windows SDK.

Last updated on 03/31/2026

What's new in the Windows SDK

In a new or existing Windows app, you can get the Windows SDK in several ways: install it from the installer or ISO, in the Visual Studio 2022 Installer, or by downloading the NuGet package. You can update the SDK by manually installing the new build, updating in Visual Studio or update the Nuget package

For the the latest builds, see Downloads for the Windows SDK.

28000 versions

Build 10.0.28000.2957

Released: **September, 2026**

▼ WinRT API additions and updates

> **Windows.ApplicationModel.Overlay**:
> New:
>
> - `OverlayContract`: New API contract for the `Windows.ApplicationModel.Overlay` namespace
> - `OverlayApp`, `OverlayAppOptions`, `OverlayCommand`, `OverlayCommandSource`: New overlay app and command types
> - `OverlayActivatedEventArgs`, `OverlayDisconnectedEventArgs`, `OverlayVisualTransitionEventArgs`: New overlay event argument types
> - `OverlayTransitionAnimation`, `OverlayVisualState`: New overlay visual state and transition types
>
> **Windows.Management.Update**:
> Updated:
>
> - `WindowsSoftwareUpdateProvider.ProviderIdentity`: Gets the identity of the software update provider
>
> **Windows.Networking.Vpn**:
> New:

- `VpnChannel.ConnectionDetail`, `VpnChannel.DisconnectReason`: Get details about the current connection and the reason it was disconnected
- `VpnChannel.SetErrorMessage(String, UInt32)`: New overload that sets an error message with an error code
- `VpnConnectionDetailsPreview`: Class exposing the connection `Type` and `Attempt` count
- `VpnConnectionTypePreview`, `VpnDisconnectReasonPreview`: Enumerations describing how a connection was started and why it ended
- `VpnManagementAgent.GetAlwaysOnActiveProfile`, `VpnManagementAgent.GetAlwaysOnActiveProfileAsync`, `VpnManagementAgent.SetAlwaysOnActiveProfile`, `VpnManagementAgent.SetAlwaysOnActiveProfileAsync`: Get or set the active always-on VPN profile

Windows.System:
New:

- `User.GetUserAgeRangeAsync`: Gets the age range associated with the user
- `User.GetAgeVerificationStatusAsync`: Gets the age verification status of the user
- `UserAgeRange`: Class exposing the `Lower` and `Upper` bounds of a user's age range
- `UserAgeVerificationStatus`: Enumeration describing the user's age verification status

> ⓘ **Note**
>
> The age signal APIs (`User.GetUserAgeRangeAsync`, `User.GetAgeVerificationStatusAsync`, `UserAgeRange`, and `UserAgeVerificationStatus`) are documented ahead of availability. They aren't enabled at runtime yet, and are planned to be turned on in a future release later this year. Calls made before then return no age data, so design your app to fall back to its default behavior.

Windows.System.RemoteDesktop.Provider:
Graduated from experimental to stable:

- `IRemoteDesktopRegistrarStatics2`: The `RemoteDesktopRegistrar` static members added in build 10.0.28000.2526 are no longer experimental

Windows.UI.Input:
New:

- `GamepadKeyRoutingConfiguration`: Runtime class that configures gamepad key routing

- `GamepadKeyRoutingConfiguration.IsSupported`: Indicates whether gamepad key routing configuration is supported
- `GamepadKeyRoutingConfiguration.IsKeyRoutingEnabled`: Gets whether gamepad key routing is enabled
- `GamepadKeyRoutingConfiguration.TrySetKeyRoutingEnabled`: Attempts to enable or disable gamepad key routing

Removed (experimental):

- `GamepadNavigationConfiguration`: The experimental class added in build 10.0.28000.2526 was removed and replaced by `GamepadKeyRoutingConfiguration`

Windows.ApplicationModel.Contacts:
Removed (experimental):

- `ContactAnnotationOperations.Activity`: The experimental enumeration value was removed

Windows.Devices.WiFiDirect.Services:
Deprecated:

- The Wi-Fi Direct Services APIs are now deprecated and might not work on all platforms

▼ Win32 API additions and updates

Cryptography (wincrypt.h)
New post-quantum cryptography (PQC) and CMS support:

- Composite ML-DSA and ML-KEM OIDs (for example `szOID_MLDSA44_ECDSA_P256_SHA256`, `szOID_MLKEM768_ECDH_P256_SHA3_256`) and matching `CRYPT32_COMPOSITE_*` algorithm names, pairing ML-DSA/ML-KEM with classical ECDSA/ECDH
- NIST AES-CCM and AES-GCM OIDs (`szOID_NIST_AES128_CCM` through `szOID_NIST_AES256_GCM`) and `CRYPT_OID_AES_EXTRA_INFO_*` values
- `szOID_RSA_HKDF_WITH_SHA256`: HKDF key-derivation OID
- Authenticated enveloped data: `CMSG_AUTH_ENVELOPED`, `CMSG_AUTH_ENVELOPED_ENCODE_INFO`, and `szOID_RSA_authEnvelopedData`
- KEM recipient info: `CMSG_KEM_RECIPIENT`, `CMSG_KEM_RECIPIENT_ENCODE_INFO`, `CMSG_KEM_RECIPIENT_INFO`, `CMSG_CTRL_KEM_DECRYPT`, `CMSG_CTRL_KEM_DECRYPT_PARA`, and `szOID_ORI_KEM`

- `CRYPT_MESSAGE_REQUIRE_OAEP_FLAG`, `CRYPT_MESSAGE_REQUIRE_AUTH_FLAG`: Message flags that require OAEP padding or authenticated encryption
- `szOID_PREHASH`: Prehash OID

Schannel (schannel.h)

New:

- `TlsSignatureAlgorithm_Mldsa`: ML-DSA TLS signature algorithm value

WebAuthn (webauthn.h)

New:

- `WEBAUTHN_COSE_ALGORITHM_ML_DSA_44`, `WEBAUTHN_COSE_ALGORITHM_ML_DSA_65`, `WEBAUTHN_COSE_ALGORITHM_ML_DSA_87`: ML-DSA COSE algorithm identifiers

Security and Identity (winnt.h)

New anti-tamper and process isolation support:

- `SECURITY_ISOLATION_CONFIGURATION` and its `_GENERAL`, `_DEBUG`, `_UI`, `_PROCESS`, and `_SIGNING` sub-structures: Isolation configuration passed at process creation
- `SECURITY_ANTITAMPER_POLICY` and related structures (`SECURITY_ANTITAMPER_POLICY_DEBUG`, `SECURITY_ANTITAMPER_POLICY_PROCESS`, `SECURITY_ANTITAMPER_POLICY_SIGNING`, `SECURITY_ANTITAMPER_POLICY_MODULES`, `SECURITY_ANTITAMPER_IDENTITY`, `SECURITY_ANTITAMPER_ENTITLEMENT`), plus `SECURITY_ANTITAMPER_POLICY_PRESENT_*` flags: Anti-tamper policy definitions
- `SECURITY_ANTI_TAMPER_AUTHORITY` and `SECURITY_ANTI_TAMPER_*_RID` values: Anti-tamper SID authority and RIDs
- `SYSTEM_MANDATORY_APP_ACCESS_ALLOWED_ACE_TYPE`, `SYSTEM_MANDATORY_ACCESS_ALLOWED_ACE_TYPE`, `MANDATORY_ACCESS_SECURITY_INFORMATION`: New ACE types and security information flag
- `TokenHasTamperProtection`, `TokenAppIdentitySid`, `TokenAppInstanceSid`, `TokenAppSuiteSid`, `TokenEntitlements`: New `TOKEN_INFORMATION_CLASS` values

Process Creation (WinBase.h / processthreadsapi.h)

New:

- `PROC_THREAD_ATTRIBUTE_ISOLATION_CONFIGURATION`: Process attribute that applies a `SECURITY_ISOLATION_CONFIGURATION`
- `PROC_THREAD_ATTRIBUTE_DESKTOP_APP_IDENTITY_POLICY`, `PROCESS_CREATION_DESKTOP_APP_IDENTITY_ENABLE`,

`PROCESS_CREATION_DESKTOP_APP_IDENTITY_DISABLE`: Control desktop app identity for a new process
- `QUEUE_USER_APC_FLAGS` now has bitwise flag operators

System Information (winnt.h)
New:

- `PROCESSOR_RELATIONSHIP.RelativePerformance`, `PROCESSOR_RELATIONSHIP.RelativeEfficiency`: Relative performance and efficiency of a processor
- `PF_APX_F_INSTRUCTIONS_AVAILABLE`, `PF_ARM_FEAT_ECV_INSTRUCTIONS_AVAILABLE`, `PF_ARM_FEAT_ECV_POFF_INSTRUCTIONS_AVAILABLE`: Processor feature flags
- `XSTATE_APX_F`, `XSTATE_MASK_APX_F`: Extended state for Intel APX
- `ARM64_CNTVCTSS_EL0`: ARM64 self-synchronized virtual count register
- `ATTRIBUTE_NODISCARD` (ntdef.h / winnt.h): Expands to `[[nodiscard]]` when the compiler supports it

DXCore (dxcore_interface.h)
New:

- `IDXCoreAdapterFactory2` with `GetReservedAcceleratorMemoryInfo`, plus `DXCoreReservedAcceleratorMemoryInfo` and `DXCoreReservedAcceleratorMemoryFlags`: Query reserved accelerator memory
- `DXCoreMemoryType::ReservedPartition`: New memory type value

Media Foundation (mfapi.h / dxva.h)
New:

- `MFT_DECODER_AUTO_POSTPROCESS`: Enables automatic post-processing after decode
- `MFSampleExtension_AverageBitrate`, `MFSampleExtension_ScenarioInfo`: Per-sample bitrate target and encoding scenario attributes
- `DXVA_PicParams_HEVC`: `EncoderBitrateHint`, `EncoderQualityHint`, and `ContentAwareHint` bit fields replace `ReservedBits6`

WinHTTP (winhttp.h)
New:

- `WinHttpConnectionSetProxyInfo`, `WinHttpConnectionDeleteProxyInfo`, `WINHTTP_CONNECTION_PROXY_INFO`, and related types: Set or delete the proxy for a connection

- `WinHttpConnectionUpdateIfIndexTable`, `WINHTTP_CONNECTION_IFINDEX_LIST`: Map connections to interface indexes
- `WinHttpConnectionSetPolicyEntries`, `WinHttpConnectionDeletePolicyEntries`, `WinHttpConnectionDeletePolicyEntriesByAppSid`, `WINHTTP_CONNECTION_POLICY_ENTRY_LIST`, `WINHTTP_CONNECTION_POLICY_TAG`: Manage connection policy entries
- `WINHTTP_CONNECTION_POLICY_ALLOW_DEFAULT_INTERFACE_SELECTION`, `WINHTTP_FEATURE_CONNECTION_PROXY_EX`: New policy flag and feature value

DNS (WinDNS.h)
New:

- `DnsConnectionDeletePolicyEntriesByAppSid`: Deletes DNS connection policy entries for an app SID
- `DNS_CONNECTION_POLICY_ALLOW_DEFAULT_INTERFACE_SELECTION`: New policy entry flag

Networking (ws2def.h / wlantypes.h)
New:

- `AI_EXTRA_UNTRUSTED_QUERY`: Reserved addrinfo hint flag
- `DOT11_AUTH_ALGO_PQC_1X` / `DOT11_AUTH_ALGORITHM_PQC_1X`: Wi-Fi PQC 802.1X authentication algorithm

Windows Hypervisor Platform (WinHvPlatformDefs.h)
Updated:

- `WHV_X64_PENDING_INTERRUPTION_REGISTER`: `NestedEvent` is renamed to `NestedGuestEvent`, and a `NestedException` bit is added

Shell Properties (propkey.h)
New:

- `PKEY_ItemFolderPathDisplayNarrowNamespaceRelative`: Namespace-relative folder path for display

Print (prnasntp.h)
New:

- `IID_IPrintAsyncCookie`, `IID_IPrintAsyncNewChannelCookie`, `IID_IAsyncGetSendNotificationCookie`, `IID_IAsyncGetSrvReferralCookie`: Interface IDs for asynchronous print notification

USB (usbspec.h)
New:

- `USB_DEVICE_CLASS_RVC`: USB device class code

Status and Error Codes (ntstatus.h / winerror.h / wuerror.h)
New:

- `STATUS_SPACES_API_VERSION_MISMATCH`, `STATUS_SPACES_METADATA_SIZE_INVALID`, `ERROR_SPACES_API_VERSION_MISMATCH`, `ERROR_SPACES_METADATA_SIZE_INVALID`: Storage Spaces error codes
- `FVE_E_WIM_TRUST_DISABLED_BY_POLICY`: BitLocker can't trust the WinRE image because the trust mechanism is disabled by policy
- `WU_E_UH_POSTREBOOT_VERSION_ABOVE_TARGET`, `WU_E_UH_POSTREBOOT_VERSION_BELOW_TARGET`, `WU_E_UH_COMMIT_FAILED`, `WU_E_UH_APPX_STAGING_TIMEOUT`: Windows Update handler error codes

---

Build 10.0.28000.2705

Released: **August, 2026**

▼ WinRT API additions and updates

Windows.Management.Update:
Updated:

- `WindowsSoftwareUpdateIdentity`, `WindowsSoftwareUpdateIdentityType`, `WindowsSoftwareUpdateCategory`: New identity and category types for describing a software update
- `WindowsUpdateManagerScanMode` and `WindowsUpdateManager.PerformScan`: Scanning now takes a scan mode instead of a scan options object (`WindowsUpdateManagerScanOptions` was removed)
- `WindowsSoftwareUpdate.IsSeeker`, `WindowsSoftwareUpdate.UpdateCategory`, `WindowsSoftwareUpdate.UpdateIdentity`, `WindowsSoftwareUpdate.Properties`, `WindowsSoftwareUpdate.GetPropertyValue`: Additional update metadata and property access

Windows.System.Profile:
New:

- `RetailInfo.IsDemoModeEnabledEx`: Indicates whether retail demo mode is enabled

Windows.UI.Notifications:
New:

- `ToastNotification.IsExpandableContentSupported`: Indicates whether expandable toast content is supported

▼ Win32 API additions and updates

Cryptography (wincrypt.h)
New:

- Endorsement-key enrollment v2 OIDs: `szOID_ENROLL_V2_CONTAINER_NAME`, `szOID_ENROLL_V2_EK_ALGORITHM`, `szOID_ENROLL_V2_EK_PARAMETER`, `szOID_ENROLL_V2_AIK_INFO`, `szOID_ENROLL_V2_ATTESTATION_STATEMENT`, `szOID_CERTSRV_V2_ATTESTATION_VERIFIED`

Media Foundation codecs (codecapi.h)
New:

- `CODECAPI_AVEncAV1EncoderOperatingMode` and `eAV1EncoderOperatingMode`: Configure the AV1 encoder operating mode

Direct3D 12 (d3d12.h)
New:

- `ID3D12DeviceStatistics` with `GetStateObjectStatistics`, plus `D3D12_STATE_OBJECT_STATISTICS` and `D3D12_CREATE_STATE_OBJECT_STATISTICS`: Query statistics for a state object

Remote Desktop Protocol (wtsprotocol.h / wtsdefs.h)
New:

- `IWRdsProtocolConnection3_Experimental` with `GetSerializedUserCredentialInteractively`: Retrieve a serialized user credential interactively
- `WRDS_CREDENTIAL_SCENARIO_INTERACTIVE_UNLOCK`, `WRDS_CREDENTIAL_SCENARIO_INTERACTIVE_LOGON`: Credential scenario identifiers

Storage (ntddstor.h / winioctl.h)
New:

- `GUID_STORPORT_PAGING_DEVICE_DUMP`, `STOR_PAGING_DEVICE_DUMP_ENTRY`, `STOR_PAGING_DEVICE_DUMP_DATA`: Paging device dump definitions for storage crash dumps

Shell Properties (propkey.h)
New:

- `PKEY_Devices_WiFiAware_IsSupported`: Indicates whether Wi-Fi Aware is supported

Status and Error Codes (ntstatus.h / winerror.h)
New:

- `STATUS_FIPS_FAILURE` and `NTE_FIPS_FAILURE`: A cryptographic operation failed FIPS validation
- `STATUS_DEVICE_NACKED` and `ERROR_DEVICE_NACKED`: The device returned a negative acknowledgement
- `UTC_E_WINRE_COMPLIANCE_CAPTURE_FAILED`, `UTC_E_WINRE_ETM_ENABLED`, and `PPF_E_*` diagnostics error codes

▼ COM API updates

Direct3D 12 (d3d12.idl)
New:

- `ID3D12DeviceStatistics` with `GetStateObjectStatistics`: Query statistics for a state object

Remote Desktop Protocol (wtsprotocol.idl)
New:

- `IWRdsProtocolConnection3_Experimental` with `GetSerializedUserCredentialInteractively`: Retrieve a serialized user credential interactively

---

Build 10.0.28000.2526

Released: **July, 2026**

▼ WinRT API additions and updates

Windows.Devices.Printers, **Windows.Graphics.Printing.PrintSupport**, **Windows.Graphics.Printing.PrintTicket**, **Windows.Graphics.Printing.Workflow**:

Graduated from experimental to stable:

- A number of IPP printing, print support, print ticket, and print workflow interfaces graduated from experimental to stable

Windows.System.Power:
New:

- `EnergySaverStatus2`: Enumeration describing the energy saver state (`Unknown`, `Off`, `Standard`, `HighSavings`)
- `PowerManager.EnergySaverStatus2`: Gets the current energy saver status
- `PowerManager.IsEnergySaverStatus2Supported`: Indicates whether `EnergySaverStatus2` is supported on the device
- `PowerManager.EnergySaverStatus2Changed`: Fires when the energy saver status changes

▼ WinRT Experimental API additions

Windows.System.RemoteDesktop.Provider:
New (experimental):

- `RemoteDesktopRegistrar.ConnectionCenterRequested`: Event raised when the connection center is requested

Windows.UI.Input:
New (experimental):

- `GamepadNavigationConfiguration`: Runtime class that configures gamepad-based navigation
- `GamepadNavigationConfiguration.NavigationEnabled`: Gets whether gamepad navigation is enabled
- `GamepadNavigationConfiguration.TrySetNavigationEnabled`: Attempts to enable or disable gamepad navigation
- `GamepadNavigationConfiguration.TryGetForCurrentProcess`: Gets the configuration for the current process
- `GamepadNavigationConfiguration.IsSupported`: Indicates whether gamepad navigation configuration is supported

▼ Win32 API additions and updates

Cryptography (bcrypt.h / ncrypt.h)
New post-quantum cryptography (PQC) support:

- `BCRYPT_COMPOSITE_MLDSA_ALGORITHM` / `BCRYPT_COMPOSITE_MLKEM_ALGORITHM`: Composite ML-DSA and ML-KEM algorithm identifiers, with corresponding algorithm handles, key blob types, magics, and parameter-set names
- `BCRYPT_PARAMETER_SET_NAMES` / `BCRYPT_PARAMETER_SET_KEY_BLOB`: Structures for enumerating and importing parameter-set keys
- `NCRYPT_COMPOSITE_MLDSA_ALGORITHM` / `NCRYPT_COMPOSITE_MLKEM_ALGORITHM` and algorithm groups: CNG key storage support for composite ML-DSA and ML-KEM
- `NCRYPT_PAD_PQDSA_FLAG`, `NCRYPT_MLDSA_EXTERNAL_MU_FLAG`: Padding and signing flags for PQC signatures

DirectX Kernel (d3dukmdt.h / d3dkmthk.h)
New:

- `DXGK_FEATURE_*_VERSION` defines and `DXGK_QUERYFEATURESUPPORT2_VERSION`: DXGK feature version identifiers
- `D3DDDI_SEGMENTPREFERENCE2`: Segment-preference structure with large-page preference support

Status Codes (ntstatus.h)
New:

- `STATUS_FVE_VOLUME_HANDLE_OPEN`: The BitLocker volume handle is open

RPC (rpcndr.h)
New:

- `USER_MARSHALUNMARSHAL_DONT_CLEANUP_UNMARSHALLED_OBJECT`: Marshaling flag that skips cleanup of an unmarshalled object

Process Creation (WinBase.h)
New:

- `PROC_THREAD_ATTRIBUTE_CONTAINMENT_CONFIGURATION`: Process/thread attribute for containment configuration

Windowing and Input (WinUser.h)
New:

- `WM_STOPINERTIA`, `WM_ENDINERTIA`: Window messages for inertia handling
- `GetPointerTouchpadInfo`, `GetPointerTouchpadInfoHistory`, `GetPointerFrameTouchpadInfo`, `GetPointerFrameTouchpadInfoHistory`: Retrieve precision-touchpad pointer information

- `CreateSyntheticPointerDevice2`: Creates a synthetic pointer device from `SYNTHETIC_DEVICE_CREATION_PARAMS`
- `SYNTHETIC_DEVICE_CREATION_OPTIONS`, `SYNTHETIC_DEVICE_CREATION_PARAMS`: Options and parameters for synthetic pointer device creation
- `TOUCHPAD_ACTION` and `InjectTouchpadAction`: Enumeration and function for injecting synthetic touchpad actions
- `ReportWindowContentInertia`: Reports the start and end of window content inertia
- `RegisterTouchpadCapableWindow`, `RegisterTouchpadCapableThread`, `SetMaxTouchpadSensitivity`: Opt in to and configure precision-touchpad input

Security (winnt.h)
New:

- `SECURITY_CONTAINMENT_CONFIGURATION`: Structure describing containment configuration

ReFS Attestation (wbcl.h / winioctl.h)
New:

- `SIPAEVENT_REFS_ATTESTATION_*` event types, headers, and structures: ReFS volume attestation SIPA event definitions
- `FSCTL_REFS_VOLUME_ATTESTATION_BIND_TO_TPM`, `FSCTL_REFS_VOLUME_ATTESTATION_QUERY_STATUS`, `FSCTL_REFS_VOLUME_ATTESTATION_RESET`: File system control codes for ReFS volume attestation

WRL Async (wrl/async.h)
Updated:

- Async completion handling reworked for thread safety

Build 10.0.28000.2270

Released: **June, 2026**

▼ WinRT API additions and updates

Windows.ApplicationModel.DataTransfer:
New:

- `DataPackageOperation.NewTarget` — Hint indicating the drag is over a new logical target (for example, individual files and folders on the desktop)

- `DataPackageOperation.BackgroundTarget` — Hint indicating the drag is over a logical target that represents the background (for example, the desktop itself)

Windows.Devices.Printers:
Graduated from experimental to stable:

- `ReplaceDevicePropertiesStatus` — Enumeration for the status of a replace-device-properties operation
- `IIppPrintDevice5` — IPP print device interface
- `IReplaceDevicePropertiesResult` — Result of a replace-device-properties operation

Windows.Graphics.Capture:
Graduated from experimental to stable:

- `IDirect3D11CaptureFrame3` — Direct3D 11 capture frame interface
- `IGraphicsCaptureSession7` — Graphics capture session interface

Windows.Graphics.Printing.PrintSupport:
Graduated from experimental to stable:

- `IPrintSupportEnterpriseManagementUIEventArgs` — Event arguments for print support enterprise management UI

Windows.UI.Input:
Updated:

- `TouchpadGlobalAction` — Renamed `ThreeFingerPressDown`/`FourFingerPressDown`/`FiveFingerPressDown` to `ThreeFingerPress`/`FourFingerPress`/`FiveFingerPress`, and `ThreeFingerPressUp`/`FourFingerPressUp`/`FiveFingerPressUp` to `ThreeFingerRelease`/`FourFingerRelease`/`FiveFingerRelease`

▼ Win32 API additions and updates

App Model (appmodel.h)
New:

- `PackageOrigin2` — Enumeration describing the origin of a staged package (adds `LineOfBusiness` and `SignedSBOM` values)
- `GetStagedPackageOrigin2` — Retrieves the `PackageOrigin2` of a staged package
- `AppModelPackageFeature` — Enumeration of queryable package features (`SignedSBOM`)

- `IsPackageFeatureSupported` — Queries whether a package feature is currently available or enabled

Updated:

- `PackageOrigin` — Adds `PackageOrigin_LineOfBusiness` and `PackageOrigin_SignedSBOM` values

Bug Codes (bugcodes.h)
New:

- `ASSERTBUGCODE_I3CHOST_DRIVER` — Assertion bug code for the I3C host driver

Codec API (codecapi.h)
New:

- `eAVDecDDOperationalMode_PORTABLE16` — Dolby Digital portable operational mode with dialogue at -16 dBFS

Device Class GUIDs (devguid.h)
New:

- `GUID_DEVCLASS_MIDIENDPOINT`, `GUID_DEVCLASS_PROXIMITY`, `GUID_DEVCLASS_SDHOST`, `GUID_DEVCLASS_SECURITYDEVICES`, `GUID_DEVCLASS_SOFTWAREDEVICE`, `GUID_DEVCLASS_USBDEVICE`, `GUID_DEVCLASS_USBFUNCTIONCONTROLLER` — Device setup class GUIDs

DHCP (DhcpCSdk.h / Dhcpv6cSdk.h)
New:

- `OPTION_CAPTIVE_PORTAL` — DHCPv4 captive portal option (114)
- `DHCPV6_OPTION_CAPTIVE_PORTAL` — DHCPv6 captive portal option (103)

Direct3D 11 (d3d11.h)
New:

- `D3D11_DECODER_PROFILE_APV_VLD_422_10`, `..._422_12`, `..._444_10`, `..._444_12`, `..._4444_10`, `..._4444_12`, `..._400_10` — Advanced Professional Video (APV) decoder profile GUIDs (RFC 9924)

DirectX Video Acceleration (dxva.h)
New:

- `DXVA_ModeAPV_VLD_422_10`, `..._422_12`, `..._444_10`, `..._444_12`, `..._4444_10`, `..._4444_12`, `..._400_10` — Advanced Professional Video (APV) DXVA profile GUIDs (RFC 9924)
- `DXVA_PicEntry_APV` and related APV DXVA structures — Picture and slice data structures for APV decode

EAP Types (eaptypes.h)
New:

- `EAP_FLAG_DISABLE_SESSION_RESUMPTION` — Flag to disable TLS session resumption

Media Engine (mfmediaengine.h)
New:

- `IMFMediaEngineVideoRendererEffect` — Interface that allows an app to set a custom video renderer effect

TPM / Key Storage (ncrypt.h)
New:

- `NCRYPT_PCP_SDDIDK_KEY` — Platform Crypto Provider SDDI DK key

Security Packages (NTSecPKG.h)
New:

- `SecpkgFailureReason_GlobalSPN` — Failure reason indicating the target name resolved to a global SPN

OLE (oleidl.h)
New:

- `DROPEFFECT_NEWTARGET` — Drop effect hint that the drag is over a new logical target (0x40000000)
- `DROPEFFECT_BACKGROUNDTARGET` — Drop effect hint that the drag is over a background logical target (0x20000000)

Windows Definitions (winnt.h)
New:

- `SECURITY_AGENTIC_PLATFORM_BASE_RID` — Base relative identifier (RID) for the agentic platform

Windows Hypervisor Platform (WinHvPlatformDefs.h)

New:

- `LowerVtlGuestRequestSupport` — Synthetic processor feature bit for lower-VTL guest request support

Windows Update Errors (wuerror.h)

New:

- `WU_E_FEATURE_CONTAINMENT_NOT_ENABLED` — Unexpected code execution when the feature is not enabled
- `WU_E_UPDATE_SESSIONDATA_CONFLICT` — The specified session data conflicts with the existing value
- `WU_E_DS_UNKNOWNSERVICE_NOTFOUND` — The service was not found in the data store
- `WU_E_DS_UNKNOWNSERVICE_DELETEPENDING` — The service is pending deletion from the data store
- `WU_E_DS_UNKNOWNSERVICE_REMOVED` — The service has been removed from the data store
- `WU_E_UH_TIMEOUT` — The update handler operation timed out
- `WU_E_UH_DRV_PREINSTALL_OPERATION_FAILED` — The driver update handler's pre-install operation failed

▼ COM API updates

Direct3D 11 (d3d11.idl)

New:

- `D3D11_DECODER_PROFILE_APV_VLD_422_10`, `..._422_12`, `..._444_10`, `..._444_12`, `..._4444_10`, `..._4444_12`, `..._400_10` — Advanced Professional Video (APV) decoder profile GUIDs (RFC 9924)

Media Engine (mfmediaengine.idl)

New:

- `IMFMediaEngineVideoRendererEffect` — Interface that allows an app to set a custom video renderer effect

OLE (oleidl.idl)

New:

- `DROPEFFECT_NEWTARGET` — Drop effect hint that the drag is over a new logical target (0x40000000)

> - `DROPEFFECT_BACKGROUNDTARGET` — Drop effect hint that the drag is over a background logical target (0x20000000)

Build 10.0.28000.2114

Released: **May, 2026**

▼ WinRT API additions and updates

> **Windows.Devices.Printers**:
> Graduated from experimental to stable:
>
> - `IppAttributeConverter.ConvertPrintTicketToIppAttributesForPrinter(String, WorkflowPrintTicket, String)` — Print ticket to IPP attribute conversion
>
> **Windows.Graphics.Printing.Workflow**:
> Graduated from experimental to stable:
>
> - `PrintWorkflowPrinterJob.IsPassthroughJobWithAttributes` — Property indicating whether a print job is a passthrough job (renamed from `IsPassthroughJob`)

▼ Win32 API additions and updates

> **Direct3D Kernel Mode (d3dkmthk.h)**
> New:
>
> - `_D3DKMT_QUERYFEATUREINTERFACE` — Query feature interface structure
>
> **Status Codes (ntstatus.h)**
> New:
>
> - `STATUS_SMB_ALTERNATIVE_PORT_CONFLICT` — SMB alternative port conflict status code
>
> **Power (poclass.h)**
> New:
>
> - `BATTERY_TEST_EXEMPT` — Battery test exemption flag
>
> **Error Codes (winerror.h)**
> New:

- `FVE_E_METHOD_MISMATCH` — Full Volume Encryption method mismatch error
- `ERROR_SMB_ALTERNATIVE_PORT_CONFLICT` — SMB alternative port conflict error

TPM / Key Storage (ncrypt.h)

New:

- `NCRYPT_PCP_SDDIDK_OPERATION` — Platform Crypto Provider SDDI DK operation
- `NCRYPT_PCP_AIKSTORE_PROPERTY` — AIK store property
- `NCRYPT_PCP_EKSTORE_PROPERTY` — EK store property
- `NCRYPT_PCP_SDDIDK_CONTEXT_PROPERTY` — SDDI DK context property

Content Index (NTQuery.h)

New:

- `CI_VERSION_QUERY_METADATA` — Content index version query metadata

Security Packages (NTSecPKG.h)

New:

- `SECPKG_CALL_AGENT_LOGON` — Security package agent logon call flag

Shell (shellapi.h)

New:

- `ABC_OVERLAYDESKTOPICONS` — Overlay desktop icons flag

Windows Filtering Platform (fwpmtypes.h / fwpvi.h)

Removed:

- `FWPM_LAYER_STATISTICS0_`, `FWPM_STATISTICS0_` — Filtering platform statistics structures
- `FWPM_LAYER_STATISTICS`, `FWPM_STATISTICS` — Filtering platform statistics defines

Terminal Services Virtual Channels (tsvirtualchannels.h)

Removed:

- `IWTSRemoteAppWindowInfoService` — Remote app window info service interface and related definitions

▼ COM API updates

Windows Filtering Platform (fwpmtypes.idl)

Removed:

- `FWPM_LAYER_STATISTICS0_`, `FWPM_STATISTICS0_` — Filtering platform statistics structures

Terminal Services Virtual Channels (tsvirtualchannels.idl)
Removed:

- `IWTSRemoteAppWindowInfoService` — Remote app window info service interface
- `GetLocalHwnd` — Get local window handle function

Build 10.0.28000.1839

Released: **April, 2026**

▼ WinRT API additions and updates

Windows.Devices.Haptics (UniversalApiContract 19.0):
New properties on `KnownSimpleHapticsControllerWaveforms`:

- `Collide` — Waveform ID for collision haptic feedback
- `Align` — Waveform ID for alignment haptic feedback
- `Step` — Waveform ID for step haptic feedback
- `Grow` — Waveform ID for growth haptic feedback

Windows.Devices.Printers:
Graduated from experimental to stable:

- `VirtualPrinterInstallationStatus` enum — Installation status values including `InstallationSucceeded`
- `VirtualPrinterPreferredInputFormat` enum — Preferred input format values including `OpenXps`
- `IVirtualPrinterInstallationParameters` interface
- `IVirtualPrinterInstallationResult` interface
- `IVirtualPrinterManagerStatics` interface
- `IVirtualPrinterSupportedFormat` interface
- `IVirtualPrinterSupportedFormatFactory` interface
- `VirtualPrinterInstallationParameters` runtime class
- `VirtualPrinterInstallationResult` runtime class
- `VirtualPrinterSupportedFormat` runtime class

Windows.Media.ClosedCaptioning (UniversalApiContract 15.0):
New types:

- `ClosedCaptionTheme` runtime class — Represents a closed caption theme with customization support
- `IClosedCaptionTheme` interface — Properties: `Id`, `DisplayName`, `FontColor`, `ComputedFontColor`, `FontOpacity`, `FontSize`, `FontStyle`, `FontEffect`, `BackgroundColor`, `ComputedBackgroundColor`, `BackgroundOpacity`, `RegionColor`, `ComputedRegionColor`, `RegionOpacity`
- `IClosedCaptionThemeStatics` interface — Methods: `GetAvailableThemes`, `GetSelectedTheme`, `TrySetSelectedTheme`; Events: `ThemesChanged`, `SelectedThemeChanged`

▼ WinRT Experimental API additions

Windows.Storage.Search (UniversalApiContract 19.0):
New interface:

- `IQueryOptionsAdditionalSearchSources` — Adds `IncludeCloudProviders` and `IncludeLocalSemanticIndex` properties to `QueryOptions`

▼ Win32 API additions and updates

Bluetooth Hands-Free Profile (bthdef.h)
Added comprehensive Bluetooth Hands-Free Profile (HFP) feature constants:

- `HFP_AG_SDP_SUPPORTED_FEATURE_*` — Audio Gateway SDP supported feature flags for three-way calling, echo cancellation, voice recognition, in-band ring tone, voice tag, wide-band speech, and more
- `HFP_AG_BRSF_SUPPORTED_FEATURE_*` — Audio Gateway BRSF supported feature flags for three-way calling, echo cancellation, voice recognition, codec negotiation, and more
- `HFP_HF_SDP_SUPPORTED_FEATURE_*` — Hands-Free SDP supported feature flags for echo cancellation, three-way calling, CLI presentation, voice recognition, and more
- `HFP_HF_BRSF_SUPPORTED_FEATURE_*` — Hands-Free BRSF supported feature flags for echo cancellation, three-way calling, voice recognition, codec negotiation, and more
- `HFP_NETWORK_NO_ABILITY_TO_REJECT`, `HFP_NETWORK_ABILITY_TO_REJECT` — Network call rejection capability flags

HID Usages (hidusage.h)
New haptics usage values:

- `HID_USAGE_HAPTICS_WAVEFORM_COLLIDE`, `HID_USAGE_HAPTICS_WAVEFORM_ALIGN`, `HID_USAGE_HAPTICS_WAVEFORM_STEP`, `HID_USAGE_HAPTICS_WAVEFORM_GROW`

NVMe (nvme.h)

Fixed typo:

- `NVME_LOG_PAGE_BOOT_PARTITON` renamed to `NVME_LOG_PAGE_BOOT_PARTITION`

Security / SSPI (sspi.h)

New GUID:

- `SEC_WINNT_AUTH_DATA_TYPE_PLACEHOLDER` — Placeholder authentication data type

Security / LSA (ntlsa.h)

New agent-based authentication APIs:

- `LsaCreateAgentAccount` — Creates an agent account
- `LsaRetrieveAgentLogonCredential` — Retrieves agent logon credentials
- `LsaEnumerateAgentAccounts` — Enumerates agent accounts
- `LsaDeleteAgentAccount` — Deletes an agent account
- `LsaGetAgentOwner` — Gets the agent owner
- `LSA_AGENT_LOGON_CREDENTIAL` — Agent logon credential struct
- `LSA_AGENT_ACCOUNT_INFO` — Agent account information struct
- `LSA_AGENT_ACCOUNT_LIST` — List of agent accounts struct

Security / Authentication (NTSecPKG.h)

New definitions:

- `KSecAllocateContextBuffer` — Function for allocating security context buffers
- Added `extern "C"` guards for C++ compatibility

Content Indexing (NTQuery.h)

New define:

- `CI_VERSION_CORRID` — Content index correlation ID version constant

Text Services (TextStor.h)

New defines:

- `TS_SD_DISABLEWRITINGSUGGESTIONS` — Flag to disable writing suggestions
- `TS_SS_MULTILINE` — Flag for multiline text store support

WRL Async (wrl/async.h)

Updated:

- Async completion handling reworked for thread safety using `_InterlockedCompareExchange` and reference counting (`cCompleteDelegateRefCount_`)

▼ COM API updates

Edition Upgrade Helper (EditionUpgradeHelper.idl)
Updated method:

- `IClipServiceNotificationHelper::ShowToast` — Parameter list simplified from 5 BSTR parameters to `void`

Text Services Framework (TextStor.idl)
New constants:

- `TS_SD_DISABLEWRITINGSUGGESTIONS` — Flag to disable writing suggestions
- `TS_SS_MULTILINE` — Flag for multiline text store support

Build 10.0.28000.1721

Released: **March, 2026**

This is a major version bump to the **28000** SDK series.

▼ WinRT API additions and updates

Windows.Devices.Haptics (UniversalApiContract 19.0):
New types:

- `HapticDeviceType` enum — Defines haptic device types: `None`, `Generic`, `Pen`, `Touchpad`, `Mouse`
- `HapticsControllerOverrideToken` struct — Token for managing haptics controller overrides
- `IInputHapticsManager` interface — Provides per-thread haptics management with methods for sending waveforms, controlling duration/play count, stopping feedback, and overriding haptics controllers
- `IInputHapticsManagerStatics` interface — Static methods: `IsSupported`, `IsHapticDevicePresent`, `GetForCurrentThread`, `TryGetForThread`
- `InputHapticsManager` runtime class

Windows.ApplicationModel.Contacts.Provider (UniversalApiContract 19.0):

New types:

- `IContactProvider` interface — Provides `GetContactFromRemoteIdAsync` method and `ContactListId` property for contact provider scenarios

▼ WinRT Experimental API additions (UniversalApiContract 20.0)

Windows.Devices.Printers:

New types:

- `IppAttributeGroupKind` enum — Defines IPP attribute group kinds: `Printer`, `Job`, `Operation`
- `IIppAttributeConverterStatics` interface — Provides `ConvertPrintTicketToIppAttributesForPrinter`, `ConvertBufferToIppAttributes`, `ConvertIppAttributesToBuffer`
- `IppAttributeConverter` runtime class
- `IPdlPassthroughProvider2` interface — Adds `IsPassthroughWithJobAttributesSupported` property and `StartPrintJobWithIppJobAttributes` method

Windows.Graphics.Printing.PrintSupport:

New types:

- `IPrintSupportPrintDeviceCapabilitiesChangedEventArgs5` interface — Adds `SetPdlPassthroughWithJobAttributesSupported` method

Windows.Graphics.Printing.Workflow:

New types:

- `IPrintWorkflowPrinterJob3` interface — Adds `IsPassthroughJob` property, `GetPassthroughJobAttributes`, and `GetPassthroughJobOperationAttributes` methods

Windows.UI.Shell.Tasks:

New types:

- `IAppTaskInfo2` interface — Adds `Id` and `HiddenByUser` properties and `UpdateDeepLink` method
- `AppTaskContract` version bumped from 1.0 to 2.0

▼ Win32 API additions and updates

Video Encoding — D3D12 Reconstructed Picture Output (codecapi.h / mfapi.h)

New enum and properties for D3D12-based video encoder reconstructed picture output:

- `eAVEncVideoD3D12ReconstructedPictureOutputMode` enum — Values: `None`, `Copy`, `Shared`
- `CODECAPI_AVEncVideoD3D12ReconstructedPictureOutputMode` codec API GUID
- `MFSampleExtension_VideoEncodeD3D12ReconstructedPicture` Media Foundation sample extension GUID

HTTP Server API (http.h)

New server property and struct:

- `HttpServerRequestInfoProperty` (=19) — New server property to enable optional request info fields
- `HTTP_REQUEST_INFO_PROPERTY_INFO` struct with `HTTP_REQUEST_INFO_FLAG_INITIAL_TTL` flag
- `HttpFeatureTlsHandshakePerformanceCounters` (=17) — New HTTP feature for TLS handshake performance counters

Graphics / Display Driver (d3dukmdt.h)

New driver feature:

- `DXGK_DRIVER_FEATURE_PANEL_BUFFER_CONTROL` (=46) and corresponding `DXGK_FEATURE_PANEL_BUFFER_CONTROL` feature ID

Event Tracing (evntprov.h / evntcons.h)

New definitions:

- `EVENT_DATA_DESCRIPTOR_TYPE_RESERVED1` (=4) — Reserved event data descriptor type
- `EventProviderSetReserved2` — New value in `EVENT_INFO_CLASS` enum
- `EVENT_HEADER_FLAG_RESERVED1` (0x0400) — New event header flag

Storage Provider Properties (propkey.h)

New property:

- `PKEY_StorageProviderUserAccountKind` — Identifies the account kind (Unknown, Consumer, Business) for the authenticated storage provider user

User Input (WinUser.h)

New function:

- `ConvertPrimaryPointerToMouseDrag` — Converts primary pointer input to a mouse drag operation

Updated struct:

- `TOUCHPAD_PARAMETERS_V2` — Improved C/C++ layout compatibility

WebAuthn Plugin API (webauthnplugin.h)
Graduated from experimental to stable:

- `WEBAUTHN_PLUGIN_ADD_AUTHENTICATOR_OPTIONS_2` (previously `EXPERIMENTAL_WEBAUTHN_PLUGIN_ADD_AUTHENTICATOR_OPTIONS_2`)
- `WEBAUTHN_PLUGIN_UPDATE_AUTHENTICATOR_DETAILS_2` (previously `EXPERIMENTAL_WEBAUTHN_PLUGIN_UPDATE_AUTHENTICATOR_DETAILS_2`)
- `WEBAUTHN_PLUGIN_USER_VERIFICATION_REQUEST_2` (previously `EXPERIMENTAL_WEBAUTHN_PLUGIN_USER_VERIFICATION_REQUEST_2`)
- `WebAuthNPluginAddAuthenticator2`, `WebAuthNPluginUpdateAuthenticatorDetails2`, `WebAuthNPluginPerformUserVerification2` functions

Rust Bindgen Compatibility (ntdef.h / winnt.h)
Updated:

- `DECLSPEC_NOINITALL` macro now excludes Rust bindgen passes via `!defined(RUST_BINDGEN)`

▼ COM API updates

Shell Object IDL (ShObjIdl_core.idl)
Updated:

- `IAttachmentExecute2` — Method comments corrected: `Save2()` renamed to `SaveNoVirusCheck()`, `SaveWithUI2()` renamed to `SaveWithUINoVirusCheck()`

26100 versions

Build 10.0.26100.9457

Released: **September, 2026**

▼ WinRT API additions and updates

Windows.ApplicationModel.Overlay:
New:

- `OverlayContract`: New API contract for the `Windows.ApplicationModel.Overlay` namespace
- `OverlayApp`, `OverlayAppOptions`, `OverlayCommand`, `OverlayCommandSource`: New overlay app and command types
- `OverlayActivatedEventArgs`, `OverlayDisconnectedEventArgs`, `OverlayVisualTransitionEventArgs`: New overlay event argument types
- `OverlayTransitionAnimation`, `OverlayVisualState`: New overlay visual state and transition types

Windows.Graphics.DirectX:
New:

- `DirectXColorSpace.RgbFullG10NoneP2020`, `DirectXColorSpace.RgbFullG22ExtNoneP709`, `DirectXColorSpace.RgbFullG22NoneP3`: New color space values

Windows.Networking.Vpn:
New:

- `VpnChannel.ConnectionDetail`, `VpnChannel.DisconnectReason`: Get details about the current connection and the reason it was disconnected
- `VpnChannel.SetErrorMessage(String, UInt32)`: New overload that sets an error message with an error code
- `VpnConnectionDetailsPreview`: Class exposing the connection `Type` and `Attempt` count
- `VpnConnectionTypePreview`, `VpnDisconnectReasonPreview`: Enumerations describing how a connection was started and why it ended
- `VpnManagementAgent.GetAlwaysOnActiveProfile`, `VpnManagementAgent.GetAlwaysOnActiveProfileAsync`, `VpnManagementAgent.SetAlwaysOnActiveProfile`, `VpnManagementAgent.SetAlwaysOnActiveProfileAsync`: Get or set the active always-on VPN profile

▼ Win32 API additions and updates

Security and Identity (winnt.h)
New anti-tamper and process isolation support:

- `SECURITY_ISOLATION_CONFIGURATION` and its `_GENERAL`, `_DEBUG`, `_UI`, `_PROCESS`, and `_SIGNING` sub-structures: Isolation configuration passed at process creation
- `SECURITY_ANTITAMPER_POLICY` and related structures (`SECURITY_ANTITAMPER_POLICY_DEBUG`, `SECURITY_ANTITAMPER_POLICY_PROCESS`, `SECURITY_ANTITAMPER_POLICY_SIGNING`,

`SECURITY_ANTITAMPER_POLICY_MODULES`, `SECURITY_ANTITAMPER_IDENTITY`, `SECURITY_ANTITAMPER_ENTITLEMENT`), plus `SECURITY_ANTITAMPER_POLICY_PRESENT_*` flags: Anti-tamper policy definitions
- `SECURITY_ANTI_TAMPER_AUTHORITY` and `SECURITY_ANTI_TAMPER_*_RID` values: Anti-tamper SID authority and RIDs
- `SYSTEM_MANDATORY_APP_ACCESS_ALLOWED_ACE_TYPE`, `SYSTEM_MANDATORY_ACCESS_ALLOWED_ACE_TYPE`, `MANDATORY_ACCESS_SECURITY_INFORMATION`: New ACE types and security information flag
- `TokenHasTamperProtection`, `TokenAppIdentitySid`, `TokenAppInstanceSid`, `TokenAppSuiteSid`, `TokenEntitlements`, `TokenAgentId`: New `TOKEN_INFORMATION_CLASS` values

Process Creation (WinBase.h)
New:

- `PROC_THREAD_ATTRIBUTE_ISOLATION_CONFIGURATION`: Process attribute that applies a `SECURITY_ISOLATION_CONFIGURATION`
- `PROC_THREAD_ATTRIBUTE_SECURITY_ENVIRONMENT`: Process attribute for the security environment

Updated:

- `ProcThreadAttributeContainmentConfiguration` and `ProcThreadAttributeDesktopAppIdentityPolicy` changed from 31 and 32 to 33 and 34

User Profiles (UserEnv.h)
New:

- `PT_COMPOSABLE_AGENT`, `PT_SHELL_AGENT`: Profile types for isolation-session agents

DirectX Graphics Infrastructure (dxgicommon.h)
New:

- `DXGI_COLOR_SPACE_RGB_FULL_G10_NONE_P2020`, `DXGI_COLOR_SPACE_RGB_FULL_G22EXT_NONE_P709`, `DXGI_COLOR_SPACE_RGB_FULL_G22_NONE_P3`: New color space values

Media Foundation (mfapi.h / dxva.h)
New:

- `MFT_DECODER_AUTO_POSTPROCESS`: Enables automatic post-processing after decode

- `MFSampleExtension_AverageBitrate`, `MFSampleExtension_ScenarioInfo`: Per-sample bitrate target and encoding scenario attributes
- `DXVA_PicParams_HEVC`: `EncoderBitrateHint`, `EncoderQualityHint`, and `ContentAwareHint` bit fields replace `ReservedBits6`

WinHTTP (winhttp.h)

New:

- `WinHttpConnectionSetProxyInfo`, `WinHttpConnectionDeleteProxyInfo`, `WINHTTP_CONNECTION_PROXY_INFO`, and related types: Set or delete the proxy for a connection
- `WinHttpConnectionUpdateIfIndexTable`, `WINHTTP_CONNECTION_IFINDEX_LIST`: Map connections to interface indexes
- `WinHttpConnectionSetPolicyEntries`, `WinHttpConnectionDeletePolicyEntries`, `WinHttpConnectionDeletePolicyEntriesByAppSid`, `WINHTTP_CONNECTION_POLICY_ENTRY_LIST`, `WINHTTP_CONNECTION_POLICY_TAG`: Manage connection policy entries
- `WINHTTP_CONNECTION_POLICY_ALLOW_DEFAULT_INTERFACE_SELECTION`, `WINHTTP_FEATURE_CONNECTION_PROXY_EX`: New policy flag and feature value

DNS (WinDNS.h)

New:

- `DnsConnectionDeletePolicyEntriesByAppSid`: Deletes DNS connection policy entries for an app SID
- `DNS_CONNECTION_POLICY_ALLOW_DEFAULT_INTERFACE_SELECTION`: New policy entry flag

Networking (wlantypes.h)

New:

- `DOT11_AUTH_ALGO_PQC_1X` / `DOT11_AUTH_ALGORITHM_PQC_1X`: Wi-Fi PQC 802.1X authentication algorithm

Print (prnasntp.h)

New:

- `IID_IPrintAsyncCookie`, `IID_IPrintAsyncNewChannelCookie`, `IID_IAsyncGetSendNotificationCookie`, `IID_IAsyncGetSrvReferralCookie`: Interface IDs for asynchronous print notification

USB (usbspec.h)

New:

- `USB_DEVICE_CLASS_RVC`: USB device class code

Status and Error Codes (ntstatus.h / winerror.h)
New:

- `STATUS_SPACES_API_VERSION_MISMATCH`, `STATUS_SPACES_METADATA_SIZE_INVALID`, `ERROR_SPACES_API_VERSION_MISMATCH`, `ERROR_SPACES_METADATA_SIZE_INVALID`: Storage Spaces error codes
- `FVE_E_WIM_TRUST_DISABLED_BY_POLICY`: BitLocker can't trust the WinRE image because the trust mechanism is disabled by policy

New headers

- `i3c.h`: I3C protocol types and definitions
- `wini3c.h`: User-mode APIs for discovering and communicating with I3C targets
- `processmodel.h`: APIs for creating, querying, and managing process security environments and learning-mode traces

▼ COM API updates

DirectX Graphics Infrastructure (dxgicommon.idl)
New:

- `DXGI_COLOR_SPACE_RGB_FULL_G10_NONE_P2020`, `DXGI_COLOR_SPACE_RGB_FULL_G22EXT_NONE_P709`, `DXGI_COLOR_SPACE_RGB_FULL_G22_NONE_P3`: New color space values

Build 10.0.26100.9169

Released: **August, 2026**

▼ WinRT API additions and updates

Windows.Management.Update:
Updated:

- `WindowsSoftwareUpdateIdentity`, `WindowsSoftwareUpdateIdentityType`, `WindowsSoftwareUpdateCategory`: New identity and category types for describing a software update

- `WindowsUpdateManagerScanMode` and `WindowsUpdateManager.PerformScan`: Scanning now takes a scan mode instead of a scan options object
- `WindowsSoftwareUpdate.IsSeeker`, `WindowsSoftwareUpdate.UpdateCategory`, `WindowsSoftwareUpdate.UpdateIdentity`, `WindowsSoftwareUpdate.Properties`, `WindowsSoftwareUpdate.GetPropertyValue`: Additional update metadata and property access

Windows.System:
New:

- `User.GetUserAgeRangeAsync`: Gets the age range associated with the user
- `User.GetAgeVerificationStatusAsync`: Gets the age verification status of the user
- `UserAgeRange`: Enumeration describing the user's age range
- `UserAgeVerificationStatus`: Enumeration describing the user's age verification status

> ⓘ **Note**
>
> The age signal APIs (`User.GetUserAgeRangeAsync`, `User.GetAgeVerificationStatusAsync`, `UserAgeRange`, and `UserAgeVerificationStatus`) are documented ahead of availability. They aren't enabled at runtime yet, and are planned to be turned on in a future release later this year. Calls made before then return no age data, so design your app to fall back to its default behavior.

Windows.UI.Notifications:
New:

- `ToastNotification.IsExpandableContentSupported`: Indicates whether expandable toast content is supported

Windows.UI.Input:
New:

- `GamepadKeyRoutingConfiguration`: Runtime class that configures gamepad key routing
- `GamepadKeyRoutingConfiguration.IsSupported`: Indicates whether gamepad key routing configuration is supported
- `GamepadKeyRoutingConfiguration.IsKeyRoutingEnabled`: Gets whether gamepad key routing is enabled
- `GamepadKeyRoutingConfiguration.TrySetKeyRoutingEnabled`: Attempts to enable or disable gamepad key routing

Windows.Devices.WiFiDirect.Services:

Deprecated:

- The Wi-Fi Direct Services APIs are now deprecated and might not work on all platforms

▼ Win32 API additions and updates

Cryptography (wincrypt.h)

New post-quantum cryptography (PQC) and CMS support:

- Composite ML-DSA and ML-KEM OIDs (for example `szOID_MLDSA44_ECDSA_P256_SHA256`, `szOID_MLKEM768_ECDH_P256_SHA3_256`) pairing ML-DSA/ML-KEM with classical ECDSA/ECDH
- NIST AES-CCM and AES-GCM OIDs (`szOID_NIST_AES128_CCM` through `szOID_NIST_AES256_GCM`)
- `szOID_RSA_HKDF_WITH_SHA256`: HKDF key-derivation OID
- Authenticated enveloped data: `CMSG_AUTH_ENVELOPED`, `CMSG_AUTH_ENVELOPED_ENCODE_INFO`, and `szOID_RSA_authEnvelopedData`
- KEM recipient info: `CMSG_KEM_RECIPIENT`, `CMSG_KEM_RECIPIENT_ENCODE_INFO`, and `szOID_ORI_KEM`
- Endorsement-key enrollment v2 OIDs (`szOID_ENROLL_V2_EK_ALGORITHM`, `szOID_ENROLL_V2_ATTESTATION_STATEMENT`, and related)
- `szOID_PREHASH`: Prehash OID

WebAuthn (webauthn.h)

New:

- `WEBAUTHN_COSE_ALGORITHM_ML_DSA_44`, `WEBAUTHN_COSE_ALGORITHM_ML_DSA_65`, `WEBAUTHN_COSE_ALGORITHM_ML_DSA_87`: ML-DSA COSE algorithm identifiers

Media Foundation codecs (codecapi.h)

New:

- `CODECAPI_AVEncAV1EncoderOperatingMode` and `eAV1EncoderOperatingMode`: Configure the AV1 encoder operating mode

Direct3D 12 (d3d12.h)

New:

- `ID3D12DeviceStatistics` with `GetStateObjectStatistics`, plus `D3D12_STATE_OBJECT_STATISTICS` and `D3D12_CREATE_STATE_OBJECT_STATISTICS`: Query

statistics for a state object

Process Creation (WinBase.h)
New:

- `PROC_THREAD_ATTRIBUTE_DESKTOP_APP_IDENTITY_POLICY`, `PROCESS_CREATION_DESKTOP_APP_IDENTITY_ENABLE`, `PROCESS_CREATION_DESKTOP_APP_IDENTITY_DISABLE`: Control desktop app identity for a new process

Shell Properties (propkey.h)
New:

- `PKEY_Devices_WiFiAware_IsSupported`: Indicates whether Wi-Fi Aware is supported
- `PKEY_ItemFolderPathDisplayNarrowNamespaceRelative`: Namespace-relative folder path for display

Status and Error Codes (ntstatus.h / winerror.h)
New:

- `STATUS_FIPS_FAILURE` and `NTE_FIPS_FAILURE`: A cryptographic operation failed FIPS validation

Networking (ws2def.h)
New:

- `AI_EXTRA_UNTRUSTED_QUERY`: Reserved addrinfo hint flag

▼ COM API updates

Direct3D 12 (d3d12.idl)
New:

- `ID3D12DeviceStatistics` with `GetStateObjectStatistics`: Query statistics for a state object

Build 10.0.26100.8876

Released: **July, 2026**

▼ WinRT API additions and updates

Windows.ApplicationModel.DataTransfer:
New:

- `DataPackageOperation.NewTarget`: Hint indicating the drag is over a new logical target (for example, individual files and folders on the desktop)
- `DataPackageOperation.BackgroundTarget`: Hint indicating the drag is over a logical target that represents the background (for example, the desktop itself)

Windows.Devices.Printers:
Graduated from experimental to stable:

- `IIppPrintDevice5`: IPP print device interface
- `IIppPrintDeviceInstallationResult`: Result of an IPP print device installation
- `IReplaceDevicePropertiesResult`: Result of a replace-device-properties operation

Windows.Graphics.Printing.PrintSupport:
Graduated from experimental to stable:

- `IPrintSupportAppInfo`: Provides information about the print support app
- `IPrintSupportEnterpriseManagementUIEventArgs`: Event arguments for print support enterprise management UI
- `IPrintSupportPrintTicketValidationRequestedEventArgs`: Event arguments for print ticket validation requests

Windows.Graphics.Printing.Workflow:
Graduated from experimental to stable:

- `IPrintWorkflowJobBackgroundSession3`: Print workflow background session interface
- `IPrintWorkflowPdlConverter3`: Print workflow PDL converter interface
- `IPrintWorkflowPdlModificationRequestedEventArgs`: Event arguments for PDL modification requests
- `IPrintWorkflowPrinterJobStatusChangedEventArgs`: Event arguments for printer job status changes
- `IPrintWorkflowXpsObjectModelProvider`: Provides access to the XPS object model
- `IPrintWorkflowObjectModelProvider`: Provides access to the print workflow object model

Windows.System.Power:
New:

- `EnergySaverStatus2`: Enumeration describing the energy saver state (`Unknown`, `Off`, `Standard`, `HighSavings`)

- `PowerManager.EnergySaverStatus2`: Gets the current energy saver status
- `PowerManager.IsEnergySaverStatus2Supported`: Indicates whether `EnergySaverStatus2` is supported on the device
- `PowerManager.EnergySaverStatus2Changed`: Fires when the energy saver status changes

Windows.System.Profile:
New:

- `RetailInfo.IsDemoModeEnabledEx`: Indicates whether retail demo mode is enabled

Windows.UI.Input:
Updated:

- `TouchpadGlobalAction`: Renamed `ThreeFingerPressDown`/`FourFingerPressDown`/`FiveFingerPressDown` to `ThreeFingerPress`/`FourFingerPress`/`FiveFingerPress`, and `ThreeFingerPressUp`/`FourFingerPressUp`/`FiveFingerPressUp` to `ThreeFingerRelease`/`FourFingerRelease`/`FiveFingerRelease`

▼ WinRT Experimental API additions

Windows.System.RemoteDesktop.Provider:
New (experimental):

- `RemoteDesktopRegistrar.ConnectionCenterRequested`: Event raised when the connection center is requested

Windows.UI.Input:
New (experimental):

- `GamepadNavigationConfiguration`: Runtime class that configures gamepad-based navigation
- `GamepadNavigationConfiguration.NavigationEnabled`: Gets whether gamepad navigation is enabled
- `GamepadNavigationConfiguration.TrySetNavigationEnabled`: Attempts to enable or disable gamepad navigation
- `GamepadNavigationConfiguration.TryGetForCurrentProcess`: Gets the configuration for the current process
- `GamepadNavigationConfiguration.IsSupported`: Indicates whether gamepad navigation configuration is supported

▼ Win32 API additions and updates

App Model (appmodel.h)

New:

- `PackageOrigin2`: Enumeration describing the origin of a staged package (adds `LineOfBusiness` and `SignedSBOM` values)
- `GetStagedPackageOrigin2`: Retrieves the `PackageOrigin2` of a staged package
- `AppModelPackageFeature`: Enumeration of queryable package features (`SignedSBOM`)
- `IsPackageFeatureSupported`: Queries whether a package feature is currently available or enabled

Cryptography (bcrypt.h / ncrypt.h)

New post-quantum cryptography (PQC) support:

- `BCRYPT_COMPOSITE_MLDSA_ALGORITHM` / `BCRYPT_COMPOSITE_MLKEM_ALGORITHM`: Composite ML-DSA and ML-KEM algorithm identifiers, with corresponding algorithm handles, key blob types, magics, and parameter-set names
- `BCRYPT_PARAMETER_SET_NAMES` / `BCRYPT_PARAMETER_SET_KEY_BLOB`: Structures for enumerating and importing parameter-set keys
- `NCRYPT_COMPOSITE_MLDSA_ALGORITHM` / `NCRYPT_COMPOSITE_MLKEM_ALGORITHM` and algorithm groups: CNG key storage support for composite ML-DSA and ML-KEM
- `NCRYPT_PAD_PQDSA_FLAG`, `NCRYPT_MLDSA_EXTERNAL_MU_FLAG`: Padding and signing flags for PQC signatures
- New TPM platform crypto provider (PCP) properties

Bug Codes (bugcodes.h)

New:

- `ASSERTBUGCODE_I3CHOST_DRIVER`: Assertion bug code for the I3C host driver

Direct3D 11 (d3d11.h)

New:

- `D3D11_DECODER_PROFILE_APV_VLD_422_10`, `..._422_12`, `..._444_10`, `..._444_12`, `..._4444_10`, `..._4444_12`, `..._400_10`: Advanced Professional Video (APV) decoder profile GUIDs (RFC 9924)

DirectX Kernel (d3dkmthk.h / d3dukmdt.h)

New:

- `D3DKMT_QUERYFEATUREINTERFACE` and `D3DKMTQueryFeatureInterface`: Query a kernel-mode feature interface
- `DXGK_FEATURE_*_VERSION` defines and `DXGK_QUERYFEATURESUPPORT2_VERSION`: DXGK feature version identifiers
- `D3DDDI_SEGMENTPREFERENCE2`: Segment-preference structure with large-page preference support

Device Class GUIDs (devguid.h)
New:

- Additional device setup class GUIDs

DirectX Video Acceleration (dxva.h)
New:

- `DXVA_ModeAPV_VLD_422_10`, `..._422_12`, `..._444_10`, `..._444_12`, `..._4444_10`, `..._4444_12`, `..._400_10`: Advanced Professional Video (APV) DXVA profile GUIDs (RFC 9924)
- `DXVA_PicEntry_APV` and related APV DXVA structures: Picture and slice data structures for APV decode

EAP Types (eaptypes.h)
New:

- `EAP_FLAG_DISABLE_SESSION_RESUMPTION`: Flag to disable TLS session resumption

Media Foundation (mfidl.h)
New:

- `IMFDXGIScheduler`, `IMFDXGISchedulerClient`, `IMFDXGISchedulerRegistration`: Interfaces for DXGI-based media scheduling
- `MF_DXGI_SCHEDULING_PRIORITY`: Enumeration of DXGI scheduling priorities

Media Engine (mfmediaengine.h)
New:

- `IMFMediaEngineVideoRendererEffect`: Interface that allows an app to set a custom video renderer effect

Storage (ntddstor.h / winioctl.h)
New:

- Structures supporting storage paging-device dump collection

Status Codes (ntstatus.h)

New:

- `STATUS_DEVICE_NACKED`: The device rejected the request
- `STATUS_SMB_ALTERNATIVE_PORT_CONFLICT`: SMB alternative port conflict

Power (poclass.h)

New:

- `BATTERY_TEST_EXEMPT`: Battery flag indicating the battery is exempt from testing

Process Creation (WinBase.h)

New:

- `PROC_THREAD_ATTRIBUTE_CONTAINMENT_CONFIGURATION`: Process/thread attribute for containment configuration

Windowing and Input (WinUser.h)

New:

- `WM_STOPINERTIA`, `WM_ENDINERTIA`: Window messages for inertia handling
- `GetPointerTouchpadInfo`, `GetPointerTouchpadInfoHistory`, `GetPointerFrameTouchpadInfo`, `GetPointerFrameTouchpadInfoHistory`: Retrieve precision-touchpad pointer information
- `CreateSyntheticPointerDevice2`: Creates a synthetic pointer device from `SYNTHETIC_DEVICE_CREATION_PARAMS`
- `SYNTHETIC_DEVICE_CREATION_OPTIONS`, `SYNTHETIC_DEVICE_CREATION_PARAMS`: Options and parameters for synthetic pointer device creation
- `TOUCHPAD_ACTION` and `InjectTouchpadAction`: Enumeration and function for injecting synthetic touchpad actions
- `ReportWindowContentInertia`: Reports the start and end of window content inertia
- `RegisterTouchpadCapableWindow`, `RegisterTouchpadCapableThread`, `SetMaxTouchpadSensitivity`: Opt in to and configure precision-touchpad input

Security (winnt.h)

New:

- `SECURITY_SHADOWADMINACCOUNT_RID`, `SECURITY_AGENTIC_PLATFORM_BASE_RID`: Well-known security identifier values
- `SECURITY_CONTAINMENT_CONFIGURATION`: Structure describing containment configuration

- Heap memory usage structures

Error Codes (winerror.h / wuerror.h / deliveryoptimizationerrors.h / slerror.h)

New:

- UTC, per-processor, device-NACKED, and SMB error codes in `winerror.h`
- New Windows Update (`WU_E_*`) and Delivery Optimization (`DO_E_*`) error codes
- `SL_ACS` licensing error code

Terminal Services (wtsdefs.h / wtsprotocol.h)

New:

- WRDS credential scenario GUIDs
- `IWRdsProtocolConnection3_Experimental`: Experimental protocol connection interface

OLE (oleidl.h)

New:

- `DROPEFFECT_NEWTARGET`, `DROPEFFECT_BACKGROUNDTARGET`: Drop-effect hints matching the new `DataPackageOperation` values

▼ COM API updates

Media Foundation (Mfidl.h)

New:

- `IMFDXGIScheduler`, `IMFDXGISchedulerClient`, `IMFDXGISchedulerRegistration` and `MF_DXGI_SCHEDULING_PRIORITY`: DXGI media scheduling interfaces and enumeration

Media Engine (mfmediaengine.h)

New:

- `IMFMediaEngineVideoRendererEffect`: Interface for setting a custom video renderer effect

Audio (mmdeviceapi.h)

New:

- `PKEY_AudioEndpoint_StableId`: Property key for a stable audio endpoint identifier

Direct3D 11 (d3d11.h)

New:

- APV decoder profile GUIDs (RFC 9924)

OLE (oleidl.h)
New:

- `DROPEFFECT_NEWTARGET`, `DROPEFFECT_BACKGROUNDTARGET`: Drop-effect hints

Build 10.0.26100.8249

Released: **April, 2026**

▼ WinRT API additions and updates

Windows.Devices.Printers:
Updated methods:

- `IIppAttributeConverterStatics.ConvertPrintTicketToIppAttributesForPrinter` — Now accepts an additional `targetPdlFormat` parameter

Graduated from experimental to stable:

- `IppAttributeConverter` runtime class
- `IppAttributeGroupKind` enum
- `IPdlPassthroughProvider2` interface

Windows.Graphics.Printing.PrintSupport:
Graduated from experimental to stable:

- `IPrintSupportPrintDeviceCapabilitiesChangedEventArgs5` — Interface with `SetPdlPassthroughWithJobAttributesSupported` method

New types:

- `PrintSupportEnterpriseManagementUIEventArgs` — Implements `IActivatedEventArgs` and `IActivatedEventArgsWithUser` for enterprise management UI activation scenarios

Windows.Graphics.Printing.Workflow:
Graduated from experimental to stable:

- `IPrintWorkflowPrinterJob3` — Interface with passthrough attribute support

Updated properties:

- `IsPassthroughJob` renamed to `IsPassthroughJobWithAttributes`

▼ WinRT Experimental API additions

> **Windows.AI.Agents.Mcp**:
> New interface:
>
> - `IMcpMessageFilterExperimental2` — Adds `Initialize` method with client/server process identifiers and IDs, plus `OnMessage` for MCP message filtering
>
> **Windows.Devices.Haptics**:
> New properties on `KnownSimpleHapticsControllerWaveforms`:
>
> - `Collide` — Waveform ID for collision haptic feedback
> - `Align` — Waveform ID for alignment haptic feedback
> - `Step` — Waveform ID for step haptic feedback
> - `Grow` — Waveform ID for growth haptic feedback
>
> **Windows.Graphics.Capture**:
> New interfaces:
>
> - `IDirect3D11CaptureFrame3` — Adds `ConfigurationIteration` property to capture frames
> - `IGraphicsCaptureSession7` — Adds `ConfigurationIteration` property and window exclusion list management
> - `IDisplayGraphicsCaptureSession` — Display-specific graphics capture session
>
> New methods:
>
> - `SetWindowExclusionList` — Sets a list of windows to exclude from capture
> - `GetWindowExclusionList` — Gets the current window exclusion list
>
> **Windows.Media.ClosedCaptioning**:
> New types:
>
> - `ClosedCaptionTheme` — Represents a closed caption theme with customization support
>
> New methods:
>
> - `GetAvailableThemes` — Retrieves available closed caption themes
> - `GetSelectedTheme` — Gets the currently selected theme
> - `TrySetSelectedTheme` — Attempts to set the selected theme
>
> New events:
>
> - `SelectedThemeChanged` — Fires when the selected closed caption theme changes

▼ Win32 API additions and updates

Event Tracing (evntprov.h / evntcons.h)

New definitions:

- `EVENT_DATA_DESCRIPTOR_TYPE_RESERVED1` — Reserved event data descriptor type
- `EventProviderSetReserved2` — New value in `EVENT_INFO_CLASS` enum
- `EVENT_HEADER_FLAG_RESERVED1` — New event header flag

HID Usages (hidusage.h)

New haptics usage values:

- `HID_USAGE_HAPTICS_WAVEFORM_COLLIDE` — Collision haptic waveform
- `HID_USAGE_HAPTICS_WAVEFORM_ALIGN` — Alignment haptic waveform
- `HID_USAGE_HAPTICS_WAVEFORM_STEP` — Step haptic waveform
- `HID_USAGE_HAPTICS_WAVEFORM_GROW` — Growth haptic waveform

Error Codes (winerror.h)

New BitLocker error codes:

- `FVE_E_MISSING_PROTECTORS` — BitLocker protectors are missing
- `FVE_E_METHOD_MISMATCH` — BitLocker method mismatch

Security / Authentication (NTSecPKG.h)

New definitions:

- `SECPKG_CALL_AGENT_LOGON` — Security package call flag for agent-based logon
- `KSecAllocateContextBuffer` — Function for allocating security context buffers
- Added `extern "C"` guards for C++ compatibility

Crypto / TPM (ncrypt.h)

New TPM property defines:

- `NCRYPT_PCP_AIKSTORE_PROPERTY` — TPM AIK store property
- `NCRYPT_PCP_EKSTORE_PROPERTY` — TPM EK store property

Content Indexing (NTQuery.h)

New define:

- `CI_VERSION_QUERY_METADATA` — Content index version for query metadata / semantic reliability

Shell API (shellapi.h)
New define:

- `ABC_OVERLAYDESKTOPICONS` — Overlay desktop icons flag for `ABM_NEW`

Rust Bindgen Compatibility (ntdef.h / winnt.h)
Updated:

- `DECLSPEC_NOINITALL` macro now excludes Rust bindgen passes via `!defined(RUST_BINDGEN)`

WRL Async (wrl/async.h)
Updated:

- Async completion handling reworked for thread safety using `_InterlockedCompareExchange` and reference counting (`cCompleteDelegateRefCount_`)

Build 10.0.26100.8038

Released: **March, 2026**

▼ WinRT API additions and updates

Windows.ApplicationModel.Contacts.Provider:
New types:

- `IContactProvider` - Interface with `GetContactFromRemoteIdAsync` method

Windows.Devices.Printers:
New types:

- `IppAttributeGroupKind` - Enum for IPP attribute group kinds
- `IIppAttributeConverterStatics` - Interface with `ConvertPrintTicketToIppAttributesForPrinter`, `ConvertBufferToIppAttributes`, `ConvertIppAttributesToBuffer`
- `IppAttributeConverter` - Runtime class
- `IPdlPassthroughProvider2` - Interface with `IsPassthroughWithJobAttributesSupported` property and `StartPrintJobWithIppJobAttributes` method

Windows.Graphics.Printing.PrintSupport:
New types:

- `IPrintSupportPrintDeviceCapabilitiesChangedEventArgs5` - Interface with `SetPdlPassthroughWithJobAttributesSupported` method

Windows.Graphics.Printing.Workflow:
New types:

- `IPrintWorkflowPrinterJob3` - Interface with `IsPassthroughJob` property, `GetPassthroughJobAttributes`, and `GetPassthroughJobOperationAttributes` methods

Windows.Storage.Search:
New types:

- `IQueryOptionsAdditionalSearchSources` - Adds `IncludeCloudProviders` and `IncludeLocalSemanticIndex` properties to `QueryOptions`

▼ Win32 API additions and updates

Bluetooth Hands-Free Profile (bthdef.h)
Added new defines for HFP Audio Gateway and Hands-Free SDP and BRSF supported features:

- `HFP_AG_SDP_SUPPORTED_FEATURE_*` - Audio Gateway SDP feature flags for three-way calling, echo cancellation, voice recognition, in-band ring tone, voice tag, wide-band speech, and more
- `HFP_AG_BRSF_SUPPORTED_FEATURE_*` - Audio Gateway BRSF feature flags for three-way calling, echo cancellation, voice recognition, reject call, enhanced call status/control, codec negotiation, and more
- `HFP_HF_SDP_SUPPORTED_FEATURE_*` - Hands-Free SDP feature flags for echo cancellation, three-way calling, CLI presentation, voice recognition, remote volume control, wide-band speech, and more
- `HFP_HF_BRSF_SUPPORTED_FEATURE_*` - Hands-Free BRSF feature flags for echo cancellation, three-way calling, CLI presentation, voice recognition, remote volume control, codec negotiation, and more
- `HFP_NETWORK_NO_ABILITY_TO_REJECT`, `HFP_NETWORK_ABILITY_TO_REJECT` - Network call rejection capability flags

Virtualization-Based Security (ntstatus.h / winerror.h)
New error codes:

- `STATUS_VSM_FW_MEASUREMENTS_SEAL_FAILURE` - VSM firmware measurements seal failure status
- `ERROR_VSM_FW_MEASUREMENTS_SEAL_FAILURE` - Corresponding Win32 error code

Video Encoding (codecapi.h)

New enum and property for D3D12 reconstructed picture output:

- `eAVEncVideoD3D12ReconstructedPictureOutputMode` enum
- `CODECAPI_AVEncVideoD3D12ReconstructedPictureOutputMode` codec API GUID

Direct3D 12 (d3d12.h)

Spelling corrections for tight alignment defines:

- `D3D12_TIGHT_ALIGNMENT_MIN_COMMITTED_RESOURCE_ALIGNMENT` (replaces misspelled `ALIGNEMNT` variant)
- `D3D12_TIGHT_ALIGNMENT_MIN_PLACED_RESOURCE_ALIGNMENT` (replaces misspelled `ALIGNEMNT` variant)

Local Security Authority (ntlsa.h)

Added new structs for agent-based logon:

- `_LSA_AGENT_LOGON_CREDENTIAL` - Agent logon credential data
- `_LSA_AGENT_ACCOUNT_INFO` - Agent account information
- `_LSA_AGENT_ACCOUNT_LIST` - List of agent accounts

Content Indexing (NTQuery.h)

New define:

- `CI_VERSION_CORRID` - Content index version correlation ID

Secure Channel (schannel.h)

Added new types for TLS extension copying:

- `SCH_COPY_EXTS_DATA` struct - Data structure for copying TLS extensions
- `SchCopyExtsOptions` enum - Options for TLS extension copy operations

Text Services Framework (TextStor.h)

New defines:

- `TS_SD_DISABLEWRITINGSUGGESTIONS` - Flag to disable writing suggestions
- `TS_SS_MULTILINE` - Flag for multiline text store support

Remote Desktop Virtual Channels (tsvirtualchannels.h)
Added new interfaces, structs, and enums for RDP window information:

- `IWTSWindowChangedCallback` - Callback interface for window change notifications
- `IWTSWindowInfoService` - Interface for querying window info, client process ID, session type, and subscribing to window changes
- `WTSWindowInfo` struct - Window information data
- `RdpSessionType` enum - RDP session type values

WebAuthn Plugin API (webauthnplugin.h)
Graduated from experimental to stable:

- `WEBAUTHN_PLUGIN_ADD_AUTHENTICATOR_OPTIONS_2` (previously `EXPERIMENTAL_WEBAUTHN_PLUGIN_ADD_AUTHENTICATOR_OPTIONS_2`)
- `WEBAUTHN_PLUGIN_UPDATE_AUTHENTICATOR_DETAILS_2` (previously `EXPERIMENTAL_WEBAUTHN_PLUGIN_UPDATE_AUTHENTICATOR_DETAILS_2`)
- `WEBAUTHN_PLUGIN_USER_VERIFICATION_REQUEST_2` (previously `EXPERIMENTAL_WEBAUTHN_PLUGIN_USER_VERIFICATION_REQUEST_2`)

▼ COM API updates

Remote Desktop Virtual Channels (tsvirtualchannels.idl)
Added new interfaces for RDP window information:

- `IWTSWindowChangedCallback` - Callback interface with `WindowChanged` method
- `IWTSWindowInfoService` - Interface with `GetWindowInfo`, `GetRdpClientProcessId`, `GetRdpSessionType`, `SubscribeWindowChanged`, `UnsubscribeWindowChanged`
- `WTSWindowInfo` struct - Window information data
- `RdpSessionType` enum - RDP session type values

Build 10.0.26100.7705

Released: **February, 2026**

▼ WinRT API additions and updates

Windows.UI.Shell.Tasks:
New namespace for managing app tasks:

- `AppTaskContract` - API contract for the task APIs

- `AppTaskState` - Enumeration for task states
- `IAppTaskContent`, `IAppTaskContentStatics` - Interfaces for task content
- `IAppTaskInfo`, `IAppTaskInfo2`, `IAppTaskInfoStatics` - Interfaces for task information
- `IAppTaskResultAsset`, `IAppTaskResultAssetFactory` - Interfaces for task result assets
- `AppTaskContent` - Runtime class for task content
- `AppTaskInfo` - Runtime class for task information
- `AppTaskResultAsset` - Runtime class for task result assets

Windows.UI.Shell.CompanionWindows:
New namespace for companion window management:

- `CompanionWindowsContract` - API contract for companion windows
- `CompanionWindowRequestResultStatus` - Enumeration for request result status
- `ICompanionWindowCoordinator`, `ICompanionWindowCoordinatorStatics` - Coordinator interfaces
- `ICompanionWindowRequest`, `ICompanionWindowRequestResult` - Request interfaces
- `CompanionWindowCoordinator` - Runtime class for coordinating companion windows
- `CompanionWindowRequest`, `CompanionWindowRequestResult` - Runtime classes for requests

Windows.Devices.Haptics:
New types and enhancements for haptic feedback:

- `HapticDeviceType` - New enum for haptic device types (UniversalApiContract 19.0)
- `HapticsControllerOverrideToken` - New struct for controller override tokens
- `IInputHapticsManager`, `IInputHapticsManagerStatics` - New interfaces for input haptics management
- `InputHapticsManager` - New runtime class for managing input haptics

▼ Win32 API additions and updates

Windows Hypervisor Emulation (WinHvEmulation.h)
Added new emulator management functions for AMD64:

- `WHvEmulatorCreateEmulator` - Creates a new emulator instance with specified callbacks
- `WHvEmulatorDestroyEmulator` - Destroys an emulator instance
- Additional emulator management APIs for memory access, I/O port handling, and virtual processor register operations

WebAuthn Plugin (webauthnplugin.h)
Updated documentation:

- Plugin RPID is now required (previously optional) for nested WebAuthN calls originating from a plugin

Windows Error Codes (winerror.h)
Updated error definitions and codes

Graphics Driver Model (d3dukmdt.h, d3dkmdt.h)
Updates to graphics driver display mode definitions and user mode types

Windows User Interface (WinUser.h)
Updates to user interface definitions

Cryptography (wincrypt.h)
Updates to cryptographic function definitions

Property Keys (propkey.h)
Updated property key definitions

HTTP API (http.h)
Updates to HTTP server API definitions

The following issue is fixed in this SDK version: **BinSkim Warning 4146 Triggered by Windows SDK 10.0.26100.7175 in Visual Studio**

Build 10.0.26100.7627

Released: **January, 2026**

▼ WinRT API additions and updates

Windows.Security.Credentials:
New interfaces:

- `IKeyCredentialManagerCreateWithWindowStatics` - Provides methods for creating key credentials with an associated window handle
- `IKeyCredentialWithWindow` - Extends key credential functionality with window association support

New methods:

- `KeyCredentialManager.RequestCreateForWindowAsync` - Creates a key credential associated with a specific window

▼ Win32 API additions and updates

WinSQLite (winsqlite3.h / winsqlite3ext.h)
SQLite version update:

New error codes:

- `SQLITE_ERROR_RESERVESIZE`
- `SQLITE_ERROR_KEY`
- `SQLITE_ERROR_UNABLE`

New IO error codes:

- `SQLITE_IOERR_BADKEY`
- `SQLITE_IOERR_CODEC`

New capabilities:

- `SQLITE_IOCAP_SUBPAGE_READ`

New file control codes:

- `SQLITE_FCNTL_NULL_IO`
- `SQLITE_FCNTL_BLOCK_ON_CONNECT`

New source control management constants:

- `SQLITE_SCM_BRANCH`
- `SQLITE_SCM_TAGS`
- `SQLITE_SCM_DATETIME`

winnt.h
New definitions:

- `SECURITY_MANDATORY_MEDIUM_PLUS_CREDUI_RID` - New security mandatory integrity level RID for Credential UI contexts

---

Build 10.0.26100.7463

Released: **December, 2025**

▼ WinRT API additions and updates

> **Windows.Management.Deployment**:
>
> - Added the `PackageOperationPriority` enum and new priority fields exposed via `AddPackageOptions` and `StagePackageOptions`.

▼ Win32 API additions and updates

> **Driver runtime reporting**
> Added new structs:
>
> - `_DRIVER_INFO_ENTRY`
> - `_DRIVER_RUNTIME_REPORT`
> - `_RUNTIME_REPORT_DIGEST_HEADER`
>
> **DNS SVCB/HTTPS record parsing**
> Added new structs:
>
> - `DNS_SVCB_PARAM`
> - `DNS_SVCB_PARAM_IPV4`, `DNS_SVCB_PARAM_IPV6`
> - `DNS_SVCB_PARAM_ALPN`
> - `DNS_SVCB_PARAM_MANDATORY`
> - `DNS_SVCB_PARAM_UNKNOWN`
>
> **Search and Query engine**
> Updated or added structs:
>
> - `CONTENTRESTRICTION`
> - `VECTORRESTRICTION`
> - `NODERESTRICTION`
> - `NOTRESTRICTION`
>
> **Audio / Device activation**
> Added the `IMMDeviceActivator` interface for new device-level activation scenarios within the audio stack.
>
> **Firmware table enumeration**
> Updated:

- `EnumSystemFirmwareTables`
- `GetSystemFirmwareTable`

Build 10.0.26100.7175

Released: **November, 2025**

▼ Updated APIs

Updates made to the following Win32 API headers (defines, structs, enums, interfaces and other changes):

- AppxPackaging.h, AppxPackaging.idl (interfaces IAppxFactory4, IAppxBundleFactory3, IAppxBundleReader2)

▼ New APIs

WinRT namespaces updated (new or modified APIs/types):

- Windows.ApplicationModel.DataTransfer
- Windows.Management.Update
- Windows.Security.Credentials
- Windows.Storage.Provider
- Windows.System.RemoteSystems
- AppxManifestTypes.xsd schema updated

▼ Known issues

BinSkim Warning 4146 Triggered by Windows SDK 10.0.26100.7175 in Visual Studio

- We are aware of an issue where builds may fail with BinSkim rule BA2007 due to warning C4146 being explicitly disabled in a small number of Windows SDK libraries included in the Windows SDK version 10.0.26100.7175, which shipped with Visual Studio 17.14.22.
- Developers may see build breaks or security-tool validation failures when using this SDK version, depending on project configuration and toolchain settings.
- A fix has been identified and is currently being prepared for a Visual Studio update. This will update the affected SDK content to restore expected behavior. Until the fix is

> available, you may use one of the following mitigations:
>
> - Retarget your project to another supported Windows SDK version, or
> - Suppress the specific BinSkim warning in your build configuration (not recommended long-term)

Build 10.0.26100.6901

Released: **October, 2025**

▼ Updated APIs

> Updates made to the following Win32 API headers, defines, structs, enums, and other changes:
>
> - networksetup.h
> - windows.system.power.thermal.h
> - windows.ui.input.preview.text.h

▼ New APIs

> Added new APIs to the following WinRT namespaces:
>
> - Windows.AI.Actions
> - Windows.Management.Update
> - Windows.Media.Core

Build 10.0.26100.6584

Released: **September, 2025**

Release to correspond with the Windows 11, version 25h2 public release.

▼ Updated APIs

> Updates made to the following Win32 API headers, defines, structs, enums, and other changes:
>
> - winnt.h

- WtsApi32.h
- wtsdefs.h
- wtsprotocol.h
- NetworkSetup.h
- FoundationManifestSchema.xsd
- AccessControlManifestSchema.xsd
- AppDataManifestSchema.xsd

Added new APIs to the following WinRT headers and idl:

- windows.security.credentials.h
- windows.system.power.thermal.h, windows.system.power.thermal.idl

▼ New Experimental APIs

Updated or added experimental APIs to the following:

- windows.ai.actions.h
- windows.ai.actions.hosting.h
- windows.ai.agents.mcp.h
- windows.ai.agents.h
- windows.graphics.printing.printsupport.h
- windows.graphics.printing.printticket.h, windows.graphics.printing.printticket.idl
- windows.devices.printers.h
- windows.applicationmodel.contacts.h

Build 10.0.26100.4948

Released: **August, 2025**

▼ Updated APIs

Updates made to the following Win32 API headers, adding new defines, structs, enums, and other changes:

- ModelContextProtocolHelpers.h
- PrintSupportManifestSchema_v3.xsd

▼ New APIs

Added new APIs to the following WinRT headers and idl:

- windows.graphics.printing.printsupport.h
- windows.storage.provider.h
- windows.devices.printers.h
- windows.applicationmodel.activation.h
- windows.ui.input.preview.text.h

Removed experimental tag from:

- windows.ui.input.preview.text (APIs previously behind ENABLE_WINRT_EXPERIMENTAL_TYPES are now stable)

Build 10.0.26100.4654

Released: **July, 2025**

▼ Updated APIs

Updates made to the following Win32 API headers, adding new defines, structs, enums and other changes:

- bugcodes.h
- d3d12.h
- d3d12.idl
- DbgEng.h
- fwpmu.h
- hidusage.h
- ksarm64.h
- minidumpapiset.h
- ModelContextProtocolHelpers.h
- ntddstor.h
- ntddvdeo.h
- ntlsa.h
- ntstatus.h
- nvme.h
- Raseapif.h
- sherrors.h
- srb.h
- WaaSApiTypes.h

- WaaSApiTypes.idl
- webauthn.h
- winbio_types.h
- winerror.h
- WinHvPlatformDefs.h
- winioctl.h

▼ New APIs

Added new APIs to the following Win32 headers:

- wincodec.h, wincodec.idl
  - **IWICBitmapFrameChainReader**
  - **IWICBitmapFrameChainWriter**
  - **IWICDisplayAdaptationControl2**
- wincodecsdk.h, wincodecsdk.idl
  - **GUID_MetadataFormatGainMap**
  - **CLSID_WICGainMapMetadataReader**
  - **CLSID_WICGainMapMetadataWriter**

Added new WinRT Preview namespaces:

- **windows.ui.input.preview.text**

Build 10.0.26100.4188

Released: **May, 2025**

▼ New APIs

Added or updated new APIs to the following WinRT namespaces:

- **Windows.AI.Actions**
- **Windows.AI.ModelContextProtocol** (experimental)
- **Windows.ApplicationModel.Background.Bluetooth**
- **Windows.Devices.Bluetooth**
- **Windows.UI.ViewManagement**

Added new APIs to the following Win32 headers:

- http.h
  - **HttpQueryRequestProperty**
- ntlsa.h
  - **LsaSetLocalSystemAccess**
  - **LsaQueryLocalSystemAccess**
  - **LsaQueryLocalSystemAccessAll**
- WinUser.h
  - **ConvertToInterceptWindow**
  - **IsInterceptWindow**
  - **ApplyWindowAction**
  - **RegisterCloakedNotification**
  - **EnterMoveSizeLoop**

▼ Updated APIs

Updates made to the Win32 CRT headers:

- corecrt_search.h
- wchar.h

Updates made to the following Win32 API headers, adding new defines, structs and enums:

- CertSrv.h
- codecapi.h
- dwmapi.h
- MDMRegistration.h
- mfapi.h
- ntddvdeo.h
- NTSecAPI.h
- NTSecPKG.h
- ntstatus.h
- overridecapabilities.h
- Propkey.h
- WindowsSearchErrors.h
- Winldap.h
- rpcndr.h
- winerror.h

Added new Win32 API header:

- ModelContextProtocolHelpers.h

▼ New Experimental APIs

Added experimental APIs to the following Win32 API headers:

- webauthn.h
- WinBio.h
- winbio_types.h

Build 10.0.26100.3916

Released: **April, 2025**

▼ New APIs

Added new APIs to the **windows.ui.viewmanagement** WinRT namespace to support the user's preferred UserInteractionMode.

Added support for semantic search to the **searchapi.h** Win 32 API header.

Added new **GamingExperience** Win32 API header.

Added new APIs to the following Win32 headers:

- dcomp.h
- http.h
- ntsecpkg.h
- winioctl.h

Build 10.0.26100.3624

Released: **March, 2025**

▼ New APIs

Added gamepad support to the CoreInputViewKind enumeration in the **windows.ui.viewmanagement.core** WinRT namespace.

Added new APIs to the following Win32 headers:

- fileapi.h
- ntlsa.h
- shobjidl_core.h
- softintrin.h
- webauthn.h
- webservices.h
- winenclaveapi.h
- winnt.h

Build 10.0.26100.3323

Released: **February, 2025**

▼ Updated APIs

Renamed the PrivacyScreen WinRT APIs added to the **windows.devices.sensors** namespace in the previous build. These are now OnlookerDetection APIs.

Build 10.0.26100.3037

Released: **January, 2025**

▼ New APIs

Added new WinRT APIs in the **windows.devices.sensors** namespace:

- PrivacyScreenOptions class.
- HumanPresenceSettings.PrivacyScreenOptions method.
- LightSensor.IsChromaticitySupported method.
- New properies and structs were also added to support these methods.

▼ New Experimental APIs

Experimental APIs were changed within the following Win32 header (please note that Expertimental APIs should not be used in a production environment):

- webauthn.h

Build 10.0.26100.2454

Released: **January, 2025**

▼ Updated APIs

Made major additions or changes to the following Win32 headers:

- windows.applicationmodel.background.h: Added many bluetooth-related APIs.
- windows.applicationmodel.calls.h: Added many VOIP call configuration APIs.

Added or modified the following Win32 APIs:

certsrv.h:

- CRL_BUILD_PROPID
- CRL_EXTRACT_KEY_INDEX
- CRL_EXTRACT_PARTITION_INDEX

clusapi.h:

- NodeSriovInfo

combaseapi.h:

- STDMETHOD_CHPE_PATCHABLE

d2d11.h:

- d3d11.D3D11_FEATURE_DATA_D3D11_OPTIONS6

filter.h:

- IPixelFilter
- IPixelFilter.GetImageInfo
- IPixelFilter.GetPixelsForImage
- IMAGE_INFO
- IPixelFilterVtbl

http.h:

- _HTTP_REQUEST_TRANSPORT_IDLE_CONNECTION_TIMEOUT_INFO

msclus.h:

- NodeSriovInfo

ntsecapi.h:

- _KERB_CHANGEMACHINEPASSWORD_REQUEST

winenclaveapi.h:

- EnclaveEncryptDataForTrustlet
- EnclaveUsesAttestedKeys

winnt.h:

- STDAPI_CHPE_PATCHABLE_

winuser.h:

- GetCurrentMonitorTopologyId

Please follow best practices to ensure an API is available on a machine before it is called.

▼ New Experimental APIs

Many experimental APIs were added to the following Win32 header (Please note that experimental APIs should not be used in a production environment):

- webauthn.h

Build 10.0.26100.1742

Released: **September 24, 2024**

Release to correspond with the Windows 11, version 24H2 public release.

Build 10.0.26100

Released: **May 5, 2024**

Initial release of the 10.0.26100 series, to correspond with the Windows 11, version 24H2 preview.

22000 versions and earlier

Build 10.0.22621.3235

Released: **February 29, 2024**

Servicing update 10.0.22621.3235.

Build 10.0.22621.2428

Released: **October 24, 2023**

Servicing update 10.0.22621.2428.

Build 10.0.22621.1778

Released: **May, 2023**

Servicing update 10.0.22621.1778.

▼ Highlighted features

> - WindowTabManager APIs allows applications with tabbed interfaces to provide information on open tabs to the Windows shell.
> - Updates to HumanPresence APIs to improve ease-of-use and add new settings for sensors that support human presence capabilities.
> - RemoteDesktop APIs allows applications to switch between a remote and local desktop.

Windows SDK for Windows 11, version 22H2

Servicing update 10.0.22621.755. Includes ARM64 support for the VS 17.4 release

Windows 10 SDK, Version 2104

▼ Updated APIs

- Removed api-ms-win-net-isolation-l1-1-0.lib. Apps that were linking against api-ms-win-net-isolation-l1-1-0.lib can switch t OneCoreUAP.lib as a replacement.
- Removed irprops.lib. Apps that were linking against irprops.lib can switch to bthprops.lib as a drop-in replacement.
- Moved ENUM tagServerSelection from wuapicommon.h to wupai.h and removed the header. If you would like to use the ENUM tagServerSelection, you will need to include wuapi.h or wuapi.idl.
- The Windows 10 WinRT API Pack lets you add the latest Windows Runtime APIs support to your .NET Framework 4.5+ and .NET Core 3.0+ libraries and apps. To access the Windows 10 WinRT API Pack, see the Microsoft.Windows.SDK.Contracts nuget package.
- The printf family of functions now conforms with the IEEE 754 rounding rules when printing exactly representable floating-point numbers and will honor the rounding mode requested via calls to fesetround. Legacy behavior is available when linking with legacy_stdio_float_rounding.obj.
- Windows App Certification Kit. Several new APIs were added to the Supported APIs list in the App Certification Kit and Windows Store. If there are APIs in the supported list that appear greyed out or disabled in Visual Studio, you can make a small change to your source file, to access them. For more details, see this known issue. Find more updates to tests.

▼ Tool updates

Message Compiler (mc.exe) updates:

- Now detects the Unicode byte order mark (BOM) in .mc files. If the .mc file starts with a UTF-8 BOM, it will be read as a UTF-8 file. Otherwise, if it starts with a UTF-16LE BOM, it will be read as a UTF-16LE file. If the -u parameter was specified, it will be read as a UTF-16LE file. Otherwise, it will be read using the current code page (CP_ACP).
- Now avoids one-definition-rule (ODR) problems in MC-generated C/C++ ETW helpers caused by conflicting configuration macros (e.g. when two .cpp files with conflicting definitions of MCGEN_EVENTWRITETRANSFER are linked into the same binary, the MC-generated ETW helpers will now respect the definition of MCGEN_EVENTWRITETRANSFER in each .cpp file instead of arbitrarily picking one or the other).

Windows Trace Preprocessor (tracewpp.exe) updates:

- Supports Unicode input (.ini, .tpl, and source code) files. Input files starting with a UTF-8 or UTF-16 byte order mark (BOM) will be read as Unicode. Input files that do not start

with a BOM will be read using the current code page (CP_ACP). For backwards-compatibility, if the -UnicodeIgnore command-line parameter is specified, files starting with a UTF-16 BOM will be treated as empty.

- Supports Unicode output (.tmh) files. By default, output files will be encoded using the current code page (CP_ACP). Use command-line parameters -cp:UTF-8 or -cp:UTF-16 to generate Unicode output files.
- Behavior change: tracewpp now converts all input text to Unicode, performs processing in Unicode, and converts output text to the specified output encoding. Earlier versions of tracewpp avoided Unicode conversions and performed text processing assuming a single-byte character set. This may lead to behavior changes in cases where the input files do not conform to the current code page. In cases where this is a problem, consider converting the input files to UTF-8 (with BOM) and/or using the -cp:UTF-8 command-line parameter to avoid encoding ambiguity.

TraceLoggingProvider.h updates:

- Avoids one-definition-rule (ODR) problems caused by conflicting configuration macros (e.g. when two .cpp files with conflicting definitions of TLG_EVENT_WRITE_TRANSFER are linked into the same binary, the TraceLoggingProvider.h helpers will now respect the definition of TLG_EVENT_WRITE_TRANSFER in each .cpp file instead of arbitrarily picking one or the other).
- In C++ code, the TraceLoggingWrite macro has been updated to enable better code sharing between similar events using variadic templates.

Signing your apps. Device Guard signing is a Device Guard feature that is available in Microsoft Store for Business and Education, which allows enterprises to guarantee every app comes from a trusted source. See the documentation about Device Guard Signing.

▼ SDK updates

- SDK headers have been updated to address errors when compiling using the standard-conformant C preprocessor in the MSVC compiler cl.exe (/Zc:preprocessor, introduced in VS 2019 v16.6).
- Fixed: "GdiplusTypes.h does not compile with NOMINMAX". See Visual Studio Feedback.
- When building with /std:c11 or /std:c17, you now get:
  - C99 tgmath.h
  - C11 static_assert in assert.h
  - C11 stdalign.h
  - C11 stdnoreturn.h

▼ Known issues

- Clang/LLVM for Windows v11 targeting ARM64 is not compatible with the latest winnt.h
  - As a workaround, use the previous version of the Windows 10 SDK (build 19041), or clang/LLVM for Windows v10 when targeting ARM64 platforms

- DirectXMath (including version 3.16 in this release) is not compatible with Clang/LLVM for Windows on ARM64.
  - As a workaround, use the latest version of DirectXMath, available from NuGet, vcpkg, or GitHub. These versions include the required hot fixes (version 3.16b).
    - //www.nuget.org/packages/directxmath
    - //github.com/microsoft/vcpkg/tree/master/ports/directxmath
    - //github.com/microsoft/DirectXMath

- The case of some header files were changed, to normalize them for case-sensitive file systems:
  - OAIdl.h, ObjIdl.h, ObjIdlbase.h, OCIdl.h, Ole2.h, OleAuto.h, and OleCtl.h were all made lower-case.
  - For Clang/LLVM for Windows builds, to support both older version and the latest Windows 10 SDK without warnings, add `-Wno-nonportable-system-include-path` to the CLI, or the following #pragma in source:

```
#ifdef __clang__

#pragma clang diagnostic ignored "-Wnonportable-system-include-path"

#endif
```

Windows 10 SDK, Version 2004 servicing update

Released: **December 16, 2020**

▼ Bugfixes

- Resolved unpredictable and hard to diagnose crashes when linking both umbrella libraries and native OS libraries (for example, onecoreuap.lib and kernel32.lib)
- Resolved issue that prevented AppVerifier from working
- Resolved issue that caused WACK to fail with "Task failed to enable HighVersionLie"

Last updated on 09/30/2026

Windows SDK downloads

This page provides links to the **.ISO** and **Installer** files for all supported and preview Windows SDK releases and updates for both Windows 10 and Windows 11. The NuGet packages are available on the Windows SDK NuGet feed.

For older versions, see the Windows SDK download archive.

Windows 11 — 28000 versions

Expand table

Release	Downloads
Windows SDK for Windows 11 (10.0.28000.2957) September 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.28000.2705) August 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.28000.2526) July 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.28000.2270) June 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.28000.2114) May 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.28000.1839) April 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.28000.1721) March 2026 Release notes	Installer ISO
Windows SDK for Windows 11 26H1 for OEMs and Partners (10.0.28000.1) November 2025	Installer ISO

Windows 11 — 26100 versions

Expand table

Release	Downloads
Windows SDK for Windows 11 (10.0.26100.9457) September 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.9169) August 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.8876) July 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.8249) April 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.8038) March 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.7705) February 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.7627) January 2026 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.7463) December 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.7175) November 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.6901) October 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.6584) September 2025 Release notes	Installer ISO

Release	Downloads
Windows SDK for Windows 11 (10.0.26100.4948) August 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.4654) July 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.4188) May 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.3916) April 2025 Release notes	Installer ISO
Windows SDK for Windows 11 (10.0.26100.3323) February 2025 Release notes	Installer ISO

Windows 11 — 22000 versions and earlier

Expand table

Release	Downloads
Windows SDK for Windows 11 (10.0.22621.5040) April 2025	Installer ISO
Windows SDK for Windows 11 (10.0.20348.3330) August 2025 (Includes servicing update for CVE-2024-29187)	Installer

Windows 10

Expand table

Release	Downloads
Windows SDK for Windows 10 2004 (10.0.19041.0) Released in conjunction with Windows 10, version 2004. Includes servicing updates 10.0.19041.5609. Updated: 4/2/2025	Installer ISO
Windows SDK for Windows 10 (10.0.17763.7010) August 2025 (Includes servicing update for CVE-2024-29187)	Installer

Preview releases

Preview releases are available from the Windows Insider Preview Download Center. To use a preview SDK via NuGet, ensure the **Preview** checkbox is enabled and select package versions with the `-preview` suffix.

Last updated on 09/30/2026

Windows versions and SDK overview

When you start building Windows apps, you'll notice references to several different version numbers — the Windows OS version, the Windows SDK, and the Windows App SDK. Don't worry: for most new projects, Visual Studio sets sensible defaults and you won't need to think about this at all. But when you do need to target a specific OS version, understand why an API isn't available, or configure your project file, this article explains how the pieces fit together and what choices to make.

Definitions

Expand table

Component	What it is	Version format	How it's updated
Windows OS version	The operating system installed on the user's device.	`YYHX` (e.g., 24H2, 25H2)	Windows Update (annual feature updates)
Windows SDK (platform SDK)	Headers, libraries, and metadata for Win32, WinRT, and COM APIs.	`10.0.BBBBB.x` (e.g., 10.0.26100.0)	Installed with Visual Studio or standalone installer
Windows App SDK	NuGet-distributed framework providing WinUI 3, app lifecycle, windowing, and other modern app features.	`Major.Minor.Patch` (e.g., 1.8.1)	NuGet package update in your project

Windows OS version naming

Windows 11 ships major feature updates using the naming pattern **YYHX**:

- **YY** = two-digit calendar year
- **H1** = first half of the year; **H2** = second half of the year

Most major releases have historically shipped in the fall (H2), but H1 releases also occur.

Expand table

Windows OS version	Year released	OS build number
Windows 11 22H2	2022	22621

Windows OS version	Year released	OS build number
Windows 11 23H2	2023	22631
Windows 11 24H2	2024	26100

Rule: The OS build number on the user's device determines which platform APIs are available at runtime.

> **Tip**
>
> Check the OS build number on any device by pressing **Win + R** and typing `winver`.

Windows SDK to OS version mapping

Each Windows SDK release family corresponds to a Windows OS version. The SDK's major build number typically matches the OS build number, though some OS feature updates (delivered as enablement packages) share the same SDK family as a prior release.

Expand table

Windows SDK family	OS build	Matches Windows version
10.0.22621.x	22621	Windows 11 22H2 / 23H2
10.0.26100.x	26100	Windows 11 24H2

> **Note**
>
> Within an SDK family, the last component (e.g., 10.0.26100.**2454**) represent monthly servicing updates that add bug fixes but not new APIs.

Rule: Installing a given Windows SDK grants compile-time access to all APIs from that OS version **and all prior versions**. APIs are cumulative.

Windows App SDK versions

The Windows App SDK is **independent** of both the Windows OS and the Windows SDK. It ships on its own cadence as a NuGet package.

Expand table

Windows App SDK	Release date	Minimum supported OS	NuGet package
2.2	2026-06-09	Windows 10 1809 (build 17763)	`Microsoft.WindowsAppSDK 2.2.x`
2.1	2026-05-21	Windows 10 1809 (build 17763)	`Microsoft.WindowsAppSDK 2.1.x`
2.0	2026-04-29	Windows 10 1809 (build 17763)	`Microsoft.WindowsAppSDK 2.0.x`
1.8	2025-09-09	Windows 10 1809 (build 17763)	`Microsoft.WindowsAppSDK 1.8.x`
1.7	2025-03-18	Windows 10 1809 (build 17763)	`Microsoft.WindowsAppSDK 1.7.x`

Rule: You can use the latest Windows App SDK on older OS versions (back to its stated minimum). The Windows App SDK version does NOT need to match the Windows OS version or Windows SDK version.

For the full support matrix, see Windows App SDK supported Windows releases.

How the three components relate

```
Your App (.exe / .msix)
  │
  ├── References: Windows App SDK (NuGet package, e.g., 2.2)
  │     • Provides: WinUI 3, app lifecycle, windowing, push notifications
  │     • Updated: independently via NuGet
  │     • Constraint: has a minimum supported OS version
  │
  ├── Compiled against: Windows SDK (e.g., 10.0.26100)
  │     • Provides: Win32, WinRT, COM API declarations
  │     • Updated: with Visual Studio or standalone installer
  │     • Constraint: determines which APIs are visible at compile time
  │
  └── Runs on: Windows OS (e.g., Windows 11 24H2, build 26100)
        • Provides: actual API implementations at runtime
        • Updated: Windows Update
        • Constraint: determines which APIs are callable at runtime
```

Key relationships:

1. Windows App SDK **does not replace** the Windows SDK — you need both.
2. The Windows SDK version can be **newer** than the OS your app runs on (use runtime checks for newer APIs).

3. The Windows App SDK version is **completely independent** — update it without changing anything else.
4. You cannot call a platform API at runtime if the user's OS build is older than the build that introduced it, regardless of which SDK you compiled against.

Project file configuration

MSBuild properties in your project file control platform targeting. The property names differ slightly between C# and C++:

C# project file (.csproj)

In SDK-style .NET projects (used by WinUI 3 and Windows App SDK templates), the Windows platform version is embedded in the Target Framework Moniker (TFM):

XML

```xml
<PropertyGroup>
    <!-- The TFM includes the Windows SDK version you compile against -->
    <TargetFramework>net8.0-windows10.0.26100.0</TargetFramework>

    <!-- The minimum OS version your app supports -->
    <SupportedOSPlatformVersion>10.0.19041.0</SupportedOSPlatformVersion>

    <!-- Or equivalently: -->
    <!-- <TargetPlatformMinVersion>10.0.19041.0</TargetPlatformMinVersion> -->
</PropertyGroup>

<ItemGroup>
    <!-- Windows App SDK (independent of the above) -->
    <PackageReference Include="Microsoft.WindowsAppSDK" Version="2.2.0" />
</ItemGroup>
```

> ⓘ Note
>
> `SupportedOSPlatformVersion` and `TargetPlatformMinVersion` are equivalent in SDK-style projects. The .NET SDK platform compatibility analyzer uses `SupportedOSPlatformVersion` to emit compile-time warnings when you call APIs that may not be available on your minimum OS version.

C++ project file (.vcxproj)

XML

```
<PropertyGroup>
    <!-- Note: C++ uses "WindowsTargetPlatform" prefix -->
    <WindowsTargetPlatformVersion>10.0.26100.0</WindowsTargetPlatformVersion>
    <WindowsTargetPlatformMinVersion>10.0.17763.0</WindowsTargetPlatformMinVersion>
</PropertyGroup>
```

Expand table

Property (C#)	Purpose	Effect if set too high	Effect if set too low
Windows version in TFM (`TargetFramework`)	Compile-time API surface	Enables platform analyzer warnings (a benefit, not a problem)	Newer APIs are invisible; compiler errors if you reference them
`SupportedOSPlatformVersion` / `TargetPlatformMinVersion`	Minimum OS for installation	Fewer users can install your app	You must add runtime checks before calling newer APIs
`Microsoft.WindowsAppSDK` version	WinUI 3 and framework features	N/A (use latest stable)	Missing newer WinUI controls or framework features

Decision guide: choosing versions

Use the following rules to select versions for a new project:

1. **Set the Windows platform version in your TFM to the latest stable Windows SDK** (e.g., `net8.0-windows10.0.26100.0`). This gives you the broadest compile-time API surface and enables platform compatibility analyzer warnings for APIs unavailable on your minimum OS.
2. **Set `SupportedOSPlatformVersion` (or `TargetPlatformMinVersion`) based on your audience:**
    - Enterprise apps with managed devices: match your organization's minimum supported OS.
    - Consumer apps (broadest reach): `10.0.19041.0` (Windows 10 2004) or `10.0.17763.0` (Windows 10 1809).
    - Apps using features from a specific OS version: set to that version's build number.
3. **Always use the latest stable Windows App SDK.** There is no compatibility penalty; it supports back to Windows 10 1809.
4. **Never use preview/experimental SDK releases in production.**

Runtime API availability checks

When the Windows platform version in your TFM is higher than `SupportedOSPlatformVersion`, some APIs visible at compile time may not exist on the user's device. The .NET platform compatibility analyzer warns you about these at compile time, but you **must** also check availability at runtime before calling them.

Pattern: Check API contract (WinRT APIs)

C#

```
using Windows.Foundation.Metadata;

if (ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContract",
15))
{
    // Safe to call APIs introduced in this contract version.
    // For example, use a newer composition API:
}
else
{
    // Fallback for older OS versions
}
```

Pattern: Check for a specific type

C#

```
if (ApiInformation.IsTypePresent("Windows.UI.Composition.SceneVisual"))
{
    // Safe to use SceneVisual
}
```

Pattern: Check for a specific method

C#

```
if (ApiInformation.IsMethodPresent("Windows.Storage.StorageFile",
"GetFileFromPathForUserAsync"))
{
    // Safe to call this method
}
```

Rule: If you skip these checks and call an unavailable API, the app will fail at runtime. The exact failure depends on the API type — common exceptions include `TypeLoadException`, `MissingMethodException`, or a platform-specific error.

Common questions

"Which Windows SDK should I use?"

Use the latest Windows SDK available (currently 10.0.26100). You can always compile against a newer SDK than your target minimum — it only affects what's visible at compile time.

"Do I need to update the Windows SDK when I update the Windows App SDK?"

No. They are independent. You can update `Microsoft.WindowsAppSDK` from 1.7 to 1.8 without changing your Windows SDK version.

"My app targets Windows 10. Can I use the Windows App SDK?"

Yes. The Windows App SDK supports Windows 10 version 1809 (build 17763) and later. Set your `SupportedOSPlatformVersion` accordingly.

"What happens if I compile against SDK 10.0.26100 but the user runs Windows 11 22H2 (build 22621)?"

APIs introduced between build 22621 and 26100 will be visible in your code but will fail at runtime unless you add API availability checks. APIs from 22621 and earlier will work normally.

"How do I know which build number introduced a specific API?"

Check the API's reference page on Microsoft Learn. The **Requirements** section lists the minimum OS build (as an API contract version or Windows version).

"Are these SDKs distributed with my app?"

No — but parts of one of them can be. The **Windows SDK** is a development-time tool only; its headers and libraries are used by the compiler and nothing from it ships in your app package. The **Windows App SDK** is different: by default, its runtime binaries are included in your app package (framework-dependent or self-contained depending on your deployment model). In framework-dependent deployment, a shared Windows App SDK runtime is installed once on the device and your app references it; in self-contained deployment, the necessary binaries are

bundled directly into your MSIX or output folder. Either way, the platform APIs (Win32, WinRT, COM) are always provided by the OS itself — you never ship those. For details on deployment options, see Windows App SDK deployment overview.

Summary of rules

Expand table

Rule	Description
OS version = runtime capability	The user's OS build determines which APIs actually work at runtime.
Windows SDK = compile-time surface	The SDK you install determines which APIs are visible in your code.
Windows App SDK = app framework	Updated via NuGet, independent of OS and platform SDK versions.
SDK ≥ OS is safe	You can compile against a newer SDK than your minimum OS target.
Runtime checks are required	If your TFM targets a higher Windows version than `SupportedOSPlatformVersion`, guard newer API calls with runtime checks.
Windows App SDK back-compat	The Windows App SDK supports back to Windows 10 1809 regardless of its release version.

Related content

- Windows App SDK supported Windows releases
- Windows App SDK release notes
- Windows SDK archive
- Version-adaptive apps
- Set up your development environment

Last updated on 08/29/2026

Help and guidance for Windows developers

Whether you're just getting started or you're a professional developer troubleshooting a tricky issue, you're not alone. Connect with the Windows developer community, access expert resources, and find solutions to help you build great apps.



Quick guide

Use this table to find the right resource for your scenario:

Expand table

If you need to...	Go here
Start building a WinUI 3 app	Getting started with WinUI
See controls, patterns, and code samples	WinUI 3 Gallery
Browse working code samples	Windows App SDK samples on GitHub
Check what's new or changed	Windows App SDK stable channel release notes
Package, sign, or distribute your app	Windows App Development CLI (winapp CLI)
Report a bug in the Windows App SDK	Windows App SDK issues on GitHub
Report a bug in WinUI	WinUI issues on GitHub

If you need to...	Go here
Ask a technical question	Microsoft Q&A or Stack Overflow
Publish an app to the Microsoft Store	Create a developer account

Getting started

New to Windows app development? These resources help you go from zero to a running app:

- **Create your first WinUI app** - Set up your development environment and create your first app
- **WinUI 3 Gallery** - Explore WinUI 3 controls, design patterns, and code samples in an interactive app
- **Windows App SDK samples** - Browse working code examples covering common scenarios
- **Create a WinUI project from the command line** - Use `dotnet new` templates to scaffold a WinUI app without Visual Studio

Developer tools

These tools help you build, package, and ship Windows apps from any editor or framework:

- **Windows App Development CLI (winapp CLI)** *(open-source, preview)* - Package, sign, and manage Windows apps from the command line. Works with WinUI, WPF, Electron, Rust, Flutter, and more. Read the announcement blog post.
- **WinApp VS Code extension** *(preview)* - Run, debug, and package Windows apps directly in VS Code.
- **WinUI agent plugin for GitHub Copilot and Claude Code** *(open-source, preview)* - AI-assisted scaffolding, building, and iteration for WinUI apps.
- **AI Dev Gallery** *(open-source)* - Interactive samples for on-device AI on Windows, including Windows AI APIs.
- **Dev Home** *(open-source)* - Set up your development environment, manage dev drives, and monitor system performance.

Debugging with GitHub Copilot

GitHub Copilot can help you diagnose and fix bugs in your Windows apps. Whether you're hitting a XAML binding failure, an unexpected crash, or confusing API behavior, try asking Copilot directly in Visual Studio or VS Code. Here are some sample prompts to get you started:

Expand table

Scenario	Sample prompt
XAML data binding not updating	*"My `x:Bind` property isn't updating the UI when the value changes. The property is in my ViewModel. What am I missing?"*
App crashes on launch	*"My WinUI 3 app throws an unhandled exception during startup. Here's the stack trace — what's the likely cause?"*
Async/await deadlock	*"My UI freezes when I call an async method. How do I avoid blocking the dispatcher thread in a WinUI app?"*
Packaging or deployment error	*"I'm getting error 0x80073CFD when deploying my MSIX package. What does this mean and how do I fix it?"*
API returning unexpected results	*"The `StorageFolder.GetFilesAsync` call returns an empty list even though the folder has files. What permissions or capabilities might I be missing?"*
Memory leak investigation	*"My app's memory usage keeps growing. How do I find which objects aren't being garbage collected in a WinUI 3 app?"*

> **Tip**
>
> For best results, paste relevant code snippets or error messages along with your prompt. Copilot works best when it has specific context about your problem.

To learn more about using AI-assisted development tools with Windows apps, see Build native Windows apps with AI agents.

Documentation and reference

Official documentation, API references, and release notes for experienced developers:

- **Windows App SDK documentation** - Comprehensive SDK docs covering APIs, deployment, and packaging
- **Windows App SDK API reference** - WinRT API reference for the Windows App SDK
- **Windows App SDK stable channel release notes** - What's new, fixed, and changed in each release
- **WinUI 3 controls and patterns** - Design guidance and XAML control reference
- **Package identity overview** - Understand packaged, packaged with external location, and unpackaged deployment options

Community support

Connect with other Windows developers to ask questions, share knowledge, and get help:

- **Windows App Development Discord** - Real-time chat with other Windows developers
- **Microsoft Q&A** - Ask technical questions and get answers from Microsoft experts and the community
- **Windows App SDK repository** - Connect with other developers and the Windows App SDK team
- **WinUI 3 GitHub repository** - Get help and share feedback about WinUI
- **Stack Overflow** - Browse thousands of answered questions tagged with Windows App SDK

Stay connected

Follow the latest news, updates, and tips from the Windows developer team.

Social media

Expand table

Platform	Accounts
Bluesky	Windows Developer Docs · Microsoft Developer · .NET · Windows Insider
LinkedIn	Windows Developers · Microsoft Developers
Instagram	@windowsdeveloper · @microsoftdeveloper · @dotnet.developers

Blogs and videos

- Windows Developer Blog - Official announcements and deep dives
- #ifdef Windows Blog - Hands-on tutorials and tool announcements
- Windows Developer YouTube - Video tutorials and conference sessions
- Microsoft Developer YouTube - Broader Microsoft developer content
- .NET YouTube - .NET tutorials and updates

Partner Center and Store support

If you're publishing apps to the Microsoft Store or need help with Partner Center accounts, submissions, or certification:

Store publishers

- **Partner Center dashboard** - Manage your apps and submissions
- **Create a support case with Dev Center support** - Get help with Partner Center account management, enrollment, app management, payouts and more - Entra ID users click here and MSA users click here.
- **Report app certification issues** - Contact the certification team about specific app failures

Enterprise support

For organizations with support contracts:

- **Create a support incident** - *(requires Microsoft support contract)*

Application compatibility

If your app isn't working correctly on Windows or you need help modernizing an existing application:

- **App Assure** - Free Microsoft engineering support to help your apps work on Windows 10, Windows 11, and Microsoft Edge

---

Last updated on 09/10/2026

Windows App Development Frequently Asked Questions

This FAQ provides answers to common questions about Windows application development, including guidance on choosing the right framework for your projects. Topics covered include:

- Getting started and the Windows app development landscape.
- Native Windows-only app development with WinUI 3, Windows Presentation Foundation (WPF), and Windows Forms (WinForms).
- Windows Software Development Kit (SDK) and Windows App SDK.
- Targeting Windows as part of your cross-platform development strategy.
- Hybrid and web app development with .NET MAUI, Blazor, and ASP.NET Core.
- How to choose an approach while understanding Microsoft's investments.

Windows app development landscape

▼ Where can I find a straightforward overview of Windows development technologies?

> For an overview of today's options for Windows developers, watch the Windows Dev Chat episode Choosing your ideal dev platform, which discusses WinUI 3, .NET MAUI, React Native, Blazor, and Progressive Web Apps (PWAs). You can find other episodes in the Windows Dev Chat playlist.
>
> You can also refer to the overview of app development options for Windows developers.

▼ Why is client app development still crucial for modern digital transformation in the era of cloud services?

> In the age of cloud services, client app development remains important for delivering responsive, meaningful interactions on user devices.
>
> Here's why client apps matter:
>
> - **Device reach:** Client apps let you bring your application directly to users on their devices of choice.
> - **Gateway to Intelligent Services:** Client apps are often the first interaction users have with your services. They offer a rich, interactive interface that allows you to showcase

intelligent features and differentiate your product from others.
- **Scalability with Cloud Integration:** A well-integrated client app can sync effortlessly with backend cloud services, enabling real-time data access and seamless scalability as your user base grows.
- **Enhanced Productivity and User Loyalty:** A thoughtfully designed app can enhance productivity and keep users engaged with your product or service over time.

Native Windows-only app development

▼ What is the Windows App SDK?

> The Windows App SDK provides independently serviced components for Windows desktop apps, including WinUI 3, app lifecycle, windowing, notifications, resources, and text APIs. It supports apps that run on Windows 10, version 1809 and later, subject to the support lifecycle of the Windows release and Windows App SDK version.

▼ What's the difference between the Windows App SDK and the Windows SDK?

> Both are software development kits (SDKs) that let you build Windows apps.
>
> The **Windows App SDK** provides components that ship independently from Windows and work across supported Windows releases down to Windows 10, version 1809. It includes WinUI 3 and APIs for app lifecycle, windowing, notifications, resources, text, and other capabilities.
>
> The **Windows SDK** provides headers, libraries, metadata, and tools for operating-system APIs such as Win32, WinRT, COM, DirectX, devices, and shell capabilities.
>
> **The Windows App SDK doesn't replace the Windows SDK.** Apps that adopt the Windows App SDK can continue to use Windows SDK APIs, and WinUI 3 apps commonly use both.

▼ I'm building a new team to develop a Windows-only app. Why should I choose to develop with a native Windows framework like WinUI 3, WPF, or WinForms?

> Here are some reasons to choose a native Windows framework for your Windows-only app:
>
> - **Performance:** Native Windows frameworks are optimized to leverage modern Windows hardware, providing fast and responsive user experiences.
> - **Integration:** Windows ships with a wide variety of APIs that enable sophisticated experiences only available on Windows. Native frameworks provide deep integration

with these features and APIs.

- **Native user experience:** Native frameworks provide a consistent experience across Windows devices, ensuring that your app looks and works great everywhere.
- **Offline support:** Native frameworks support offline scenarios, allowing apps to function even without internet connectivity.
- **Support and tooling:** Microsoft maintains the native frameworks and provides current SDKs, documentation, debugging tools, and samples.

▼ Which framework should I use to leverage Microsoft's latest investments in Windows app development?

If you're building a new general-purpose Windows desktop app, we recommend using WinUI 3. WinUI 3 is the native UI framework delivered with the Windows App SDK. It supports Windows desktop apps and provides access to current Fluent controls and Windows platform capabilities.

▼ Can I use Windows App SDK / WinUI 3 in my existing Windows app?

Note that WinUI 3 (a UI framework) ships with the Windows App SDK (a Windows platform development framework).

You can migrate an app's UI to WinUI 3, or use WinUI XAML Islands to host Windows App SDK controls in a supported existing desktop host. Legacy system XAML Islands host UWP XAML controls and use different APIs.

Elements of the Windows App SDK can often be used in desktop apps, depending on how the existing app was built. UWP apps are not supported by Windows App SDK.

This means **WPF/MFC/WinForms** apps can use Windows App SDK APIs that are unrelated to WinUI 3. Examples include app lifecycle, windowing, and app notifications.

See Use the Windows App SDK in an existing project for more info.

▼ Do I need to use Visual Studio to build WinUI 3 apps?

No. WinUI 3 XAML builds use MSBuild, but you can build with the .NET SDK and current WinUI 3 templates from the command line in another editor. See the command-line quickstart.

Visual Studio 2026 provides the richest integrated editing, debugging, profiling, and XAML Hot Reload experience. Use the workflow that matches your tooling requirements.

▼ I get an "Unable to load DLL 'Microsoft.ui.xaml.dll'" error when running my app. How do I fix it?

> This error usually occurs in **unpackaged** app scenarios where the Windows App SDK runtime hasn't been installed on the machine. Try the following:
>
> - If you're running a **packaged** app (the recommended default), ensure you're launching via Visual Studio with the **MsixPackage** launch profile selected (not the plain executable profile). The MSIX packaging step installs the required runtime components.
> - If you're running a framework-dependent **unpackaged** app, install the matching Windows App SDK runtime. A self-contained deployment includes its Windows App SDK dependencies.
> - Confirm that your project matches your deployment model. For a normal .NET unpackaged app, setting `<WindowsPackageType>None</WindowsPackageType>` enables Windows App SDK runtime auto-initialization. Use the bootstrapper API directly only when you need explicit control over dynamic dependency initialization.
>
> See Deploy apps that use the Windows App SDK for more details on deployment requirements.

▼ What is the difference between WinUI 3 and WinUI 2 for UWP?

> **WinUI 3** is Microsoft's current native UI framework for Windows desktop apps and is delivered as part of the Windows App SDK.
>
> **WinUI 2**, also called **WinUI for UWP**, is a control and styling library for UWP apps. WinUI 2 and WinUI 3 use different XAML namespaces and aren't binary-compatible.

▼ When I build an app using Windows App SDK and WinUI 3, am I building a "WinUI app"?

> Yes. **WinUI 3 app** is the clearest term for an app whose UI uses WinUI 3 and the Windows App SDK. **WinUI app** is also commonly used when the context is unambiguous.

▼ Can I incrementally update my UWP app with WinUI for UWP controls to WinUI 3 by gradually replacing the controls?

> No. Windows App SDK can't be used in UWP apps, and WinUI for UWP can't be mixed with WinUI 3. See Migrate from UWP to the Windows App SDK.

▼ How hard is it to migrate a UWP app to WinUI 3?

> UWP and WinUI 3 share many XAML concepts, but migration isn't a direct namespace change. The cost depends primarily on:
>
> 1. **Project file and MSBuild customization:** Migration effort varies depending on advanced MSBuild usage.
> 2. **.NET API migration:** UWP apps using .NET Native can move to a currently supported .NET release with Native AOT. This modernization is separate from migrating the UI to WinUI 3.
> 3. **UI component libraries:** Libraries must have versions targeting WinUI 3.
> 4. **Windowing and application-model APIs:** UWP APIs tied to concepts such as `CoreWindow`, `ApplicationView`, or `GetForCurrentView` require Windows App SDK replacements or another desktop approach.
> 5. **C++ language projection:** If the UWP app uses the superseded C++/CX projection, port that code to C++/WinRT.
>
> For more info, see Migrate from UWP to the Windows App SDK and the UWP to Windows App SDK API mapping.

▼ If I have an existing UWP app in the Store, can I publish a new packaged WinUI 3 app using the same identifiers?

> Yes, upgraded apps can be published without updating the application identity. Users of the old version will be updated to the new version. This applies to desktop apps only. Xbox, HoloLens, and standard Surface Hub apps cannot migrate to WinUI 3.

▼ How do I package or distribute my WinUI 3 app?

> See Package and deploy overview.

▼ Where can I find Windows App SDK migration guidance?

> See Migrate from UWP to the Windows App SDK.

▼ Do I need to use XAML markup if I want to use WinUI 3?

> No. UI controls can be created in code. However, representing the UI in declarative XAML markup provides many benefits, including an improved developer experience.
>
> - Migrating from UWP to WinUI 3: Many XAML and UI concepts carry over, but the namespaces, project model, and some APIs differ.

- Migrating from WPF to WinUI 3: Many concepts carry over, but the control set and APIs differ.

▼ Does Visual Studio have a design surface or UI designer for WinUI 3?

Not currently. Use XAML Hot Reload, Live Visual Tree, Live Property Explorer, and related runtime tools to inspect and update XAML while the app runs.

For a complete walkthrough of the runtime design tools available for WinUI 3, see XAML runtime design tools for WinUI 3.

▼ Does Windows App SDK include WinUI 3?

Yes. WinUI 3 ships as part of the Windows App SDK.

▼ Does Windows App SDK include WinUI for UWP?

No. WinUI for UWP is part of the UWP platform.

▼ Are WinUI for UWP and WinUI 3 built on the same technology?

Not quite. Although WinUI 3 started from the WinUI for UWP codebase, they are distinct technologies. Both are XAML-based UI frameworks that work across .NET and C++, but WinUI for UWP and WinUI 3 aren't compatible with each other.

▼ Can I use WinUI 3 without using Windows App SDK?

No. WinUI 3 ships as part of the Windows App SDK.

▼ Can I use WinUI 3 in an unpackaged app?

Yes. WinUI 3 and many Windows App SDK APIs work in unpackaged apps. However, some Windows capabilities require package identity, and framework-dependent unpackaged apps must initialize the Windows App SDK runtime. Compare the options in Packaging overview and Features that require package identity.

▼ What's the difference between XAML Islands and WinUI 3?

WinUI 3 is the UI framework included in the Windows App SDK. XAML Islands are a hosting technique that lets an existing desktop app place XAML content alongside UI from another

framework.

The term can refer to legacy system XAML Islands that host UWP XAML controls, or to WinUI XAML Islands that host Windows App SDK controls in supported desktop hosts. The APIs, namespaces, and host requirements differ.

▼ If I create a WinUI 3 app, will it look modern on both Windows 11 and Windows 10?

WinUI 3 controls use Fluent styling on supported versions of Windows 10 and Windows 11, in both packaged and unpackaged apps. Some operating-system effects and behaviors differ by Windows version. For example, Mica is available on Windows 11 and falls back to a solid color on Windows 10.

▼ Can I use Mica or Acrylic backgrounds in apps built with Windows App SDK?

Yes. Desktop Acrylic is supported on Windows 10, version 1809 and later. Mica requires Windows 11 and falls back to a solid theme color on Windows 10. Call `MicaController.IsSupported` or `DesktopAcrylicController.IsSupported` at run time before applying a backdrop. See Apply Mica or Acrylic materials in desktop apps for Windows 11.

▼ Where can I find WinUI 3 samples?

See Sample and resources. Some notable repositories:

- WindowsAppSDK-Samples: Demonstrates how to use specific Windows App SDK API sets.
- Windows topic-specific samples: Contains the sample used in the Build a WinUI 3 notes app tutorial.
- WinUI 3 Gallery: Showcases WinUI and Windows App SDK. Also available in the Microsoft Store.

▼ If I have already invested heavily in WPF, should I continue to use WPF or consider migrating to WinUI 3?

If you've already invested heavily in WPF, you can continue using it for existing apps. WPF is a mature, stable framework widely used to build Windows desktop apps.

Use GitHub Copilot upgrade to assess and upgrade a .NET Framework WPF app to modern .NET. Review the generated plan and validate each change in your app.

▼ If I build a new WPF app, will it look dated compared to other new Windows apps?

> When developing a WPF application with .NET 9 or later, you can ensure your app matches the sleek, modern look of Windows 11. The new Fluent theme for WPF introduces a contemporary Windows 11 aesthetic, with integrated Light/Dark mode and system accent color support. This modernizes your app's appearance and delivers a polished, cohesive user experience.

▼ My team is comfortable building WinForms apps, and it suits our needs. Should we consider migrating to WinUI 3 or another framework?

> If WinForms meets your needs and your team is comfortable with it, you can continue using WinForms for existing apps. WinForms is a mature and stable framework widely used for Windows desktop development.
>
> The WinForms team continues to invest in the platform. Recent and ongoing work includes:
>
> - Asynchronous form and dialog APIs
> - Dark mode and visual-style support
> - Accessibility, high-DPI, layout, and designer improvements
> - Clipboard and `DataObject` modernization

Cross-platform native development

▼ What are some reasons for building cross-platform, native apps that target Windows?

> If you're targeting users across multiple OS platforms, building cross-platform apps with .NET MAUI or React Native can offer several benefits:
>
> - **Reach:** Cross-platform apps reach a larger audience across different devices and operating systems.
> - **Code reuse:** Reusing code across platforms reduces development time and cost. Building separate apps for Windows, Android, iOS, and macOS can be prohibitively expensive.
> - **Consistent user experience:** Cross-platform frameworks help provide a consistent look and feel across platforms.
> - **Integration:** Cross-platform apps can still integrate with platform-specific services to deliver a comprehensive experience.

▼ Can I be confident that .NET MAUI apps will run well on Windows?

> When you build a .NET MAUI app for Windows, the output uses WinUI 3. During development, .NET MAUI offers a single .NET experience across platforms, but it generates platform-specific code under the hood.

▼ How can .NET MAUI provide native device APIs across every platform?

> .NET MAUI provides a unified .NET experience across Windows, iOS, Android, and macOS. It offers cross-platform APIs for common capabilities such as storage, networking, and device sensors. You can also call platform-specific APIs or provide specialized implementations for each platform.

▼ Can I start with WinUI 3, and later integrate .NET MAUI if I eventually want to target cross-platform scenarios?

> Not at this time. Although .NET MAUI uses WinUI 3 when running on Windows, teams expecting to target multiple platforms should start with .NET MAUI or React Native for Desktop.

▼ Our team has strong web front-end development skills. Should we consider using React Native for Desktop?

> Teams with strong web development experience may want to consider React Native for Desktop. It includes React Native for Windows and macOS. With the "Learn once, write anywhere" approach, existing JavaScript, TypeScript, and React skills can be used to build native Windows and macOS apps.
>
> React Native for Desktop renders UI directly to native primitives, delivering native performance and platform capabilities.
>
> See the React Native for Desktop documentation to get started.

▼ Are any other Windows devices supported by React Native for Desktop?

> React Native for Windows supports the Windows versions listed in its compatibility documentation. Verify device-family support for the React Native for Windows version you target rather than assuming that every Windows device is supported.

▼ What should I use if I want to build apps that work on Windows and Xbox?

> For an Xbox app, use UWP and account for the Xbox-specific UWP limitations. For game development, use the Microsoft Game Development Kit.

▼ What should I use if I want to build apps that work on Windows and Surface Hub?

> For a Surface Hub running the standard Teams Rooms or Surface Hub environment, use a UWP app that meets the Surface Hub app requirements. A Surface Hub 3 configured with Windows 11 Pro or Enterprise can run supported desktop app technologies, so UWP isn't the only option in that configuration.

Hybrid and web development

▼ What are hybrid apps, and why should I consider building one?

> Hybrid apps blend the best of web and native app development. Their core is built using web technologies like HTML, CSS, and JavaScript, and wrapped in a native container that gives access to certain native platform features and hardware. They can also be distributed through app stores.
>
> The main advantage is that hybrid apps allow you to build a single app that can run on multiple native platforms and on the web, reducing development time and cost. Examples of hybrid app development platforms include:
>
> - Electron for desktop apps
> - Ionic for mobile apps
> - .NET MAUI Blazor Hybrid for cross-platform apps

▼ How do I build native-feeling progressive web apps (PWAs) on Windows?

> See Web development on Windows and Overview of Progressive Web Apps.

▼ What is a .NET MAUI Blazor hybrid app?

> With .NET MAUI, Blazor apps can run natively on Windows, iOS, Android, and macOS. This allows you to create hybrid client apps that combine Blazor and .NET MAUI components in a single native client app, with full access to native platform capabilities.

> Learn more at ASP.NET Core Blazor Hybrid.

▼ Do the web components of a .NET MAUI hybrid app need to be created with Blazor?

> No. Starting with .NET 9, .NET MAUI includes a HybridWebView control that allows hosting other JavaScript-based UIs inside a native app.
>
> This allows you to host Angular, React, Vue, or other HTML/JavaScript apps inside a .NET MAUI app. The hybrid control provides interop between C# and JavaScript, so C# code can call JavaScript functions and vice versa.

▼ Can any other native app types host Blazor hybrid components?

> Yes. WPF and WinForms apps can also host Blazor hybrid components, enabling the addition of modern web UI to existing apps. This is not supported for WPF or WinForms apps built on .NET Framework.

▼ Does my entire app need to be a hybrid app, or can I mix and match native and hybrid components?

> Native and hybrid components can be mixed within an app. For example, the core of an app may be built with .NET MAUI components while hybrid components provide additional functionality. This allows combining the performance and capabilities of native components with the flexibility and cost efficiency of hybrid components.

▼ What are my choices for building .NET-based web apps that look great on modern browsers on Windows?

> Web apps offer the broadest reach of any client app platform. Options for creating beautiful .NET web apps include:
>
> - ASP.NET Core apps with Razor Pages
> - ASP.NET Core MVC apps
> - ASP.NET Core Blazor apps, with hosting model options:
>   - Blazor WebAssembly
>   - Blazor Server
>
> Blazor hosting models can now be configured at the component level, enabling scenarios like hosting a Blazor WebAssembly component within a Blazor Server app.
>
> See the ASP.NET Core documentation for more details.

Choose an approach and understand Microsoft's investments

▼ There are so many framework options for building apps that target Windows! How do I decide?

> Windows is an open platform that supports many technologies. Here are some criteria that can help you choose a platform:
>
> - Are you building Windows-first or cross-platform?
> - What languages or skills do you already have — .NET, JavaScript, something else?
> - Do you need access to Windows-specific APIs?
> - Which framework's capabilities best match your app's requirements?
> - See this table for additional comparison factors.
>
> For many business apps, teams often choose based on existing skills and what the team is most comfortable using.

▼ How do I choose the best development approach for my web app?

> Consider the following when choosing a development approach for your web app:
>
> - Blazor is recommended for building front-end web apps with .NET. It lets you build both the front-end and back-end using .NET, saving time and cost, and it's especially good for enterprise apps.
> - JavaScript web apps still make sense if you want to leverage existing JavaScript skills or need to integrate with established JS libraries or frameworks.
> - Existing apps using older frameworks like Web Forms, MVC, or Razor Pages remain supported and can continue to be developed and maintained.

▼ Who is building apps with WinUI 3 today?

> Microsoft Photos is one documented example. The app migrated from UWP to the Windows App SDK and continues to use WinUI 3. For details about the architecture and migration, see Microsoft Photos: Migrating from UWP to Windows App SDK.

▼ Who is building .NET MAUI apps today?

Organizations use .NET MAUI to build cross-platform apps for Android, iOS, macOS, and Windows. See examples in the .NET customer showcase.

▼ Who is building WPF apps today?

Most of the Microsoft Visual Studio UI is built with WPF. The Visual Studio IDE itself is a major example of a complex, high-performance WPF app.

▼ Who is building Blazor apps today?

GE Digital's FlightPulse airline system uses Blazor for the backend configuration of everything pilots see, bringing sensor data and analytics directly to pilots to improve safety and efficiency.

See more Blazor customer stories on the .NET site.

Language choice (.NET vs C++)

▼ Should I use C# or C++ for my Windows app?

Use C# (.NET) in most cases. C# offers faster development, memory safety, rich libraries, and excellent tooling. Most Windows apps — including WinUI 3, WPF, WinForms, and .NET MAUI apps — are best built with C#.

Use C++ when you need direct hardware access, minimal runtime overhead, or interop with existing C++ codebases. Common C++ scenarios include game engines (DirectX), drivers, system-level utilities, and performance-critical components.

Expand table

Factor	C# (.NET)	C++
Development speed	✅ Faster — managed memory, rich ecosystem	⚠️ Slower — manual resource management
Runtime performance	✅ Excellent with modern .NET (AOT, Span<T>)	✅ Best possible — no GC pauses
Memory safety	✅ Garbage-collected	⚠️ Manual — risk of leaks and vulnerabilities
Windows API access	✅ Via C#/WinRT projection	✅ Via C++/WinRT projection

Factor	C# (.NET)	C++
WinUI 3 support	✅ Full support	✅ Full support via C++/WinRT
Cross-platform	✅ .NET runs on Windows, Linux, macOS	✅ With platform-specific code
Best for	Business apps, CRUD, services, UI-heavy apps	Games, drivers, system tools, low-latency

You can also mix both: build your app in C# and call performance-critical native code via P/Invoke (CsWin32) or a C++/WinRT component.

▼ How do I call Win32 APIs from C#?

Use CsWin32, a source generator that creates type-safe P/Invoke signatures at build time. You add the `Microsoft.Windows.CsWin32` NuGet package, list the APIs you need in a `NativeMethods.txt` file, and call them through a generated `PInvoke` class.

CsWin32 replaces hand-written `[DllImport]` declarations and works in any C# project, including WinUI 3, WPF, WinForms, and console apps. See Call Win32 APIs from a C# Windows app (CsWin32) for a step-by-step walkthrough.

▼ What is C++/WinRT and when should I use it?

C++/WinRT is a standard C++17 language projection for Windows Runtime APIs. Use it when building Windows apps in C++ that consume or author WinRT APIs. It replaces C++/CX and the Windows Runtime C++ Template Library (WRL).

Choose C++/WinRT when:

- You're building a C++ WinUI 3 app
- You need to author Windows Runtime components consumed by other languages
- You're porting from C++/CX

▼ What is C#/WinRT and when do I need it?

C#/WinRT provides WinRT projection support for C#. In most cases you don't interact with it directly — .NET apps targeting Windows automatically get access to WinRT APIs through target framework monikers (TFMs). You need C#/WinRT explicitly when authoring Windows Runtime components in C# or when generating interop assemblies for third-party WinRT components.

Packaging, deployment, and updates

▼ What's the difference between apps that are packaged, unpackaged, and packaged with external location?

> A **packaged app** contains its files, identity, and deployment information in a package such as MSIX. An **unpackaged app** uses an installer or deployment process outside the Windows package system and doesn't have package identity by default. An app **packaged with external location** uses a small identity package while retaining externally located binaries and its existing installer and update process.
>
> See Packaging overview for requirements and tradeoffs.

▼ Do I need package identity?

> It depends on the Windows features your app uses. Package identity is required for scenarios such as packaged background tasks, share targets, startup tasks, custom context-menu package extensions, manifest-based file-type and protocol associations, and many Windows AI APIs. Windows App SDK push notifications support limited foreground scenarios without identity, but background delivery and COM activation require identity. WinUI 3 and local app notifications can work without package identity.
>
> See Features that require package identity. If you need identity but must retain an existing installer, consider packaging with external location.

▼ What's the difference between framework-dependent and self-contained deployment?

> A **framework-dependent** app uses Windows App SDK runtime packages installed separately on the device. This reduces the app's deployment size and lets the installed framework receive servicing updates. A **self-contained** app carries its Windows App SDK dependencies with it, which increases deployment size and makes the app publisher responsible for distributing Windows App SDK servicing updates with new app versions.
>
> APIs that depend on additional MSIX packages, such as the Singleton package, can require separate deployment or runtime support checks even in a self-contained app. Packaging and runtime deployment are separate decisions. See Windows App SDK deployment overview.

▼ Will my WinUI 3 app automatically update for end-users?

A WinUI 3 app can be delivered through the Microsoft Store, an `.appinstaller` file, or an MSI or setup executable. Store packages can be updated through Microsoft Store servicing, subject to Store and organizational settings. An `.appinstaller` deployment supports automatic updates only when its `UpdateSettings` configure launch-time or background checks. MSI and setup deployments must provide or integrate their own update mechanism.

▼ Can I use Windows App SDK without using MSBuild?

Yes, for some scenarios. WinUI 3 XAML projects currently require MSBuild, although Visual Studio isn't required and `dotnet build` can invoke MSBuild from the command line. You can use non-XAML Windows App SDK APIs from C++ and CMake projects through the preview Windows App Development CLI, or integrate the runtime manually.

Windows AI

▼ How do I choose between Windows AI APIs, Foundry Local, and Windows ML?

The first three technologies are part of **Microsoft Foundry on Windows**. You can combine them with each other and with cloud models in the same app:

- Use Windows AI APIs for ready-to-use capabilities whose models and hardware acceleration Windows manages.
- Use Foundry Local to discover, download, and run supported open-source language and speech models locally.
- Use Windows ML to run your own ONNX models with execution providers for available CPU, GPU, and NPU hardware.
- Use Microsoft Foundry, a separate cloud AI platform, when you need cloud-hosted models, retrieval, centralized governance, or capabilities that aren't available on the target device.

Compare the options in Choose your Windows AI solution. Consider model capability, privacy, connectivity, latency, hardware coverage, deployment size, and operating cost.

▼ Do Windows AI features require a Copilot+ PC?

Not all of them. Many Windows AI APIs require a Copilot+ PC, but some APIs also support specific GPUs or CPUs. Foundry Local and Windows ML support broader hardware configurations, subject to their current operating-system, model, runtime, and execution-provider requirements.

> Check the Windows AI API hardware table and the requirements for the specific API or model. Detect support and model readiness at run time, and provide a non-AI, local-model, or cloud fallback when the feature is unavailable.

▼ Can Windows AI features run locally and offline?

> Yes. Windows AI APIs, Foundry Local, and Windows ML can run inference on the user's device, which can reduce latency and keep input data local. Some models or execution providers must first be downloaded or provisioned and can require an internet connection during setup or servicing. Cloud AI services require connectivity and send data to the service according to its data-handling terms.
>
> Tell users when a model download is required and when data leaves the device. Don't describe a feature as offline-capable until you've tested its complete first-run, update, and fallback experience.

▼ Can AI tools help me build or modernize a Windows app?

> Yes. AI coding agents can help scaffold projects, explain APIs, migrate code, generate tests, and diagnose build problems. Use the AI-assisted Windows development guidance for GitHub Copilot, the WinUI agent plugin, the Microsoft Learn MCP Server, migration workflows, and AI-assisted testing.
>
> Review and test generated code as you would any other contribution. In particular, verify API names and versions, package capabilities, security-sensitive code, accessibility, and any UWP-to-WinUI 3 substitutions.

▼ What should I consider before shipping an AI-assisted feature?

> Define the feature's intended use and limitations, evaluate quality and safety with representative data, disclose AI behavior where appropriate, protect user data, and provide a fallback when the model or required hardware isn't available. Keep secrets and privileged service credentials out of client apps, and require user confirmation before consequential or irreversible actions. See Responsible generative AI development on Windows and Security and responsible AI for Windows development.

Performance and optimization

▼ What can I do to make my Windows app feel great to end-users?

> See Windows application development - Best practices and Windows app performance and fundamentals overview.

Compatibility

▼ Will my users ever have to update Windows to use my WinUI 3 app?

> The Windows App SDK has a minimum compatible OS of Windows 10, version 1809, build 17763. Microsoft support requires a supported Windows App SDK release with its latest servicing update and a Windows edition, version, and servicing channel that is still supported. Individual APIs can require a newer Windows version or specific hardware. See Windows App SDK support and Release channels.

▼ Can I target Arm64 with my WinUI 3 app?

> Yes. Build a native Arm64 app for the best performance and efficiency. For a large C++ codebase with x64 dependencies, Arm64EC lets you migrate modules incrementally. Windows 11 on Arm can also run many existing x86 and x64 apps through Prism emulation, but you should test performance and compatibility on representative Arm devices.

Deprecations and migrations

▼ Are UWP / WinUI for UWP deprecated?

> UWP and WinUI 2 aren't formally deprecated. Visual Studio 2026 supports UWP with modern .NET and Native AOT, while WinUI 2.8 remains the latest stable WinUI release for UWP. However, Microsoft recommends WinUI 3 and the Windows App SDK for new general-purpose Windows desktop apps.
>
> UWP support for modern .NET with Native AOT is generally available and is the default C# UWP project type in Visual Studio 2026. Moving an existing UWP app from .NET Native to modern .NET is a separate modernization step from migrating its UI to WinUI 3. See Modernize your UWP app with .NET and Native AOT.

▼ When should I migrate a UWP / WinUI for UWP app to WinUI 3?

> UWP developers should not feel pressured to migrate if they are satisfied with UWP and its feature set — for many apps, the right choice may be to stay on UWP.

> Apps that want to benefit from the latest Windows platform and .NET investments should consider moving to WinUI 3 and the Windows App SDK. See Migrate from UWP to the Windows App SDK.

▼ When should I *not* migrate a UWP + WinUI for UWP app to WinUI 3?

> Continue using UWP when your target device or app model requires it, such as Xbox apps, HoloLens 2D apps, or apps for the standard Surface Hub environment. Windows IoT Enterprise supports desktop app technologies, including the Windows App SDK, so an IoT target isn't by itself a reason to use UWP.

▼ Is WPF deprecated?

> No. WPF is supported and continues to receive feature, performance, accessibility, and Fluent-style improvements in modern .NET. It remains a good choice for existing WPF apps and for new apps whose requirements fit WPF. For new general-purpose Windows desktop apps, Microsoft's primary recommendation is WinUI 3 with the Windows App SDK. See the WPF roadmap on GitHub.

▼ Is WinForms deprecated?

> No. WinForms is supported and continues to receive feature updates. See the Windows Forms Roadmap on GitHub.

▼ Is the Windows Runtime (WinRT) deprecated?

> No. WinRT is an application binary interface (ABI) that enables interop across multiple languages. WinRT is the evolution of COM, and the Windows App SDK provides most of its functionality through WinRT APIs.

Release notes

▼ Where can I find release notes for Windows App SDK?

> See the Windows App SDK release notes for stable, preview, and experimental releases. The What's new for Windows developers page summarizes the latest Windows SDK, Windows App SDK, WinUI 3, tooling, and platform updates.

Related content

- Windows developer glossary
- Overview of app development options

Last updated on 10/02/2026

Windows developer glossary

This glossary promotes a common vocabulary among Windows developers.

Advanced Windows Settings

The Windows 11 settings surface for developer-focused options such as Developer Mode, sudo, long paths, file extensions, Dev Drive, PowerShell script execution, and the default terminal. On Windows 11, version 25H2 and later, open **Settings** > **System** > **Advanced**. See Advanced Windows Settings.

App lifecycle management (ALM)

The processes and APIs that manage how an app starts, activates, runs, restarts, and terminates. UWP apps can transition between foreground, background, and suspended states. Windows App SDK desktop apps use the Win32 process lifecycle and aren't suspended by the UWP lifecycle manager. See UWP app lifecycle and Windows App SDK app lifecycle.

Application model

Often referred to as "app model." The application model combines deployment, isolation, lifecycle, and presentation components that are unique to a given application development technology. For example, Windows App SDK and WinUI 3 apps run on the Win32 app model, while WinUI for UWP apps run on the UWP app model.

Application packaging

Describes the way in which your app is packaged before being deployed and installed by users. An app can be packaged, unpackaged, or packaged with external location (see the Windows developer FAQ).

Bootstrapper

A redistributable component that provides an API to find and load the Windows App SDK framework package for the calling process. In a packaged-with-external-location or unpackaged app, you can load the Windows App SDK framework package explicitly by calling Bootstrapper APIs such as MddBootstrapInitialize. See Reference the Windows App SDK framework package at run time.

CsWin32

A .NET source generator that produces type-safe P/Invoke signatures for Win32 APIs at build time. You list the APIs you need in a `NativeMethods.txt` file and CsWin32 generates the correct interop code automatically. It's the recommended replacement for hand-written `[DllImport]` declarations. See Call Win32 APIs from a C# Windows app (CsWin32).

C++/WinRT

C++/WinRT is a standard C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-based library, and designed to provide first-class access to modern Windows APIs. See C++/WinRT.

Dev Drive

A storage volume optimized for developer workloads. Dev Drive uses the Resilient File System (ReFS) and provides faster performance for common development I/O operations like builds, package installs, and source control. See Set up a Dev Drive on Windows 11.

Dev Home

An open-source Windows developer dashboard that was retired in May 2025. Its repository is archived. Use Advanced Windows Settings, WinGet Configuration, and Dev Drive for current setup workflows.

DirectML

A high-performance, hardware-accelerated DirectX 12 API for machine learning workloads on supported GPUs. For higher-level ONNX inference across available CPUs, GPUs, and NPUs, see Windows ML. See DirectML overview.

Dynamic Dependencies

Dynamic Dependencies make framework package contents available to packaged and unpackaged apps at run time.

Fluent Design

Fluent Design is the design system for Windows experiences. WinUI 3 provides controls, materials, typography, and interaction patterns that implement Fluent guidance.

Foundry Local

A lightweight runtime for running small language models locally on Windows devices using available hardware (CPU, GPU, or NPU). Enables offline AI scenarios with low latency. See Get started with Foundry Local.

GitHub Copilot

An AI pair programmer that helps you write code faster and with less work in Visual Studio or Visual Studio Code (VS Code). See AI-assisted development in Visual Studio for more information.

Hot Reload

An app development feature that allows you to update your application's code and observe your changes while your application runs, eliminating the need to stop, rebuild, and re-run your apps while developing. See Write and debug running code with Hot Reload.

Hybrid app

An app that uses multiple technologies. For example, a .NET MAUI app that uses Blazor to render web content in a WebView2 control. See ASP.NET Core Blazor Hybrid for more information.

Hybrid CRT linkage

A C/C++ runtime library linkage technique that simplifies deployment. Also referred to simply as *Hybrid CRT*. See Hybrid C/C++ runtime library linkage (hybrid CRT linkage).

Large language model (LLM)

A transformer-based model trained on large corpora to understand and generate natural language (and sometimes images/audio). Supports tasks like chat, summarization, and code generation. See Azure OpenAI models.

Managed apps

"Managed" refers to the "managed runtime" of .NET, which provides managed services such as garbage collection and security assurances. If you're building an app with .NET, you're building a managed app.

MCP Servers

MCP is an open protocol designed to standardize integrations between AI apps and external tools and data sources. Model Context Protocol (MCP) Servers.

Microsoft Foundation Classes (MFC)

You can use Microsoft Foundation Classes (MFC) to create complex user interfaces with multiple controls. You can use MFC to create applications with Office-style user interfaces. See: MFC desktop applications.

Microsoft Foundry

A managed platform for building, evaluating, and deploying generative AI applications with foundation models. Formerly Azure AI Foundry. See Microsoft Foundry documentation.

MSIX

MSIX is a Windows app package format for reliable installation, update, and removal. You can use MSIX with apps built using Windows App SDK, Win32, WPF, or Windows Forms. An app deployed with MSIX is a *packaged app*. See What is MSIX?.

Native apps

A Windows-specific app that integrates with Windows UI and platform capabilities. A native Windows app can use managed C# and .NET, unmanaged C++, or both. This meaning is distinct from *native code*, which means code that executes outside a managed runtime.

P/Invoke (Platform Invocation Services)

The .NET mechanism for calling unmanaged (native) functions from managed C# code. Modern Windows apps should use CsWin32 to generate P/Invoke signatures automatically rather than writing `[DllImport]` declarations by hand.

.NET MAUI

.NET Multi-platform App UI. A cross-platform framework for creating native mobile and desktop apps with C# and XAML. An evolution of `Xamarin.Forms` extended from mobile to desktop scenarios, with UI controls rebuilt from the ground up for performance and extensibility. What is .NET MAUI?.

Neural Processing Unit (NPU)

A dedicated on-device AI accelerator optimized for machine learning workloads. Windows apps can use NPUs through supported Windows AI APIs, Foundry Local, or Windows ML execution providers.

ONNX Runtime (ORT)

A high-performance, cross-platform inference engine for models in the ONNX format. See Run ONNX models.

Packaged app

An app whose files, manifest, identity, and deployment information are contained in a package such as MSIX. Packaged apps have package identity and can use package-managed installation, update, and removal. See Packaging overview.

Packaged app with external location

An app that uses a small identity package while keeping its binaries outside the package and retaining its existing installer and update process. Also called an app with external location or a sparse package. See Packaging overview.

Package identity

A system-managed identity defined by a package manifest. Some Windows features, including certain background, push notification, shell integration, and Windows AI scenarios, require package identity. See Features that require package identity.

Framework-dependent deployment

A Windows App SDK deployment mode in which the app uses Windows App SDK runtime components installed separately on the device. It produces a smaller app deployment but requires the matching Windows App SDK runtime on the device. See Windows App SDK deployment overview.

PowerToys

A set of utilities for power users to tune and streamline their Windows experience for greater productivity. See PowerToys documentation.

Progressive web app (PWA)

An application that you build by using web technologies, and that can be installed and can run on all devices, from one codebase. See Overview of Progressive Web Apps (PWAs) for more information about building PWAs.

Project Reunion

The codename for the Windows App SDK. No longer in use.

React Native

React Native is a development platform from Meta which allows developers to build fully native cross-platform apps using JavaScript, TypeScript, and React.

React Native for Desktop

React Native for Desktop brings React Native support to the Windows SDKs, enabling developers to use JavaScript to build native Windows apps for devices supported by Windows 10 and Windows 11. This includes PCs, tablets, 2-in-1s, and Xbox. The term React Native for Desktop encompasses both React Native for Windows and React Native for macOS.

Responsible AI (RAI)

A set of recommended responsible development practices to use as you create applications and features. See Developing Responsible Generative AI Applications and Features on Windows to learn more about the RAI principles and how they apply to Windows development.

Small language model (SLM)

A compact model (for example, Phi Silica) designed to run efficiently on client devices (CPU, GPU, or NPU) with lower latency and cost, suitable for many on-device AI scenarios. See Get started with Phi Silica.

Text recognition

Text recognition, also known as optical character recognition (OCR), is supported by Windows AI APIs that detect and extract text within images and convert it into machine-readable character streams. See Get started with AI Text Recognition.

Universal Windows Platform (UWP)

An application development platform and application model that uses Windows Runtime (WinRT) APIs to deliver packaged apps. UWP apps run in a sandboxed environment, and they inherit the security of the UWP platform. Learn more about UWP.

> **Note**
>
> Build Windows apps with **Windows App SDK and WinUI 3**. You can also use **WPF**.

Unmanaged app

Apps that aren't managed by the .NET runtime. If you're handling your own memory management, you're building an unmanaged app.

Unpackaged app

An app installed and updated outside the Windows package deployment system, such as through MSI, EXE, ClickOnce, scripts, or xcopy deployment. An unpackaged app doesn't have package identity. To retain externally located binaries while adding identity, use the packaged-with-external-location model. See Packaging overview.

Self-contained deployment

A Windows App SDK deployment mode that includes the framework components with the app instead of using separately installed runtime packages. It increases deployment size and makes the app publisher responsible for distributing Windows App SDK servicing updates with new app versions. See Windows App SDK deployment overview.

Visual Studio extension (VSIX)

Lets you create, package, and deploy Visual Studio extensions. Get started with the VSIX Project template.

WebView2

A control that allows app developers to embed web content (HTML/CSS/JS) in their native apps using the Microsoft Edge (Chromium) rendering engine. You can use WebView2 in WinUI 3, Win32 C++, WPF, and WinForms. See Introduction to Microsoft Edge WebView2.

Microsoft Foundry on Windows

The umbrella for Windows AI APIs, Foundry Local, and Windows ML. Operating-system, hardware, model, and package requirements vary by technology and API. See Microsoft Foundry on Windows and Choose your Windows AI solution.

Windows API

Refers to the entire set of Windows APIs including Win32 APIs, COM APIs, UWP WinRT APIs, and the WinRT/Win32 APIs that are part of the Windows App SDK and WinUI 3.

Windows App Development CLI (winapp CLI)

An open-source, public-preview command-line tool for managing Windows SDKs, package identity, manifests, certificates, builds, packaging, and UI automation across Windows app frameworks. See Windows App Development CLI.

Windows App SDK

A set of independently serviced components and tools for Windows desktop app development. It includes WinUI 3, app lifecycle, windowing, notifications, resources, text, and other APIs. The Windows App SDK complements rather than replaces the Windows SDK. See Windows App SDK.

Windows App SDK runtime

The framework, Main, and Singleton packages used by framework-dependent Windows App SDK apps. Packaged apps normally acquire the runtime through package deployment; unpackaged apps use automatic or explicit dynamic dependency initialization. See Windows App SDK deployment overview.

Windows Forms

Also known as WinForms. A UI framework for building Windows desktop applications. It is a .NET wrapper over Windows user interface libraries, such as User32 and GDI+. It's a battle-tested way to create desktop applications using a visual designer within Visual Studio. See Desktop Guide (Windows Forms .NET).

Windows Presentation Foundation (WPF)

A UI framework for building Windows desktop applications. WPF applications are based on a vector graphics architecture. This enables applications to look great on high DPI monitors, as they can be infinitely scaled. See What is Windows Presentation Foundation (WPF)?.

Windows SDK

The Windows SDK is a collection of headers, libraries, metadata, and tools that allow you to build desktop and UWP Windows apps. The Windows SDK is not the same as the Windows App SDK.

WinUI 3

Microsoft's current native UI framework for Windows desktop apps. WinUI 3 is delivered through the Windows App SDK and is decoupled from the Windows operating system. It uses Fluent Design and will feel familiar if you've worked with WinUI 2 for UWP. See WinUI 3.

WinUI for UWP (WinUI 2)

WinUI 2, also called WinUI for UWP, is tightly integrated with Windows SDKs and provides native Windows UI controls and other user interface elements for UWP applications and desktop applications using system XAML Islands. WinUI 2 and WinUI 3 use different XAML namespaces and aren't binary-compatible. See WinUI for UWP.

XAML Islands

A hosting technique for placing XAML content inside an app that uses another desktop UI framework. Legacy system XAML Islands host UWP XAML controls. WinUI XAML Islands host Windows App SDK controls and use different APIs, namespaces, and host requirements.

Windows ML

Windows APIs for running ONNX models locally in Windows apps, with automatic execution provider management across CPUs, GPUs, and NPUs. See Windows ML.

Related content

- Windows developer FAQ
- Overview of app development options

Last updated on 09/16/2026

What's new for Windows developers

This section curates the latest platform capabilities, SDK and API additions, AI integration options, performance and diagnostics improvements, design guidance updates, and productivity tooling enhancements. Microsoft refreshes these highlights as new capabilities ship.

Latest SDK and WinUI releases

Find the latest downloads, release notes, and updates for the Windows SDK, Windows App SDK, and WinUI 3.

Versions below are current as of September 30, 2026. Use the release notes to review the changes and requirements for each channel or SDK build.



Windows App SDK

Latest stable: 2.5.1

Latest experimental: 2.5.4-experimental

Stable release notes

Experimental release notes

View downloads



Windows SDK

Latest 28000-series build: 10.0.28000.2705

Latest 26100-series build: 10.0.26100.9169

View release notes

View downloads

Highlights – September 2026

SDK releases and developer blog posts

- **Windows App SDK 2.5 Experimental (2.5.4-experimental)**: The September 29 experimental release adds AI compute-device selection, updated speech-recognition and language-model APIs, programmatic theme-resource binding, delegate-based `DataTemplate` construction, experimental `TableView` and Chart controls, and updated inking APIs. See the Windows App SDK 2.5 experimental release notes.
- **WSL Containers is generally available**: Build, run, and deploy Linux containers on Windows by using the `wslc.exe` CLI, its `container.exe` alias, or the WSL Containers API. The GA release adds container lifecycle, networking, health-check, storage, Microsoft Defender for Endpoint, and Microsoft Intune capabilities. See WSL Containers now generally available and the WSL Containers architecture deep dive.
- **Visual Studio September update**: The update introduces Bring Your Own Model in preview, Copilot assistance for NuGet vulnerability fixes, pull-request exploration in the Git agent, improved branch-condition debugging, and Podman support for Attach to Process. See Visual Studio September Update: Power Your Workflow with Your Model.

- **Inside MSIX: identify packaged processes**: Learn how to use Task Manager's **Package name** column to distinguish packaged processes from unpackaged processes. See How to tell if a process is packaged.
- **Windows App Development CLI 0.7.0**: The release adds Windows Sandbox execution, Native AOT support, one-step project packaging, single-file apps with package identity, the `winapp find-api` command, Reactor templates, and improved UI automation. See WinAppCLI v0.7.0 release announcement.
- **Windows App SDK 2.5.1**: The September 16 stable release adds support for the `windows.diagnosticServiceModule` package-manifest extension, limited-access App Content Search APIs for lexical and semantic search scenarios, and reliability fixes across WinUI, input, and composition. See the Windows App SDK 2.5.1 release notes.
- **.NET 11 performance improvements**: Review runtime, JIT, garbage collection, Native AOT, libraries, and SDK changes that improve application performance in .NET 11. See Performance improvements in .NET 11.
- **Intelligent Terminal 0.2.2572**: The update includes more than 60 performance and reliability fixes, faster startup and context gathering, API keys for local models, optional automatic approval, and native agent slash commands. See Intelligent Terminal 0.2.2572: Performance & Reliability.
- **.NET 11 Release Candidate 1**: The September 8 prerelease includes SDK tooling, C# 15, and Windows Forms updates, and comes with a go-live support license. Review the release notes and tool requirements before upgrading. See Announcing .NET 11 Release Candidate 1.
- **September .NET servicing updates**: Security and non-security updates are available for .NET 10, .NET 9, .NET 8, and .NET Framework. See .NET and .NET Framework September 2026 servicing updates.
- **Project Zenith**: The September 4 announcement describes a developer-focused Windows experience with preinstalled tools and development settings on new developer-class devices. Initial availability is planned for AMD's Ryzen AI Halo. See Announcing Project Zenith.
- **Inside MSIX: servicing packages that are in use**: Learn why deployment can return `ERROR_PACKAGES_IN_USE`, and how application shutdown and deferred updates affect servicing. See Servicing While In Use and ERROR_PACKAGES_IN_USE.
- **Inside MSIX: package storage**: Learn how `PackageVolume` manages staged packages, default and offline volumes, and storage on non-system volumes. See MSIX PackageVolume: How Package Storage Works.

Documentation highlights

- **WinUI 3 line-of-business apps**: A new hub brings together guidance on tabular data, validated forms, database access, design, migration, and AI for business applications. See Build line-of-business apps with WinUI.
- **AI-assisted WinUI development**: Build a task-tracking app while learning to inspect generated projects, check APIs against Microsoft Learn, and test the resulting UI and data behavior. See Build a WinUI 3 task app with an AI assistant.
- **AI-assisted modernization planning**: Use an updated, tool-neutral prompt that asks an assistant to inspect an existing desktop app, clarify constraints, and propose an incremental plan grounded in documentation. See Plan your modernization with an AI assistant.
- **WinUI XAML Islands**: Updated hosting guidance explains how to add Windows App SDK controls to an existing desktop app and distinguishes WinUI XAML Islands from legacy UWP system XAML Islands. See Host WinUI controls with XAML Islands.
- **WinUI 3 performance workflow**: The performance overview now connects measurement, startup optimization, responsiveness and rendering, and remeasurement, with links to focused guidance for each bottleneck. See Improve your WinUI 3 app's performance.

Highlights – August 2026

- **Windows App SDK 2.4 Experimental (2.4.1-experimental)**: The August 25 experimental release adds **inking support for WinUI 3** — the new `InkCanvas`, `InkToolbar`, and `InkPresenter` APIs bring pen and touch input, stroke input, unprocessed input, and clipboard and high-contrast support to WinUI 3 apps. It rolls the 2.4.0 stable changes forward on the experimental channel — Release notes | Install: `Microsoft.WindowsAppSDK` version `2.4.1-experimental` from NuGet
- **Windows SDK build 10.0.28000.2705**: The latest Windows SDK flight (August 2026) adds new WinRT and Win32 APIs, including `Windows.Management.Update` scan-mode and software-update identity/category types, `RetailInfo.IsDemoModeEnabledEx`, `ToastNotification.IsExpandableContentSupported`, a Direct3D 12 `ID3D12DeviceStatistics` interface, an AV1 encoder operating-mode codec API, and endorsement-key enrollment v2 cryptography OIDs — Windows SDK release notes | Downloads
- **Inside MSIX – What Kind of Packaged Process Is It?**: Howard Kapustein breaks down the three kinds of packaged process — Universal, Centennial/Desktop Bridge, and Win32alacarte — and how the `RuntimeBehavior` manifest attribute (`windowsApp`, `packagedClassicApp`, `win32App`) trades off compatibility against modern packaged behavior — What Kind of Packaged Process Is It?

- **PowerToys 0.101**: Adds **Window Hopper** for quickly moving between windows from the same app, a redesigned update experience with **Stable and Insider channels**, **compact mode** for Command Palette, on-device Phi Silica transformations in Advanced Paste, and continued WinUI 3 modernization — PowerToys 0.101 release
- **AppxManifest IntelliSense in the WinApp VS Code extension**: The WinApp VS Code extension now provides IntelliSense — completions, tooltips, and validation — for `AppxManifest.xml`, making it easier to author package identity and capabilities without leaving the editor — Announcing AppxManifest IntelliSense support
- **Windows App SDK 2.4.0**: The August 13 stable release adds touchpad and mouse haptics for WinUI 3 apps (via `Windows.Devices.Haptics`), touchpad single-finger panning on supported hardware, and more precise `LanguageModel` response statuses — `UnsupportedLanguage` and `LanguageMismatch` — so apps can distinguish language errors from generic failures. Also includes targeted reliability fixes for input, Storage Pickers (keyboard focus now restores after a dialog closes), MRT Core, app runtime isolation, and composition — Release notes
- **Windows App Development CLI v0.6.0**: Adds `winapp new` to scaffold a WinUI 3 app from official templates with a single command (blank, NavigationView, TabView, MVVM, class library, and unit-test starters), `winapp find-ui` to search and fetch copy-pasteable XAML and C# samples from WinUI 3 Gallery and Windows Community Toolkit without leaving the terminal, run-from-`.csproj` with no manual build step, cloud-based artifact signing via Azure, and sparse-package workflow support — WinApp CLI v0.6.0 announcement | Install: `winget install Microsoft.WinAppCli`
- **Intelligent Terminal 0.2**: The biggest release yet of the experimental AI-powered terminal, adding local model support in the Agent Pane, per-tab agent selection with `/agent`, OpenCode in the built-in agent lineup, WSL agent execution, and a richer Agent Pane UI — Intelligent Terminal 0.2 announcement | Install: `winget install Microsoft.IntelligentTerminal`
- **Inside MSIX – Package Lifetime**: How MSIX uses a garbage-collection design for staged package lifetime — why destaging is not a direct result of a single API call, what holds a reference, and when payloads are automatically removed from disk — MSIX Package Lifetime
- **The Old New Thing**: Raymond Chen's long-running blog on practical Windows development — internals, Win32, shell behavior, and the history behind platform decisions — The Old New Thing

---

Highlights – July 2026

- **Windows App SDK 2.3.1**: The July 16 stable release adds the **Structured JSON Output API** for generating language-model responses constrained to a caller-supplied JSON Schema, the **XamlOptionalChanges API** for opting into optional breaking changes before XAML initialization, a **CompositionEngine API** (Limited Access Feature) that apps can use to make the OS the engine for Composition APIs, Video Super Resolution improvements (NPU detection fix + CPU support), ARM64EC support for Windows ML, and a massive batch of XAML performance optimizations — Release notes
- **Visual Studio 2026 is here**: Now with AI-driven development improvements, integrated Copilot agents for debugging and profiling, and backward compatibility with VS 2022 projects — Visual Studio 2026 announcement
- **.NET 11 Preview 6**: Features union type support in C#, async validation APIs, AI/agentic workflow enhancements, NativeAOT improvements, and enhanced runtime/JIT performance — .NET 11 Preview 6
- **PowerToys 0.100**: A milestone release featuring a redesigned Shortcut Guide that detects the active app and shows relevant shortcuts as a side pane, a new Command Palette Extension Gallery for browsing and installing extensions, multi-monitor Dock support, and an upgrade to .NET 10 — PowerToys 0.100 release notes
- **WSL Containers**: Run Linux containers on Windows by using the new `wslc.exe` CLI or the `Microsoft.WSL.Containers` NuGet package, which provides C#, C++, and C projections for programmatically pulling, running, and interacting with Linux containers — including stdin/stdout, file mounts, networking, and GPU access — from your Windows app — WSL Containers overview

---

Announced at Build – June 2026

- **Windows Developer Configurations**: Get from a fresh Windows install to a ready-to-code environment in minutes with curated, open-source configuration files for toolchains, OS settings, and shells — Windows Developer Configurations.
- **Coreutils for Windows**: A collection of essential Unix-style command-line utilities, now available natively on Windows — Coreutils overview.
- **Intelligent Terminal**: An experimental, open-source fork of Windows Terminal with native agent integration, automatic error detection, and a built-in agent pane for pair-programming in the shell — Announcing Intelligent Terminal version 0.1.
- **WSL Containers (public preview)**: A CLI and API for creating, running, and interacting with Linux containers on Windows. Public preview became available on June 29. See WSL container is now available for public preview.

- **Windows Development Skills**: Structured knowledge that enables AI agents to build native Windows apps end-to-end by using WinUI 3 skills and WinApp CLI — Get started with Windows Development Skills.

Highlights – June 2026

- **Build 2026: Windows as the trusted platform for development**: Major announcements including developer-optimized Windows 11 and the Microsoft Execution Containers (MXC) SDK. For additional Build highlights, see **Announced at Build – June 2026**, earlier in this article — Build 2026 blog post
- **Developer-optimized Windows 11**: One-command setup for your dev environment with VS Code, GitHub Copilot, WSL, and PowerShell 7. Now generally available.
- **Microsoft Execution Containers (MXC) SDK**: Secure, policy-driven execution containers that let agent apps set containment boundaries on resources like files and networks.
- **Windows App SDK 2.0 features**: AI Video Super Resolution API (VideoScaler), ApplicationData for unpackaged apps, XamlBindingHelper improvements, TitleBar custom drag regions, and semantic versioning — Release notes
- **Inside MSIX – Staging: Sharing is Caring**: Deep dive on how MSIX staging works — pkgdir creation, ACL enforcement, and why the immutability guarantee lets the system safely share the same staged package across all users. Part 1 of a series by Howard Kapustein, MSIX Principal Engineer — Staging: Sharing is Caring

---

Highlights – May 2026

- **Inside MSIX blog**: New Microsoft engineering blog with deep dives on MSIX architecture, package identity, deployment operations, and the Stage/Register model — written by Howard Kapustein, MSIX Principal Engineer — Inside MSIX
- **WinUI agent plugin for GitHub Copilot and Claude Code**: A new plugin with eight specialized skills and a dedicated `winui-dev` agent for end-to-end WinUI development — scaffold, build, run, test, package, and migrate from WPF — in 70% fewer tokens than generic agents. Install with `/plugin install winui@awesome-copilot` — Introducing WinUI agent plugin for GitHub Copilot and Claude Code
- **`dotnet new` WinUI templates** (preview): New open-source project and item templates for WinUI that let you create and run apps directly from the command line without Visual Studio. Includes Blank, NavigationView, TabView, and MVVM templates, all built around

Windows app silhouettes with modern Fluent Design defaults — Introducing dotnet new WinUI templates

- **WinApp VS Code extension**: Brings the full Windows App Development CLI into VS Code — initialize, run, debug, package, and sign Windows apps from any framework (WinUI, WPF, C++, Electron, Rust, Tauri, Flutter) without leaving the editor — Announcing the WinApp VS Code Extension

---

Highlights – April 2026

- **WinUI 3 Gallery 2.9**: First release of WinUI Gallery built on Windows App SDK 2.0, with new control samples, updated defaults, and improvements across the board — Announcing WinUI 3 Gallery 2.9
- **Windows App Development CLI v0.3**: New `winapp run` and `winapp ui` commands, `dotnet run` support for packaged apps, and a full run-and-debug experience outside Visual Studio. Unlocks agentic scenarios where agents can run, see, and interact with a running Windows app — Windows App Development CLI v0.3
- **Agentic AI tools for Windows development**: New guide on enhancing AI coding agents with Windows-specific context — including the Microsoft Learn MCP Server for live documentation access, and the WinUI 3 development plugin for GitHub Copilot to generate accurate, modern WinUI 3 code — Agentic AI tools for Windows.
- **WinApp CLI cross-framework guides**: The Windows App Development CLI (public preview) now has step-by-step guides for adding Windows capabilities to apps built with **.NET, C++, Rust, Flutter, Electron, and Tauri** — including packaging, identity, AI integration, and notifications — WinApp CLI guides.
- **MapControl for WinUI**: New documentation for the MapControl, an interactive map powered by Azure Maps with support for pins, layers, and user interaction — MapControl guide.
- **Materials documentation**: New dedicated pages covering how to use Mica and Acrylic in your WinUI apps — Materials overview and In-app acrylic.
- **Store API updates**: New sections documenting how to check whether an unpackaged app is installed and how to open the Store product detail page for a product — Send requests to the Store.
- **Iconography documentation reorganized**: App icon and iconography docs now live in a dedicated Iconography hub for easier navigation.
- **Windows notifications overview**: Confused by `AppNotificationManager` vs `ToastNotificationManager`? New overview page explains which notification API to use for

your app type, with a feature comparison table and links to samples — Windows notifications overview.

- **Performance docs for WinUI**: The full set of Windows app performance documentation — startup, memory, XAML layout, animations, ListView/GridView optimization, and more — is now available under the WinUI developer section at Windows app performance.
- **Visual layer and Composition docs for WinUI**: Documentation for the Visual layer (`Microsoft.UI.Composition`) — including visuals, animations, effects, brushes, lighting, and shadows — is available in the WinUI/Windows App SDK developer section at Visual layer overview.
- **Command Palette extension toolkit**: New API reference documentation for the PowerToys Command Palette extension toolkit, covering built-in commands (`CopyPathCommand`, `OpenFileCommand`, `OpenInConsoleCommand`, and more) and layout types — Command Palette extension toolkit.
- **Java getting started for Windows**: New guide for setting up a Java development environment on Windows, covering JDK installation, `JAVA_HOME` configuration, editor options, and WSL considerations — Java on Windows.
- **Windows app development overview**: Landing page for Windows app capabilities, with entry points to accessibility, AI, files, notifications, UI, and more — Windows app development overview.
- **PowerToys 0.99**: New Power Display utility for controlling monitors from the system tray, Grab And Move for resizing windows from anywhere, and improvements to Command Palette and the Dock — PowerToys 0.99 release — Microsoft PowerToys: Utilities to customize Windows.
- **Windows Terminal 1.25**: New settings page for extensions, improved multi-language Command Palette suggestions, and a completely rebuilt windowing architecture with more reliable tray icon and Quake mode — Windows Terminal 1.25 release
- **Segoe Fluent Icons Font updated**: The Segoe Fluent Icons Font documentation now includes the latest icon additions and usage guidance — View icons.
- **WinUI terminology clarification**: Updated terminology across documentation for clarity — "WinUI 2" is now "WinUI for UWP" and "WinUI 3" is now just "WinUI" to reflect current naming conventions.
- **Windows Developer Support hub**: New centralized support page with quick help actions, community channels, and Microsoft support contacts — Get support.
- **Windows App SDK release notes**: Refactored and consolidated history from 0.5 through 2.0 — find the latest fixes and APIs in one place (release notes hub).
- **Windows SDK updates**: New overview and detailed release notes to track SDK changes (overview · release notes).

- **WinAppCLI public preview**: The Windows App Development CLI is a command-line interface for managing Windows SDKs, packaging, generating app identity, manifests, certificates, and using build tools with any app framework — (GitHub repo).
- **Cross Device Resume (XDR) overview**: Introduces Windows app continuity across devices and technologies available to enable XDR scenarios (overview).
- **Implement XDR using WNS raw notifications**: Step-by-step guide to integrate app continuity via WNS, including prerequisites and Python/JavaScript examples (how-to).
- **AI on Windows**: Fresh overview for building AI-powered experiences on Windows, with entry points to APIs and tooling (Windows AI overview).
- **Packaged vs. unpackaged guidance**: Updated guidance for choosing and configuring packaged or unpackaged apps, including WinUI scenarios (decision guide).
- **UWP capability declarations**: Revised topic clarifying capability types, privacy-sensitive capabilities, and Store submission considerations (declare capabilities).
- **Microsoft Store**: The latest news from the Microsoft Store including waived fees and updated analytics.

Windows app development documentation and resources

Expand table

Feature	Description
Create and run a WinUI 3 project	Comprehensive starting point for Windows app development.
WinUI agent plugin	8-skill GitHub Copilot / Claude Code plugin for end-to-end WinUI development.
dotnet new WinUI templates	Command-line project templates for WinUI (Blank, NavigationView, TabView, MVVM).
WinApp VS Code extension	Run, debug, and package Windows apps from any framework directly in VS Code.
Win32 app isolation overview	Security and reliability benefits of isolating Win32 apps.
AppWindow.SetIcon	API reference for setting a window icon (Windows App SDK).
Get started with Windows AI APIs	Quickstart building apps by using Windows AI.
Agentic AI tools for Windows	Connect AI coding agents to Windows docs and WinUI 3 best practices.
WinApp CLI overview	Command-line tool for packaging, identity, and SDK management across frameworks.

Feature	Description
MapControl	Interactive Azure Maps-powered map control for WinUI apps.
Materials in Windows apps	Overview of Mica and Acrylic materials for WinUI.
Windows notifications overview	Which notification API to use: `AppNotificationManager` vs `ToastNotificationManager`.
Windows app performance	Performance docs for WinUI apps: startup, memory, XAML layout, and more.
Windows app development overview	Landing page for Windows app capabilities: accessibility, AI, files, notifications, and more.

Last updated on 10/08/2026